

A++ 敏捷开发

Contents

前言 Prologue	19
I 案例分析	21
1 如何改善	23
敏捷开发过程改进案例	23
如何改善	26
丰田故事	27
水面下的管理思路	30
改善质量	39
结束语	41
附件	42
现代汽车生产	42
XP	43
“ 5 Why ” 实例	45
5S 法	45
参考 References	46
II 团队与自我改善基本功	47
2 改进从团队开始	51
美国费城 Weisbord 故事	51
问题处于 X 思维的主管	51
启发	53
芬兰计划	55
参考 References	58
3 克服拖延症	61
结束语	65
附件	66
锻炼之道 The Truth about EXERCISE	66
参考 References	67

III 自我管理开发过程	69
4 三点估算	73
估算是一个范围，不是一个数	73
从单点到三点估算	73
总结 + 解读分析结果	75
利用蒙特卡洛模拟 10 个步骤（三角形分布）的总分布	75
分析 10 个步骤模拟结果	77
问答 Q&A	78
附件	79
蒙特卡洛 (Monte Carlo) 模拟	79
5 量化管理从个人开始	83
如何降低项目进度偏差	83
从量化培训开始	84
从估算、策划到监控	84
应如何改善	86
附件	88
挣值分析法 (Earned Value)	88
如何简单记录工时	89
个体软件过程 Personal Software Process (PSP) 简介	89
使用挣值分析监控案例	91
参考 References	94
6 估算软件规模	95
为什么要估算规模	95
为什么不应用代码行数 (LOC)	95
怎样估算简化功能点 (SiFP)	97
从故事点转简化功能点	98
附件	100
简化功能点 (SiFP) 简介	100
逻辑文件 (Logical File)	105
基本过程 (Elementary Process EP)	106
实例：识别基本过程 (EP)	107
案例 A: 开发 (Development) 项目	108
答案与解读	109
案例 B: 优化维护 (FEM) 项目	110
答案与解读	111
与国际功能点 (IFPUG) 的偏差	112
参考 References	113
7 估算工作量	115
估算：靠模型，还是靠专家？	116
为什么估算开发工作量/工期这么困难	117
估算软件开发的困难	117
结束语	121

附件	122
估算砌乐高积木时长 / 2015 年学生实验数据	122
参考 References	127
IV 开始过程改进之旅	129
8 获取高层支持	133
案例	133
第一版方案书	134
第二版与中层确认, 然后向高层汇报	135
9 寻找改进机会	137
评估中访谈公司培训专员	138
总结以上访谈的问题	138
管理层的理解与支持	139
结束语	140
附件	141
CMMI 评估案例	141
V 如何做好迭代回顾	143
10 二八原则	147
怎样开始	148
附件	150
二八原则 (80/20)	150
开发项目工作量 (成本) 分布	152
公司高层不一定关注质量	152
11 团队协作分析根因	153
回顾流程	154
缺陷排除率	156
根因分析的误解	158
附件	162
FMEA 实例	162
如何分析与对应各类由于人员引起的缺陷	164
12 软硬兼施	167
团队迭代回顾根因分析常见问题	170
附件	171
Asch 1951 群众压力实验	171
Brehm 1956 决策影响实验	173
KJ 分析方法步骤	174
游戏: 应与否	175
13 从团队实验到持续改善	177

项目团队分析根因	177
内部教练如何做好现场辅导	182
从迭代复盘到持续改进	183
与 Y 公司总经理汇报	184
改进效果	184
附件	185
功能性分组帮助团队成长实例	185
敏捷回顾互动练习 1: 时间表 (Timeline)	186
敏捷回顾互动练习 2: 颜色点 (Color Code Dots)	187
缺陷排除预测模型实例	188
参考 Reference	193
VI 代码质量改进的基础	195
14 测试驱动开发	199
为什么我们要测试驱动开发	199
每小步验证	200
用 TDD 开发程序	201
单元测试与 TDD 的好处	202
结束语	203
附件	203
从 MIT OCW 学写程序	203
15 代码重构	207
如何学好 SOLID, 打好基础做重构	207
如何提升学员的兴趣与动力	207
什么时候做重构	208
代码规范	209
持续集成	209
附件	209
小孩利用乌龟玩几何游戏 (Turtle Geometry) 的例子	209
温伯格的教学实验	216
参考 References	216
16 持续集成	217
验收测试	217
测试自动化	218
持续集成	219
团队协作	220
持续交付	221
附件	221
持续集成	221
技术债务例子	222
参考 References	223

17 结对编程与同行评审	225
培训	226
结束语	229
附件	230
代码可读性例子	230
18 软件需求	231
附件	233
英国煤矿生产问题的研究	233
19 客户参与	239
常见问题	239
识别利益相关者	240
用户故事卡	240
用例与场景	241
举例	243
结束语	244
附件	245
利益相关者	245
客户声音: Kano Diagram	246
参考 References	247
20 团队协作	249
各自负责自己开发的模块	250
团队合作	252
按时上下班	252
什么导致系统崩溃	253
技术团队的持续性	254
附件	256
平衡心态 (Cognitive Dissonance)	256
参考 References	257
VII 度量与分析	259
21 做问卷调查	263
案例: 为某全国快餐连锁做问卷调查	263
结束语	266
22 教小孩统计分析	269
从考试成绩到贸易逆差	269
分析例子: 连续与分组数据	277
结束语	279
附件	280
如何画茎叶图 (Stem and Leaf Plot)	280
箱线图 (Box and Whisker Plot)	280

参考 References	281
23 假设检验	283
假设检验的步骤	284
检验单个正态总体的均值	284
比较两个正态总体的均值	287
检验单个正态总体的均值, 方差未知	288
方差分析 (ANOVA test)	289
分组 (分类) 数据	292
附件	293
中心极限定理 (Central Limit Theorem CLT)	293
检验单个正态整体均值相关方程式	294
比较两个正态总体的均值	294
检验单个正态总体的均值, 方差未知	294
Tukey-Kramer 相关方程式	295
24 控制图	299
建立基线	300
噪音还是信号	300
控制图的应用	304
结束语	306
附件	306
如何画 ImR 控制图	306
各种控制图	308
过程能力 (Process Capability)	309
细分例子	310
参考 References	312
VIII 总结	313
25 创新: 从个人到公司	317
引言	317
400 年前的重大数学发明	317
创新 Creativity	320
原创精神 Spirit of Creating	320
公司如何创新	323
音乐播放器	323
iTunes 网上商店	326
万事起头难	329
从濒临破产变为回报最好的投资	330
公司创新成功要素	331
结束语	334
个人经验教训	337
附件	338
Pixar 制作第一部电脑动漫: 玩具总动员 (Toy Story)	338

CONTENTS	11
参考 References	340
26 北京手记	341
创新来源于生活	342
27 根与翼	345
根	345
翼	346
共通点	349
TED 演讲式总结	352
参考 References	356
IX 附录	357
28 A: 绅士俱乐部过程改进	359
参考 References	365
29 B: 分析迭代客户满意度调查数据	367
案例背景	367
迭代数据	367
数据分析	367
改进措施	368
改进效果	368

Dedication

To Maria: Thank you for your support and love. Otherwise, I could not complete this project.

----- Edmond

序 Preface

这是一本从事敏捷开发和过程改进的人员的必修之书，作为本书的早期读者，有两个创新让我印象深刻。其一是本书的写法，类似《金刚经》体，全文多是以对话的方式，让读者读起来轻松流畅，少了长篇大论的说教，而是加入了真实的、与不同 IT 企业高管、中层及基层开发和技术人员的对话，在问答中将普遍问题进行归纳总结，并提出解决方法。其二是案例和数据的引用，虽然很多是引用书籍中的案例和数据，宋老师能结合自己多年经验，对案例和数据的引用力求精益求精，每个案例都颇为经典。阅读和理解这些案例能加深对会话中问题的领悟，也加深了对相关建议和解决方案的理解。

读过很多管理类的书籍，有些书籍将管理数据自始至终贯穿如一，例如德鲁克的《卓有成效的管理者》；有的则以故事的形式给企业管理者以启示，例如《目标》。而本书的内容始终如一地贯穿了过程改进的主题，通过引用一些小故事，宋老师将多年来在该领域的培训和咨询的经验跃然纸上，让书籍活跃起来，使读者读起来不至于太累。诚然，一千个人有一千个哈姆雷特，具体有什么收获，就请大家在本书的阅读中去体会吧。

除此以外，给我感触更深的是宋老师严谨的写作态度，每次他写完一章，都会发给不同的管理者阅读，邀请他们提供改进的意见和建议，每一个建议都会得到反馈并在相关章节中加以完善。这种态度是非常值得尊敬的，毕竟在他才是该领域货真价实的专业人士，而我们这些先期读者只是普通的从业者。最后，预祝本书能帮助到那些有志于在管理上提升的从业人员。

杨立东《微管理》作者，四维世景科技（北京）有限公司总经理
2023 年 3 月于北京

第二版序 Preface to the Second Edition

从 2023 年底第一版定稿后，在使用《A++ 敏捷开发》的材料与团队交流时，发现有些内容容易引起误解，第二版针对这些部分做了修正和完善，但总体架构和章节基本没有变动。改动较大的部分包括：

第五部分《如何做好迭代回顾》：

里面的 4 个章节都做了大幅修改，按企业领导解读的视角，先用《二八原则》选择针对软件开发的方面去分析，然后介绍了不同的软件根因分析的技巧后，第三章针对人性部分，如何确保团队成员的投入和积极性，最后第四章里加入了实际的团队常见的问题，对应解决方法和成功案例等。

最后第八部分：

在公司创新的具体实例后加入了 26 章《北京手记》，利用 3 场表演说明个人每天听、看、读、写对改善、创新的重要性。最后 27 章《根与翼》也加了实例说明根与翼，最后总结这部分的概念如何帮助敏捷团队和公司持续改善、保持竞争力。

感谢 Acknowledgments

首先感谢我的“写作伙伴”杭州的徐洪洋，北京的洪一海、王国军、王绍军、赵丽在百忙之中也抽空看我的初稿，指出错误，提出建议。因自己是难以看到自己的不足，如果没有他们，难以确保作品的质量。

也感谢朋友圈里各位老师、行业精英们分享经验和意见，包括北京的杨立东、武宏伟、纪钟涛，天津的韩淑军，上海的周青龙，杭州的程文进、姚钧、胡蕊莉，成都的杨杰等。

如果没有与这些老师、客户、学生和朋友的面对面详细讨论，并在项目过程中反复试验与反馈，不可能写出这本书。这些宝贵经验帮我验证了各种敏捷思路与量化管理的可用性，也让我更加清晰地理解了质量改进的重点。

最后感谢无锡的陈镜庆、秦瑜不断提出修改意见，并帮助我最后完成本书的编辑；

也感谢我香港大学老同学李启良先生对 Mediawiki 服务器、Linux、LATEX 等平台的技术支持。

前言 Prologue

前言 Prologue

We are uncovering better ways of developing software by doing it and helping others do it.

—Agile Software Development Manifesto

敏捷宣言已经有超过 20 年的历史，国内越来越多软件开发团队开始采用敏捷和迭代，但总体效果参差不齐。有些年轻的团队成员听到敏捷不需要文档，以为也不需要注重代码质量，包括代码可读性，导致后面发布的软件产品问题多多，难以维护。要编写出高质量的代码，人本身能力非常关键，但软件工程快速发展，导致程序员人数快速增长，其中很多缺乏专业工程师素养，所有若要敏捷开发真正起作用必须先提升程序员能力。

没有数据就无法管理，但很多敏捷团队只是走流程（每天站立会议、看板并非敏捷的重点），缺乏度量，所以管理者一听到敏捷就觉得不靠谱，要求团队用回传统的瀑布开发方式。

这 20 年来，中外都出版了很多关于敏捷的书，但绝大部分都没有深入去探索以上的问题。这本书就是希望通过解读各种敏捷最佳实践，加上敏捷以外的其他知识，帮助大家理解并更好使用敏捷，提升软件的质量和总生产率，让团队成员与公司管理层获得双赢。同时也希望管理层通过这本书能了解敏捷开发的要素，并能使用敏捷开发模式，帮助公司提升软件产品质量，同时降低成本，来提高公司的竞争力。

某技术总监陈总在 5 天差距分析的最终总结会里问开发团队：

1. 你们知道自己是每天产出多少代码行吗？（生产率）
2. 你们知道平均修复一个系统测试的缺陷花费多少工作量（人时）吗？
3. 有没有发生过：代码开发完成，系统测试人员尝试运行，但跑不动的情况？（开发人员不能借口说不懂业务，或需求不清，因为你写完代码后，自己连最基本确保代码能运行都没有去做。我还未问你们做开发的有没有做好单元测试/集成测试。）
4. 你们做测试的知道测试覆盖率是多少？

陈总有丰富开发经验，他最后总结说：“我以前自己做开发时也是被问过这些问题，我当时也不懂，但我能知耻而后勇，XXXX。先知道现在的不足，然后按评估老师的最佳实践，持续改进。”

- 如果你像陈总下面团队一样，不懂以上问题的答案，这手册可以帮你自己找答案。
- 如果你（是管理者或客户/用户）不满意团队的软件开发成果，这手册可以帮你找出根因，启发团队改善。
- 如果你已经很了解，恭喜你！但应还可能从这手册里找到一些你未知但有用的方法。

1980 年，STROUSTRUP 先生不满意当时很流行的 C 语言，因为在 70 年代，已经有如 SmallTalk 之类的面向对象语言，C 是基于传统步骤式的语言，没有面向对象的功能，所以 STROUSTRUP 先生就在 C 的基础上，加上有类功能的 C (C with Classes)。过了 4 年，他同事觉得这个名字不好，就改成 C++，也出了一本书叫《C++》，与本来经典的 C 很相似。后面 C++ 就成为了面向对象的常用语言。

A++ 继承 40 年前 C++ 的思路，希望在敏捷 (Agile) 开发的基础上，加上增值的元素，帮助团队与管理者做好软件开发，让客户满意。

Part I

案例分析

Chapter 1

如何改善

敏捷开发过程改进案例

5 月

A 公司一直专门为某电信公司提供针对客服、线上播放等业务。张工是公司的中层管理者，管理好几个开发团队，有 5 位项目经理向他汇报。他听说老同学的团队都开始用敏捷开发，很感兴趣，便参加了几次敏捷交流会，觉得可以解决很多开发团队的问题，尤其是可以快速交付给客户。他便提建议给部门经理推动敏捷开发，找咨询公司做相关培训，例如 SCRUM Master 内部培训，然后全面开展实施。开始时，部门经理有些怀疑，问：“听起来很吸引，但后面那些工程文档都不做，会不会影响质量和交付？客户都是专业做电信的，不缺钱，但是对质量要求很高。”张工解释说：“只要利用敏捷把过程变成迭代，快速交付、改善工程的问题不难，主要是人的问题。”部门经理听到敏捷可以比以前更快速交付，因之前客户经常抱怨项目延误，他也希望可以改变，就答应了。

8 月

开发组长王工，三十出头，毕业后一直做开发，一年前晋升为组长。周五下班后与朋友喝酒，很开心地说：“太兴奋了。我们团队刚参加了两天 SCRUM 培训，并指出我们以前按传统瀑布式开发的种种问题。我们会两周一次迭代并割接上线，还会与用户代表定期交流，不再会像以前几个月交付后才发现开发出来的功能不合需求。现在团队合作不像以前只按工种工作，也会跟产品经理、业务方面更充分合作，给客户带来更高价值。工作方式也会改变，以前要写需求、规格说明书，现在简单化成产品需求卡片，以前我们要做详细项目计划和甘特图。现在会改成用燃烧图和“KANBAN”。每

天用便利贴去写要开发什么内容贴在白板上面，我们会改名叫 SCRUM Team。工作区周围都有 SCRUM 的海报，提醒我们 SCRUM 的重点。团队也没项目经理了，自己管理自己。部门经理会变成产品负责人，敏捷开发方式让我们团队自己做决策，不仅仅是技术方面，项目相关的也由我们项目组一起讨论决定。”

9 月

王工的朋友老杨周五晚喝酒时问他：“你们团队使用敏捷 SCRUM 后，效果如何？”

王工立马面露笑容，充满自信地跟大家说：“我们培训后就使用 SCRUM 的方法，每 2 周一个冲刺，每次冲刺前都会用故事点来估算每个功能多大，然后按本次冲刺的资源，估计可以完成多少功能。然后用看板来监控模块完成的情况，哪些正在开发中？哪些已经完成？团队和管理者都可以一目了然，不用像以前……。”

“听你这样说，我也要尽快提议领导引入敏捷开发。”

“我还没说完，我们每天早上也按照 SCRUM 的规定站立会议，每人说自己完成了哪些任务，今天做什么……。”

10 月

喝酒时老杨问王工：“你们项目如何？”

王工听完，没有立马回应，把注意力放在窗外的大街上，想了一下，然后说：“我们本应上周要完成一次冲刺后的割接上线，但被推到下次的。”

“为什么？”

王工说：“我们按培训学到的做冲刺计划会议，按照产品的待办事项列表，团队利用扑克牌一起估算每一事项所需要的时间。我们总共 8 位开发人员，其中有一半是刚毕业不久，但大家刚上完培训，很有信心，虽然技术主管张工对我们出来的估算有些顾虑，觉得我们太理想，但大家刚培训完敏捷，张工也希望让部门经理尽快看到敏捷开发可以加快速度，我们就按这‘进取式’估算开展两周冲刺。但因新人多，编码水平有限，虽然大家已经尽快把开发出来的代码交给系统测试人员手工测试，依据测试发现的缺陷修正再测试，但割接上线期限前 4 天还有很多 Bug 没改好，最后 3 天，基本是天天加班，最终到上线期限仍然有不少问题，最终割接前测试，还是不能达到客户要求的水平，没办法上线。大家确实都尽力冲刺了，但未能达到我们本来希望的结果。”

“你们开发人员在提交代码到系统测试之前，有没有自己先自测？”

王工说：“我们敏捷开发每天早上站立会议上都用看板，记录每个模块的进度完成情况，开发编码人员都说自测过。”

“如果开发人员能自己把握好交付代码的质量。不应该等到系统测试才发现这么多问题要解决。你们团队是怎么做自测？有记录吗？”

王工说：“这我就没有详细问了，因为我们学完 SCRUM 后，大家更关注开发速度，用看板保证按计划的进度，不延误。但没想到后面缺陷会这么多。而且有些问题改完以后还会导致另外一些 Bug 发生。到了最后，因为赶时间，大家对修改 Bug 已经麻木了。”

“你们之前不是都有代码评审和扫描的习惯吗？”

王工说：“之前我们确实比较多评审，除了评审核心设计和代码，我们还要求组长抽查新人开发的代码，并且要求提交代码前必须通过扫描。但我们为加快速

度，没有硬性要求必须先通过扫描。经你一问，我想这也可能是出现很多 Bug 的原因之一。”

“你们有没有事后迭代复盘？复盘可以帮团队分析之前有哪些不足，后面改进。”

王工说：“我们都交付不了，哪里有心情来做回顾、复盘。最后几天冲刺，大家每天都没睡好，大家都只想回去好好睡一觉。”

11 月

部门经理之前收到客户总监电话，投诉一些技术缺陷，导致好几次不能按计划上线，问为什么正在交付的软件质量变差了？

张工被问到是什么原因时无法回答，只能说立马回去探索原因，尽快汇报。

张工从部门经理办公室出来后，找其中一位项目经理李工喝茶，问他对今次敏捷改革的意见。

两人都赞同敏捷开发应能帮助团队提升，当张工问李工有什么可以改善的地方，李工说：“让团队自主是件好事，但因为我们专注做这块业务已经很多年了，本来业务的变化不多，只是一些功能微调，所以开发人员尽量不去修改核心代码，怕改动了反而会影响投产，无法切割。为了不触及原始代码，新代码都只能在核心的外围去写，但这种做法效率很低，无法有效长久维持，到时会被迫重写整个产品。而且懂老代码的开发人员大部分都离开了，代码维护越来越困难。

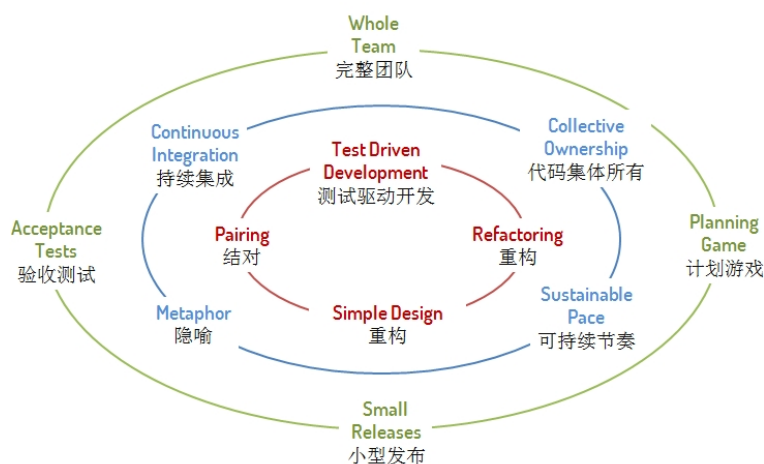
敏捷教练强调团队自主管理，给团队空间发挥。但 SCRUM 的教练缺乏软件工程的基础，只懂项目管理过程。所以他们也解决不了软件相关的问题。只是把精益管理怎么做迭代、怎么做回顾这些基本过程再解读一下，解决不了实际问题。”

张工说：“很赞同，这是必须要解决的问题。但现在燃眉之急是要解决主管提出的客户投诉不能按时交付这紧急问题。不然我们以后也不敢再提敏捷开发了。”无论张工或李工也没有能去总结出什么好的解决方案。现在推行敏捷才刚刚 3 个月，绝不能打退堂鼓，回到本来的状态。但应怎么解决敏捷带来的问题，挽回部门经理与客户的信心呢？

从上面的 SCRUM 案例看到，本来管理层希望利用敏捷开发，加快软件开发的交付、减少延误、令客户更满意。但因为只注重项目进度是否延误，但团队没注意如何改善软件开发本身的质量，也因为团队成员能力不足，开发出来的软件缺陷比以往还多，导致后面大量返工，恶性循环，后面更导致延误和客户投诉。因为软件本身设计有问题，导致软件难以修改，开发人员都不敢改动任何代码，怕可能会引起系统崩溃。

怎样才能确保开发出来软件的质量？

敏捷开发有很多种方法 (SCRUM 只是其一)，因为目的不仅仅是管好项目进度，也要确保软件产品的质量。所以 SCRUM 只包括项目管理部分，不全面。反过来，例如极限编程 (XP eXtreme Programming)，因它的发明者 Kent Beck 本身是一位精通面向对象的程序员，所以 XP 不仅仅关注项目管理，也包含编程的最佳实践。下图是 Ron Jeffery 把 XP 的重点画成从外到内 3 层：



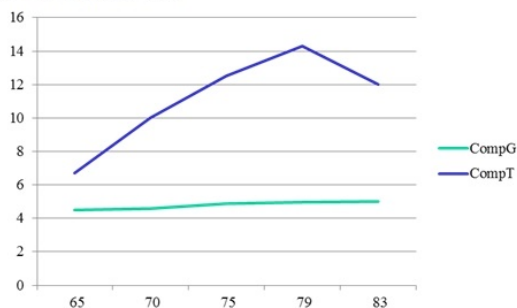
SCRUM 只包含了外层的部分，缺乏中间和内层元素。按 XP 的 12 实践（详见附件）都做到了便可以解决张工的问题吗？

如何改善

既然 SCRUM 方法有不足，XP 方法能解决开发质量问题，是否团队学好 XP 便能帮助团队做好敏捷开发？怎样才能不断完善？先问你以下关于汽车公司的问题：

CompT vs CompG

- 1955年前后, 生产一辆汽车的成本: $\text{CompT 成本} = \text{CompG 成本} \times 2$
- 全球销售额: $\text{CompG 销售额} = \text{CompT 销售额} \times 140$
- 平均每员工年汽车生产量



T 公司在 60 年代，在销售、生产成本都远远不如 G 公司，但它每年生产率都一直提升。

请猜猜 T 和 G 是那家公司？

(提示：两家都是世界级汽车公司，现在你还能买到它们生产的汽车。)

=====

G 是美国通用,T 是日本丰田。

为什么丰田能从一家战后小公司提升为世界最大 (#) 的汽车公司？

(#：2023 年底，总收入可能是大众第一、丰田第二，但丰田销售数量世界第一。)

丰田故事

二战后 50 年代, 丰田汽车规模很小, 经营很困难, 在破产边缘。但丰田创始人喜一郎先生明白美国生产线的弊端, 意识到未来的汽车生产必须是 Just-In-Time:

- 每一辆都是按客人订单订制：例如，颜色，配置，左右钛等。
- 从钢材原料开始，整个生产线零等待、零浪费。

每个工作步骤所需配件按生产需要到达（不晚到也不早到），把生产过程中的配件降到零。



* 0 defect 零缺陷

100% value added 百分百的增值

One-piece flow, in sequence, on demand

按订单的流水线生产，过程中零报废

喜一郎先生安排总工大野耐一去美国汽车公司考察。

为什么福特 Model-T 出名？

它是第一辆一般人买得起的汽车首辆现代汽车是由德国奔驰 (Benz) 于 1885 制作，但是这些第一代汽车都非常昂贵。

老福特是一名工程师，对机械特别感兴趣，白天是爱迪生公司的总工程师，到了下班后就研究汽车发动机经过十年的不断优化，最终开发出 4 冲程自动汽车发动机，他的发明大大降低了汽车组件的成本

1908 Model-T 首次推出市场，售价是 850 美金，虽然已经比同期的其他厂家汽车便宜，但还是超出一般工人家庭可以负担的水平。

为了进一步降低成本，老福特觉得必须想其他办法。汽车组装一直都是作坊式运作，所以组装汽车需要超过 12 个工时。为了降低组装时间，必须要学其他工厂，如啤酒，麦粉厂，它们都已经是流水式生产线生产 (Assembly Line) 福特把整个 Model-T 组装拆分成 84 步骤，并培训工人只做一项工作，还聘用了科学工程师 Taylor 先生设计整个流程，提高效率，最终可以一个半小时完成组装一辆汽车。

生产线生产帮助福特公司把成本和售价降下来，在 15 年期间 (1913 - 1927)，福特公司总共生产了 15,000,000 辆 Model-T。虽然大量生产能直接降低生产成本，但这种做法也要付出代价：

- 为了生产所有都是同一个款式同一个颜色，黑色
- 为了提高生产的效率，也增加了组件的库存量 (为了提高效率，降低成本，大部分组件都要批量生产 (Batch production)，例如轮胎。例如上面右下图看到工厂里存在大量轮胎。)

福特 Ford Model T 生产 (~1910 - 1920)



↑
Ford
Model-T →



Federick Taylor 与科学管理 (Scientific Management)



十九世纪末美国很多工业快速扩张,Taylor 先生发现当时很多工厂生产力很低,例如钢铁厂。他是很落地、实干的人,他就想有什么好方法可以提高生产力,工人也多赚些钱。他发现很多工作都未设计好,未能好好利用工人,他针对工作本身做很多科学研究,测量每个工作应该多长时间,怎么做最好,然后按他的最佳设计把工作细分,也同时要求公司提高员工的奖励,希望工人看到因科学管理,提高生产,个人也受益。1890 年,他加入了 Bethlehem Steel,钢铁业的大公司,帮它设计了很多奖励制度,也设定了一些叫工业工程师 (Industrial engineers) 岗,发现生产力可以提升 30-40%。后来公司被收购,他也被辞退,他便成为顾问,当时都没有顾问这个职业,他可算是全球首位工程顾问。

总工大野耐一先生从美国的超市(非汽车公司)得到如何做 Just-In-Time 的启发,回国后就开始在丰田全力推动。开始时,很多人都觉得 Just-In-Time 这个愿景好像是远不可及的梦想。而到了今天,现代汽车生产都基本做到了当年喜一郎先生和大野耐一先生的梦想。(详见附件)

水面下的管理思路



为什么丰田能成功地把汽车生产做到 Just-In-Time，超越西方的巨头，使日本汽车制造过程成为世界“标准”？

但我们不能单从表面看这“系统”的方法和技巧，例如大家都熟悉的看板管理（他的竞争对手通用、福特肯定都学过），更重要是了解背后的管理思路。我们看看下面丰田七个习惯，开始了解水面下的“系统”：

1. 容易实现的目标不是好目标
2. 不以“我们公司”作主语
3. 重复五遍“为什么”
4. 持续改善没有停止
5. 成长比成功更重要
6. 以忙碌为耻
7. 从心里相信“大家的力量”

容易实现的目标不是好目标

不是“削减一成”，而是通过“取消一个零”来发现浪费

“如何把原先需要三小时的工作，改用三分钟完成”

如果听到上司以上要求，你该怎么回答呢？如你回答：“这太强人所难了”、“绝对做不到”，请读以下丰田案例。

{| class="wikitable" |“单分换模 (Single Minute Exchange of Die)” 例子：
1965 年，丰田汽车在推行丰田生产方式时遭遇一个瓶颈：装置更换时间太长，其中特别是 500 吨冲压机和 1000 吨冲压机的模具，更换时间长达 2~4 小时。如果不缩短这两个模具的更换时间，就不可能实现多品种少量生产方式。
挑战由大野耐一先生统领，在生产管理的先行者，新乡重夫先生的指导下，分

两个阶段展开。

改善开始之前，新乡先生凭借自己多年的工作经验，了解到装置更换有两种方式：

内部装置更换——必须在机器停下来以后，才能进行的装置更换。

外部装置更换——能在机器运转过程中，或是在运转起来以后，进行的装置更换。

要缩短时间，把内部装置更换和外部装置更换清楚地分开来是一个关键。能在外部装置替换作业中进行的工作，就全部在外部装置替换过程中实施。同时，分别对内部装置替换和外部装置替换进行改善。通过这种方法，装置更换时间缩短为一个半小时。

完成这一改善花费了半年时间。通常能有这样的成果就可以告一段落了。但是，大野先生仍要求进一步缩短时间。

他要求把更换过程“缩短为 3 分钟！”通常通过改善能把原先的 2~4 小时缩短为一个半小时，就可以很满意地说“已经很好了”。但是，大野先生不这么想。他认为：“既然能缩短到这个水平，那么继续改善肯定可以把时间缩短为 3 分钟。”

改变汽车生产的单分换模的秘密

对于这一要求，在以新乡先生为中心的技术小组中，自然有人提出“3 分钟绝对干不完”。但是，新乡先生认为：“如果能把内部装置更换全部转化为外部装置更换，3 分钟也不是不可能……”

之后，他着手对多达 100 个以上的项目进行了改善。

首先，进一步对内部装置更换和外部装置更换进行细分，彻底把内部装置更换转化为外部装置更换。同时，想方设法对各种切割工具和模具进行设置，使得更换时用一个动作即可完成。此外，在紧固件上也动了很多脑筋。这样，终于创造出了无数项不花时间、能够简单完成同时可以在作业时保持稳定的改善。

紧接着，对作业顺序反复进行改善，实施标准化（制订没有多余工序的作业标准）。这样，在挑战进行了三个月后的某一天，真的只要 3 分钟就能完成了！这个结果让所有人都大吃一惊。

}}

标杆管理 (Benchmarking)

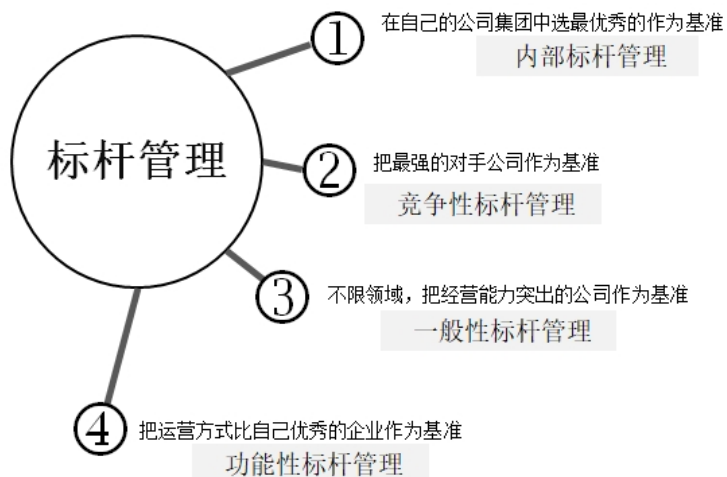
很多公司都会用百分比来设定改善目标，但改善了百分比，不一定代表质量有改善，例如某快递公司去年送包裹未按时送达占 8%，今年是 6%，好像已经改善了 2%。但去年托运的数量是 500 万件，所以 8% 是 4 万件。今年的数量是 750 万件，所以今年的 6% 就是 4 万 5 千件。

所以今年包裹延误增加了 5000，这样非但没有任何改善，反而服务变得更差了。所以更重要是看数字本身。容易达到的目标，不是好目标。所以丰田一般会选行业里最强的对手作为基准（标杆）。

1965 年，美国通用汽车公司是世界顶尖。例如销售额的规模，丰田与通用是 1:60，完全不在同一个层次上。成本上丰田对通用是 1:0.5，成本是通用的两倍。丰田把当时顶尖的通用公司当成了自己进行标杆管理的对象。

如果丰田某零件的成本价格是 1000 日元，通用是 400 日元。丰田会把通用的 400 日元，作为基准成本价，把原料价定为 400 日元，把差额 600 作为“不必要花费”，让团队立马改善，尽早达到标杆。

丰田（如下图）很注重各种标杆，例如内部标杆、竞争性标杆等。软件开发也应该同样利用数据来制订量化目标。



某家专门开发金融软件产品的公司:

研发总监问我“你接触这么多国内的企业, 觉得有哪家优秀的, 可以作为我们的标杆? 我们尝试寻找, 但还未找到。有些优秀的公司, 但它们业务跟我们不同, 我们的产品是面对企业, 也非嵌入式软件(我们不生产硬件)。”

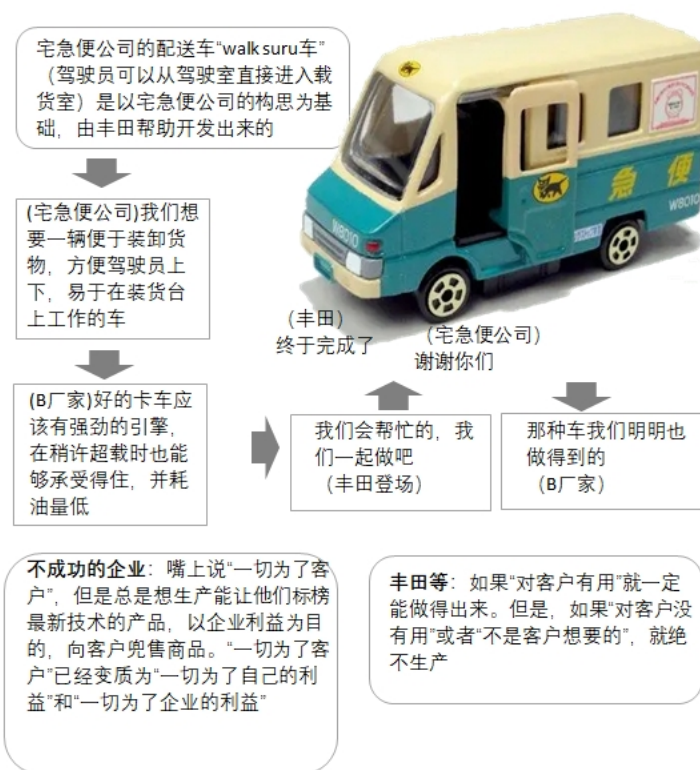
“确实难找, 我真没遇到过, 但为什么你只看国内公司?”

从以上对话看到越来越多高水平开始用标杆管理, 但标杆必须是比自己高很多才有作用。

不以“我们公司”作主语

- 不是从“专业”的角度, 而是从“顾客”的角度生产产品。

创业大忌——闭门造车, 所以丰田的原则, 对客户有用就一定做出来, 但对客户无用或者不想要, 就绝不生产。



要创新必须从“顾客”的角度看,不单从工程师的视角看——这不仅适用于丰田和汽车制造:

征服太空,踏上月球。六七十年代,美国开始阿波罗太空计划。首批宇航员强烈要求工程师加上窗户、逃生门、可人手控制等(现代我们会觉得这些是太空船(spacecraft)基本要求,但当时是闻所未闻)。

1980,一位 NASA 工程师回顾当时工程部的想法:“你们又不是火箭专家,航天专家,只是飞行员。希望可以在火箭驾驶舱人工控制火箭飞行,开玩笑!太极端,你们可不了解成本多高,我们应可以很轻易用工程的理由拒绝。”

重复五遍“为什么”(5 Why)

最重要的不是追究责任而是找出根因。

五个“为什么”是一种找根因的方法,实例可详见附件:

- 不停留在“原因”上,而要找出“真因”,彻底改善。
- 不追究责任,而是追究原因。

丰田要求检查人员不仅仅检查过程与结果,做统计报告,交给负责的工程师,也必须分析产生不合格的原因,使同类的问题不再发生。检查人员不仅仅是“考官”,也作为“内部教练”,指导生产的持续改善。

反过来,如果下属因出错,导致失败,立马被上司骂,觉得很难受,就会从反省变为排斥。如果公司有这种“责难,追究责任”的风气,坏消息就会被隐瞒。所以丰田有一条原则是“Hard news first (先说坏消息)”。

当工程师找出了问题就必须要求他查找原因、提供数据,但很多时候这个工作并不简单,但大野耐一先生决不放弃,必须找到根因。他说:做到一半是不行的,只有一个期限——“到完成为止”

持续改善没有停止

A 公司用丰田方式进行了两年多的生产改革,高层觉得成果令人瞩目,举办改善报告会,邀请集团内其他公司高层来参加。

在参观工厂时,A 公司的负责人利用图表展示如何成功地缩短了零件替换时间。C 公司高层听了说明后问:“你说把 90 分钟缩短为了 30 分钟,最初的一年半,你们确实一口气实现了时间的缩短,但是,看一下这半年,你们好像只缩短了几分钟。对于这点,你是怎样考虑的呢?”

开始实施改善后的一年左右,成果会很快出现。因为以前有问题的地方太多,所以值得改善的地方多不胜数,只要努力实施改善,零件替换时间、库存、生产效率等等,各方面都会眼看着变得越来越好。但是,到达某个水平以后,改善的步伐经常就会停止下来。

到了这个阶段以后,有很多企业都会因为“改到这样已经很不错了”而停止改革的脚步。

着手执行丰田式的改革方式本身并不是一件太难的事。只要解决问题,就可以获得成果。难的是,必须要把改善持续搞下去。

人有的时候很想“往前跑”,但有的时候也想“停下来休息”。对丰田而言,比起以为“已经胜利了”而在睡午觉的兔子而言,他们更推崇不倦不息一直向前进的乌龟。

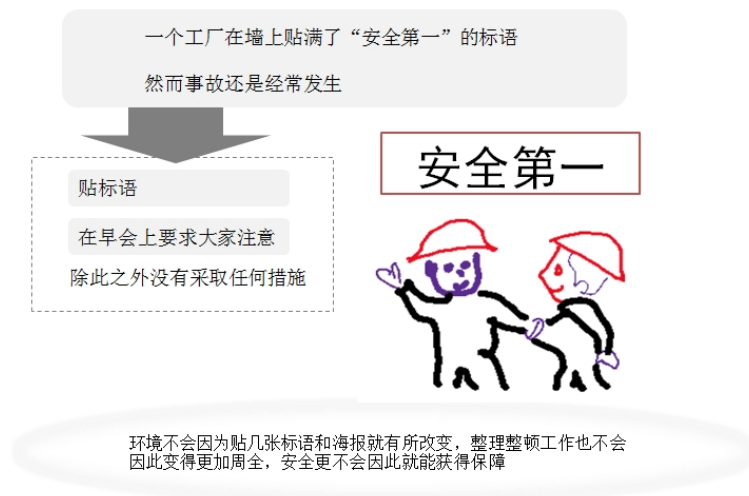
报告会结束的时候,A 公司的员工说:“课题依旧堆积如山。我们的目标是成为世界一流的工厂,所以我们会永远改善下去”这也是 A 公司全体员工的心声。听到此,B 公司和 C 公司的高层也总算放下心来。

能否坚持下去是能否成功的关键因素。

成长比成功更重要

- 要培养人才,“改变体制”比“改变人”更有效。

有些工厂只依赖张贴标语、海报,希望可以减少工地的事事故发生率,但丰田不注重喊口号,而是动手干实事,包括机械保安、设备保安等。



所谓 5S 指的是：整理、整顿、清扫、清洁及修养。无法做到这些基本原则的企业及个人是无法生产出优质产品。

在这些基本原则中，丰田尤其注重“整理和整顿”。

“处理掉不需要的东西是整理。使得任何时候都能把想要的东西拿出来叫整顿。只是把东西放整齐的则叫做排列。我们要求在生产现场必须要做好整理和整顿。”

实例：

B 先生受命去子公司 A 公司实施重建工作。当他第一天到达工厂的时候,他看到的是一副脏得无法形容的情景。机器上布满了油和灰尘,地板和墙壁上沾满污渍。飞散开来的机油上又落了灰尘,积成厚厚的一层。半成品和零件随便地东一堆西一堆地放着,工具扔得到处都是。工厂里想找块站的地方都很困难。这样的状态不用说也会降低丰田的“开动率”。仓库里也是堆满了零件和产品,但看那样子就知道,若要问起某个东西在哪里的话是不会有人知道的。他在晨会上问大家:“你们想带自己的家人或恋人来这个工厂吗?”

A 公司的员工们一直戒备着从总公司来的 B 先生新官上任后不知道会烧什么火,听到这句话后面面相觑。看来 B 先生好像对工厂的这个样子很不满意,他们一个个相视苦笑。就在大家不出声苦笑的过程中,不知道是谁轻轻说了一句“怎么可能带他们来呢”。声音虽然不大,但是 B 先生却没有漏听掉。

“为什么不想带自己最亲近的人来看一看你们工作的地方呢?”“这么脏,哪好意思给人看啊。”“那为什么就让它一直这么脏着呢?”

这样追问以后, B 先生听到了很多理由。“太忙了,没时间打扫”、“没有打扫的工具”、“没钱买重新粉刷的油漆”、“收拾好以后马上又脏了,扫了也是白搭”……。

“心情”改变工作

在 B 先生听来,这些理由中其实就蕴含了“改善的线索”。

每当人们不想干什么事的时候,就会想出理由来辩解为什么干不了。大部分人都会因为这样的理由,而在是否要行动时犹豫再三。

但是, B 先生身体力行丰田方式。他认为,找借口不如去思考该如何做。“用思考借口的脑袋来想怎么做”,“借口”其实和“之所以能做到的理由”是一致的。B 先生逐一攻破了 A 公司员工的“做不到的理由”。“没有时间”他就在工作时间中拿出几分钟将其设定为“清洁时间”。没有打扫工具和油漆,他就早早地买好。对于员工所说的“白搭”, B 先生身先士卒地去捡垃圾、擦墙壁。

没过多久,原本很脏的工厂开始慢慢干净起来了。而当工厂开始变干净以后,员工们都产生了一种要把这种干净的状态维持下去的心情。他们开始自己收拾机器的漏油,而当初觉得“扫了也是白搭”的员工开始觉得“打扫也是件不坏的事”。

随着工厂开始变得干净起来,业绩也开始恢复了。工作的环境变好以后,在那里工作的人心情也就会随之出现了变化。

大野耐一在他的《现场经营》中写过:

如果仅仅觉得 (机器人) 用起来很方便, 或是能代替我去工作, 那就说明根本还没有能够有效地使用机器人。如果引进机器人, 就要从引进的那一刻起对机器人进行改善, 或者要使自己的做法能和机器人合拍。

如果公司老有人在嘀咕做这样的事买个机器人回来就成了, 甚至在不明白目前的情况下有没有比引进机器人更好的方法, 就成天想着如果不买机器人, 改善就不可能进行得下去, 那么, 这家企业就糟糕了。首先要弄清楚目前企业里的机器能做到哪一步, 或是为了使现在的机器能发挥更大的作用, 而先充分地去使用现在的机器。这样当以后有先进的机器引进来的时候, 就可以依靠其获得“基于智慧的 +”(注)了。丰田最终决定要在最后的组装工序中引进机器人也是经过了一段很长的准备期的。

比如说公司计划要把过去由自动机械完成的作业换成由机器人来完成。一般的做法无非就是直接用机器人把自动机械替换掉。但是, 丰田首先挑选出典型的生产线, 将这条生产线上原本由自动机械干的工作暂时换为人工作业。以此来详细地分析人会采用什么样的方法去做。并执行彻底的作业改善。在此基础上, 探讨有什么工作是机器人做得了的, 又有什么是机器人做不了的, 以及为什么做不了。有的情况下还特意开发一些更容易能和机器人配合得起来的零件。他们是在经过了这样的过程以后, 才和机器人的生产商共同开发, 并最终决定导入机器人的。

实行机器人化并不是为了能够减少人数。在丰田不但“人的工作与机械 (机器人) 的工作”泾渭分明, 而且两者的存在很和谐, 推行机器人化是为了能让人去做附加值更高的工作。

(注: 指不仅要以现在的技术、价格、服务等为基准进行标杆管理, 还必须预想一个包含了将来的变化的“现在 +”, 作为标杆管理的对象。)

第一个例子, 说明工作环境的重要性, 主管通过逐步改善工厂的环境, 团队的士气与生产率都提升了。

从第二个例子看到成长不是仅仅按既定方向去做, 必须思考动脑筋, 并了解背后的目的才能真正取得效果。

这些丰田的成长故事也适用于软件开发团队, 例如, 有些软件开发团队盲目地想推自动化测试, 却没有想好哪一类测试应自动化, 哪一类更合适手工测试, 最后因效果与投入不匹配, 以失败告终。

以忙碌为耻

- 不吝惜智慧, 但要吝惜汗水

把“动作”转变为“工作”以前,当大家批评某机关的工资太高时,职员会以上班时间很长,“一直在努力工作”为由来反驳人们的批评。这其实是一种对于“有动作”和“在工作”的混淆。不管上班时间有多长,如果没能够创造出利润,那么就不能称之为工作,也不能称之为一直在努力。

丰田是把“动作”和“工作”分开考虑的。在丰田看来,“就算一直在动也不代表那个人在工作”。省掉徒劳动作,把“在动着”转化为“在工作”。

大野耐一先生曾经问过年轻员工:“每天工作一小时左右,你们能做到吗?”听了这句话后,有人抱怨:“算上加班时间,我们一天工作 9 个小时,他那句话是什么意思?”

确实,他们在公司待了很长时间,但如果把“有动作”和“在工作”分开考虑,9 个小时中,真正在工作的时间可能只有 1 小时。大野先生指的是这一点。

关键的不在于流着汗在公司转了多长时间,而要把自己的工作区分成“徒劳作业”和“有价值作业”。

二战后,日本经济萎缩,丰田辞退了不少员工。1950 年,朝鲜战争爆发,需要大量增产,但丰田选择了只增加设备,而不增加人手,大野先生也借此机会,完善丰田生产方式,成功找到了以不增加人手为前提的增产办法。

从心里相信“大家的力量”

- 不是靠“一个不平凡的人”,而是依靠“一百个平凡人”来创造亮眼的成绩。
- 生产产品就是培养人才。

“做事最关键是人……‘培养人才’为基础” 总裁丰田英二先生。

有一位丰田员工被问到“丰田生产方式到底是什么”的时候,这样回答:“就是在人的智慧建起的基础上,立起了自动化和及时生产这两根支柱。”

他所说的“人的智慧”是指“在一线工作人员的智慧”。

人了不起的智慧

所谓丰田生产方式其实就是要建立起一种体制,把“人了不起的智慧”引导出来,使得这些智慧能在生产一线得到充分发挥。所以说“丰田生产方式源自人的智慧”。

企业相信员工的智慧提升以后,员工们的干劲、使命感、责任感都会随之而生,最重要的是,员工们会因此逐渐对工作抱有自豪感。一旦员工们的意识改变了,理所当然地,企业的竞争力也将获得很大的提高。

大野耐一先生说:“改良是指通过投入资金使情况变好,改善是指通过动脑筋使情况变好。”

带诚意去赢得协作

B 先生所在的 A 公司曾以丰田生产方式为基础进行了生产改革。

要进行生产改革没有技术部门的配合是行不通的。但是, 任凭 B 先生怎么要求, 技术部门依然毫不合作。B 先生实在没有办法了, 只能向总裁要求: “请加大我手上的权力, 让我可以支配技术部。” 总裁回答: “你去给我请教了大野(耐一) 先生以后再说!”

于是 B 先生去找大野先生, 在听他诉说了自己面临的窘境以后, 大野先生对他说: “你这两天跟我一起去工厂转转吧!” 并于百忙之中抽出时间带 B 先生参观了丰田的工厂以及附近的协作企业的工厂。这期间, 大野先生什么话都没说, 只在第二天下午问 B 先生: “怎么样, 你明白了吗?”

B 先生回答: “我觉得在工厂听到的关于厂长的改善事例, 跟丰田方式所强调的重点好像不太一致。” 听了 B 先生的回答以后, 大野先生点头: “就连我, 也是一直都在忍耐的啊。工作并不是有权力就能解决问题的。要想得到对方的理解和信任, 拿出诚意去找人家吧。”

从那以后, B 先生再也不找 “因为我手里没有权力” 之类的借口, 而总是带着诚意去找对方协商, 不久以后, 他成功地对 A 公司实施了生产改革。

每当听到有人感慨 “下属不听话” 的时候, 一位曾在丰田工作的人就会说: “你要求自己的孩子” 每天学习三小时’ 时, 他会听话去学习吗?”

对方的回答是: “估计没用。”

“连自己的孩子都这样, 更何况那些成年的员工呢?”

改善质量

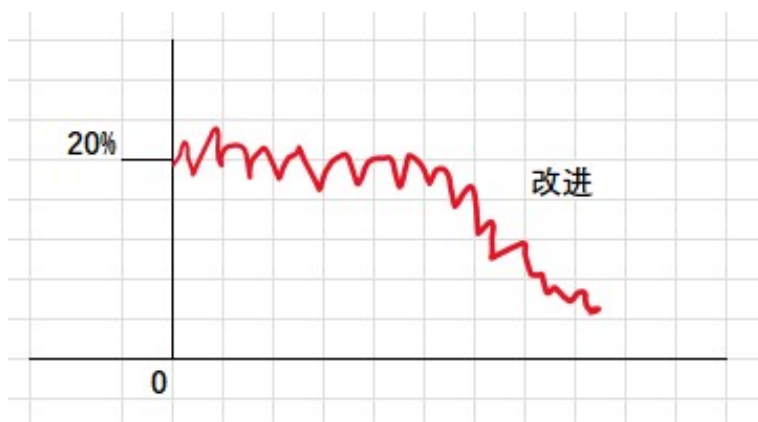
质量管理与财务管理类似, 同样也有质量策划、质量控制和质量改进, 基于这大框架再细分: 如何制订质量目标、度量等。

质量策划包括:

1. 设定目标, 包括外部和内部目标。
2. 识别内部需求。
3. 依据客户需求制订产品的功能特征。
4. 制订产品和过程的目标。
5. 设计过程来达到这些目标, 并验证过程能力。

质量控制和质量改进

下图的左面是在过程之前的策划部分, 例如发现缺陷比率为 20%, 这就是过程的能力, 这是策划的时候已经定好的, 过程控制没有什么可以做, 只是当缺陷有变化, 比如特别高的时候, 需要做一些纠正措施。例如希望把缺陷从 20% 降到 3%, 这就必须驱动一系列的改进计划。改进计划也必须按项目管理方法推行与监控, 没有其他办法。



所以如果希望敏捷开发有效，不仅仅是走迭代过程，确保进度没偏差，还要确保软件产品质量，也应该用裘兰博士的质量管理思路去看敏捷过程，才能更全面地了解如何确保软件开发的质量，控制交付工期。

最佳实践，如果没有质量目标、度量衡量和策划，都只是空想，对长远提升企业文化、员工素质没有任何帮助。

例如我们想改善团队的策划和估算。首先要识别客户，哪些是主要干系人——甲方有什么要求、内部部门经理有什么要求。然后从他们的诉求变成过程的功能和特征，但如果特征只是描述，没有数字也没有意义，所以要配合可衡量的度量单位，和用什么方式去收集那些数字，然后依据目标定过程要怎么去做，怎样改。

不是单纯“空降”某敏捷流程（例如 SCRUM），便能加快团队的发布速度。所以虽然极限编程里每一条实践都是最佳实践，也必须配合质量策划，和监控改进才会有效果。

乔布斯 85 年离开苹果，自己开创 NEXT 公司，他很注重质量，便邀请了裘兰博士 (Dr Juran) 从美国东岸飞到西岸，帮他们公司做辅导，改进过程。以下是节录他接受访谈时，对美国企业、质量和裘兰博士的观点：

美国已经富裕了很多年。很多企业都忘记要获得成功，还是要关注基本功，包括教育。现在我们美国很多企业面临困难，处处感觉被日本领先了。其实不是日本针对我们，而是我们作为美国企业家应反思一下，为什么我们的战略比日本差，为什么我们的策划不如日本？我们知道裘兰博士多次去日本，帮助日本企业提升质量。现在他回到美国，希望把他的经验带到美国企业，提升竞争力，可以再一次成为世界第一。

我觉得裘兰博士很实在，不是泛泛而谈。我们的工程师也深受他这种风格影响。无论我们之中谁问他问题，总裁还是工程师，他都会全心全意用自己的知识解答。所以工程师和我都很希望用他那套方法来做提升。质量提升的道理其实很简单，是一个重复的过程，然后我们需要不断去看，有哪些无效的环节要省略，哪部分要重新设计，不断试验、提升，就这么简单。重点是所有的提升都应该是科学化的，有数据而不是泛泛而谈。

一般管理层的思路是：我是领袖，你们应该听我命令。但应该是反过来，让应如何做好的决定权利放在团队手上，做改进不需要请求管理层的批准。改进是工作的一部分，整个架构扁平化，自己管控日常过程，每位工程师应像以前的工匠，愿意花精力不断做好。然后能以自己最后做出的优质工艺、产物自豪。

结束语

按质量大师裘兰博士定义，质量包括两部分：

- 满足客户需要（包括外部客户和内部客户）
- 没有缺陷

这定义不仅仅适合于制造业，也适用于服务业、IT 业。

丰田生产方式是 TQM (Total Quality Management 全面质量管理) 的最佳例子，TQM 强调专注客户、持续改进、以数据说话、员工参与等，丰田生产方式覆盖了 TQM 原则的七项（除了战略）。



从以上丰田故事看到丰田方式如何帮公司培养知识工作者，发挥人的无限智慧，

为公司增值。

丰田汽车，从 50 年代开始，沿用裘兰博士的质量管理思路，成为世界最大的汽车企业。

针对软件开发，如何基于以上质量管理、精益管理思路，配合敏捷开发最佳实践，提升产品质量与团队竞争力？

所以团队成员，包括团队组长与组员的能力是过程改进的基础，不仅仅依赖领导。

虽然 XP 比 SCRUM 更全面列出敏捷团队的最佳实践，但还必须依赖团队持续改善才会有效果。

下一部分，我们探索“团队与自我改善基本功”与成功要素。

附件

现代汽车生产

今天 Just-In-Time 已成为汽车制造的主流，例如：日产在英国牛津郡 (Oxfordshire) 专门生产 mini 车的工厂便能做到：

- 从钢材原料到生产出汽车只需要 24 小时。
- 整个生产线没有任何中间等候，每 68 秒出一辆。
- 每天生产 1000 辆车。

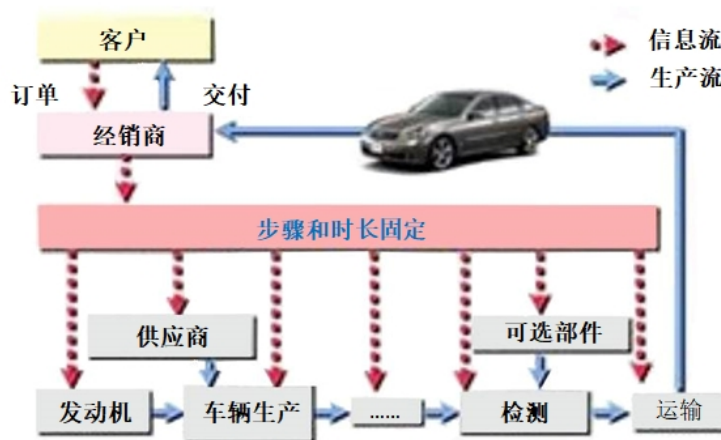


挑战：每一辆汽车都不同——颜色、设备、左右驾驶座等

- 生产线上每一辆汽车都按照客户需求订制。
- 组件不早不晚按需求准时到达生产线。
- 这便需要信息化系统把客户订单转换成生产信息。

客户订单与生产交付同步

<生产线的步骤固定，每一步的时长也固定，支持从客户下单到交付的生产线流程。>



例如生产线上每一辆的颜色都可以不同

**XP**

编码实践 (Coding Practices)

CP1: 简单地编码和设计 (Code and Design Simply)

- To produce software that is easy to change 使软件易于更改

CP2: 无情地重构 (Refactor Mercilessly)

- To find the code's optimal design 找到代码的最佳设计

CP3: 制订编码标准 (Develop Coding Standards)

- To communicate ideas clearly through code 通过代码清晰地传达想法

CP4: 共同的词汇 (Develop a Common Vocabulary)

- To communicate ideas about code clearly 清楚传达软件设计的想法

开发实践 (Develop Practices)**DP1: 测试驱动开发 TDD (Test-Driven-Development)**

- To prove that code works as it should 来证明软件正常工作:

- Test-first programming(prim practice#)

DP2: 结对编程 (Pair Programming)

- To spread knowledge, experience and ideas 传播知识、经验和想法:

- Pair Programming(prim practice#)

DP3: 集体负责写好代码 (vs 只顾虑自己的代码) Collective Code Ownership (vs individual own code)

- To spread the responsibility for the code to the whole team 将写好代码的责任扩展到整个团队:

- Whole team(prim practice#)
- Share code(corollary practice#)

DP4: 持续集成 (Integrate Continually)

- To reduce the impact of adding new features 降低添加新功能的影响:

- Incremental Design(prim practice#)
- Single code base(corollary practice#)
- Ten-minute Build(prim practice#)
- Continuous Integration(prim practice#)

商务实践（**Business Practices**）

BP1: 将客户添加进团队（Add a Customer to the Team）

- To address business concerns accurately and directly 准确、直接地解决业务问题:
- Real Customer involvement(corollary practice#)

BP2: 计划游戏（Play the Planning Game）

- To schedule the most important work 安排最重要的工作:
- Weekly cycle ; Quarterly cycle ; Slack (prim practice#)

BP3: 定期发布（Release Regularly）

- To return the customer's investment often 尽早交付，让客户看到投资回报:
- Incremental Deployment(corollary practice#)
- Daily Deployment(corollary practice#)

BP4: 以可持久的速度工作（Work at a Sustainable Pace）

- To go home tired, but not exhausted 回家时虽然很累，但不筋疲力尽:
- Slack (prim practice#)

“ 5 Why ” 实例

大野耐一先生见到生产线上的机器总是停转，虽然修过多次但仍不见好转，便上前询问现场的工作人员。

(1-Why) 问：“为什么机器停了？” 答：“因为超过了负荷，保险丝就断了。”

(2-Why) 问：“为什么超负荷呢？” 答：“因为轴承的润滑不够。”

(3-Why) 问：“为什么润滑不够？” 答：“因为润滑泵吸不上油来。”

(4-Why) 问：“为什么吸不上油来？” 答：“因为油泵轴磨损、松动了。”

(5-Why) 问：“为什么磨损了呢？” 答：“因为机器打磨金属零件，空气混进了铁屑等杂质，并掉进机器油缸里。”

经过连续 5 次不停地问“为什么”，找到问题的真正原因（润滑油里面混进了杂质）和真正的解决方案（安装过滤器）。由现象推其本质，因此找到永久性解决问题的方案，这就是 5 Why。

5S 法

本来 5S 是用于工业生产，例如日本的生产工厂很注重洁净，东西要放在容易找到的固定位置。其实这对生产线员工有很重要的心理作用。如果整个环境都很脏，必然会影响人工做好的动力，如果东西乱放，也容易找不到。5S 不仅仅适

用于制造业，也适用于服务业，每件东西必须放在固定位置；这也可以用于个人管理，我以前常常在出差去客户现场时，经常忘记把一些东西，如鼠标或插头。但后面我固定了每件东西都应放哪里，临走收拾时就确保那些东西不会遗漏掉。平常工作有一个洁净的环境，也能降低工作压力，提高工作效率。

		实例 Example
Organization	整理 <u>Seiri</u>	倒掉垃圾、长期不用的东西放仓库
Neatness	整顿 <u>Seiton</u>	30秒内就可找到要找的东西
Cleaning	清扫 <u>Seiso</u>	谁使用谁清洁、管理
Standardize	清洁 <u>Seiketsu</u>	维持无污染状态
Discipline & training	修养 <u>Shitsuke</u>	严守标准、遵守已决定工作习惯

参考 References

- 1. 若松义人. 《为什么是丰田：成为第一的方法和 7 个习惯》

Part II

团队与自我改善基本功

改善都应从团队开始，并要改善本来的工作习惯的不足，但改变工作习惯并不容易。首章用实例介绍公司团队如何从“破冰”到变革，到稳定，以及成功要素。

团队的提升依赖于个人的成长与提升。敏捷开发假定每位成员都具备极高的经验与能力，如果团队成员都是刚毕业的新手，不建议使用敏捷开发。后面第 3 章介绍提升个人效率的方法。

Chapter 2

改进从团队开始

上一章介绍了丰田方式水面下的七个习惯，但公司应如何有效开展与推行？有哪些误区要注意？我们先看美国东岸某家小印刷公司的故事。

美国费城 Weisbord 故事

60 年代复印机还未普及，很昂贵，所以有不少公司专门为各类公司客户提供印刷服务，这个故事的主人翁是某美国东岸一家印刷公司老板的儿子 Weisbord。他一直都是没有接受什么正式的管理培训。他的好朋友 Don 在国际大公司已工作了 20 年，Don 推荐 Weisbord 说：“很多大公司已经开始推进团队自主管理，提升生产率，你可以先读 X-Y 理论（McGregor, *Human side of Enterprise*）一书。”

X 理论：

对 (员工) 人性的假定

- 懒散 / 不想工作，尽可能避开责任
- 依赖 / 被动希望被指挥，工作安稳最重要
- 因此主管必须要把工作细分，并要紧密管控才会达到结果

Y 理论：

假定

- 关注工作，希望负责任 (take responsibility)
- 希望成长，有成就
- 如给予机会，会尽力把事情做好

问题处于 X 思维的主管

Weisbord 本来只想看看有多厉害，但他结果一口气一周末读完全书。他发现自己公司到处都是 X 理论管理：

- 工作分得很细，每个员工不清楚全局

- 任务都是依赖主管分配
- 员工遇到问题、困难，交给主管处理
- 员工上下班打卡

他觉得这方式应该可以帮公司提升，但担心单靠自己推动不了这个改革，便请 Don 过来正式加入公司。

他们分析公司业务最大问题是订单处理部门 (Order Processing)，每个订单处理工作分到 5、6 个任务，员工只管被分配的任务，例如：

- 筛选客户邮寄地址，邮寄印刷样本
- 输入订单。
- 检查客户的信用
- 发起内部生产订单
- 用打字机开发票邮寄出去
- 收到的支票，对上那些应付账

公司平均每天要处理 200 到 300 个订单，但因为工作分得很细，如果某个任务（如输入订单）有一、两人缺席，便严重影响整个流程，效率会降低一半。

针对这问题，Weisbord 计划改革，把这部门的人分成多个 4~5 人的团队，把公司的 2 万个客户按地区分到各团队，每个团队有自己的客户名单，打字机等设备，希望以此增加部门的灵活度，并提高生产率。

Weisbord 与 5 位小主管 (supervisor) 商量，2 位很赞成 2 位中立其他 1 位觉得不会成功。Weisbord 决策启动这改革，与 Don 开始重组部门。

开始团队制

开始时问题非常多：

- B 物流公司运送出错，送到另一个城市，怎么办
- D 团队误解了生产的步骤，怎么办
- C 团队的样板工人，刚入职 3 周，对我们公司产品线不熟识，怎么办

原因很简单：本来每个人以前都只懂自己的一小块工作，以前问题都是由小主管处理。现在自主团队没有主管，都要靠团队自己想办法，像瞎子摸象。

但问题确实太多了，没办法，Don 建议每周开会讨论。他们从未有每周开会的习惯，Weisbord 本来以为开会浪费时间，把时间用于生产更实际。问题一直都非常多，延续了一个月。

Weisbord 开始怀疑这 X-Y 理论，只是大学象牙塔里的玩意，难以真正用于实际公司环境。Don 也没有更好的建议。

Weisbord 开始有撤销整个改革的想法，返回本来的组织架构，Don 还希望 Weisbord 稍等一两周，看看有没有好转。但 Weisbord 觉得一直这么多问题，严重影响公司运作，也难以与父亲交代，想在下次开会时向大家宣布变回本来架构。

到了第五周，与以前开会一样，Weisbord 先问：大家有什么问题？

沉默。几分钟后，某组长说：没有问题。

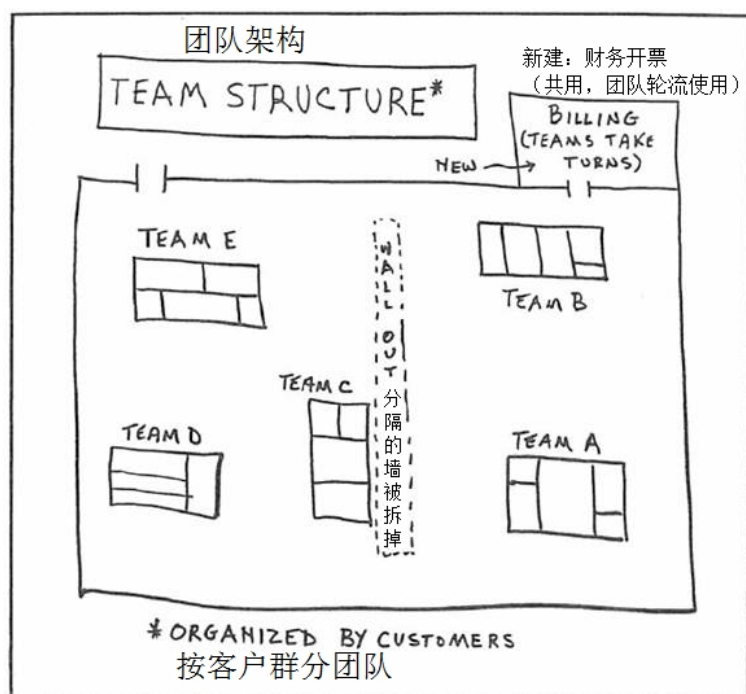
Weisbord: 为什么会没有? 一直这么多问题 (心里想, 一定是你们也放弃了, 连问题都不愿提了)。

另一组长回答: 我们今周没有什么问题, 遇到的问题都在以前的会议里面被解决了。

效果

改革的结果是从原来每天低于 300 个订单提升到了 400 个。员工的出勤率也得到了改善, 缺席率降低到接近零。

改革后的订单处理部平面图:



(团队自主改革前, 小主管们为了减少各功能之间在处理订单引起的争吵, 特意在中间加了一道墙, 后面团队们一致建议把墙拆掉。)

Weisbord 事后回顾: 所有改变, 重组都需要时间磨合, 过程中管理者的支持非常重要, 如果管理者不能坚持, 面对并解决面对的困难, 便会以失败告终。

启发

如果员工具备知识技能, 管理者便应尽力给员工平台, 让团队自主发挥, 才能不断提升竞争力, 应对变化万千的市场需要, 让员工与公司都受益。管理者不仅仅是制订目标、计划, 发号施令的角色, 而更应该是一位导师, 辅助团队成长。这故事也引出推行丰田式改善的 4 个常见误区:

- 不要指望一蹴而就
- 只能坚持到有结果，不能半途而废

本来 Weisbord 和 Don 误以为会很快就有效果，预估不了会出现这么多问题，如果他们没有坚持到底，就会以失败告终。

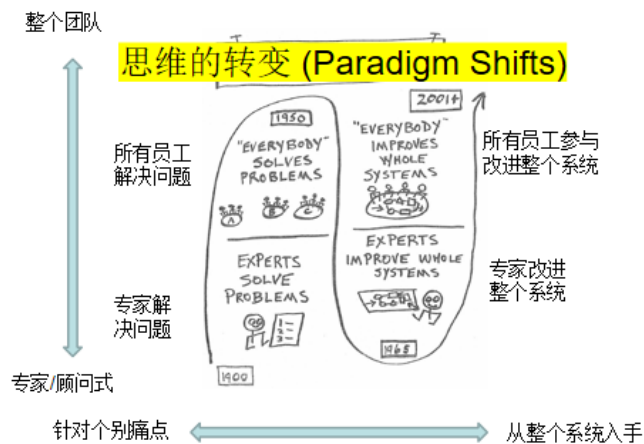
- 最重要是培养人才，但训练比教育更重要

大野先生认为：教育式教对方他们本来不知道的东西。训练是让对方重复练习已经知道的东西，让他们牢牢记住该怎样做。要做好训练，领导或内部教练必须能够身先士卒带头动手做。不然会容易被批“光会嘴巴说，但什么都不干”。

- 让员工自己找答案

过程改进需要每一个部门都参与，要由团队自己去找答案，而不是直接告诉他们怎么做，才可以提升公司的最终目标，

所有员工参与改进整个系统



(上图参考 M. Weisbord, Productive Workplaces Revisited)

大部分已经是九零后了，很多员工都有自己的想法，很难要求他们像以前只听从主管指挥。所以也不能再依赖专家（外部或内部）和顾问解决问题，必须所有员工参与改进整个系统。

上面费城故事就是领导让团队记住自主和学习，完成订单处理部门从传统主管式管理改组成团队自主管理的例子。

当时，改成团队制后开始每周开会，不仅仅解决团队的疑问，其实也促进团队之间的交流，了解其他团队的问题。

如果管理层认同这种新的管理模式，组织架构从本来的功能划分变为小组独立自主管理外，还有什么具体方法促进公司走向所有员工参与改进整个系统？

可以定期举报了一些工作坊集体培训，请各个团队主要岗位的代表都参加：

回顾过去，放眼未来

最后依据培训的主题，让大家讨论并制定具体的改进目标和行动计划。

但若要有效果，管理者必须定期监控：

改进效果如何？

遇到什么困难计划？

计划要不要调整？

若不及时监控，团队会觉得只是一般的培训，学完便完事了。

工作坊可以促进团队间的了解，例如：

- 产品开发部原来不知道现场实施工程师对新产品的安装和使用还是有很多误解。
 - 后面开发部除了做安装手册也录视频，减少现场工程师误解。
- 开发工程师从现场实施工程师了解到有些因为现场测试不通过，导致不能按时发新版本的主因。
 - 原来客户验收有一套标准的流程，包括他们的测试数据和步骤，但测试人员交付前的检测都只是按自己理解测试，与客户验收流程有出入。

芬兰计划

要具备什么条件才能让敏捷开发团队有好成绩？

再邀请你来听我讲 ACP 的第一堂课。

(ACP = Agile Certified Professional 是美国 PMI 推的敏捷管理个人资质。)

???	???
94 年 由两位史丹福研究生创立	96-98 年 由两位史丹福研究生创立
1996年上市	2004年 上市
员工数 超过 8千 (2016)	员工数 超过 7万
产品领域:	一些产品包括:
Mail, Search, Web portal, Directory,	Android
News, Messenger.....	Chrome

“请问大家左右两家美国西岸 IT 公司是什么公司？”

估计你们都能猜到右面的是谷歌 Google，但不一定能认出左面的是 Yahoo！因为后者在 2016 年已经被收购。在 2001 年时，谷歌是刚成立不久的小公司，雅虎是首批互联网公司，当时已经是有名的上市大公司，市值比福特、通用汽车公司还高，并不断收购公司扩大业务。但也养成了很多“大公司”病——官僚化管理。例如很多决策或改进都需要管理层多层审批，有很多 X 管理的味道。

谷歌 (Google) 公司是由两位史丹福大学研究生 Larry Page, Sergey Brin 创办。在 2000 年, Eric Schmidt 加入并担任 CEO (他之前是 Novell 的 CEO, 以前在 SUN Microsystems 工作), Jonathan Rosenberg 也加入并出任产品经理 (他之前在 Apple 当产品经理)。

谷歌的管理很扁平, 当 Eric Schmidt 第一天进公司当 CEO, 他本来是 Novell 的 CEO。因为谷歌公司快速发展员工很多, 办公空位不够, 人事部经理好不容易才为他找到一个办公室, 但过了几天有一位工程师来问: “我那边的办公地点是在太挤了, 我看你的办公室应该还可以容立多一个人。我可否搬过来过来?” 虽然 Eric Schmidt 觉得很奇怪, 但因为刚进公司, 就接受了。

当时谷歌 (Google) 还是很小的公司, 但因在搜索引擎上有所突破, 预计会被微软攻击和收购。董事会主席 Mike Moritz 便请刚进公司的 CEO Eric 和产品经理 Jonathan 准备了一份详细的商业计划书来应对, 代号“芬兰计划”。俩人经过两个多月细心准备, 约创始人 Larry 开会解释计划详细内容, Larry 拿到计划书快速看了一下, 便立马问他们俩: “你们以往有试过超出你们的计划吗? 如果没有, 建议还是先跟我们那些工程师聊聊吧。”

后面他们俩发现 Google 的工程师都是精英, 无论技术或业务方面的能力都很强, 都有很多创新思路。了解到如果还是按传统的计划驱动反而会限制团队的创新能力。

Eric 与 Jonathan 后面把谷歌的独特文化写成书 “How Google Works”, 他们把公司里这些优秀人才统称为聪明创意人 (Smart Creatives)。

当我讲到这里时, 便把书的目录派给学员, 然后讲解书里相关章节, 介绍谷歌公司文化:

七的规则 (The Rule of Seven)

一般公司组织架构都会限制主管直属下属人数, 谷歌当时也规定人数为 7, 但规定直接下属不能少于 7, 可以看到他们组织架构多扁平。

河马 HiPPO

河马 (Hippopotamuses) 是攻击力很强, 非常危险的动物。

谷歌公司用河马的英文简称 HiPPO (Highest Paid Person's Opinion), 比喻企业类内的“大王”按自己的主观判断, 雷厉风行也是非常危险。

例如在谷歌的内部讨论会, 做决定不看提出意见者在公司的地位、权威。高管提出的建议, 如果缺乏充足的数据支持, 也可以被工程师推翻, 例如创始人之一 Sergey 想某工程部协助开发某新产品, 但他的方案缺乏有力数据支撑, 工程主管不同意。Sergey 没有用他的权威强制要求 (他绝对有这权力), 他让步妥协, 只需要一半人手参与, 但工程主管还是不同意, 最终大家客观比较各种方案的优劣, Sergey 的方案未被采纳。

谷歌有个特点, 每次开会都会用 2 个投影仪, 其中一个专门投放数据的。很多公司管理层都会嘴巴说用数据说话。

两个披萨的规则 (Bezo's two pizza rule)

两个披萨应该让团队够吃, 团队人数不能太多, 妨碍有效沟通。

员工工作饮食生活在一起 (**Work, eat, live together.**),(Keep them crowded)

这跟敏捷开发要求团队必须面对面沟通，一起工作，效率才会高如出一辙。

所有重组在一天内完成 (**Do all reorgs in a day**)

因为市场变化快，组织必须频繁重组才能跟上。为了减少重组对工作的影响，规定每次重组必须快速完成。

针对客户 (**Focus on the user**)

制定几乎达不到的目标 (**Set {almost} unattainable goals**)

这 2 条与前面丰田的文化类似。也验证了持续改善的原则不仅适用于制造业。

70/20/10 规则

- 70% 的时间用于和核心业务
- 20% 用于具备潜力的新产品
- 10% 是用于全新的产品（例如在初步构思阶段）

20% 规则

员工都可以用 20% 的时间用于自己有兴趣的项目。

以前惠普公司（创始人 Hewlett 和 Packard 本身都是工程师）也规定公司实验室七天 24 小时开放。欢迎工程师任何时间进实验室做自己感兴趣的项目，不一定局限于公司安排的项目。很多创新的产品不是高层可以通过想象得出来的。例如谷歌的 Android，就是其中一个员工特别喜欢玩手机操作系统，开始的时候，他是用自己时间创建出来，后面成为谷歌产品的其中一条主力。

如项目不能达到预期效果，便立马叫停 (**Fail well**)

例如谷歌 Sydney 分公司推出新一代电子邮箱服务，虽然里面有很多创新，但一直没有达到预期的用户数量。便被叫停。

频繁交付和迭代 (**Ship and iterate**)

因为市场和需求快速变化，企业必须跟上节奏，尽量减少浪费。
这条与上一条类似，都是精益和敏捷开发的原则。

====

2000 年时，谷歌在快速成长期。我通常会在敏捷培训课的第一天借用谷歌的故事说明团队敏捷开发的成败依赖于公司文化。

讲敏捷课时，某学员听完敏捷精益的概念后，说她的经理要求她输出一份 3 个月的详细工作任务分解，且要求活动的颗粒度细到 5 天之内，因为她的经理按计划管理的思路很重。她很困惑，软件开发变化这么多，怎么可以做出这种详细计划呢？

这其实有点像我们要在一个足球赛还没开始之前，要预先写赛后报告一样没意义，都是猜。

我说：如果老板有这种思路的话是无法推动敏捷开发，其实敏捷开发有一个很重要的概念，因为需求变化这么大，我们的范围是不固定的，所以基于精益的概念，在有限的时间里，我们要制订这个冲刺在 2 周里面可以完成哪些故事。做完以后再依据过去的的数据进一步策划下一个阶段就更实际了，不是全都靠猜测。这种思路更能让客户尽早看到实际的项目进展，所以敏捷特别重视可以运行测试好的故事，而不是一堆需求文档或者没测试过的代码，因为需求没有给客户看过，还不算是完成，只是一个中间的状态。所以如果经理知道软件开发的这种特性后，就会在那个策划的三角形——时间、成本和规模范围之间适当取舍。如果我们规定时间为 2 周，应该可以在那些故事里面挑选最有价值，可以在 2 周完成的故事，完成它并展示给客户。这就有了 MVP（最小有价值产物）的概念，就会避免团队为了只追求进度，而导致最后做了一大堆没价值的东西。

早上讲完谷歌故事，学员下午做互动练习时便说自己公司里确实有不少“河马”横行。

从以上两故事看到，优秀的公司高层，无论东西方，都理解公司的发展依赖“大家的力量”和各人的智慧，同心协力，“大王”文化会妨碍公司持续改进，难以改善。从以上故事看到，如果公司缺乏敏捷精益的文化，管理层也不理解，单靠团队参加培训，不会有改进效果。这也是开头案例里张工推行敏捷开发失败的原因之一。

从 Weisbord 先生的故事，了解到 XY 理论，管理层要赋能与团队，公司才有机会提升，但这只是第一步。

从谷歌例子看到，公司文化、员工的能力都是关键因素。当时谷歌每位员工骨子里都已经包含了敏捷开发和精益，不仅仅是公司材料的条文，而是实际落到员工的行为上。当你踏进谷歌公司范围，跟员工接触，立刻能感受到。但有多少会像谷歌这样，用行动表现。所以我在培训 3 天 ACP 敏捷开发管理时，首先就以谷歌公司为例，说明如果公司没有类似的敏捷文化来支持团队，学了各种精益和敏捷的原则和方法，也会不会起什么效果。

团队的成长与能力取决于成员的能力，下章先探索一些能帮助个人提升的方法。

参考 References

1. McGregor, D. *The Human Side of Enterprise*.(1960)
2. Weisbord, M.R. *Productive workplaces revisited : dignity, meaning, and community in the 21st century*. (2004)

3. Schmidt, Eric *How Google Works*.(2001)

Chapter 3

克服拖延症

技术总监问：现在我遇到最大的难题就是如何提升下面技术人员的能力，如果他们全都是高手，我就很轻松了，但实际上高手最多只有 1/3，其他都是中低水平。你接触过这么多软件开发团队，有什么好方案？

我：你可以先听听以下故事。

=====

小李：你平常办公时间一直都很忙，还可以腾出晚上和周末时间，把客户遇到的问题，如何解决等，汇总成分享文章，每两周公众号发布，很厉害呀。

我：其实你也可以做到。要成为专业软件工程师，除了要学习软件工程相关的知识与技能外，个人有没有高效率的习惯其实更重要。

我在 5 年前教项目管理时参考过一本效率小册（详见参考 Reference），这本小册罗列了 99 个小技巧，每个技巧都不超过一页纸，我自己也一直用这小手册提醒自己。

目录

Part 1	Part 6
Beating Procrastination	Productivity Hacks
克服拖延症	产生效率黑洞
Part 2	Part 7
Becoming Organized	Doing the Right Work
做事更有条理	做对的工作
Part 3	
Staying Energized	
保持活力	
Part 4	
Getting Things Finished	
把事情完成	
Part 5	
Automate Your Routine	
让你的日常工作自动进行	

例如，第一章克服拖延症，这里的内容几乎全部都有帮助。

周/日目标 (Weekly/ Daily Goals)

我每天都会订计划，早上希望完成哪些功能，下午完成哪些。当然这个计划也会按实际的进展调整。

周 / 日目标是个人时间管理的基本功。每一天第一件事不是回邮件，而是仔细想想今天要完成什么任务，每一周的开始，也应该想我本周希望完成什么任务。不然的话，每天的时间就很容易被琐碎的小事吃掉，一事无成。

背后体现的道理很简单，要把时间花在重要、但非紧急的活动上，效率才会体现出来。

	紧急	非紧急
重 要	危机 紧迫的问题 受截止时间驱使的项目	针对风险未雨绸缪 建立关系 挖掘新机会 规划、娱乐
非 重 要	打扰，电话 邮件，报告 会议 紧迫的事情 受欢迎的活动	费时的琐事 邮件 电话 轻松快慰的活动

限定时间 (Timeboxing)

把每天的任务安排成时间段，每一段不应超过 1.5 小时。
一般人可以专心集中的时间段都不会超过 60 分钟，小孩可能更短。如果老师叫你星期五 5 点钟交卷，你不会提前交，都会等到最后 10 分钟，甚至最后 5 分钟。所以如果我们把一天的时间切开，分成 1~1.5 小时时间段，自然有动力，希望在时间之内完成任务。
我们写代码的时候应该也是用同样的原理。例如某些编程活动尝试了多次，但没有进展，有时总共会花超过 10 小时。所以每次当我发现某编程工作超过了 2 小时，我就会先做其他事情。

分解任务 (Dissolving tasks)

个人学习编程经验：因为都是练习题，所以每一个功能都比较细，不会超过 20 行。如果我们平常做开发时，也必须要把一些大、复杂的功能预先细分才有效率。

加强自律 (Building Self-Discipline Muscles)

晨礼 (Morning Rituals) 日常运动 (Make an Exercise Routine)

不要以为编码是一个单纯的脑力活，整天坐在屏幕前面敲代码就可以。如果人的体力、精力没有配合上也会出问题，好在我每天早上一直坚持 30 ~ 40 分钟的轻量运动，然后晚饭前 0.5 ~ 1 小时的骑单车或者慢跑的习惯。中间也不是整天坐着，一段时间会走一走、喝点儿橙汁等，以确保身体不断在动，这样才不会困，保持动力。
贝多芬每天都会通过去外面散步来获得一些创作的灵感，然后他会立马把这些写在本子上，用于后面的音乐创作。

我：身体健康，精神状态也同样重要，你每周有锻炼的习惯吗？

小李：没有，每天都太忙了，虽然一直觉得身体不如几年前了，也知道锻炼好，但无法抽出时间。

我：我也很忙，但深知定期运动对身体非常重要，我一直按以下 2 种方法保持个人身体状态—

- 工作时尽量避免长期坐下来，因我主要做培训、咨询、评估，所以可以大部分时间站着或在走动。(NEAT#)。
- 尽量每天 7 点吃早餐前跑圈，疾跑 1.5 ~ 2 分钟，休息半分钟，重复这循环 4 ~ 6 轮。(HIIT#)。

不会分心的工作场所 (**Create a Distraction-Free workplace**)

轻策划、迭代、再策划 (**Ready , Fire, Aim!**)

30 年前，软件开发都是一些大型的项目，整个架构要设计好才动手去写代码。现在反过来，需求变化极大，开发都需要敏捷，轻文档、轻计划，尽快写好代码，做一些功能给客户，从反馈优化下一轮。我这次的几天开发也是用同样原则，没有花时间在有些设计或者文档。想直接把代码写出来，并通过单元测试，节省了很多耗时间的工作。把有限的时间都放在写好代码上。

不断清洗 (**Churning**)

万事起头难。我假期重学编码时也是遇到同样问题，不知如何入手，太久没看写代码的书了，很多基本的都不知如何入手。所以我开始的时候不会直接尝试写题目里面的功能，而是重写一些书本的代码，看看结果怎么样，然后逐步提升。写一些基本功能，慢慢有了习惯，调整过来了，后面就越来越顺。好比一台旧的水泵，刚开始抽上来的水总是有难喝的铁锈，只要不停止抽水，当污水最终都从系统中抽出后，就能发现底下的净水。

要有好的土壤 (**Remove your Hidden Roadblocks**)

在含盐量高的土壤里种植物是结不出果实的。浇水、平衡在阴凉处和阳光下的时间都抵不过根部吸入的毒素。如果我们没有积极性，就可能是土壤的问题。如果没有足够的积极动力，就不会在长假专注写程序，也不会定期要求自己写分享文章。所以要有明确、很想达到的目标驱动。像作曲家希望写出很多经典的优秀作品一样，会不满足于现在的状态。觉得自己的灵感或者创造力没有发挥出来，成为可以保留下来的东西。也是这种驱动力让我可以一直努力做这件事。

摒弃拖延恶习 (**Quit your Procrastination Vices**)

长假里，大部分人都会把时间用于看视频或电视剧，而我正好没有这个习惯，也一直没有玩网络游戏的习惯，否则肯定完成不了。

最终我用日程记录 (Timelogging)，把整件事和什么活动、时间花在什么地方都记录下来。

小李：我看你上面列出的技巧，我大部分都还没做到。

我：不要紧，我 6 年前刚开始定期写文章时跟你一样，但只要不放弃，一直往既定目标努力，不良习惯最终都会改正过来。我常常说人的潜力是极大的。舒伯特你听过吗？

小李：好像是很有名的作曲家。

我：是的，但他 31 岁就去世了，你猜他一生一共写了多少首歌和音乐作品。

小李：我记得中学时，老师介绍过他的艺术作品，如《鳟鱼》，但他 31 岁就死了，我猜 100 ~ 200 首歌？

我：他一生写了超过 460 首歌曲（时长 >24 小时）。除了歌曲，他还写了其他作品，如 9 首交响曲（1 首未完成，1 首只有草稿）、20 室内乐、120 钢琴曲等，每一类都包括大量经典作品，对后世影响深远。

小李：如果粗算一下，他一生约有 600 个作品，算他有 16 年时间作曲，平均每月要完成 3 个作品，真是不得了。

我：虽然他的作品有大有小（从一首歌到 45 分钟的交响曲），他确实生产率极高，而且他最后的 7 年身体一直都不好，所以他那个时候肯定不会像我们现代“996”方式工作。他每天主要是早上用来写作，傍晚便去休息散步。但他会同时做多个创作项目。如果项目没有灵感，就暂时放下来，创作其他作品。他著名的未完成交响曲就是个例子，只有两个乐章（一般交响曲都是 4 个乐章）。所以他是使用高效技巧的一个成功例子。每个人都有自己的理想，但如果没有高效率来执行，理想只是天马行空、天方夜谭，不会有任何成就。除了以上这些技巧外，保持整洁也重要。你有没有试过想找某东西，找半天都找不着？

小李：确实经常发生，而且还会遗漏东西。我上次出差便忘记了 iPhone，后面回北京后电话联系当地酒店前台后，我找当地同事去酒店取，然后快递给我，烦死了。

我：有听过 5S（5S 法 #）吗？例如，如果你把东西都放固定地方，就可以避免同类问题再发生。如果你一直在一个很乱的环境工作，会导致心情烦躁，对工作、身体都不好。

（# 详见附件“锻炼之道”的多了解 NEAT、HIIT；5S 法详见第 1 章附件。）

总监：我大概懂你的意思了，要提升技术人员的能力应先改变他们的习惯——如时间管理，才有机会提升。

=====

总监：我大概懂你的意思了，要提升技术人员的能力先要改变他们的习惯，有良好的习惯—如时间管理，才有机会提升。

即时笔记 (The Capture Device)

总监边听边在本子上记下那些重点。高效的人都会有工具帮他记录想到的灵感、想法、项目、待做事项等，不会仅仅靠大脑记忆。你提出一个要求，他会立马写在小本子上，你会觉得他应该会按你要求去处理，但反过来如他只是口头说会处理，你会担心很可能没有下文。但我看有些领导，身边只拿个手机，除非他们的记忆力超人，否则我估计他每天都会忘记不少重要事项。

结束语

很赞同杭州某高级经理的总结：

人一定要自律！你说的小技巧确实能起到很大帮助，而且我基本都会使用，但如果不养成习惯，想起来使用下，最终还是改不了拖延症。所以要解决拖延症，一定从根源做起，还是得靠自己，需要培养自己意志力、专注力，坚持好习惯，改掉坏毛病。

我相信人分高低，但并非取决于基因、种族，主要取决于她后天的习惯、自律与努力。要养成良好习惯要从小开始，深受家庭和教育的影响，所以百年树人。

与公司改进一样，改变个人习惯很难，这些技巧可以帮助个人改善。

附件

锻炼之道 The Truth about EXERCISE

想大家都同意和相信：“多运动，便多烧耗卡路里，便能帮助减肥，降低体重。”

某国家给市民的健康指南：每周起码做 150 分钟中强度锻炼，或 75 分钟高强度锻炼。

但不是每个人都能每天抽时间做锻炼，有什么更好方法？

不一定只依赖去健身房锻炼，平常工作生活少坐多走、站着工作、开会，甚至小动作等都有帮助。

N.E.A.T. (Non-exercise activity thermogenesis) 小实验：教授使用有电子传感器的底裤，记录记者、咖啡厅女服务员、商务人员 3 人一周每天正常工作中消耗多少卡路里。

发现：

- 女服务员最好。因每天都非常忙碌（尤其是早餐时段），送餐、接单、做咖啡等等。
- 商务人员第二。虽然有很多时间坐下来，但每天都会走一公里路见客户，而且每周二、五下班后会去健身锻炼。
- 记者最差。每天无论工作或家里，大部分时间都是坐下不动，所以他看起来不胖，但其实体内存有大量脂肪，集中在肝、肾等内脏。

这实验告诉我们：如果每天一直坐下不动很不好，就算每天下班后晚上都去健身锻炼也帮不了。

后面记者听教授建议，改变习惯，定期站起来走动。如与同事交流与尽量边走边路边交流，减少长期坐下来；多爬楼梯，少用电梯；骑单车，不开车等方式。后面，从实验数据分析，发现这些改变帮他增加每天卡路里消耗接近一倍，到 500 水平。

研究发现不断大量健身锻炼不一定对每个人都有效。有 20% 会没有效果，另一端对 15% 的人会非常有效。这跟人的基因密切相关，所以多锻炼不一定都有效。

=====

实验发现，如锻炼能快速提升心跳率到最高，然后休息，反复做 4-6 轮，效果不会比大量健身锻炼差。例如每天做几轮 40 秒的冲刺，把心跳速度快速提升到极限，效果可以比长时间的缓步长跑更好。

HIIT (High Intensity Interval Training) 小实验: 教授与助教记者使用运动单车做 HIIT, 用尽全力练 20 秒, 休息, 再同样做两轮; 每周 3 次。记者按教授要求完成了 4 周 HIIT 锻炼, 虽然帮他提高了血液分解糖份的能力, 减少糖尿病风险; 但提升不了他的最高带氧运动量。教授解释这是因记者的遗传基因是属于没有效果的 20%。

参考 References

1. Young, Scott. "The Little book of Productivity." 《超效率手册》

Part III

自我管理开发过程

如何估算工作量一直都是软件开发的挑战。大家都知道应分析历史项目数据，作为后面项目估算的参考。但首先要不同项目的数据可以相互比较。如能客观衡量项目规模大小，便可用规模作为分母把不同大小的项目归一，让不同大小的项目可比。本部分会讨论应使用什么来估算项目规模。

软件开发工程师自己收集数据并分析，是团队量化管理的基础。敏捷开发团队可以从个体软件过程 (PSP) 开始。本部分也会简单介绍估算工作量、工程师如何收集数据并分析等的挑战与最佳实践。

估算本应是一个范围，不应单是一个数，会先用一个例子介绍三点估算法，并与模拟结果比较，衡量三点估算法的正确性，是否可用。

Chapter 4

三点估算

估算是一个范围，不是一个数

唐工：你估计完成开发用户登录模块要多少天？

小李：3 天。

唐工：能在 3 天完成的可能性有多高？

小李：可能性很高。

唐工：可否量化一点？

小李：可能性为 50% 60%。

唐工：所以很有可能不止 3 天，要 4 天了。

小李：对的，其实也有可能要 5、6 天，但我估计机会不大。

唐工：你信心有多少？

小李：难说，有 95% 的信心可以在 6 天之内完成。

唐工：所以有可能要用上 7 天了？

小李：这样说吧，如果所有可能出问题的都出了问题，甚至会 10 天或 11 天，但这种概率很低。

唐工再问小李：是能否给我一个确实能完成这个模块的日期？

小李：正如我前面说，很可能 3 天可以完成，但有可能 4 天。

唐工追问：你可以说 4 天吗？

小李：也有可能 5、6 天。

唐工结束对话：OK，请你尽力 6 天之内完成这个模块。

唐工貌似请求，但实际是要求小李承诺这个模块要在六天之内开发完。假如这个模块的开发时间超过 6 天，唐工就有依据说小李没有尽力导致延误了。

所以从以上对话，可以看到作为开发专业人员，必须分清估算和承诺。作为专业人士，我们不应该给一些没有把握的承诺，误导对方。中国老话说“一诺千金”就是这个道理。

从单点到三点估算

从上面的例子可以看到，一般的单点估算是很容易被误导，以为那个天数是有把握达成的，所以我们最好从单点估算变成三点估算，除了估算最可能的天数，

还有最佳和最差共三点。但项目是由一系列的任务组成（如第二任务依赖于第一个任务的完成），如何计算所有任务的总天数？下面用例子说明如何用 3 种使用三点估算估计的方法（A、B、C）估算总天数：

A) 假定都是正态分布，用模型估计：

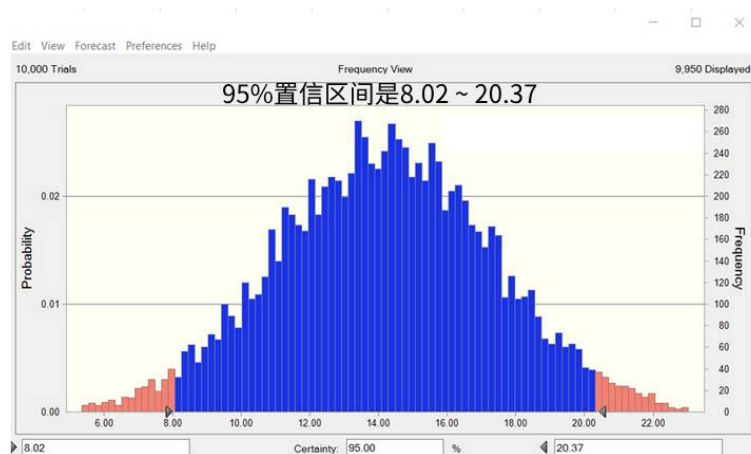
先用 PERT 方程式计算每一步的预计值与标准差：

预计值 (Expected Value EV) = (Best + 4xMost Likely + Worst) / 6

标准差 (Sigma) = (Worst - Best) / 6

Step	Best 最佳	Most Likely 最可能	Worst 最差	EV	Sigma
1	1	3	12	4.167	1.833
2	1	1.5	14	3.5	2.167
3	3	6.25	11	6.5	1.333
		10.75		14.168	

如果假定是正态分布，按以上预计值和标准差，使用蒙特卡洛模拟，从下图可看到，95% 置信区间是 8.02 ~ 20.37



B) 直接用 PERT 方程式计算总天数的均值与标准差：

如不用模拟，直接把 3 步的均值加起来：

$$4.2 + 3.5 + 3.6 = 14$$

计算 3 步总方差：(方差 = Sigma^2)

假定：总方差 = 每步方差的总和

总方差 = 9.77

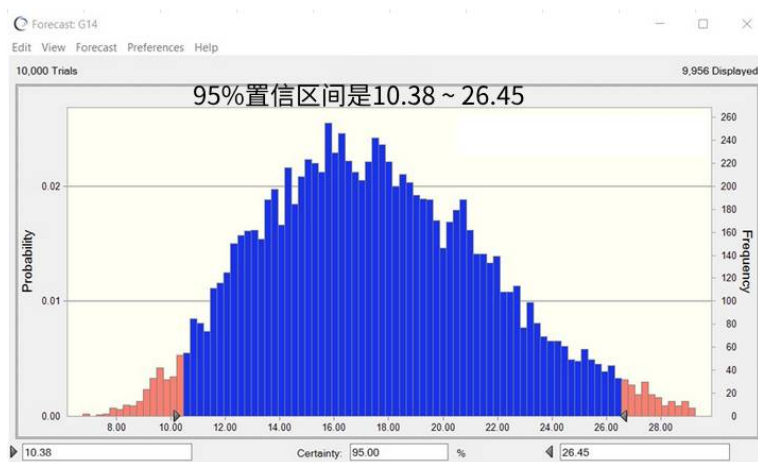
Sigma σ = 3.13

95% 置信区间计算公式为：均值的总和 $\pm 2\sigma = (4.2 + 3.5 + 3.6) \pm 2 \times 3.13 =$

$$14 \pm 6.26 = 7.74 \sim 20.26$$

结果与蒙特卡洛模拟预测类似。

C) 假定都是三角形分布，用模型估计：
如果用三角形分布，95% 置信区间是 10.38 ~ 26.45



总结 + 解读分析结果

- 如果假定每一步的分布都是一个正态分布，就可以用头两个方程式计算每一步的平均值跟标准差和方差，用方程式可计算 3 步的总均值大概是 14。也可以用方程式计算标准差，总的标准差 (sigma) 是 3.13 左右。
- 也可用蒙特卡洛模型（假定步骤都是正态分布），得出很类似的正态分布，总的平均也接近 14，95% 的置信区间是从 8.02 20.37，接近上面算出的均值 $\pm$ 两个标准差数值。
- 但因 3 个步骤都是明显往右偏，所以不能假设它们是正态分布，更合适的是使用三角形分布，然后用蒙特卡洛估算“加”起来的分布，看见最后的图明显是类似往右有个尾巴，能更正确反应 3 个步骤加起来的总数的估计分布。
- 跟假定正态分布的结果比较，很明显看到用三角形分布结果往右偏，上限是 26.45（比正态分布的 20.37 高）。不是正态分布的话，左面就没有长尾巴，所以就会比本来正态分布的下限高，下限是 10.38（比正态分布的 8 高）。
- 从这简单例子看到，如果我们要把三点估算加起来，尤其是非正态分布的话，就不能用简单的方程式，或者假定它是正态分布来计算，需要用蒙特卡洛模型假设三角形分布才能真正反应总体的分布。

从这 3 个偏左分布步骤例子看起来好像有些偏差，但不是很严重。如果我们看见用十个步骤都是偏一边分布，总分布会如何？是否相差会更远？

利用蒙特卡洛模拟 10 个步骤（三角形分布）的总分布

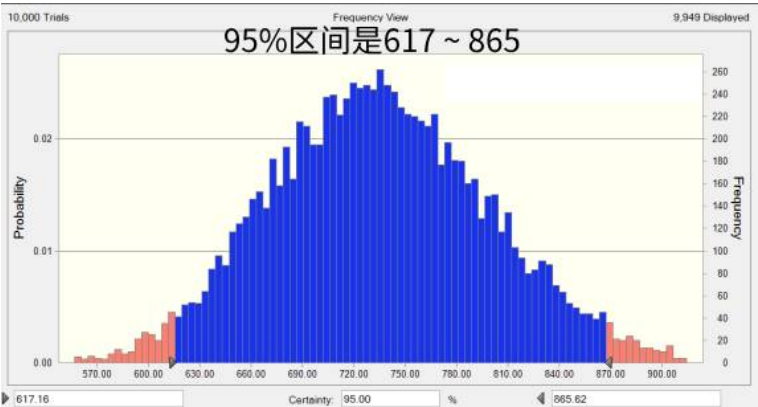
如果每步都估算天数，10 个步骤的总天数就是 500 天（把 10 个估算值加起来）。

但如果每个步骤都是三点估算：

Process 过程	天数 Durations		
1	27	30	75
2	45	50	125
3	72	80	200
4	45	50	125
5	81	90	225

很明显看到每一步都是偏左的分布，所以可预计总天数应不止 500 天，但估多少才合适？

假定每步骤是三角形分布，用模型估计重复 5000 次，得出下面分布：



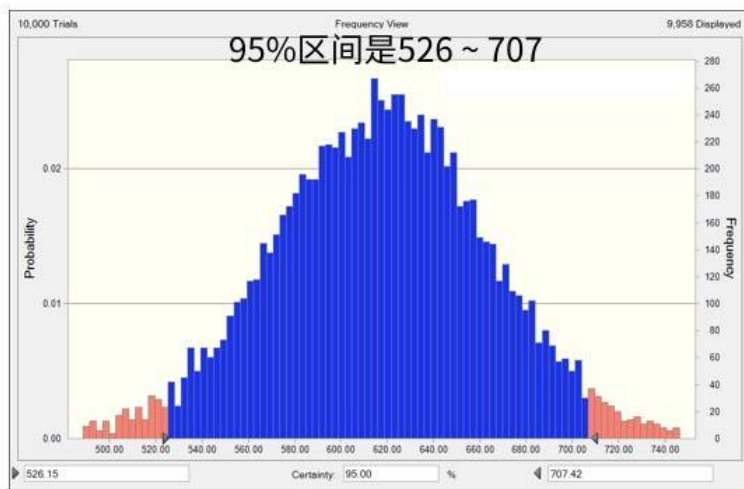
Process 过程	天数 Durations				
1	27	30	75	37	80
2	45	50	125	61.66	13.33
3	72	80	200	98.66	21.33
4	45	50	125	61.66	13.33
5	81	90	225	111	24
6	23	25	63	31	6.66
7	32	35	88	43.33	9.33
8	41	45	113	55.66	12
9	63	70	175	86.33	18.66
10	23	25	63	31	6.66
		500		617.33	

A) 用 PERT 方程式计算每一步的预计值与标准差：

Step	Best 最佳	Most Likely 最可能	Worst 最差	Expected	Sigma
1	27	30	75	37	8
2	45	50	125	61.6666667	13.333333
3	72	80	200	98.6666667	21.333333
4	45	50	125	61.6666667	13.333333
5	81	90	225	111	24
6	23	25	63	31	6.666667
7	32	35	88	43.3333333	9.333333
8	41	45	113	55.6666667	12
9	63	70	175	86.3333333	18.66667
10	23	25	63	31	6.666667
		500		617.333333	

得出 95

B) 假定是正态分布，按以上预计值和标准差，使用蒙特卡洛模拟，得出的总分布的结果几乎一致，都是左右平均分布的正态形。



分析 10 个步骤模拟结果

- 为什么用三角形分布模拟出来不是偏左的分布（类似前面 3 步结果），而是一个正态分布。

以上实验验证了“中心极限定理”，无论本来是什么形状分布，如果随机抽样够多，样本的平均值分布接近正态分布。所以如果本来只是 3 个步骤的时候还是可以看出是三角形偏左，但到了用 10 个步骤相加时，得出的分布便非常接近正态分布。

（中心极限定理会在后面数据分析里用上，例如通过画控制图判断过程是否稳定）。

- 实验结果也验证了当每一步都类似正态分布可以用 PERT 公式计算每一

步的预计值和标准差，然后计算总结结果的分布（不需要蒙特卡洛模拟），但如果非正态分布（如偏左的三角形分布）便需要使用蒙特卡洛模拟，不然预估会有偏差（类似上面 3 步模拟的结果）。

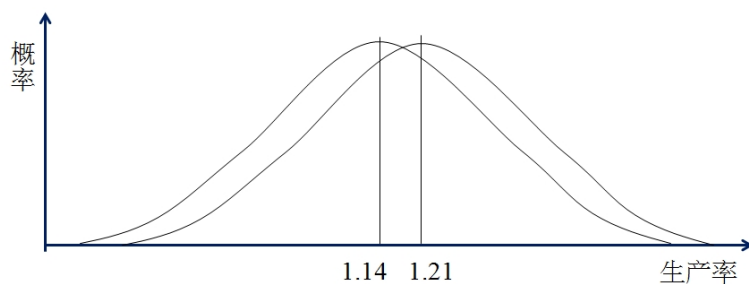
问答 Q&A

问：为什么要花这么多精力去研究分布，我们日常不都是单点估算吗？

答：例如你觉得把整个公司的人均生产率从 1.14 一年后提升到 1.21 算不错吗？

(注) 问：不是非常好，还算可以。

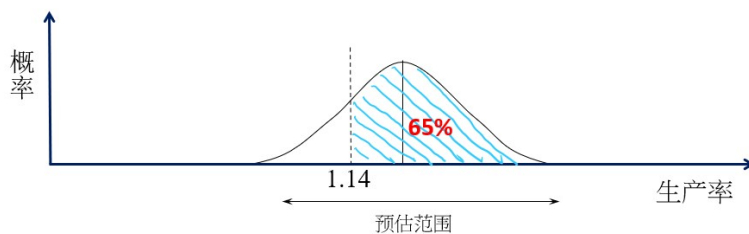
答：如果本来的分布和提升后的目标是如下图，你觉得怎么样？



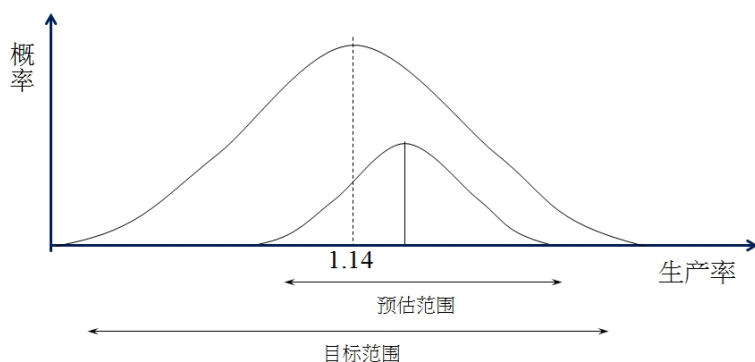
问：提升就太微小了。

答：从这简单例子看到所有估算都应包含两部分：分布和中间趋势（例如平均值）。

另一例子：假如我们预估生产率的分布是如下图，达到或超越 1.14 这目标的概率是 65



但如果告诉你目标是 1.14 只是目标平均值，分布是如下图：



预测的生产率分布完全在目标范围之内。

(注：生产率单位每人天产出代码的功能点数，类似有效代码行数都是衡量软件规模的单位。)

在下一部分，我们会看到如何使用 PERT 三点估算的实例。

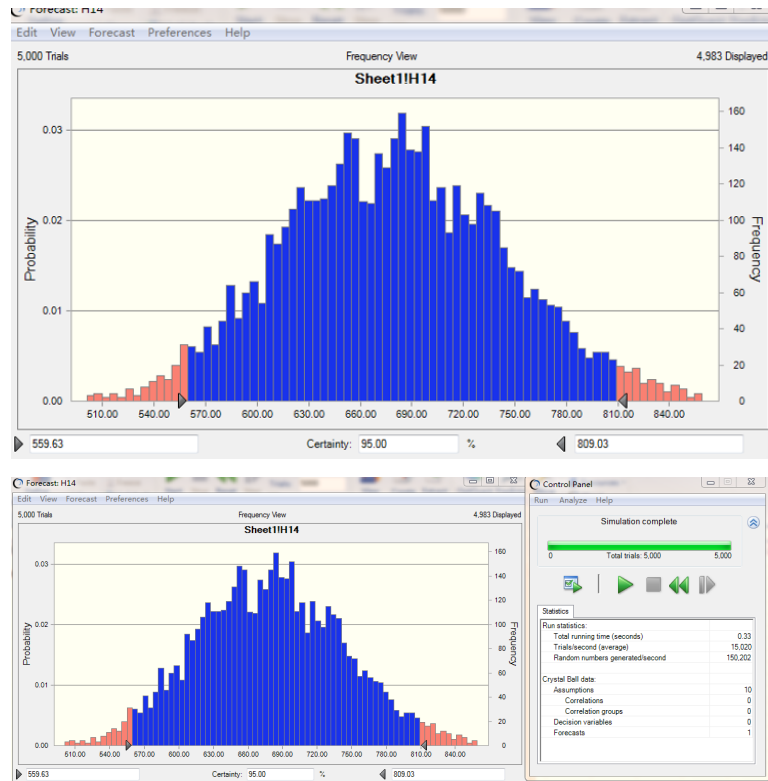
附件

蒙特卡洛 (Monte Carlo) 模拟

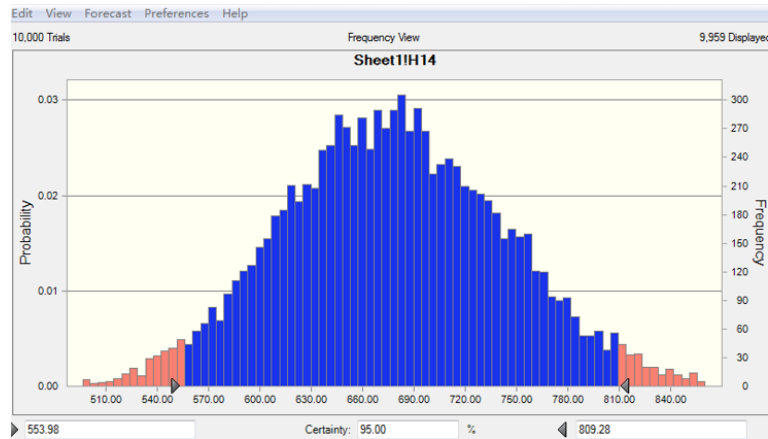
当结果不能用数学公式计算的时候（例如是三角形分布），可以用电脑随机模拟结果。例如：

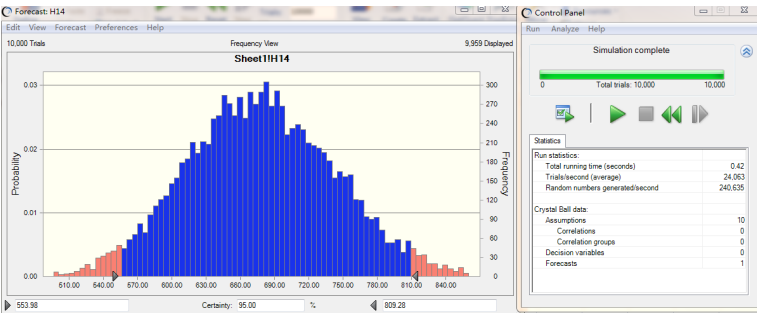
- 计算 3 个步骤的总共人天，每个步骤都是三角形分布，我们就用电脑的随机功能模拟，让随机功能的结果按三角形分布。
- 第一次模拟：步骤 1 得出 1.3，步骤 2 得出 1.2，步骤 3 得出 2.0，得出 3 个步骤的总工期是 4.5 人天。
- 第二次模拟：步骤 1 得出 1.5，步骤 2 得出 1.15
- 如果我们模拟 1000 次、10000 次，便能模拟出总分布。
- 因为是电脑随机模拟，出来的结果会有些偏差，但差异不会太大。（例如上面 10 步三角形分布的模拟结果偏差都没有低于 0.3

5000 次



10000 次





Chapter 5

量化管理从个人开始

我：你们管理层和客户都比较关心项目的进度，项目是否能按时完成？请问你们过去的项目如何？

开发：我们现在就是走敏捷开发，两周一个迭代。每次迭代前，我们聚一起开会，把所有用户故事按优先级排序，估计这个迭代可以完成哪些任务？然后放在看板的待办事项里。每天我们会做站立会议，监控实际的进展。

我：你们都能按计划冲刺迭代里把所有任务开发好吗？

开发想了一下，说：我们也有延误出现，有些模块不能在计划的冲刺里完成。

我：为什么？

开发：我们花了很多时间修正系统测试暴露的问题，有一些是因为前面没有把需求分析透导致的返工。

我：请问你们是怎么估计每个任务应该花多少时间？

开发：我们会一起讨论，然后用敏捷的扑克牌方式，集思广益来估计。

我：除了你们每天站立会议监控项目的进展，你们要不要统计一下项目延期的水平是多少？

开发：其实我们也不算是延误，有些模块不能在本迭代完成，就放在下一个迭代去做。

我：软件开发有一个系数叫每人的生产率，就是开发人员一天能产出多少有效代码？你们有统计吗？

开发：你说的是否是敏捷开发燃烧图里的速度？我们有画，但发现变化波动很大，所以后面我们也没有再用。

如何降低项目进度偏差

从以上对话可以看到，敏捷开发不一定能帮助团队按期交付，开发人员也不清楚自己的生产率是多少。要减少项目的延误，首先取决于估算是否准确。但如果只是依赖个人经验、头脑风暴去估计，很可能低估，因为可能没有考虑开发以外的一些因素，比如缺陷问题、需求问题等。在同一个冲刺里，不能交付所有计划的模块，等同于延误。要做好估算，就需要有数据。数据需要从个人收集的历史数据作为参考。IT 公司是否有注意这方面？我们可以从他们的新员工入职

培训探索一下。

我：请问你们如何培训新入职的开发人员？

培训师：我们会先上一些基础的理论课，培训公司的代码规范、框架和复用的模块。然后进行个人或小组练习，我们会简化一些简单的开发项目里面的某个模块，让他们照着做个小项目，然后我们就会对他们的成果进行反馈，让他们知道自己的开发水平如何，是否掌握了我们所培训的内容。

我：他们会收到什么反馈？

培训师（想了一下）：依据他们是否掌握了培训内容的重点来打分。

我：怎样打分？

培训师：我们依据评判标准打分，分数从 1 到 5，1 最低，5 最高。

我：评判的标准可以说说吗？

培训师（想了一下）：依据他们是否掌握了培训内容的重点打分。

我：你们公司不是已经开始推行量化项目管理，是否应该也配合量化管理，不仅仅靠老师主观判断来打分？

培训师：可否举些例子？

我：例如他们质量方面的缺陷数，如果有做单元测试或者评审相关阶段的缺陷排除，项目管理相关所花费的工时，从而可以反算出编码的效率等客观的量化指标。

从量化培训开始

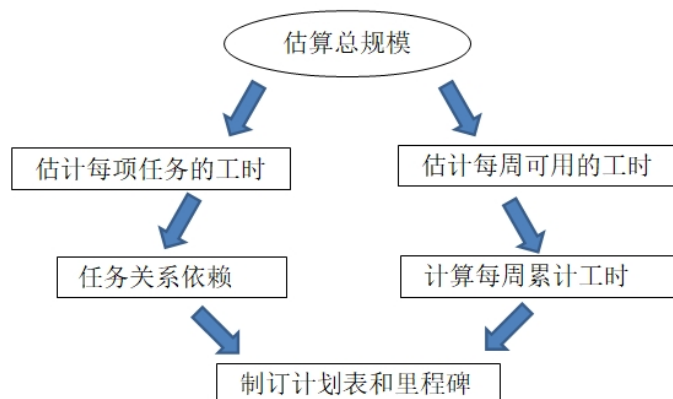
从以上对话可以看出很多公司都没有与量化软件开发项目管理相关的培训，这样如何能希望他们在项目中做好量化管理？入职不仅需要培训开发的技巧、怎么写好程序、做好面向对象设计，也应该学到要一直度量自己的开发过程，包括所花的工时、过程中发现的缺陷数等。

培训师会问：在新员工培训时，如何可以加入这些量化的元素呢？

很简单。首先，在先培训质量相关的技术指标，比如什么叫质量成本、什么叫排除率等，然后做实战练习时，要求他们除了写程序，还需要记录一下所花的工时，自己项目检查时发现的缺陷数、测试的缺陷数、评审所花的时间等。也要求开发人员除了提交程序以外，提交一个开发计划报告，内容包括实际所花工时、本来预估的工时、过程里发现的缺陷数、在哪一类过程等。帮助他们一进入公司就培养一些量化度量的习惯，以避免过了试用期，开发的习惯已经养成，后面是很难改变的。

从估算、策划到监控

挣值分析 (Earned Value, EV) 是常用的项目监控技巧，如果简单用百分比监控项目的进度或工作量偏差，因缺乏明确定义，难以比较，挣值分析把计划、实际等都统一用成本来计算，没有项目管理工具也可以简单手工计算（EV 公式与示例，详见附件）。Humphrey 先生出版了不少书，他也用 EV 分析来监控进度，他在 PSP 书中以此为实例，介绍如何用挣值分析策划与监控。我也试用挣值法来监控写这本书的进度（详见附件案例）。下图是 PSP 估算与监控的流程：



可以把敏捷的燃烧图方法看成挣值分析法的简化版，挣值分析也可以按现在的速度来预计完成的时间（燃烧图假定每一轮迭代的速度不变）。

传统大型项目每个时间段投入的工作量会有变化，挣值分析法可以按当前的进展和投入预计完成时的总成本。

想了解如何使用高中低三点估算和 PERT 方程式估算总工作量与范围，详见附件里的案例。

很多软件开发项目延误的原因是以前没有收集历史数据，导致估计所需人时都很理想。我也有同样问题：从我的挣值分析实例看到，因我是第一次写书，之前也没有养成统计章节所花的实际工时的习惯，所以在 11 月初做估算时，理想地认为最多花一个月完成，但过了 3 周就发现，实际比本来计划延误很多，原因包括：

- 实际可用工时没有想象这么理想，有很多意外事情都要处理。
- 低估了完成一章节所需时间：
 - 例如第一个章节 TDD 实际所花的工时与估计差不多，原因是这个章节是现有的，只是做了一些调整，所花时间不多。
 - 但到第二个章节 MGR 就不一样了，全部内容都重新写；如何从实际客户现场的场景总结，与理论结合也很花心思。

所以下次当你遇到一些团队项目延误，可以问项目经理以下问题：

1. 如何估算项目工期
2. 如何估算每一项任务所需的工时，依据是什么？
3. 如何收集实际工作量？
4. 如何监控累计到现在的完成情况？
5. 有没有统计以往的历史数据？如何用来帮助做好估算？

你可能会发现类似我用挣值分析估算和监控暴露出的问题。

应如何改善

软件开发和工业生产不同，工作量数据必须靠个人自己收集，量化管理依赖个人习惯，所以 Humphrey 先生在 90 年代推出了 PSP 个体软件过程，教导开发人员如何一步步自己收集数据、自己估算、自己监控。例如记录花了多少工时、发现多少缺陷、完成多少代码（从最基础开始，逐步改善），尽量减少缺陷返工，提升开发效率。（详见附件 PSP 简介。）

也正因为大部分的团队都没有收集数据的良好习惯，没有 PSP 的基础，所以就很难要求他们在冲刺回顾时拿出数据，分析根因、在下一轮迭代改善，从定性提升到量化管理。当开发人员养成好习惯，就会更清楚知道自己的速度和质量水平，不会出现本章开头的场景，开发人员对自己的质量水平或生产率一无所知。

有些企业深知量化管理对企业发展很重要，所以在新员工培训时，不仅仅教他们技术技能，也教他开始自己每天记录工时、缺陷数等习惯。因为工作习惯养成后，便难以改变。反之，如果周边的人都有记录数据，自己也觉得应做记录，逐渐形成公司文化。

问答 Q&A

资深敏捷顾问：我大约 20 年前对这个东西比较熟，PSP 包含 2 个重点：

1. 关于个体的估算内容，这个我看您的文章里边表达得比较充分。
2. 关于缺陷记录分类统计和根因分析的内容，这一块实际上相当于是 CMMI 里需要个体配合的地方。我看您几乎没有提到。

我：非常赞同，若要做到本手册后面提到基于缺陷数据的量化根因分析并改进，就要依赖个人记录缺陷与返工工作量，碍于篇幅有限，这里先用人时的估算与监控带出 PSP 的思路。

顾问：我自己也是在某军工航空项目里边用过一次。我们当年计划少、跟踪多，大家本来就缺少生产率的概念，所以其实也不知道每天能干什么事，但只要记下每天干了什么事，然后比对这个基数做就可以了，无论快或慢，都需要看看到底是什么原因。

我：您觉得这种方法有用吗？

顾问：因为开发依赖人的创造力，写新程序其实很难准确估计所需时间，但我后来发现如果估算是由组长负责去做，就能起很大的作用。例如有一次，我看有一个程序员花了一个月时间用 Java 写了某模块，接近 1 万行代码。虽然我不熟悉 Java 语言，但觉得代码有点问题。后面我就找另外一个比较资深的组长，我们 3 个人一起看代码。评审并删除无用的代码，最后剩下不到 100 行代码就可以实现了。如果当初我们先做好这模块的估计，就可以避免一个月人时的浪费。所以我建议还是需要估算，但是应该由有经验的技术组长负责。

我：很好，你很熟悉敏捷，例如 SCRUM 里用扑克牌方法估算，你觉得怎么样？

顾问：我先跟你讲个故事。我曾参加某冲刺估算会议，他们用扑克牌方法估算“数据显示”故事点，其中一位估计是 8，另外一位估计是 40。为什么会差这么多呢？第一位解释只需要展示数据，确实该工作量不大。但第二位理解就不一样了，包括整个数据的分析、输入和展示等，整个过程工作量确实很大。从这里可以看出，如果没有统一理解需求，就难以估算。所以那次以后，需求分析应分成“实体”和“行为”，清晰地写出功能需求，减少误会。（这是功能点估算规模的概念。）

我：赞同。Humphrey 先生在 PSP 书里强调，要参考历史数据来做好估算，其实也可适用于敏捷估算。不应靠人的主观判断。

顾问：是的，所以我前面强调必须要由经验的主管去负责估算。估算很重要，但是要程序员自己去估很难。所以 PSP20 年前的思路，还能适用于现在的敏捷开发团队，帮助他们做好估算。

PSP 里强调要评审设计、代码并分析缺陷，当时因为只靠人手，所以会很耗时。现在，我们有类似 SONAR 的静态扫描工具，可以像机器人一样，做本来耗时的代码评审工作。但 SONAR 只能针对一些基本语句问题，针对整个 OO 设计，我们会建议用我们的工具去扫描，补充 SONAR 扫描的不足。跟 PSP 代码评审的思路一样，所有扫描出的问题都必须修正。有些团队觉得问题太多了，只处理掉那些重大的问题，剩下一些可能不会直接影响到功能的问题暂时不处理。我不赞同这思路，这么多的问题其实都是累积出来的，如果从一开始都一直有定期处理清空，就不应该累积大量问题，而且这些问题遗漏下来还是对程序有隐患。

我：很赞同，归根到底还是开发人员缺乏保证质量的意识。最近有个团队也是用工具扫描代码，然后我问他为什么找出的部分问题不处理。他说例如定义某个变量，但是没有用，虽然被扫出是问题，觉得不影响程序的运行就不处理。我解释说，这样就好像放了一个地雷，后面还是会爆炸的，所以还是应该要处理。

PSP 要求评审代码，分析每个发现的缺陷，讨论原因，然后改进。有些管理者听过 PSP，但认为 PSP 成本太高，只适用于高价值、高质量要求的项目。如果他们理解 PSP 能最终帮团队把缺陷返工减半，能降低开发成本 20%，便不会再觉得 PSP 不适用于公司。（后面会有相关例子。）

估算很重要，要做好，不仅仅要参考历史数据，也需要对需求有共同的理解。功能点方法是估算软件规模的一种标准，下一章会用实例介绍。

附件

挣值分析法 (Earned Value)

用最简单的例子，来说明挣值分析中 PV、EV、与 AC 的意义。

PV：计划完成多少？

AC：完成工作的实际成本是多少？

EV：实际完成了多少工作？

公式：

进度偏差 $SV = EV - PV$ ， $SPI = EV / PV$ 完成占计划完成的百分比

成本偏差 $CV = EV - AC$ ， $CPI = EV / AC$ 完成的价值占实际已花成本的百分比

本文实例主要关心进度偏差，所以没有计算 AC

以下面“使用挣值分析监控案例”为例：

截止到 22.11.20，PV 累加本应是 27.5，EV 只有 3.1

所以进度偏差 $SV = EV - PV = 3.1 - 27.5 = -24.4$ 工时

$SPI = 3.1 / 27.5 = 11.27\%$

用 PERT 公式估算头三周任务累积工时的 95% 区间（工时）：

不用 MonteCarlo 模拟，直接把 12(=1+4+7) 章节任务的 ExpectedValue 加起来

计算 12 步总方差：(方差 = σ^2)

总方差 = 每步方差的总和

总方差 = 1.69 ($\sigma =$ 总方差的平方 (Sq.Root))

EXCEL 公式为：SQRT(SUM(所有方差范围)) = SQRT(SUM(H3:H15))

95% 范围下限 (=55.8)

EXCEL 公式为：SUM(所有 Expected 范围)-2× 总方差 = SUM(F3:F15)-2×I15

95% 范围上限 (=62.5)

EXCEL 公式为：SUM(所有 Expected 范围)+2× 总方差 = SUM(F3:F15)+2×I15

	A	B	C	D	E	F	G	H	I	J	K
1										95%区间(工时)	
2		章节	最佳	最可能	最差	Expected	Sigma	Sigma^2		下限	上限
3	22.11.7	TDD	2	2.5	3	2.50	0.17	0.027778	0		
4	22.11.14	MGR领导的任务	4	5	8	5.33	0.67	0.444444	0		
5		PSP	3	4	5	4.00	0.33	0.111111	0		
6		M6	4	5	6	5.00	0.33	0.111111	0		
7		M5	4	5	8	5.33	0.67	0.444444	1.07	20.0	24.3
8	22.11.21										
9		IC	4	5	6	5.00	0.33	0.111111			
10		CM	8	9	12	9.33	0.67	0.444444			
11		SiFP	3	4	5	4.00	0.33	0.111111			
12		INNOVATE创新	4	5	8	5.33	0.67				
13		MitOCW与Python	2	2.5	3	2.50	0.17	0.027778			
14		克服拖延症	2	2.5	3	2.50	0.17	0.027778			
15		CI	6	8	12	8.33	1.00	1	1.69	55.8	62.5

如何简单记录工时

如前面“克服拖延症”里提到，每人根据自己的习惯，采用每日 ToDoList 的方式，然后在开始前，简单记一下时间，结束时也记下时间。然后每日/每周统计每任务总工时：

日期	章节	时间	产出物	工时
11.25	PSP	8:15~10:10	capScreen	2hr
	SiFP	14:00~17:00	Q.A draft	3hr
	INNOV	20:15~21:00		1hr
11.26	PSP	9:00~11:00	PV~95%range(x/s) Screen.Caps	2hr
	SiFP			
	INNOV	12:30~17:30	INNOV... (1126) PROD... (1126)	5hr
	PROD	18:00~19:00	whs	

如只是个人，不一定需要项目管理工具；如果是团队，可把自己手工记录上传到项目管理工具，方便团队记录与监控。

个体软件过程 Personal Software Process (PSP) 简介

PSP 的简单介绍

- **PSP0** 基础 - 工时：计划与实际对比；每阶段引入多少缺陷；排除了多少缺陷
- **PSP0.1** 加入代码行统计 - 计划与实际对比；代码规范
- **PSP1** 加入使用 PROBE 方法做规模估算
- **PSP2** 加入设计与代码评审的计划与统计
- **PSP3** Cyclic process 先做策划与总体设计，然后多轮开发，有点类似迭代开发。

PSP 跟 CMMI 成熟度模型类似，也是按部就班一步步，帮助软件工程师利用度量改进自我的开发过程。

PSP 0.1

第一步 PSP 先估计写某个模块所需要的时间和实际花的时间，记录所有的缺陷，包括因为需求、设计或者实现引起的问题，记录修复时间，从开始发现问题直到缺陷被解决，提升到 PSP0.1 策划不仅仅是估计所需时间，加入了规模的概念，本来 PSP 是用代码行数来估计规模，也可以使用简化功能点方式估计规模大小。

在 PSP0.1 的阶段还没有用规模做策划、估算，只是记录实际的规模，里面包括基本规模，就是在开发之前软件系统本来有多大。也记录删除、改变、增加、复用的规模数等。最后总的规模数应该是等于基本、新增、删除、复用相加。

PSP1

下一步，PSP1 就有规模计划的概念，跟依据 0.1 的规模定义一样，我们在做开发之前，先估计刚才的所有规模数，并用回归方程估算对应的工作量或者进度，包括偏差的范围。

在 PSP0.1 增加了一个过程改进建议的环节，依据上一次迭代的数据，发掘下一轮可以完善的改进过程。也加入了编码标准，这个跟我们前面章节提到代码规范的概念相同。所以在 PSP0.1，除了计划，也需要有一个叫过程改进经验教训的部分，每一轮都应该有复盘回顾的概念，来提升个人的开发过程。

PSP1.1

PSP1.1 基于的 PSP1 的基础，加入挣值分析法去监控实际的项目进展，加入了 PV 和累加的 PV，计划多少和 EV 挣值和累计的挣值，反应实际完成多少。从那些系数也可以算出一个叫 CPI 成本性能指数，来反应完成与计划怎么比较，是否有延误，预计什么时候可以完成。详细可以看用挣值分析法估计写书完成时间的实例。

PSP2

到 PSP2 就开始增加评审的概念，包括设计评审、代码评审。这些同行评审是很有效的方式，避免缺陷等到出事才暴露、出现问题，因为如果只依赖测试暴露缺陷的话，只是告诉你这个程序跑不通，你还是要看代码才知道问题在哪里，怎么修正？但评审就直接看代码，首先它可以让很多缺陷在评审就发现，不用等到测试才暴露问题。评审也可以找出一些不一定测得出来的问题，例如有些银行对系统质量要求很高，核心的算法不能有任何错误，他们都会要求核心模块必须走专家评审，因为他们也知道很多核心算法的问题不能单靠测试来发现和处理。也是这个原因，很多公司也会要求代码必须走静态扫描，利用扫描机器人评审代码，预先发现一些明显代码违规的语句问题，弥补单靠人手去评审的不足。

有了基础，再加入缺陷密度的概念，即缺陷数除以模块的规模大小，本来 PSP 是使用代码行的，我们也可以用功能点取代代码行，因为单看缺陷数无法比较，无论个人或者整个团队或者团队之间，所以必须除以规模大小，才可以把系数归一，变成一个项目组之间，人之间可以比较的系数。也引入了排除率的概念，如果我们只是做了评审，但是都是 0 发现，你的缺陷排除率就是零了，一点效果都没有，所以排除率可以看成是当前评审发现的缺陷数除以当前系统内总共

引入的缺陷数，比率越高越好。也有阶段或者过程的排除率，就是这个阶段开始时总共有多少缺陷，然后在这个阶段里我们找出多少，作为分子的一个比例。还有一个就是缺陷排除的速度，按每小时评审的时间找出多少缺陷来判断评审中，可以找出缺陷的速度有多快，评审的效率有多高。

评审质量是 PSP2.1 的概念，2.1 也增加质量成本（COQ）概念，与前面敏捷回顾篇里强调软件开发缺陷发现越晚，返工量越会以几何式上升的思路一致。

PSP3

接近敏捷迭代的思路，把程序分成不超过几百行的模块，然后把模块按优先级分到不同的迭代，但还需要有需求与设计。敏捷增加了精益的思路，因为需求可能会有变，所以可以暂时不做后面迭代的估算，只先针对当前的迭代去做策划、估算。敏捷里面的迭代回顾跟 PSP 的回顾如出一辙。

大家可以参考 PSP 书里面的教材和练习，比如在书中要求学员自己编写程序，自己记录时间。虽然现在开发语言统计方法跟 90 年代有差异，但原理还是共通的。如果对新员工难以安排学校式的一周课程，也可以考虑用自学的形式，按要求去记录学习、写报告。

学 PSP，培养好习惯所以编程人员可以用 PSP 的原则，要他们记录。因为现在是写程序，不是玩游戏，所以更贴近实际。也要求后面的程序，让他分析自己的返工成本、质量成本、缺陷排除等，让后面项目团队回顾时有真实数据做分析。

例如在新员工培训时，练习中要求他们有编码规范的概念，按照公司的规范参考来写程序，在每一次做开发之前，都先简单计划一下时间和预估缺陷数。开发完后，记录实际的时间与缺陷数，因为虽然不仅仅是写一个程序，下一次开发时，可以参考以往练习的数据来更好估计，会看到自己的估计越来越准。

使用挣值分析监控案例

今天 11 月 8 日，离年底要交付全部初稿的时间不到 2 个月，书的内容也完成了超过一半，但剩下的时间因为年底密密麻麻很多评估，而且评估就是早上 9 点开始到下午 5、6 点结束，只有中间一些空档时间和周末可以自由安排，所以我就用同样的思路来策划估算是否能在年底完成要求。第一步算出现在到年底可以用在写书的时间，按每周估计：

	计划工时	累积工时
	Plan Hrs	Cum. Hrs
22.11.7	4	4
22.11.14	24	28
22.11.21	38	66
22.11.28	16	82
22.12.5	21	103
22.12.12	38	141
22.12.19	26	167
22.12.26	21	188
23.1.2	38	226
23.1.9	14	240

第二步从历史的数据估算剩下的章节，每个章节需要多少小时，写出计划的完成顺序，然后依据原计划可以使用的时间，按最佳使用算出每个章节可以在哪周完成，写在表格的右面空白处，从而估算何时可以完成所有章节，如果确实不能在 12 月底完成，就可能要重新调整、策划，是否有一些章节优先级比较低，暂时不放在第一版。

因觉得单点估算很难定一个点数，便用三点估算法。每一行估计都有三点：最差、最佳、最可能，用 PERT 方程式估计预计 (Expected) 和标准差 (Sigma)。挣值分析法所有东西 – EV、PV、AC 都是用钱来结算（这个例子我们就是用所花工时），例如我们估计了每个章节所需要的工时，利用三点估算计算出每个章节预计的工时，加起来得出预计总工时是 80.33 工时（用 PERT 公式计算）。例如第一个章节 TDD，预期工时是 2.5，除以 80.33 就得出 0.031，我们就用 3.1 作为 TDD 的 PV（所有任务完成的总 PV 大概是 100）。同样方式算出 MGR

章节的 PV 是 6.6。得出每一个章节对应的 PV 后，我们就可以计算累计的 PV 数：

Task 章节	最佳	最可能	最差	Expected	Sigma		PV	Cum. PV
TDD	2	2.5	3	2.50	0.17	0.03	3.1	3.1
MGR领导的任务	4	5	8	5.33	0.67	0.07	6.6	9.7
PSP	3	4	5	4.00	0.33	0.05	5	14.7
M6	4	5	6	5.00	0.33	0.06	6.2	20.9
M5	4	5	8	5.33	0.67	0.07	6.6	27.5
IC	4	5	6	5.00	0.33	0.06	6.2	33.7
CM	8	9	12	9.33	0.67	0.12	11.6	45.3
SIFP upd	3	4	5	4.00	0.33	0.05	5	50.3
INNOVATE创新	4	5	8	5.33	0.67	0.07	6.6	56.9
MitOCW写Python	2	2.5	3	2.50	0.17	0.03	3.1	60
克服拖延症	2	2.5	3	2.50	0.17	0.03	3.1	63.1
CI	6	8	12	8.33	1.00	0.10	10.4	73.5
RDM	6	8	12	8.33	1.00	0.10	10.4	83.9
Opening开章	3	3.5	4	3.50	0.17	0.04	4.4	88.3
最终检查	8	9	12	9.33	0.67	0.12	12.6	100.9
								100.9
		70.5	Sum_Exp	80.3333333		0.9024896		
						Overall Sigma		

估计哪一周可以完成哪些章节，得出总体的进度计划。例如第三周累积有 66 工时，应可完成总共 12(=1+4+7) 个章节，因用 PERT 三点估算，累积工时 95% 范围是（55.8 ~ 62.5）。

	章节	95%区间 (工时)		计划工时 Plan Hrs	累计工时 Cum. Hrs	策划价值 PV	累计策划价值 Cum. PV	计划哪一周 Plan Wk
		下限	上限					
22.11.7	TDD			4	4	3.1	3.1	1
22.11.14	MGR领导的任务			24	28	6.6	9.7	2
	PSP					5	14.7	2
	M6					6.2	20.9	2
	M5	20.0	24.3			6.6	27.5	2
22.11.21	IC			38	66	6.2	33.7	3
	CM					11.6	45.3	3
	SIFP					5	50.3	3
	INNOVATE创新					6.6	56.9	3
	MitOCW写Python					3.1	60	3
	克服拖延症	48.1	53.6			3.1	63.1	3
	CI	55.8	62.5			10.4	73.5	3

监控

按实际完成的情况就可以填 EV 挣值，挣值的算法很简单，必须整个章节完成才算有挣值，不接受部分完成。比如第一周虽然 TDD 章节已完成九成，但实际是到第二周开始才完成，所以第一周的挣值为 0。到了第二周才把本来的第一章完成，所以里面才放上它的 PV(=3.1)。监控的好处，好比要准备半年后的马拉松，开始长距离慢跑（LSD）锻炼，再进一步做连续马拉松配速训练，每周 3 天，每次 5-15 公里，速度是多少？比如平均每公里花费 6.25 分钟。有了数字才知道现状与标准的差距，才有动力对自己说现在离比赛不到 12 周，必须加强锻炼才能赶上。

要让项目有更大的机会按期完成，便需要监控，这样可以预计自己离最后交付差距多少，如果按现在的进展要到哪一周才可以完成，都可以从数字预计出来。但是我们写书也靠灵感，跟写程序类似，可能顺序不一定能按本来的执行，怎么办？

其实道理也一样，你可以按新的顺序更新一下本来的计划，把实际的填上。如果你是用简单的 Excel 表，这个改变可能只花几分钟，很快就能调整好顺序。例如现在看到的第二章节，本来计划放在后面才写，但因刚好与客户接触后产生了灵感，便放在前面写了。

	章节	计划工时 Plan Hrs	累计工时 Cum. Hrs	策划价值 PV	累计策划价值 Cum. PV	计划哪一周 Plan Wk	实际工时 Actual Hr.	实际哪一周 Actual Wk.	挣值 EV	累计挣值 Cum. EV
22.11.7	TDD	4	4	3.1	3.1	1	2	2		
22.11.14	MGR领导的任务	24	28	6.6	9.7	2	7	2	3.1	3.1
	PSP			5	14.7	2	5.5			
	M6			6.2	20.9	2				
	M5			6.6	27.5	2				
22.11.21		38	66							
	IC			6.2	33.7	3				
	CM			11.6	45.3	3				
	SiFP			5	50.3	3				
	INNOVATE创新			6.6	56.9	3				
	MitOCW写Python			3.1	60	3				
	克服拖延症			3.1	63.1	3				
	CI			10.4	73.5	3				
22.11.28		16	82							
22.12.5		21	103							

要估算任务工时便要依赖以往类似活动的历史记录，如果不是每天记录，过后无法记住。每周依据个人的习惯，每天记录实际所花小时数，然后统计每周累计数。

参考 References

- 1.Humphrey, Watts S. "A Discipline for Software Engineering."
- 2.Humphrey, Watts S. "PSP: A Self-Improvement Process for Software Engineers."

Chapter 6

估算软件规模

为什么要估算规模

规模可以帮助我们：

1. 依据历史数据策划，例如估算工作量、工期
2. 归一 (Normalize) 不同项目作比较
3. 知道现在水平

依据历史数据策划先把项目分成组件，参考以往类似的组件所花工作量，估算整个项目的总工作量。规模大小可简单看成是组件的数量。如果是新开发，以前从未做过同类开发，就只能靠个人经验直接估算工作量或工期，但是如果以前做过类似的工作，就可以参考以前的历史数据估算。

规模可以帮我们把不同项目归一。例如验收测试缺陷数无法比较，但缺陷率（缺陷数/规模）便可以比较；生产率（规模/所花总工时）便可比。

有了归一后可比较的系数，个人/团队便更清楚当前的水平（质量或生产率）是否在上升或下降。

为什么不应用代码行数 (LOC)

在 1996 年前,IBM 一直使用代码行数估算规模。之前一直都使用近似机器语言的 Assembly Lang, 但为了提升效率，引入了高层语言 PL/S, 发现不能再代码行数来估算规模，因 PL/S 只需要更少代码行数也能完成同样功能：

	Assembly	PL/S
代码行数	17,500	5,000
工作量（月）	30	12.5
工作量（工时）	3,960	1,650
每月代码行数	583.33	400.00
等同的 Assembly 代码行数	17,500	17,500
等同的 Assembly 人月	583.33	1,400.00
功能点	100.00	100.00
功能点/人月	3.33	8.00

上表是 IBM（1968 ~ 1975）对两种编译器的统计：

每个月 PL/S 产出代码行数 (400) 反比 Assembly(583) 少，但是如果用功能点数便能真正反映生产率的提升：

PL/S:8.00 对比 Assembly:3.33

只适用于编码

代码行数只能反应编程的工作量，但编码仅占项目总工作量的一部分，如果把项目按工作量分成以下 5 个部分，代码行数只能用于第二部分编码 (25%)，其他 4 部分 (30% + 20% + 15% + 10%) 都不合适。

	活动	成本 (%)
1	发现与修正缺陷	30
2	编码	25
3	支持类文档	20
4	会议沟通	15
5	项目管理	10
	总计	100

与功能点比较

能估算好项目规模可帮助我们更好估算工作量，规模应具备以下条件：

1. 要和工作量密切相关
2. 要容易数得出来
3. 容易在项目早期可以估算到

功能点比较符合以上条件，只要功能需求明确，就可以估算出对应的功能点数。因功能点估算已经是国际标准，基于功能点的度量数据可以与其他国家的标杆对比。

怎样估算简化功能点 (SiFP)

有了功能性需求，并识别出系统范围，就可以开始估算功能点。

1. 数据功能（简称实体）的数量 $\times 7$ （实体是系统要管理的数据）
2. 业务功能（简称行为）的数量 $\times 4.6$ （行为可以简单看成是增删改查等功能）

简化功能点数 = 上面 1 和 2 的总数

例如，附件例子案例 A 新开发项目：

- 识别出 2 个实体和 6 个行为，得出简化功能点数：

$$\begin{aligned}2 \times 7 + 6 \times 4.6 \\&= 41.6\end{aligned}$$

- 案例 B 功能优化维护项目：增加 1 个实体和 3 个行为，使用功能优化与维护公式：

$$\begin{aligned}1 \times 7 + 3 \times 4.6 \quad (\text{增加 1 个实体和 3 个行为}) \text{ 因可能有些功能删除，有些功能变更，先} \\&= 20.8\end{aligned}$$

- 以上优化维护功能点数主要是用于估算本期开发项目的工作量，因为无论变更功能或者删除功能，都会导致有开发工作量，所以不能是零或负数。国际功能点的规定很简单，都加起来。

- 然后再用应用 (Application) 公式计算优化维护后系统的功能点数：

$$\begin{aligned}41.6 + 20.8 \\&= 62.4\end{aligned}$$

- 因案例 B 没有功能删除，也没有功能变更，优化后的功能点数是功能点数只需要首次开放功能点加上优化维护的功能点数 (41.6 + 20.8)，但如果案例 B 有功能变更，那些功能点便不能加到最后的系统功能点数。(如果有功能点删除，便需要减。)

从故事点转简化功能点

因故事点只是每个团队自己定义，无法用来作为组织级标准衡量规模，所以很多公司（尤其是银行）转用功能点来衡量软件开发规模。它不仅可用于公司内，也适用于行业标杆，功能点也适用于公司内或公司间结算，例如软件维护期开发工作量变化很大，也难以事前预估，所以有些银行会按最终开发出来的功能点数结算使用部门付开发部的费用，减少争议。

虽然功能点分析源自 70 年代，但由于计算较复杂，一直未普及。有些人觉得国际功能点协会 (IFPUG) 的功能点算法 (FPA) 太复杂。针对这问题，国际功能点协会简化本来的功能点算法，推出简化功能点 (SiFP)，减少了估算的工作量与学习难度 (例如培训可以从以往的 2 天半降到半天)。因简化功能点不考虑实体/行为的复杂度，它与本来功能点的估算有 $\pm 15\% \sim 20\%$ 偏差 (详见附件)。因敏捷团队是按每轮迭代估算 (而不是一次性估整个项目)，偏差就可以接受 (例如 2 周一个迭代，偏差大概 1.5 天)，所以 SiFP 能适用于敏捷多次迭代估算。所以越来越多团队开始从故事点转简化功能点，但由于不熟悉功能点估算是从用户角度估算，而非从开发工程师的视角，团队初次估算 SiFP 通常有误。下面是某团队从故事点转简化功能点的案例：

客户：我们以前一直使用故事点，为了更好做量化管理，我们新的项目开始使用简化功能点 SiFP。

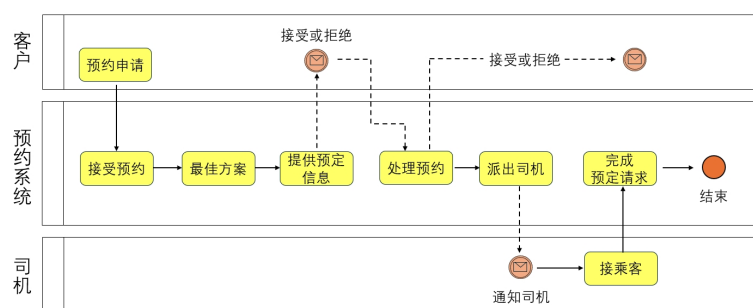
我：好的，看一下你的估算表，为什么这两个功能要分成两个行为？

客户：这两个功能都挺复杂，估计需要很多开发工作量。我们以前用故事点估算时，也会分成两个故事点。

我：请注意，在功能点估算是否是一个行为，取决于它算不算是基本过程。如果俩行为互相依赖，不能单独作为基本过程，就不应该分开为 2 个功能，不然便可能不断细化，无法得出公认的功能点数。很多团队刚开始用功能点，与你们一样，没有弄清楚基本过程的概念，还是从工程师的角度，估计开发时间的工作量来判断是一个功能还是两个功能？在故事点用这种方式估算习惯，但因为功能点是要同一套功能需求，不应有功能点数估算差异，所以不能用估计开发难易程度来判断。

客户：可否举个实例？

我：以我们日常的网约车为例，是否可以分成以下 8 个“行为”？



1. 预约申请
2. 接收预约
3. 选择最佳方案
4. 提供预约信息
5. 处理预约
6. 分派司机
7. 接乘客
8. 完成预约请求

客户：是的，每个行为都要花一定开发工作量。

我：其实按 SiFP 只有 4 个行为，因开头 4 个和最后 2 个都要合并才正确（详见附件），因如果要能独立成行为，必须符合基本过程条件。

基本过程不是随便定的，它有规则，比如从网约车案例我们可以看到，“预约申请”本身不能算一个基本过程，虽然预约申请可能要很复杂的开发工作，但是因为它不能独立存在，必须依赖其他的行为才算完整。

例如某公司财务系统有打印支票功能（用来付供应商），你觉得打印支票本身是否算基本过程？

客户：听完你网约车例子，应该不算。因打印支票只是付款流程的一部分。

我：正确，好比银行账户转账，更新账户余额和展示账户余额这 3 个用例不能单独，必须合起来才算完整。算实体也应使用同样概念。例如某人事管理系统，除了管理职工信息外，还有员工家属信息，因家属信息必须关连到员工信息，不能独立存在，所以家属信息本身不算是一个实体。你是否觉得你这表里的某些实体应合并？

客户：听完你的解释，我们这次计算里确实有些实体应合并。

但如果客户需要管理员工家属信息，我们在建数据库时确实要增加额外表格管理相关的字段，会增加不小工作量，所以如果和只需要管理员工信息，同样是算一个实体，同样的功能点数，是否不合理？

我：很理解你的疑问，也正是很多工程师功能点的困惑。你在之前不是说过在估算开发工作量时常常与客户谈不拢，客户常常觉得你们估算的工作量过大大，应不需要这么多。所以功能点估算的目的是希望从客户的角度，依据功能需求，按已定的规则，让任何人都能算出同样的功能点数（不仅依赖工程师估算）。所以有些工程师学功能点时，像你一样未能抛开软件开发工作量的想法，不觉得应只按功能点的规则来计算。

客户：理解你的意思，但你刚才这个例子确实有额外的工作量，是否会导致会低估了项目的工作量？

我：预算工作量除了考虑功能点也要看团队的历史生产率数据。如果以往项目里，很多实体都比较复杂，都包含几个相关的数据表，反映在项目的生产率历史数据里，作为新项目工作量估算的参考。

我：你们的功能点估算表没有明确区分新的迭代怎么处理变更与删除。

客户：我们表中有计算每次迭代的总功能点数。

我：你们可能有计算每迭代的功能点，但难以看清后面迭代对应之前的变化。优化与维护 (Enhancement) 功能点是用来估算本迭代的工作量，而应用 (Application) 功能点就是迭代后产品的功能点数。而且应该用表格形式把变更和删除累加在原本上一轮的功能上面，才可以更好看到迭代与迭代之间功能上的变化。

所以不要误以为可以像之前故事点估算，按个人理解写上各种不同复杂程度的故事点例子便可。国际功能点手册里包含很多计算实例，来解释功能点计算，才能确保对同一套功能需求，每人都能估算出同样的功能点数。我刚刚的例子全都可以从国际功能点手册里找到，你们不需要自己研究再‘发明’公式。学功能点估算与写程序类似，必须多动手试，例如按我发你们的 6 个案例，按案例的功能需求，计算 SiFP 功能点数。

（大家也可以看附件里有 2 个简单案例：

1. 首次开发：管理潜在客户（公司和联系人）信息
2. 优化功能与维护：首次开发后，添加帮助 (Help) 页功能，和添加报表

)

总结

虽然简化功能点 (SiFP) 比传统国际功能点 (IFPUG) 简单易学，但开发人员还是容易用工程师的视角来估算（本应用用户的视角），导致计算错误。所以要多看案例并做练习，才能把握（能参加培训会更好）。

附件

简化功能点 (SiFP) 简介

它是做什么的？

应用软件开发的客户需求可分成 3 类：

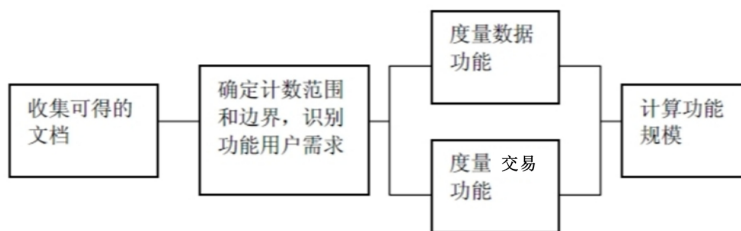
1. 功能性需求
2. 技术需求
3. 质量需求

第二类和第三类归为非功能性需求。功能点主要是针对功能性需求，目的是提供对客户有意义的功能点数，来客观地衡量软件规模。

该如何去做？

简化功能点 (SiFP) 主要分为两类度量：

1. 数据功能——实体 (逻辑文件 Logical File)
2. 交易功能——行为 (基本过程 Elementary Process)



简化功能点估算步骤：

1. 确定功能点分析类型
2. 识别分析范围和应用边界
3. 计算数据类型功能点
4. 计算交易类型功能点
5. 计算功能点

1 确定功能点分析类型

3 种计算类形：

1. 开发 (Development)
2. 优化功能与维护 (Functional Enhancement Maintenance FEM)
3. 应用 (Application)

开发 (Development) 功能点数包括首次应用开发的功能点，加上数据迁移或转换的功能点。

优化功能与维护 (FEM) 计算对现有应用程序的优化与维护，包括增加、删除和变更功能，也加上数据迁移或转换的功能点。

应用 (Application) 计算已经安装应用程序，例如优化与维护后的软件，提供给用户的功能点数，

2 识别分析范围和应用边界

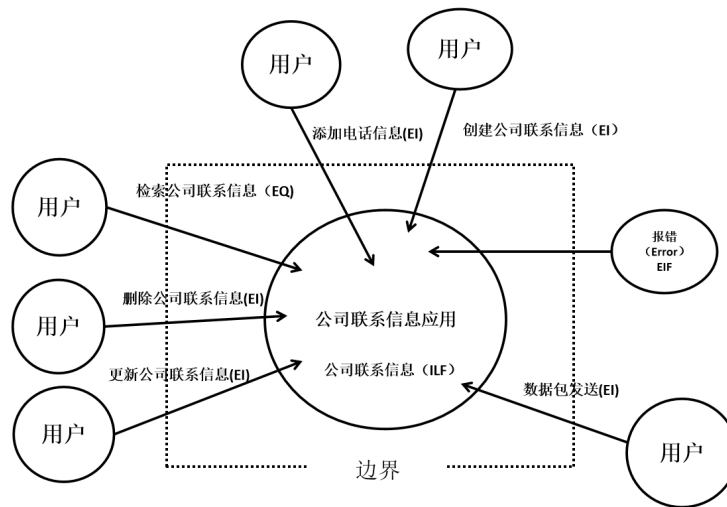
IFPUG 包括两类数据功能点 (因 SiFP 是基于 IFPUG 功能点)：

- 内部逻辑文件 (ILF)
- 外部接口文件 (EIF)

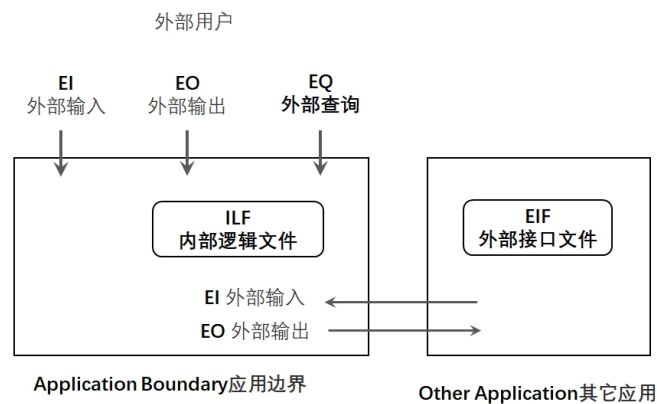
和 3 类交易类功能点：

- 外部输入 (EI)
- 外部输出 (EO)
- 外部查询 (EQ)

以下例子表示数据类和交易类功能点与应用软件系统（参考案例 A）的关系（虚线标示系统边界）：



== == ==



计数范围由计数目的决定。它标识将被划分大小的系统、应用程序或应用程序

的子集。它可以包括通过购买软件包来满足的功能；它可以包括所有将被外包的应用程序；它可以被限制为应用程序中执行特定目的的函数，例如报表。应用程序边界是被测量的应用程序与外部应用程序或用户域之间的边界。

IFPUG 制定了确定边界的具体规则：

- 边界是基于用户的观点用户应该能够用用户的语言定义应用程序的范围和业务功能
- 相关应用程序之间的界限是基于单独的业务功能，而不是基于技术考虑举例，微软办公软件目前包括 Word、Excel、PowerPoint 和 Access；每个都是 Microsoft office 套件中的独立应用程序
- 被增强的应用程序的应用程序边界保持原样当然，除了添加的功能可能会扩展边界，删除的功能可能会缩小边界。功能优化不会成为它自己的应用边界，而是按计数应用程序的领域定义（如微服务）。如开发项目和优化项目可能包括不止一个应用程序，功能点计数时应参考上图例子先画好边界，然后独立计算每个应用程序的功能点数

有关应用程序中制定 3 边界定义的混淆。

虽然功能点计算的一致性非常重要，但如果计算一致但不正确，功能点计算几乎没有什么好处。例如，在确定应用程序边界时。边界不应该在软件程序或软件模块级别建立。功能点似乎只在更高的应用程序级别上才相关。

- 但也可能在过高的层次上进行计数，例如，某会计系统可能包含几个独特的和独立的应用程序（例如，应收帐款、总账和应付帐款），它们可能具有内部关系。这些应用程序应该分别计算
- 反过来，某工厂生产系统可以由许多从属但独立的应用程序（例如，车间计划，物料库存，工作计划）组成，这些应用程序分别开发，维护和使用，并且可能具有内部关系

3 计算数据类型功能点

- 先计算逻辑文件数
- 关于计算规则，详见“逻辑文件”
- 按对应公式计算功能点数：(每个逻辑文件 = 7.0 简化功能点)

4 计算交易类型功能点

- 计算基本过程数
- 关于计算规则，详见“基本过程”
- 按对应公式计算功能点数：(每个逻辑文件 = 7.0 简化功能点；每个基本过程 = 4.6 简化功能点)

5 计算功能点

- 功能点数 = 数据类型功能点数 + 交易类型功能点数

Development Project Function Point Count 开发项目功能点计数

开发项目功能点计数由两个功能点数组成：

1. 应用程序的功能点数，由 EI、EO、EQ、ILF 和 EIF 组成
2. 转换功能，通过软件将以前的数据转移到新的 ILF 中（该组件通常包括旧数据文件的输入作为 EI 计数或将数据输入到已计数的新 ILF 中，可能还包括转换报告的 EO）

$$DFP = UFP + CFP$$

- DFP 是开发项目功能点计数。
- UFP 为功能点计数。
- CFP 是数据转换所包含的功能点的数目。（注 1）

Enhancement Project Function Point Count 优化维护项目功能点计数

功能点计数也由两个功能数组成，但它们有些不同：

1. 应用程序未调整的功能点计数，由 EL、EO、EQ、ILF 和 ELF 组成，它们是：
 - * 由增强项目增加（以前不存在的功能，例如：一个新的 EQ，三个新的 EL，一个新的 ILE，一个新的 EQ 和一个新的 EO）

- 由增强项目更改（以前存在的功能，但现在有不同的字段，FTR 或需要不同的处理）
- 由增强项目删除（已从应用程序中删除的功能，例如：已删除的报告）

2. 转换功能，通过软件将以前的数据转换为新的 ILF（该组件通常包括旧数据文件的输入 [作为 EL 计数或将数据输入到已计数的 ILF 中]，可能还包括转换报告的 EO）

增强项目功能点计算公式如下：

$$EFP = (ADD + CHGA + CFP) + DEL$$

- EFP 是增强项目功能点计数
- ADD 是增强项目增加的功能的未调整功能点数
- CHGA 是经增强项目修改的功能的未调整的功能点计数
- CFP 是数据转换所包含的功能点的数目（注 1）
- DEL 是那些在增强项目中被删除的功能的未调整功能点计数

（注 1：这简介不包含数据转换的计算与实例。）

Application Function Point Count 应用功能点计数

应用程序功能点计数由一个功能组件组成（转换工作功能不包括在应用程序计数中，因为它是开发工作的一部分，而不是应用程序一旦建立）：

1. 应用程序的功能点计数，包括 Els, EOs EQs, ILFs, EIFs

有两种不同的时间段估算应用程序的功能点数：

1. 当应用程序最初交付
2. 当增强项目改变了应用程序的功能值时（当安装增强项目时，必须更新以前的应用程序功能点计数，以识别对应用程序的修改）。增强项目可以修改应用程序

下面是两种不同的计算公式：

Initial Application Function Point Calculation 初始应用功能点计算

$AFP = ADD$

- AFP 是初始应用程序功能点计数。
- ADD 是开发项目所安装功能的未调整功能点数。

Application Function Point Calculation after an Enhancement 优化与维护后的应用功能点计算

- 增加（新）功能，增加应用程序的功能点大小
- 变更，通过增加或减少本来开发功能点，但不影响应用程序的功能点数，只影响了优化维护功能点数
- 删除功能，会减少应用程序的功能点，但反而会增加优化维护 (Enhancement) 功能点。

$AFP = (UFPB + ADD + CHGA) - (CHGB + DEL)$

- AFP 是应用程序调整后的功能点计数
- UFPB 是增强项目前应用程序的功能点数
- ADD 是增强项目增加的功能的未调整功能点数
- CHGA 是经增强项目更改的功能的未调整功能点数（此部分反映更改后的功能点值）
- CHGB 是改进前未调整的功能点数（此分量反映改进前的功能点值）
- DEL 是那些被增强项目删除的功能的未调整功能点计数

逻辑文件 (Logical File)

下面在计算实例里简称“实体”以方便理解

- 逻辑文件满足内部和外部数据存储要求的功能。它是用户可识别的逻辑相关数据组或控制信息组，
- 2 类逻辑文件：
 - 内部逻辑文件 (ILF)：在度量应用程序的边界内维护和/或引用。是在被度量应用边界内部维护的
 - 外部接口文件 (EIF)：被度量应用程序所引用，但在另一应用边界内维护

术语定义：

- 用户可识别 (**User Identifiable**)：对过程和/或数据组的需求，得到用户和软件开发人员双方的认同和理解，并达成一致。例如，财务应用程序可能需要一个支票账户记录
- 逻辑相关 (**Logically related**)：要求每一组与相关描述逻辑上一致。逻辑文件不应为了维持其存在而依赖或归因于另一个逻辑文件，这些相关逻辑文件应合并（特别是因业绩或执行原因而建立的组）。例如，地

址表可能属于更高级别, 如客户文件、账单文件、库存位置文件或员工文件

- **数据 (Data)**: 应用程序中维护的描述和/或数据组。例如, 每张支票的支票号码、金额、日期、收款人、备忘录记录和账户号码都可能在支票账户记录中维护
- **控制信息 (Control information)**: 应用程序里影响基本过程的数据。它定义处理那些数据 (what)、何时处理数据 (when) 或如何处理数据 (how)。对于 ILF, 这些数据、规则或参数在应用程序中存储和维护。举例, 在恒温器上设置温度以控制加热或空调的时序
- **维护 (Maintained)**: 通过应用程序的基本过程修改数据。数据或控制数据可以通过诸如添加、更改、删除等。逻辑文件可以被多个应用程序维护, 但在每个应用程序只能计算为一个 ILF
- **基本过程 (Elementary process)**: 具有意义的最小活动单位。它可以更新多个逻辑文件

注意:

逻辑文件包括两类不同的用户需求数据:

1. 功能性数据
2. 非功能性数据

功能性数据是用来满足用户功能需求的数据。例如销售、银行账号、供应商、人员等信息。

非功能性数据主要是为了满足易用性 (如支撑下拉菜单所需的数据, 可输入数据的上下范围等), 或性能 (如用于查询数据的索引), 或可维护性 (配置参数)。

只有第一类功能性数据才算是逻辑文件。

功能点估算只包括功能性需求, 国际功能点 (IFPUG) 有 SNAP 计数手册, 专门针对非功能需求。

基本过程 (Elementary Process EP)

基本过程是对用户有意义的最小活动单元。例如: 添加员工的用户需求包括建立工资和家属信息。只有添加所有员工信息, 才能创建员工信息记录。单独添加一些信息使添加员工业务处于不持续状态, 只有员工工资和家属信息都添加后, 这个活动单元才能完成且业务处于稳定状态。

识别基本过程

为了识别基本过程, 需要执行以下活动, 把功能用户需求分解为最小活动单元, 使其满足下面条件:

- 对用户有意义
例如功能用户需求要求在教育中添加新员工的能力

- 构成一个完整的事务
例如: 用户定义的员工信息包括工资和家属信息。如果家属人数大于零, 添加员工信息时必须包括家属信息。本例中, 添加员工信息 (不包括添加地址、工资和家属信息) 不满足本规则
- 自包含
例如: 除非输入所有的必需信息并且完成所有处理步骤, 如验证、计算、更新 ILFs, 添加过程才是自包含的
- 让应用程序的业务保持持续状态
例如添加员工的用户需求包括建立工资和家属信息。只有添加所有员工信息, 才能创建员工信息记录。单独添加一些信息使添加员工业务处于不持续状态, 只有员工工资和家属信息都添加后, 这个活动单元才能完成且业务处于持续状态

识别活动单元为基本过程需要满足以上所有规则。

识别基本过程主要目的

基本过程的主要目的可识别为下列情形的一种:

- 改变应用行为
- 维护一个或多个 ILFs
- 呈现信息给用户

实例：识别基本过程 (EP)

下面是预约网约车过程：

活动名称	描述
预约申请	客户发出预约请求到预约系统，预约信息包括：日期、时间、上车的位置和目的地位置。
接收预约	预约系统收集客户预约请求，并把预约数据记录在数据库。
检查是否有可用的车/司机	预约系统从预约数据库中查看是否有合适的车/司机，如果能找到合适的时间、日期，能配合预约请求的话，看看司机是否有空档。如能找到，在系统更新内部状态为是，否则为否。
寻找其他可选	如果状态是没有，预约系统继续在数据库搜索有没有接近的预期时间，否则写没有合适请求的出发地点和目的地。
提供预约信息	预约系统自动发出通知到客户，是否有合适空档或者提供接近的日期时间。
处理预约	客户回复预约系统接受或拒绝，预约系统把反馈记录在预约数据库。如果反馈是接受，预约系统会继续统计客户详细预约信息。如果客户拒绝，预约系统就会回复收到，并终止过程。
分派司机	如果客户接受，预约系统会指派司机到已确认的日期、时间、上车地点和目的地。预约系统把这个记录在数据库中，并发送信息通知相关司机。
接乘客	司机按约好的日期时间、上车的地点接乘客，然后发信息到预约系统，通知乘客已经上车。
完成预约请求	预约系统在数据库中记录已经完成整个过程。

分析能否满足所有 EP 识别规则，判断能否独立成为基本过程 (EP)，部分例子

预约申请

1. 是否对用户有意义，是客户功能需求的一部分。是
2. 是否构成一个完整的事务？否：预约申请本身不是一个完整的交易，因为过程必须也包括预约请求信息，收到其他可选的档期这些步骤，都不可以分离
3. 是否自包含，可以独立存在？否：例如接受预约申请，查看是否有档期，查看有没有其他接近的档期等，都是一些必需的相关步骤去完成这个基本过程
4. 是否让应用程序达到稳定状态？否：整个业务需求只能在收到预约信息，发送、接受、处理、反馈给客户才算是完成稳定状态

分派司机

- 1. 是否对用户有意义，是客户功能需求的一部分。是
- 2. 是否构成一个完整的事务？是：分配到司机是一个完整的交易，包括收到司机的确认，把信息记录在系统中并通知司机
- 3. 是否自包含，可以独立存在？是：分配到司机本身可以独立存在
- 4. 是否让应用程序达到稳定状态？是：因为当司机被分配后，是完全满足业务的需要

接乘客

- 1. 是否是客户功能需求？是
- 2. 交易是否完整？否：接乘客本身不算一个完整的交易，因为预约系统必须也记录这个信息
- 3. 是否自包含，可以独立存在？否：确认预约申请是下面一个必须执行的过程，来完成这个基本过程
- 4. 是否让应用程序达到稳定状态？否：整个业务需求只能在预约系统确认沟通，已经接到乘客，然后系统也把记录更新到预约系统才算完成

业务过程/活动	基本过程 (EP) 必须满足所有EP识别规则
• 预约申请 • 接收预约 • 检查是否有可用的车/司机 • 寻找其他可选 • 提供预约信息	如果单独来看每个活动，不能满足基本过程的条件。它们必须要结合在一起，才能满足所有基本过程的条件。基本过程包括左面所有过程/活动。
• 处理预约	这业务流程、活动满足所有基本过程的要求，可以当成一个基本过程。
• 分派司机	这业务流程、活动满足所有基本过程的要求，可以当成一个基本过程。
• 接乘客 • 完成预约请求	如果单独来看每个活动，不能满足基本过程的条件。它们必须要结合在一起，才能满足所有基本过程的条件。基本过程包括左面所有过程/活动。

案例 A: 开发 (Development) 项目

公司计划构建应用程序来维护对其功能点课程感兴趣的公司信息。

要维护的公司联系信息，按逻辑分组，包括以下数据：

- 公司
- 联系人姓名
- 职位
- 初次联系日期
- 街道地址
- 城市
- 状态
- 邮政编码
- 电话号码
- 传真号码

该数据最初将在收到查询表示对课程感兴趣时创建。员工将能够通过屏幕，使用以下命令创建、更新和删除信息。创建和更新将维护以上所有的 10 个字段。删除只需要输入公司和联系人的姓名。

下面是公司联系数据中包含的其他字段，但要用另外单独更新：

- 发送材料日期
- 电话联系日期
- 笔记、备注

这些字段将由两个独立的基本流程维护：

- (1) 发送信息包时，发送信息包的个人将使用一个单独的屏幕，使用功能键输入公司、联系人姓名和发送数据包的日期。
- (2) 邮件寄出后两周内，回电确认收到并答复问题。合同完成后，来电者将使用屏幕，使用功能键输入联系人的公司名称、电话联系日期和备注。电话联系人的日期将用作辅助键（第二种记录类型），以保存多次出现的信息并更新公司联系人数据。

由菜单驱动的用户界面导航，提供以下六个功能：

- 创建公司联系人
- 检索公司联系
- 更新公司联系
- 删除公司联系
- 已发送的材料包
- 已完成的电话联络

除了检索公司联系之外，其他所有功能都已经描述。输入公司、联系人姓名和功能键后，检索功能便显示公司联系数据中维护的所有字段。

任何功能都可以从外部维护的错误（Error）文件返回错误信息，该文件有四个字段，其中一个字段包含具体错误信息。

答案与解读

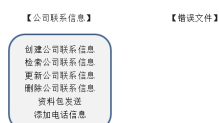
共个实体：

1. 公司联系信息
2. 报错文件

你可能会问：那些电话信息是否也应该是一个实体？（因需要花工夫开发）

这不应该是一个实体。因为电话信息必须依赖公司联系信息挂在一起，不可以独立存在，就好比我们要电话信息，假如也要维公司联系信息，这个电话信息就不能算另外一个实体，因为没有公司联系信息，电话信息是不能单独存在的。区分原则不是根据是否要产生开发的工作量，而是从用户角度看，这个实体能否独立存在和维护。否则功能点的估算就只是根据个人对开发工作量的估计，而不是从用户角度看功能的客观判断。

创建、检索、更新、删除等 4 个功能和管理发送资料包和电话联系 2 功能，共 6 行为，加 2 实体。为方便理解，建议先画下图，把行为与实体关联：



[illegible]

应用程序交付后，Caren Garmus 希望利用已经由外部应用程序维护的帮助子系统来提供两类帮助页面：

- 每个屏幕上都可用这两个级别的帮助功能，并且将从单独维护的外部帮助文件访问帮助文本。

Mary Herron 说她想要一份每周的逾期联系报告（详见下表），识别出：

- 该报告将每周从公司联系数据文件中检索数据，不需要更改公司联系数据文件。

日期: xxxx.xx.xx 页码: xx
从 xxxx.xx.xx :
公司
联系人
首次联系日期
数据包发送 xxxx.xx.xx
计划电话联系日期 xxxx.xx.xx
过期总数

答案与解读

另外加了一个要管理的实体:

1. 帮助文件

也可参考 IFPUG 关于 ILF/EIF (实体) 的识别要求, 必须符合以下条件:

1. 数据的集合必须是逻辑相关的并且是用户可以识别
2. 这些数据或者控制信息必须是在本应用的边界内被维护

总结:

- 实体方面增加了 1 个实体,
- 行为增加 4 个行为

所以动态功能点是增加的功能点 $25.4 (=1 \times 7 + 4 \times 4.6)$

数据类功能	Data Type	类型	TYPE	SiFP	MULT	操作类型	SiFP 合计
公司联系信息	实体	ILF		7	1		
错误文件	实体	EIF		7	1		
帮助文件	实体	EIF		7	1 ADD		7
交易类功能 Transactional functions							
Company contact data							
查公司联系信息	行为	EI		4.6	1		
按公司联系信息	行为	EQ		4.6	1		
新公司联系信息	行为	EI		4.6	1		
录公司联系信息	行为	EI		4.6	1		
数据包发送	行为	EI		4.6	1		
添加电话信息	行为	EI		4.6	1		
Error file							
Help file							
现场帮助	行为	EQ		4.6	1 ADD		4.6
屏幕帮助	行为	EQ		4.6	1 ADD		4.6
逾期	行为	EQ		4.6	1 ADD		4.6
联系报告	行为	EO		4.6	1 ADD		4.6
总计					1		25.4

与国际功能点 (IFPUG) 的偏差

以案例 A 为例子：

- 使用 IFPUG 数 EI、EO、EQ、ILF、EIF 每类都有规则判断复杂度，高中低，对应得出功能点数。例如 ILF 公司联系信息可分成 2 组，公司信息和联系信息如打电话、资料包等；总共包括 13 类字段，数据组 (Record Elements Type RET) =2, 字段 (Data Element Types DET) =13, ILF 复杂度为低。同样思路，创建、查询、更新也都是 2 组共不超过 15 DET 字段，按定义都是属于中等，但删除只需要共 4 字段，所以复杂度低，加起来得出调整前功能点数 FP=35 (详见下表)
- 使用 SiFP 估算实体和行为数，计算得出 FP=41.6

Complexity 复杂度	Function type 类型				
	ILF	EIF	EI	EO	EQ
Low 低	7	5	3	4	3
Average 平均	10	7	4	5	4
High 高	15	10	6	7	6

IFPUG	低 L	中 M	高 H	UFP	SiFP	数量 Count	FP
外部输入 (EI)		1	4	19	交易类功能 (行为)		
外部输出 (EO)							
外部查询 (EQ)			1	4		6*4.6=27.6	
内部逻辑文件 (ILF)		1		7	数据类功能 (实体)		
外部接口文件 (EIF)		1		5		2*7=14	
IFPUG 功能点数				35		简化功能点数	41.6

- SiFP 得出的实体数量，与行为数量都一样（实体 (=ILF+EIF)=2；行为 (=EI+EO+EQ)=6）
- 因为简化功能点只是不区分实体与行为的复杂度 (高中低)，取平均值。因这案例 A 复杂度偏低，所以用 IFPUG 考虑复杂度时功能点数比 SiFP 低，

但如果综合公司多个项目，复杂度有高有低，平均的差异便会降低

参考 References

1. Brigido, Sergio. "FPA Rules interpretations for Elementary Processes ", IFPUG Whit paper. (2022)
2. SiFPA Assoc. "Simple Function Point Functional Size Measurement Method, Measurement Examples ", SiFP-0.1.00-EX-EN-01.01 (2014)
3. Garmus, David. 'Function Point Analysis' , Addison-Wesley (2001)

Chapter 7

估算工作量

这几年大数据很火，很多高科技公司都推相关的工具或者方案，很多软件开发项目经理觉得应该也用数据分析，分析历史数据，准确预估项目工作量、工期。

”但实际上，虽然预测模型已经有超过 50 年的历史，过千份研究报告、教材、指南，但使用在项目不多。更多研究发现如果用专家估算可能更准确。”

以上是 2009 年 IEEE 杂志中的文章，Jorgensen 先生的结论。在文章里，Jorgensen 与 Boehm 两位专家讨论模型与专家估算法的长短：



北欧学者 Magne Jorgensen（左）过去 10 多年集中研究软件估算，发现专家估算法虽然很常用，但相关研究却不多。

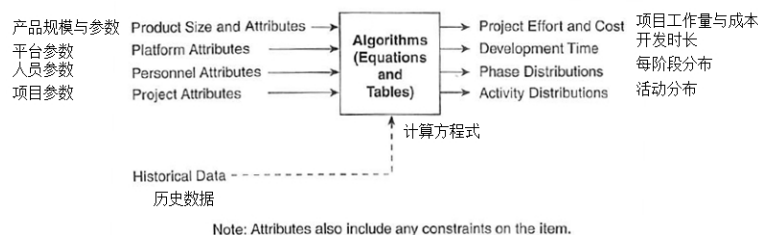
Barry Boehm（右）81 年出版”Software Engineering Economics”，后面又推出 COCOMO 预测模型，是模型估算的经典人物。

估算：靠模型，还是靠专家？

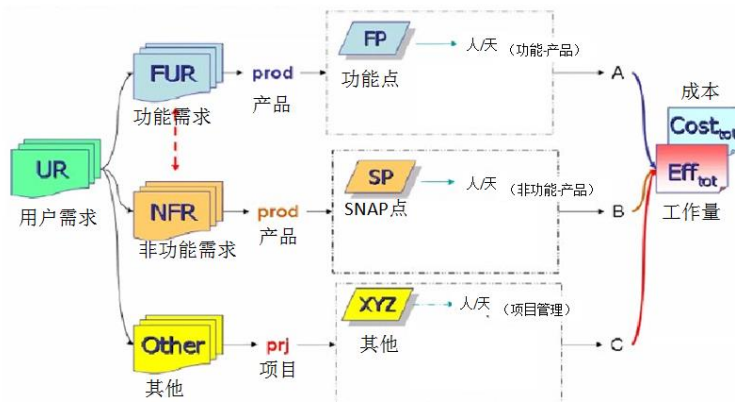
什么是专家估算？最典型的例子就是 WBS 估算，我们先把所有的任务，系统地分解出来，再召集团队和专家估算工作量（或工期）。

模型则是反过来：利用一些参数模型，如利用功能点数来做其中的一个输入，也包括其他因素，如复杂度、人员能力、平台等。

这两种方法的主要区分在于最后估算工作量的步骤：模型（例如 COCOMO）通常是利用方程式根据输入自动估算出结果，如下图：



或基于国际功能点的 ABC 模型，依据功能点数、非功能点数，和其他（例如项目管理),ABC 三部分综合估算总工作量（或成本）：



另一方面，专家估算法，例如 WBS 估算，是以专家主观判断为主（例如判断活动完成的最可能人天数）。模型估算一般比较客观、具有可重复性，不像第一种（专家估算）那样主要依赖人的主管判断，且依赖估算专家的能力。

Jorgensen 先生批评模型派，40 多年这么多研究，为什么模型估算还不准确，甚至不如用 WBS 估算。反过来，Boehm 先生说如果没有模型，整个估算就像个黑盒，无法知道规模、工作量、进度之间的关系，永远靠专家，尤其是在开发的初期，模型特别有用，因为早期需求模糊，无法很准确估算 WBS 工作量。

专家估算也有不同的变种，有些纯依赖专家主观判断，有些也会依赖历史数据和检查单，并非仅依赖挑选最佳的专家。有些估算方法，例如我们一般把扑克牌估算 (Planning Poker) 归属于专家估算，但如果最后估算主要依据上一轮冲刺的速度 (Velocity) 或生产率 (Productivity)，它就更类似模型估算了。所以虽然模型估算和专家估算看起来有明显差异，但有时候不容易明确区分。下表通过比较专家与模型估算，总结了两种方式的主要区分：

估算步骤	专家估算	模型
评判不同信息的比重	主观判断	基于对历史数据的统计分析
从信息估算工作量	主观判断	基于正式重复性的过程和方程式
回顾/讨论工作量估算	主观判断 + 分析过程，参考检查单、指南和专家意见	主观判断 + 分析过程，参考检查单、指南和专家意见，可能再主观判断更新工作量估算

所以当我们了解了模型与专家估算两者各有好处，没有绝对的对与错，就能理解为什么不能单靠模型预测，而是两者都要参考。比如模型可以更好地在早期判断，估计整个项目成本，还可以帮我们了解哪些因素会影响工作量（进度），需要关注。

为什么估算开发工作量/工期这么困难

你可能会质疑以上只是大学教授基于学术研究的结论，实际上数学模型的估算不一定比专家估算差。我们可以回顾美国 SIT 的项目管理估算模拟实验。

软件项目管理学生用各种方法估计完成某乐高积木的时长，利用项目数据得出的预算模型的估算准确度还不如戴尔菲法专家估算。（详见附件）

由于软件开发主要靠人来做，非常依赖人，这导致影响生产率的因素很多，如积极性、经验、工具、团队合作等等。用数据挖掘、机器学习、更适合研究一些科学的问题（物理/化学/生物，如药品的化学成分），相关因素、效用等都好衡量。但是人的因素就没有这么简单。所以过去几十年虽然有不少关于工作量的估算模型被建立，但准确度一直不太理想。反过来，也不能单靠专家估算，它非常依赖专家的选择，因为人是主观的。虽然主观判断有可能帮助专家做好估算，但也正因为人的主观性，导致估算的偏差。例如某人做某类软件开发很有经验，他能更准确地估出这类开发的工作量，但如问她能否用一条方程式表达出来，他肯定不知道。

所以，想做好软件开发的工作量估算，首先要理解估算本身的偏差会很大，影响因素很多，也因为无法预测所有（尤其是跟人相关的）因素的影响，个人经验就可以补充估算的不足。

估算软件开发的困难

我们有哪个软件开发像玩乐高积木这么直接简单？例如：

- 有明确固定的答案，也有明确的步骤，可以照着，按部就班完成。
- 如果中间过程有错，很容易立马看出来。

但软件开发往往是：

1. 需求和规范经常有变动（有哪个项目从需求制定到最后开发完成，中间没有变化？）。
2. 软件开发写错一句代码可能影响全盘。

因软件开发的需求往往是不确定的，所以敏捷大师们反对传统项目估算方式——需求（规模）是固定，估算出对应的工期与工作量，并制订整个项目计划与进度表，然后监控，确保项目没有超出计划工期/工时。他们觉得这种以计划驱动的传统管理模式直接影响了软件开发团队的生产效率，太多项目因为经理都只是管是否按期完成，是否没有超人时，而不考虑软件开发的质量是否给客户提供了价值，反而影响到很多软件项目都被用户诟病。

在香港讲敏捷课时，某学员听完敏捷精益的概念后，说她的经理要求她输出一份 3 个月的详细工作任务分解，且要求活动的颗粒度细到 5 天之内，因为她的经理按计划管理的思路很重。她很困惑，软件开发变化这么多，怎么可以做出这种详细计划呢？

这其实有点像我们要在一个足球赛还没开始之前，要预先写赛后报告一样没意义，都是猜。

我就回应她说：如果老板有这种思路的话是无法推动敏捷的，其实敏捷有一个很重要的概念，因为需求变化这么大，我们的范围是不固定的，所以基于精益的概念，在有限的时间里，我们要制订这个冲刺在 2 周里面可以完成哪些故事？做完以后再依据过去的进一步策划下一个阶段就更实际了，不是全都靠猜测。这种思路更能让客户尽早看到实际的项目进展，所以敏捷特别重视可以运行测试好的故事，而不是一堆需求文档或者没测试过的代码，因为需求没有给客户看过，还不算是完成，只是一个中间的状态。所以如果经理知道软件开发的这种特性后，就会在那个策划的三角形——时间、成本和规模范围之间适当取舍。如果我们规定时间为 2 周，应该可以在那些故事里面挑选最有价值，可以在 2 周完成的故事，完成它并展示给客户。这就有了 MVP（最小有价值产物）的概念，就会避免团队为了只追求进度，而导致最后做了一大堆没价值的东西。

然后我跟她说了下面两个谷歌故事：

How Google Works 谷歌文化



➤ **Larry Page – co-found Google with Sergey**
(创始人: Larry + Sergey)



➤ **Eric Schmidt , Jonathan Rosenberg**



CEO, 以前是 Novell CEO
曾在 SUN micro 工作



以前在 Excite@Home 和 Apple
当产品经理



Smart Creatives in Google 聪明的创意人

9

例如当时谷歌 (Google) 还是很小的公司, 但因在搜索引擎上有所突破, 预计会被微软攻击和收购。董事会主席 Mike Moritz 便请刚进公司的 CEO Eric 和产品经理 Jonathan 准备了一份详细的商业计划书来应对。俩人经过两个多月细心准备, 约创始人 Larry 开会解释计划详细内容, Larry 拿到计划书快速看了一下, 便立马问他们俩: “你们以往有试过超出你们的计划吗? 如果没有, 建议还是先跟我们那些工程师聊聊吧。”

后面他们俩发现 Google 的工程师都是精英, 无论技术或业务方面的能力都很强, 都有很多创新思路。了解到如果还是按传统的计划驱动反而会限制团队创新能力。

这些优秀人才被统称为谷歌内的聪明创意人 (Smart Creatives)。

河马 (Hippopotamuses) 是攻击力很强, 非常危险的动物。

谷歌公司用河马的英文简称 HiPPO (Highest—Paid Person's Opinion), 比喻企业类内的“大王”按自己的主观判断, 雷厉风行也是非常危险。

例如在谷歌的内部讨论会, 做决定不看提出意见者在公司的地位、权威。高管提出的建议, 如果缺乏充足的数据支持, 也可以被工程师推翻, 例如创始人之一 Sergey 想某工程部协助开发某新产品, 但他的方案缺乏有力数据支撑, 工程主管不同意。Sergey 没有用他的权威强制要求 (他绝对有这权力), 他让步妥协, 只需要一半人手参与, 但工程主管还是不同意, 最终大家客观比较各种方案的优劣, Sergey 的方案未被采纳。

学员下午做互动练习时便说自己公司里确实有不少“河马”横行。

从以上两故事看到, 优秀的公司高层, 无论东西方, 都理解公司的发展依赖“大家的力量”和各人的智慧, 同心协力, “大王”文化会妨碍公司持续改进, 难以改善。

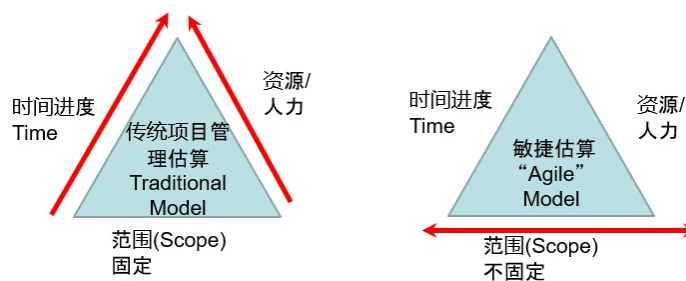
2000 年时, 谷歌在快速成长期。我通常会在敏捷培训课的第一天借用谷歌的故事说明团队敏捷开发的成败依赖于公司文化。

如果团队有能力，应让团队自主创新，传统计划反而会局限了团队发挥，是敏捷开发核心思想。

- 因为每次只估算后面迭代（2-4周）的工作量，偏差不会太大。
- 因为只估算本迭代需求相关工作，减少需求变更的风险。

因为按范围估算软件开发工作量很困难，所以只能根据有限的资源和时间，识别出那些对客户最重要的功能，评估能否在项目交付期限内完成。如果不能，再按功能对客户价值，选择那些较低的功能可以先不做。

传统项目管理假定规模是固定的，敏捷开发则反过来，时间和资源是固定的，但生产率是根据团队前面的迭代速度得到的数据。



这个时间固定，但范围可变的思路，不仅仅适用于软件开发。例如，计划必须在月底前将整本书全部交稿，但草稿中有不少小错误，如果要全部都修正好，肯定不能按月底期限交付，所以需要把一些可以删掉的部分删掉，确保书的质量，也把影响降到最低。与软件开发不同，书可以比较轻松删掉一两段，但软件之间的耦合性比较大，可能删掉一行语句整个程序就跑不通，所以更需要在交付前规划好开发的范围，最终才有机会可以按期交付。

敏捷是依据精益 MVP 概念、时间（资源）固定、范围可变，如果估计不能在交付期限前完成所有客户需求，便要尽早跟客户协商，选择哪些功能放在这次交付之内，哪些放在后面，希望在有限的时间和资源条件下，能给客户提供的最高价值。很多敏捷团队会用以下方式逐步做估算：

先把所有客户的需求写成故事卡片，作为整个项目的交付物 (Backlog)，然后按每次迭代（例如从第一次迭代）：

* 先挑选最重要的故事，放在本次迭代，估计相关的故事点，故事点是一个简单的规模概念，团队按照类似的模块的工作量，估计每故事的故事点数（例如，可以用扑克牌举牌方式多轮估算，最终得出一个团队都赞同的故事点数）。

- 按本迭代可允许的人天数估计本迭代可以完成多少个故事点
- 然后每次迭代结束后记录完成多少故事点，形成燃烧图 (Burndown chart)，看功能点的降低速度来预估整个项目最终完成的时间，需要多少个迭代。从燃烧图，故事点减少的速度计算项目速度 (Velocity)。

+++

有些敏捷大师甚至反对估算，觉得估算只是凭个人经验拍脑袋，没有科学的依据，反而导致团队只关心进度不能延误，从而影响开发人员，使得开发人员反

而不能专心专业地做好软件开发。

他们反对使用故事点估算，故事点按个人经验来估算，不同人的理解不一样，可能导致差异很大。同样，他们也反对敏捷 Velocity 速度的概念，觉得这只是另一个形式的传统策划思维，会导致团队只是关注在一个冲刺迭代后交付了多少个故事点，反而忽视了交付的软件是否对确实客户有价值。

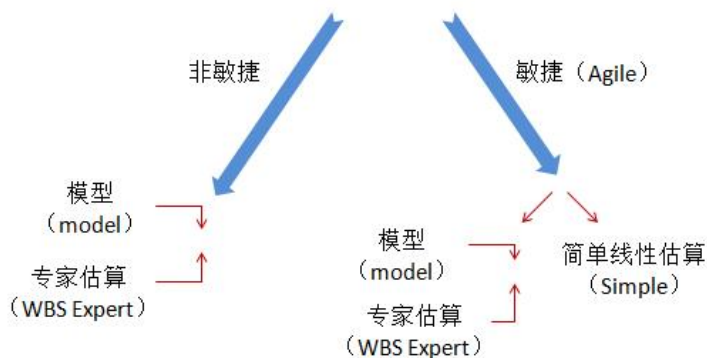
但他们也理解到团队还是要预估可否在客户定的期限内交付，或者交付多少，因为这是客户关注的。他们建议直接把 backlog 里要交付的所有需求，细分成小故事，每个故事最好是一人天左右。这样很简单，可以按完成故事的速度，更客观地预计客户要最后交付的日期（例如，10 周后），可以完成多少故事。

例如，共要求 300 个故事，按团队速度，10 周后只能总共完成 200，就要提前按优先级、重要性跟客户协商，挑选哪些在这个期间前交付，哪些放到下一次再做。（前提是：团队必须已经做了一些开发，知道每次两种迭代大概可以完成多少故事。）

结束语

选择哪种方法估算软件开发工作量，跟选择最合适的项目生命周期方法类似，取决于项目的特性和团队的能力。如果需求很明确且固定，也没有客户代表在每个迭代（2-4 周）后进行确认，例如，政府投标定制开发项目，就不一定选择精益思路的敏捷估算方法，而应依据范围 (Scope) 和规模 (Size)，估算工期与工作量，除了使用 WBS 分解从下而上靠专家/团队估算工作量与工期外，也要利用规模（功能点数），利用基于历史数据的估算模型从上而下估算。（千万不要认为哪种估算最准确，其实都不准，但综合考虑可以减少偏差）

对于需求很可能会变，客户参与较多，团队能力强的产品性开发项目，则采用时间/资源固定，范围可变的精益敏捷思路，就更能为客户创造价值。如果可以把范围细分到很小的估算（每故事 ~1 人天），可以依据以往历史速度，估算要完成所有需求一共要多少次迭代（或周）。但如果项目复杂，之间相互依赖，有些模块要多人合作，难以细分，就难以用前面的简单线性估算，需要团队估算工作量，但不应只依赖从下而上按任务估计工作量，也要参考规模大小，利用估算模型从上而下估算，然后综合考虑。



附件

估算砌乐高积木时长 / 2015 年学生实验数据

2015 年, 17 位有工作经验的兼读学生参加 SIT 的软件估算与度量课程 (一个学期, 共 13 节课)。为了让学员亲身感受软件估算的困难, 老师要求学员用以下方式估算要完成乐高“Sopwith Camel” 积木 (详见下图) 的时长:



1. 每位学员独自做估算 (I)。
2. 使用戴尔菲法 (Wideband Delphi), 多轮后更新 (个人) 估算 (II)。
3. 分成 3 组, 每组也使用戴尔菲法, 制定本组的 (团队) 估算 (II (团队))。
4. 依据实验数据 (注 2) 调整、更新 (个人) 估算 (III)。
5. 团队也依据实验数据调整、更新 (团队) 估算 (III (团队))。
6. 完成以上 4、5 步后, 立马告知会选那位学员实际做 LOGO 积木 (注 3), 让个人调整、更新 (个人) 估算。(IV)
7. 也让团队调整、更新 (团队) 估算 (IV (团队))。

注 1:

- 之前在课程里，已正式学过戴尔菲法。
- 加上规模，复杂度等对工作量、生产率的影响。

注 2: 让学员自己做乐高实验（因有些学员可能从未玩过乐高积木），提供 4 对同样的积木：

- 大笨钟 Big Ben(下面有完成后的照片)
- 比萨斜塔 Leaning Tower of Pisa
- 埃菲尔铁塔 Eiffel Tower
- 自由式小轮车 General Grievous Wheel Bike

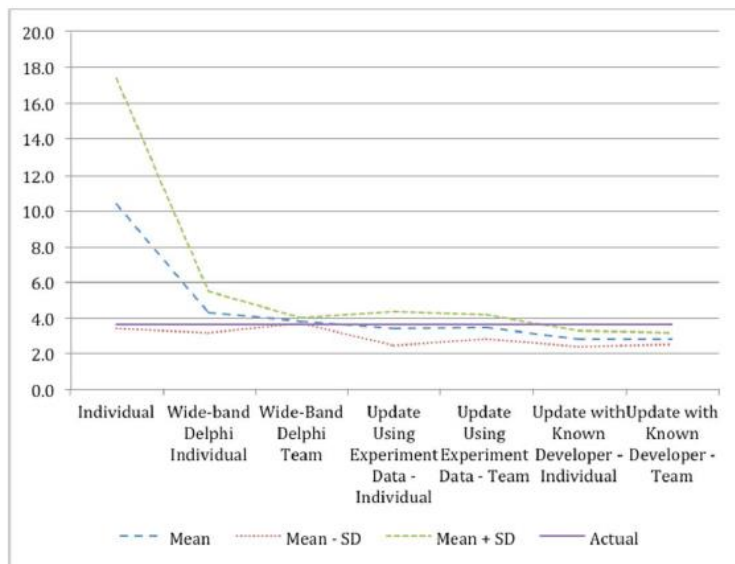


- 每位学员记录实际完成的时长，把所有数据汇总并公开给所有人。
- 之前在课程里，已正式学过预测模型和数据统计基础，如何利用以往数据，建立模型（附件里有如何建模的介绍）。

注 3:

- 大家都有这位学员之前乐高实验所需时长的数据。

实验结果与讨论



- 那学员用了 213 分钟（差不多 3.55 个小时）完成，他自述困难主要在于颜色都很类似，很多时候找不到合适的积木。（跟其他体验组比较，其他 2 3 组都在 210 220 分钟范围之内完成）。
- 戴尔菲法显著减少了个人估算的偏差，用戴尔菲的团队估算其实是最接近实际的，估得最好的，偏差不超过 15 分钟。开始时，尤其个人都会多估，但后面数据越来越多，反而开始有点偏低了。
- 可能导致后面估算偏低的原因：之前实验用的 4 组积木都是建筑物，复杂度与颜色搭配都较简单。

经验教训

1. 学生经过这个过程可以从实验亲身感受到估算的局限，没有想象这么准确，因太多因素影响，也很难建立一个预测模型，比如偏差可能到了 0.3 以上。
2. 从这个过程里面也让他们感受到主观判断的重要性，不要以为仅仅客观用一些数据、用数学方程式就可以准确估出正确的答案。
3. 了解复杂度和人对生产力的影响，比如积极性跟经验都是很重要的因素。例如，有一个学生经验很少，但竞争力和积极性非常高，所以有好的成绩。有两个团队也是完成同一套模型，发现本来自己判断为高经验的团队反而比中经验的团队花多一倍时间，最慢的团队是一个两人团队，因为基本没有经验，所以就很小心，一步一步去做，尽量减少错误，导致用的时间最长。
4. 团队规模大小也有影响，团队人数越多其实是对整个生产力有负面影响，人数少反而会好一些。（其他关于软件开发的研究也有类似的发现）。

很多人以为软件项目估算与其他项目（如土木）类似。以上实验，虽然不是真

正做软件开发，学生自己做估算，从亲身体验学习，更好了解软件项目估算的常见误解：

1. 以为软件估算可以精确到正负几个百分点。
2. 以为在项目时间不够的时候，增加人员对项目的进度会有帮助。
3. 以为团队人越多，工期就按比例缩短，像建筑工程一样。
4. 以为估算是可以单靠一些预测模型简单地估出一个数值。

学生都没有软件项目的经验，我们也不可能在上課的时候做软件项目，于是用乐高积木，让他们做估算，比如我们让他们依据一个结果图，估算要多少人时来完成积木，也要看人的乐高经验，首先第一步是他们单独做估算。我们在课程中用了戴尔菲估算法，每个人多轮估算。找人确实来建乐高，看看实际用了多少时间。然后我们在课程中加了一些估算模型，用一些历史数据给他们进行参考，下面就是我们的一些发现：很多人在没有经验的时候都会高估，用了戴尔菲的方式会确实比较准确。后面有了实际数据，要求学生利用软件项目估算模型方式来估，他们就发现这不容易，很多时候，有学生直接改用简单方程式做估算。学生自己动手来估算，才会有感觉，真正了解软件估算的各种困难。

学生 ID	经验水平 (自估)	I	II	III	IV
1	0	14.3	3.7	2.25	2.25
2	Low	10	4	2.5	2.5
3	Low	20	4.5	3.75	3.75
4	Medium	3	5	4.5	3
5	Medium	2.5	4	3.2	3.2
6	Low	8	4.1	4.1	3
7	0	20	7	5	3.5
8	Medium	20	5	4	2.75
9	Medium	4.5	5	4.5	3.5
10	Medium	3	3	3	2.5
11	High	4.6	3.8	2.8	2.4
12	Medium	2	2.5	2.7	2.3
13	Low	6.5	3.5	2.5	2.5
14	0	18	6	5	3
15	Low	20	3	3	3
16	High	5	5.5	2.3	2.5
17	High	15	3.8	2.7	2.4
Mean 均值	-	10.4	4.3	3.4	2.8
Median	-	8	4	3	2.8
SD 标准差	-	7	1.1	0.9	0.4

注：上面 I、II、III、IV 与下面 II、III、IV 的定义，对应哪种场景，已经在上面本文里说明。

团队 ID	II (团队)	III (团队)	IV (团队)
1	3.8	2.7	2.4
2	3.7	3.4	3.16
3	4	4.3	2.85
Mean 均值	3.8	3.5	2.8
Median	3.8	3.4	2.9
SD 标准差	0.1	0.7	0.3

Team ID	# Pieces	对象人群岁数	团队人数	实际总工作量 (分钟)	模型估计总工作量 (分钟)	偏差 MRE
1	346	12+	1	37	46.21	0.25
2	321	12+	1	48	45.39	0.05
3	261	7 to 12	1	32	43.19	0.35
4	345	12+	1	45	46.18	0.03
5	261	7 to 12	1	59	43.19	0.27
6	345	12+	2	114	73.48	0.36
7	346	12+	3	84	96.48	0.15
8	321	12+	4	104	114.90	0.10

注：偏差 MRE(Magnitude of Relative Error) = | (实际 - 估算) | / 实际

参考 References

1. Jorgensen, Magne, with B. Boehm. "Software Development Effort Estimation: Formal Models or Expert Judgment? ", IEEE SOFTWARE, March/April 2009
2. Laird, Linda, with Y. Yang. "Engaging Software Estimation Education using LEGOs: A Case Study." 2016 IEEE/ACM International Conference on Software Engineering Companion
3. Duarte, Vasco. "NO ESTIMATES: How to Measure Project Progress without Estimating." (2016)

Part IV

开始过程改进之旅

所有有效的改进，团队必须先获得管理层的关注与支持。

第 8 章先介绍如何以高层的“语言”分析改进对公司的价值，获取高层支持，投入资源，成立改进组，跟其他项目一样立项管理。

第 9 章继续用实例说明过程改进项目的主要成功要素，包括要做好根因分析。

Chapter 8

获取高层支持

我：对过程改进来说，最重要的成功要素是什么？

客户：最难的是如何得到高层的支持，这不仅仅是嘴巴说说而已，而是要切实地给人、给时间。高层往往不清楚什么是质量改进的重点，但他们对员工的人均收入、利润（比如员工可为公司盈利的时间占比。如果少，就表示这个员工对公司的盈利贡献不够。）等这些财务指标都非常清楚。

我：非常赞同。我们可以利用评估机会来引起高层对质量改进的重视，但往往评估组只说软件开发的各种问题，难以引起他们注意。一般人，尤其是高层，一听到问题都会觉得比较烦，没有动力听下去，更不要说有对应的改进行动了。

客户：你说的挺有道理的。确实我们每年都会有一些评估，发现了不少问题。高层也都会参加评估会，并同意这些问题需要改进。但很多时候过后就没下文了。

我：质量大师裘兰博士 (Dr Juran) 有丰富的过程改进经验，他深知要说服高层真心投入、支持改进，就必须用高层的语言打动他们。如果与高层交流，不能把团队关注的事转化成高层关注的，就难以获得立项，做改进。

我：所以在最近的某评估里，我会引导公司内部的质量经理，跟公司高层汇报时，着重汇报能为公司节省成本的初步方案。你有兴趣听听这案例吗？

客户：非常有兴趣。

案例

我问公司内部评估组，高层有哪些关注点？他们就列出来，如人均收入、人均利润等商务指标。

然后我再问影响这些指标的因素很多，有些是软件开发团队无法控制的，例如销售人员与客户的关系，所以必须从这些高层关注的指标细分到一些团队实际可以影响到的指标。有哪些呢？

质量经理：项目进度的偏差，遗漏给客户的缺陷数。

我：这些我们都叫性能目标。这些指标是如何影响高层目标的？

质量经理：比如项目延误，成本就会超支。

我：是的。如果我们过程改进希望改善质量、减少缺陷，你们觉得会对成本有

什么影响？
质量经理：改进要投入工作量，肯定会增加成本。但长远应可降低成本。
我：很多高层都不熟悉质量管理，所以很少关注和监控团队的缺陷。他们通常还是会觉得质量改进是好事，但要花成本。我做到客户满意的水平就可以了，不要追求十全十美，也正是这个误解，才导致他们认为大部分缺陷在系统测试或验收阶段才发现是正常、是常态。如果是开发软件产品的公司会好一些。在软件工程领域，因为缺陷发现越靠后，返工工作量是前期发现同样缺陷的 15-40 倍。所以如果我们能把返工缺陷预先发现并处理，必然会大量降低返工工作量，与相关成本。

理论上也可以用缺陷排除率来估算改进后的缺陷分布（会在后面迭代回顾里细讲），但高层一般不会注意这些细节，沟通越直接越简单越好，所以只需要简单估计改进后的数量，大家觉得合理便可。

第一版方案书

质量经理做了第一版的方案书。



我：挺好的。你在投资方面是保守型吗？
质量经理：不是啊，我还会定期买股票，因为单是靠银行定期，利息太低了。
我：但是看你在质量改进方面的目标很保守。比如你看，说如果做了改进以后，后面的缺陷可以降低 10% 到 20%，然后最后算出来节省不到 5 个点。你估计高层会买单，投资你这个项目吗？
质量经理：确实有道理。
我：你这里有几个地方没做对，例如从我们过去的经验，因为很多团队在前面几乎没做好单元测试或代码扫描，反之，系统测试或者验收测试的缺陷几乎可以减少一半。低的目标不是好目标。还有，你评估在后面阶段返工的工作量也太低了，只是比前面发现的高一点点。你尝试依据这 2 个思路再调一下吧。
质量经理：你有所不知，我们虽然有些行业参考数据，但没有实际项目数据支撑，是否需要先找些实际数据才可以做方案，不然好像说不过去。
我：在初步方案阶段，其实不需要实际数据的支撑。你可以想象，目的是要让管理者有依据做决定，你可以利用实际数据把节省算得更准确，但对高层决策没有太大帮助。所以在初始阶段，合理的估算就足够了，不需要花精力在没有价值的事情上。

第二版与中层确认，然后向高层汇报

做了第二版后，大家觉得这确实比之前更有说服力了，质量经理再跟中层确认。

项目A：规划120人月，总成本300w					
里程碑	单个缺陷返工工时/h	原缺陷数	原返工工时/h	改善后缺陷数	改善后返工工时/h
需求评审	1	10	10	15	15
设计评审	1	5	5	8	8
UT阶段	1	45	45	60	60
SIT阶段	20	240	4800	120	2400
UAT阶段	30	100	3000	50	1500
总计			7860		3983
成本占比			39.30%		19.92%

改善效果：成本下降19.39%，约58w

问题	需求变更 评审不充分（例如，设计评审） 单元测试（自动化脚本） 开发人员技能不足 版本管理问题
影响	项目延期、成本增加
总人月	原计划120人月 影响后130人月
总成本	原计划300w 影响后325w

造成损失：成本增加8.3%，约25w
改善后的返工工时是原来的 51%

质量经理跟高层汇报，并得到高层的认可后，高兴地说：以前都以为过程改进应先自己默默耕耘，先在试点取得效果，再跟高层要资源后再推广。现在看，还是应该一开始便提具体改进方案并立项，才有机会成功。

我：管理层是过程改进最重要的干系人，必须一开始便得到他们支持。你后面有什么计划？

质量经理：既然他们都同意，我就找项目试点，对吗？

我：虽然试点很重要，但不是第一步。你们一直有过程改进小组吗？

质量经理：懂你的意思，但实际没有。团队有关质量问题便会直接找我。我：应让团队自己依据迭代数据分析根因，下一轮做改进。但有些需要跨功能或组织级改进，如完善复用框架，或完善规范/指南与相关检查单等，便需要依靠过程改进小组来推动。如果你们没有，便应尽快举办工作坊互动培训，邀请所有过程的利益相关者参加，一起讨论未来改进重点并制订具体短期长期目标与计划。

质量经理：你说的有道理，请发我相关案例，打铁趁热，我尽快跟高层说。

结束语

从以上简单案例，得到的经验教训：

- 1. 有效的质量改进必须从获取高层支持开始。
- 2. 中层要从高层关注点（¥）提改进方案。
- 3. 在做改进方案时应根据现状估算，暂时不需要实际项目统计数据，以节省资源和时间。
- 4. 如果确实有高层真正支持的话，就必须正式立项，再配上具体的团队、分工和详细计划，否则整个改进计划还是一纸空文。
- 5. 立项后，可以用 2 天工作坊培训启动，大家一起制定目标和行动计划。

反馈

某 CTO：方向很好，但高层不仅关注如销售额这些商业指标，也关心交付效率和质量相关指标。

公司老板针对于研发除了关注成本外，也关注以下内容：

- 人员：人均薪资、人均产能、人员参项率（有收入的项目）、人员闲置率
- 进度：是否能正常交付
- 质量：缺陷率、严重缺陷数

例如，公司都会用人均产值，人均交付金额等指标。但有些公司业务很稳定，公司不一定关注创新：例如，针对重复性工作，是否能基于以往项目形成公共组件或低代码平台，来大幅度提升生产率。

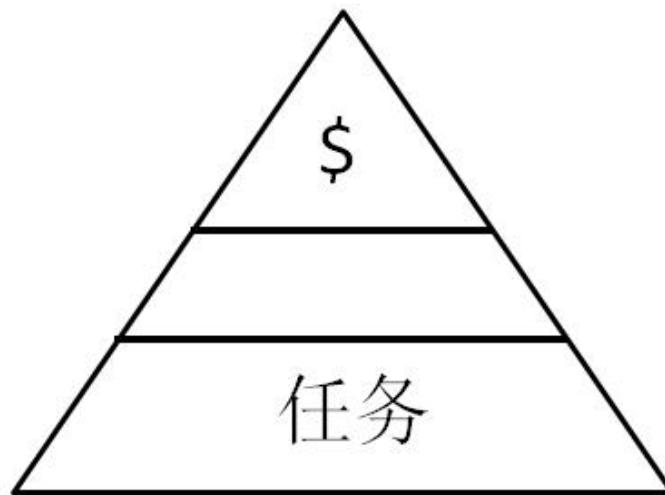
质量方面，如果客户能参与使用，例如产品定义，需求的质量会大幅度提升（不仅依赖评审、测试）。为什么客户不愿意深度参与？是否因为没有让客户可以高效参与的工具。

例如，需求调研过程，90

所以不应只关注如何尽量提前发现缺陷

我：是的。想要过程改进成功，首先必须得到高层的支持，很多高层不一定关注质量，只关注项目是否按时交付和财务指标。

所以质量改进方案必须强调能节省多少人力成本，不仅仅是质量提升，才有机会得到高层支持。



大量缺陷在后期才暴露是最常见的质量问题。提前发现并消除这些缺陷最容易能提升项目质量，并降低成本（因大幅减少返工）。所以上面案例使用这质量改进作为例子，但如果面对像你这种通情达理的高层，就可以更全面，从质量、创新等多维度估计改进能如何为公司带来价值。

Chapter 9

寻找改进机会

客户：想改善我们的敏捷开发过程，但没有概念应该怎么开始？

我：可以借用评估，例如 CMMI 评估。

客户：CMMI 不是和敏捷打对台吗？

我：这是很多人的误解，把 CMMI 等同于瀑布式开发，而且误认为 CMMI 评估只是认证（类似 ISO 认证）。本来 CMMI 的目的是帮助企业，例如美国国防部，评价自己和供应商的软件开发过程质量、诊断软件工程软件开发过程有哪些不足。所以 CMMI 很全面地覆盖项目管理、软件工程等领域。

敏捷过程，例如 SCRUM、XP 等都能对上大部分的 CMMI 实践。但反过来敏捷的实践只包括部分 CMMI 实践，不全面，例如缺少组织级部分，最新的 V2.0 版本更精简，更适合敏捷团队。

客户：怎么可以利用 CMMI？学习一下资料就可以了么？

我：这也是很多团队的误解，跟培训学习的道理一样，若只是吸收了知识，没动手动脑筋，不会有什么作用。我刚开始做 CMMI 评估时，有些企业说：“CMMI 是挺好的，但我们不需要花这么多钱、精力去做评估，平时按照你发的模型与参考材料，学习一下就可以了，我们团队学习力很强。”但过了 6 个月、或一两年后，再次拜访客户，99

但请注意，虽然评估能帮我们识别出一些改进点，但过程改进必须做好相关纠正措施。

客户：请解释一下。

我：首先要理解纠正措施 (Corrective Action) 和改正 (Correction) 的区别。这一点我刚开始接触 CMMI 的时候，已经听过，但没有充分了解。

- 改正 Correction 只是针对问题的表面，去处理，不能长期避免同类问题再发生。
- 纠正措施 Corrective action 是针对问题分析和发现背后的根本原因并制订对应改进计划。

很多时候团队仅仅利用问题跟踪表，把评估发现的问题一一解决，只做 Correction，做事习惯没有变化，后面同类问题还是会重复出现。所以更应分析问题根因，并识别哪些改进项目会有效果，然后以项目方式立项做改进。

客户：你说的好像还是挺有道理，是否可以举些例子。

下面就举一个关于组织培训的实例。

评估中访谈公司培训专员

评估师：请问你们如何与上级和管理层汇报公司培训的情况？

培训专员：我们每次都有培训报告，内容包括学员的考试成绩、反馈、出席情况等。也有每年的总结报告，包括整年所有的培训，比如实际与计划对比、费用与预算比较等等。

我：是否可以举例？

培训专员：例如我们比较关注信息安全。因为很多客户尤其是政府客户也注重这一点，所以我们就找一些外面的培训课，专门对员工做一个线上的信息安全培训。大概有 100 个人参加，为了衡量培训效果，有考试题库，及格分数线是 80 分，但是第一轮考试只有 60 多位及格，我们觉得效果不好，要求那些不及格的重考。第二次还是有 20 位左右不通过。

评估师：你们觉得效果如何？满意吗？

培训专员：不太满意，但后面该怎么做呢？

。。。。。

评估师：请问你们如何培训新进公司的开发人员？

培训专员：我们都有一系列的新员工技术培训课，上午会讲一些原理要求，然后会有一些编码练习，比如如何设计跟开发一个人事管理系统。我们会把用的框架和复用组件放到内网，方便他们获取使用。也会定好系统的主要模块和相关的测试用例，让他们可以在编程的时候用上那些原则和理论。评估师：挺好的，你们如何对他们做反馈？

培训专员：我作为老师会对他们的开发进行评价、打分，告知他们不足之处。

评估师：是指我们平常年终评价那种，按表现的优秀程度打分吗？

培训专员：差不多，我们会从不同的角度去评价。比如我们强调是否按公司的编码规程？是否满足面向对象不耦合原则？从多个角度去评价，进行综合打分。

总结以上访谈的问题

大家可以从以上访谈发现下面 2 问题，并针对每一条问题制定改正措施：

1. 发现培训反馈靠主观，只是靠老师主观打分，可以加考试就可以满足不客观了

2. 发现有些课学生成绩及格率低，下一次同类外部培训要求学员备课，以提高学生的通过率

是采取这些行动，对公司级培训质量改进作用吗？几乎没有，因为只是对问题做 correction, 没有改变系统与习惯，类似问题还是会再出现。

几乎没有，因为只是对问题做 correction, 没有改变系统与习惯，类似问题再出现，还是以前的做法。

但如能深入探索根因，就发现原来新员工正式上课的培训不长，只有一天，大部分还是靠新员工在工作中，跟有经验“老”员工导师，老带新，边做边学。但因为很多有经验的都忙于做项目，没有时间带领新员工，所以主要依赖新员工自学，导致新员工培训的效果差异很大。

也正因为员工都没有持续学习的习惯，都是边做边学，参加比较正式的课程前都没有备课的习惯，导致及格率比较低。也发现原来公司有给客户开发并提供线上学习平台。

评估师问：为什么没有类似的内部平台用于内部培训？

公司总监：平台我们是有的，但是没有课件。我们工作也很独特，没有一些可以买回来的培训可以用。所以一直都没有计划用这个平台。

评估师：现在很多公司，尤其疫情原因，都倾向利用培训平台帮助员工学习，虽然没有传统面对面的培训效果好，但肯定会比自学好多了，同时还有上课记录、考试等度量。但须要公司花时间、精力去准备课件，才对公司级培训有作用。

后面总监在评估后，与大家讨论这个评估的发现，大家都同意这做法会有帮助，并且评价这个改进项目的投入和价值，最终成功立项，作为下半年的主要改进项目之一。

客户：回顾我们每年都会有各种评估，但评估后只是针对发现的问题做修正，对整个公司的质量确实没有任何提升的作用。大家做事的习惯还是一样，把评估和处理问题当成一些常规性应付的工作。

问题描述	类型	解决方案	责任人	状态	完成日期
员工外训考试通过率	弱项	要求员工培训前都要备课		✓	2022.10.8
新员工培训考老师主观评价，不客观	弱项	增加考试打分， 2022.11.15李工开始准备 新员工考试的试题	李工		

我：所以要质量改进，必须针对弱点找出根因，并有相关改进项目才有效果。如果希望利用评估做改进，首先大家的心态要改过来，如果只追求通过评估的心态，只给评估组看最好的一面，不让问题暴露，对公司的质量改进没有作用。但是反过来，如果大家希望这些模型能帮助发现不足，识别改进的机会，评估可以很全面地帮助我们，就好像去医院做体检一样，各方面都会帮我们看。所以每次评估时，我都会让大家放心，千万不要担心问题被发现，我们发现的问题都不是个人或者项目组的问题，都是系统的问题，系统有不足，才导致有这些问题发生。所以每次评估也必须遵守保密跟不追究原则，让大家可以放心说出不足，才可以起改进的作用。

管理层的理解与支持

评估发现会分成弱项与建议两类：

- 弱项是指跟模型的要求有明显差距
- 建议比弱项程度轻，没有明显差距但可以改善的点

评估过程要求标弱项必须要得到内部评估组成员认同。某次评估，某些内部评估组成员对弱项就特别担心，尽量想办法解释。有些弱项可能要反复讨论十几二十分钟，他们确实没办法才接受。

”我们理解你说项目量化管理计划模板中写公司基线的地方应该是公司目标，公司目标里应该是写总体的年度目标，我们同意模版有不足，应该后面改善，但应该算是建议，不能当成弱项。“

但是如果提的是建议，一点问题都没有。为什么发现弱项跟建议反应区分这么大呢？

后来发现，原来领导关注评估，看弱项多少，觉得如果弱项多就表示相关人没做到位，但恰恰是管理者这种心态妨碍了企业的过程改进，因为要改进，首先要觉得本身有不足，才有动力挖掘不足的根因并做改进。

评估师问：您觉得你们配置管理做得如何？有没有一些你不满意的地方？
研发总监：挺好的。没有问题。
(但实际上，从文档检查已发现在配置审计、建立基线、基线变更、如何保证文档、环境配置与代码同步等都有不足。)

所以如果想利用评估来做改进，首先管理者要觉得本身有不足，并理解评估的目的，就好比是我们做代码走查，如果没发现是坏事一样道理。

如想了解更多应如何分析根因，并在敏捷项目中使用，例如冲刺回顾，请看下部分。

结束语

CMMI 过程改进的成功要素：

1. 时间

如果公司希望借用 CMMI 确实做到一些具体的改进，因为要足够的时间来累积数据，整个项目时间不能太短，比如小于 6 个月就很难达到改进效果。

2. 计划

一开始就要做好计划，企业要有过程改进组人员的投入，包括负责度量与分析。而且还要有合适的工具，单靠手工很难收集真正的项目数据。

3. 方法

培训不仅仅是教 CMMI 模型，更重要是有一些具体的方法、模板、工具，可以让项目组立马用起来。项目实战辅导很关键，听得懂并不一定会用，老师的实战辅导才能真正将方法落到实处。

4. 使用功能点估算项目规模使项目间可比

直接从需求估算功能点数，让不同项目的度量数据可比，不然无法建立公司基线。

5. 工具与平台

尽量用工具取代人手工作，例如，提供了从需求计算功能点数的模板工具等。

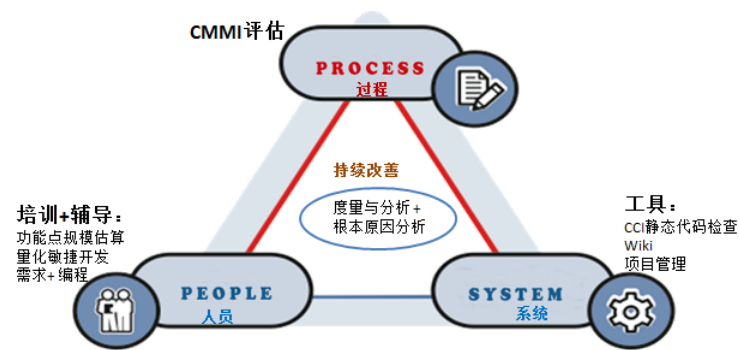
6. 高层的支持

高层与事业部领导的支持非常重要，可以想象如果他们不认同这些对企业长远的帮助，很难要求他们投入这么多人力资源在这个项目中。如果领导不关心，很可能评估过后便恢复到改进前的状态，不能持续。

附件

CMMI 评估案例

过程改进要有效，离不开 3 个方面的相互配合：培训 + 评估 + 自动化工具。



某咨询顾问的回顾：

我们提供 wiki 服务器，里面除了提供模型、Q&A 问答、视频等，帮助团队了解 CMMI 的最佳实践。并要求每个项目组按模型的每一条实践，写出是如何在项目组体现。要顺利通过 CMMI 评估，项目组不仅需要提供数据，也需要参加访谈，把项目的故事讲出来。

因 Wiki 服务器是个交互平台，公司每个人都可以随时访问，所以评估组和我可以随时了解到他们对 CMMI 实践的理解是否有错误。

项目经理与大家分享/汇报

客户完成了 CMMI V 评估后，举办项目团队评估后总结会，安排了其中的 3 个项目团队汇报他们在过去半年，如何利用所学到的方法达到 CMMI5 级评估要求。非常高兴地看到他们已经掌握了老师所培训的技能——利用模板从需求计算功能点，然后从功能点估算出项目工作量、测试用例数，再利用小工具测量代码质量，并且预估最终的缺陷密度等。

3 位项目经理每人都针对以上内容发表了 1 小时的演讲，分享实战心得。

项目团队把学到的东西用于项目中，配合度量数据，取得具体的质量提升，且达到了相应的 CMMI 评估要求。



这些项目经理都非常忙，每天可能工作到晚上 10 点/11 点；如果没有 CMMI 五级评估驱动，很难在短短几个月时间内达到这效果。其他项目组也因为听到有改进效果，产生兴趣专门抽空过来听。所以若要有改进效果，大家必须把评估看成是大家学习、提升的好机会。

Part V

如何做好迭代回顾

怎样避免同类问题的重复发生？因为有很多问题背后的原因是整个系统的不足，而不仅仅是个人不关注、不努力，做好根因分析可以帮助我们找出背后的根因。

怎样做好根因分析？二八原则、五个“为什么”（5Why）等，大家可能都听过。在网上也可能找到相关的理论知识。但很多人还是有误解，没有弄清楚应该如何利用这些技巧来找出根因。

针对软件开发的过程改进，也可以使用根因分析，帮助软件开发团队避免问题的重复发生，以提升团队的质量和效率。

迭代回顾 (Retrospective) 可以让团队每走一小步（一个冲刺），便立马回顾过程中有哪些不足、分析根因，以便在下一小步中得到改善。

要做好迭代回顾，除了要了解根因分析的技巧，还要了解回顾背后的原则，配合团队互动练习，再配合工具做好数据分析，才能使团队有实际的提升，让老板满意。

这部分会讨论：

- 根因分析的主要元素。
- 软件开发团队应针对哪方面入手，才能最容易取得效果。
- 迭代回顾前应如何做好准备。
- 内部教练引导团队做回顾时应注意什么。
- 如何可延续，让团队（甚至组织级）开始建立标杆。

Chapter 10

二八原则

团队成员协作，利用项目数据，分析根本原因，制定纠正措施，并立马尝试，判断是否有效，是改善的“基本功”。10-12 章会探索里面的注意事项，13 章会看两家公司的实施情况和常见问题。

如果已经获得高层支持，高层赞同目前返工工作量太多并愿意投入资源进行过程改进，我们首先应该针对哪些方面进行改进呢？

软件开发的特点是，超过 95% 的成本都是人力成本。因此，我一般会从二八 (20/80) 原则开始（附件里有二八原则简介），识别应针对什么做改进，才能最容易取得效果。软件开发里什么工作的占比最高？

请把过去一年的软件开发项目，按照不同工种占项目总工作量的比例，从多到少排个序。

1. 编码与代码设计
2. 交付后的所有工作，包括维护、更新与缺陷修正
3. 交付前的评审，静态扫描，测试与缺陷修正
4. 项目管理与监控

可以参考软件开发度量专家卡珀斯 • 琼斯 (Capers Jones) 先生在 2012 年的研究（详见附件中的“开发项目工作量 (成本) 分布”），看看你的选择与典型分布有多大差距。

软件开发项目最大的工作量通常是花在找出与修正缺陷上（开发里的“测试与评审”），Capers Jones 发现，对于一个超过 10,000 个功能点、计划使用 25 年的大型系统来说，有接近一半的工作量是与找出并修正缺陷相关。

很多项目中的缺陷大部分还是到后期才发现，这导致了大量返工。如果能提前发现并解决这些缺陷，就可以大大降低研发成本（不仅仅是提升产品质量）。

对于一个软件项目来说，发现缺陷最多的是在系统测试阶段，其次是在验收测试阶段（很多公司都是如此）：

: _20231023085822.png

但很多开发人员误以为编码是项目的主要工作，却忽视了大量因质量问题引起的返工。

前面在“获取高层支持”那章里缺陷越后面发现，返工量越高，如果是在前面评审或单元测试发现的缺陷返工量不会超过一小时，但到了系统测试或验收，返工量平均不会低于 20 人时。所以在软件开发，大量缺陷在后面测试或客户使用时才发现不单影响产品质量，也会导致大量返工。

只有当管理层意识到尽早发现并排除缺陷的重要性并引起重视，公司才有机会改进。

有些偏业务的领导可能会质疑，客户不一定关注缺陷密度，只要达到可接受的水平就可以了。

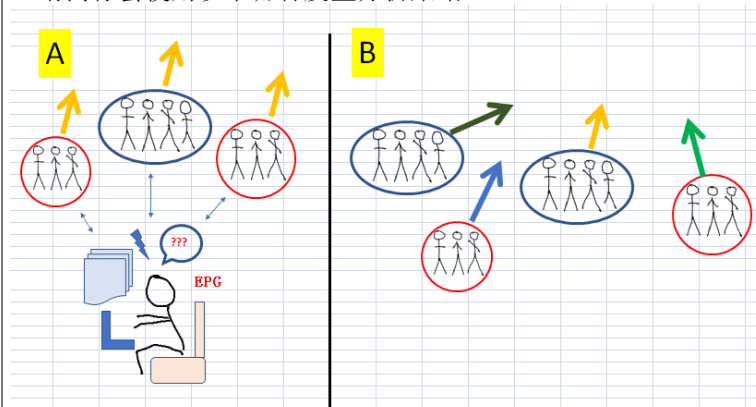
我解释：“在软件开发中，减少后期测试才发现的缺陷，不仅仅能提升产品质量，更重要的是能降低研发的总工作量，所以最终能帮助公司省钱，减少开发人员加班。”（详见附件“公司高层不一定关注质量”）

怎样开始

没有度量便无法谈改进，所以应先考虑如何采集修复缺陷相关的数据。然而，在探索数据收集与分析之前，我们更需要先确定度量分析的策略。

但在探索数据收集与分析之前，我们更需要先想好度量分析的策略。应怎样做？我与质量部经理张工和项目经理讨论了上面工作量排序和那阶段发现缺陷数最多后，现场问他们以下问题：

”” 请问你会使用以下哪种度量分析策略？



A：由公司的过程改进组（EPG）专员设计并执行过程改进的度量计划，包括收集哪些数据、如何收集以及如何分析。然后按计划收集各项目团队的数据，进行总体分析，并制定公司的改进方案。

B：将数据收集与分析的任务下放到各团队，让团队制定自己的改进方案。

项目经理：我认为度量与分析需要专业知识，要由专人来做，我选 A。

我问：但提供数据的人（图中 EPG）需要很长时间才能收到反馈。例如，从团

队成员提交里程碑数据，到项目经理填报，再由度量分析人员收集、整理多个项目的数据，可能需要一到两个月时间。再经过数据分析，管理层评审，最终到团队成员收到反馈可能共两三个月。由于每位提供数据者都希望尽快得到反馈，如果反馈的时间很长，那么团队成员还会有动力继续收集数据吗？

项目经理：估计确实可能会导致缺乏动力。

我：无法确保数据的准确性，会导致分析没有意义。A 场景很难确保数据的正确性。但 B，因团队自己分析数据，会好些。

质量经理：从去年开始我们进行度量分析以来，就发现员工会对数据造假，导致结果失真。度量什么，他们就造假什么，所以我们想通过工具代替人工，不仅能提升效率，还能帮助判断数据是否合理。现在我们的主管很反感度量，因为每次度量就会有人造假——大家了解算法原理后，就开始造假。比如我们搞了红黑榜，但很多人聪明得很，就会想办法作弊。我们有很多数据统计分析，比如看测试用例与需求的比例。尽管客户发现的缺陷比例降低了，但由于我们对质量特别注重，产品经过多年的逐步演化，过程很复杂，导致软件缺陷的修复很耗时，客户不太满意。而且公司要求交付的频率要比以前高了很多，我们团队做这些分析都忙不过来，所以希望利用自动化工具加快速度。

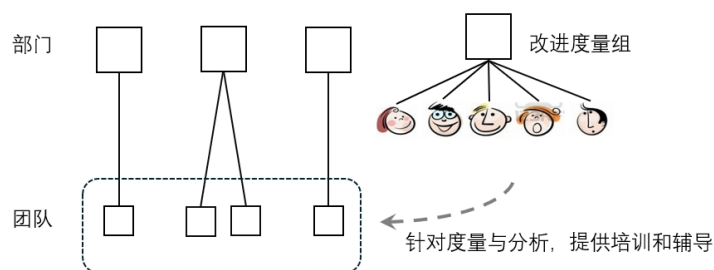
我：确实有不少管理层误解了独立的用途，当度量数据被用于公司考核指标，便里面出现你说的情况。度量分析人员需要什么数据，提供数据的人都会按理想的指标填写，反正公司级的总体分析人员也无法验证数据的准确性。

除了数据收集问题，你们这种总体分析策略可能还有另一个问题：没有考虑各个项目的特性并不一样。现在你们对几十个项目的总体趋势进行分析，但每个项目的差异可能很大，根本原因很可能不同。例如，你上次展示过一些测试数据，同样是测试用例比例数，你们采集的数据范围从最低的 0.3 到最高的超过 200，但你们取平均值 5.1 来做分析。所以自动化工具不一定能帮到你。

但如果把数据分析下放到团队自己做，便灵活多了；也正因为他们有参与收集和分析讨论，过程改进组便可以节省很多沟通（数据收集）成本与失真。反而应该重新定位自己是内部老师，辅导团队怎么做好数据分析，效果会更好。

项目经理问：团队自己讨论便可以得到改进吗？不需要领导监督？

我：如果团队有能力，就应该放手让他们自己收集数据、利用数据进行分析并做出改进。但大部分团队还是没有做过度量分析，没有概念。所以质量经理应把自己重新定位为团队的教练，去辅导他们如何收集与分析数据，如下图的分工：右面的内部教练熟识各种度量分析技巧，辅导左面团队做改进。千万不要以为这项工作会比以前自己做分析更轻松，其实需要更熟悉整个过程，才能真正辅导好团队。要公司从定性管理提升到定量管理，最大的挑战是如何让项目团队动起来，所以应要让管理层了解并赞同需要让团队自主分析数据，自己寻找纠正措施。最终也要团队自己定目标，定计划，定期与上层汇报改进效果。例如，当你要健身锻炼减肥，也请了教练，最终因自己没有恒心，最终没有效果不应该把责任推卸到教练一样道理。

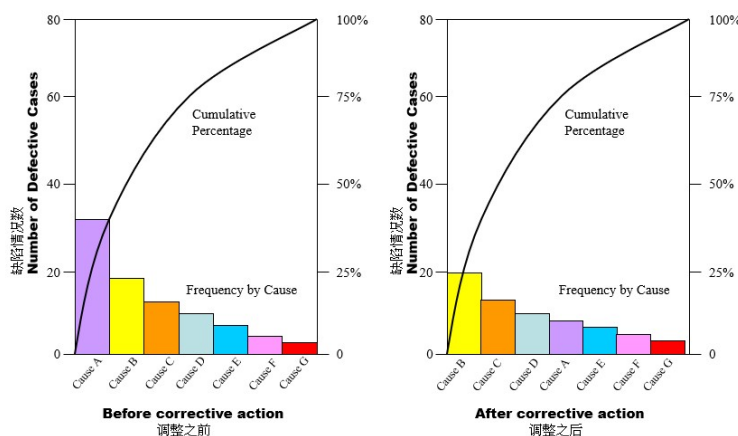


下一章会继续探索团队协作与分析根因：团队如何利用迭代冲刺数据，在敏捷冲刺回顾，做好根因分析，得到提升。

附件

二八原则 (80/20)

帕里托 (Pareto)，16 世纪意大利威尼斯人，他发现虽然威尼斯很富有，但财富并非平均分布，80% 财富在 20% 手里，他也发现很多其他分布都非平均。

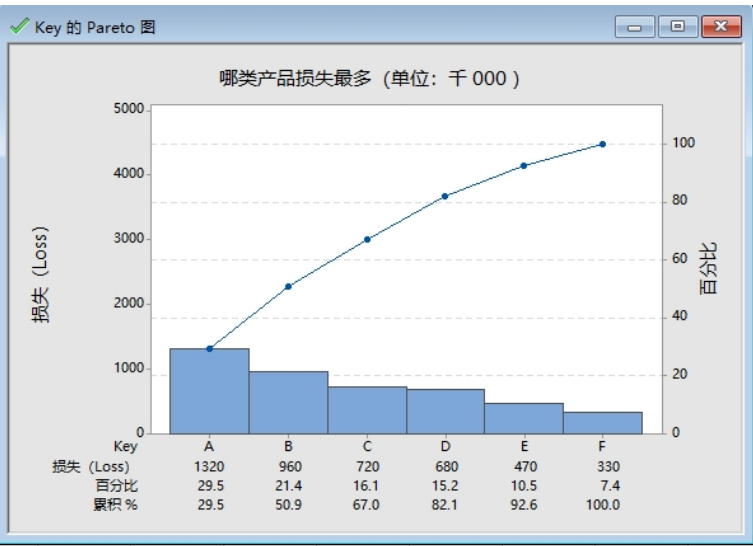


因为过程改进需要公司额外的资源投入，必须有针对性地找出最容易取得改进效果的原因，才可能拥有最大的成功机会。所以，应该利用二八原则去识别影响最大的几个因素。以上图为例，原本 A 类问题出现最多，针对这类问题做过程改进之后，A 类问题就少了很多，下一轮便应针对最多的 B 类问题做改进。有些人以为，只需要对某个维度做分析即可，并没有从多个维度进行分析。例如，某纸制产品工厂会计数据显示，8 成的产品问题相关成本都归属于 5 类。例如，某纸制产品工厂会计数据显示，因纸质量问题 (Broke) 被客户退货类的损失最大，共 5560 千美元，占总损失的 61%。（其他损失类包括材料成本高、生产线停顿等。）针对这类损失，发现里面八成是由 6 个产品引起（共有 53 个产品），共 4480 千美元，占 5560 的 80%。针对这 6 个产品把损失按缺陷种类细分，发现 B 产品容易断裂缺陷 (Tear) 的占比最高，612 千美元，我们就应该

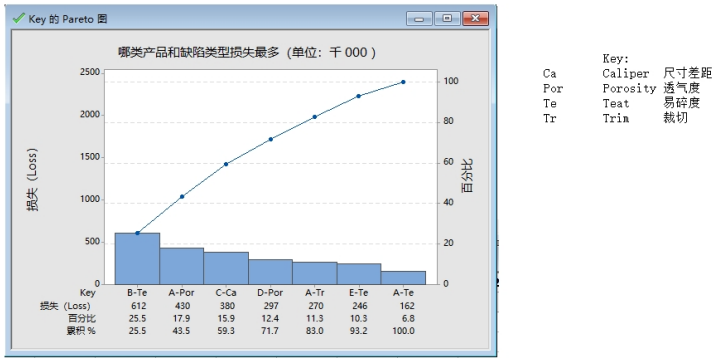
针对这一问题研究如何改善。

质量成本(千美元)							
产品 类型	Trim 裁切	Visual defects 视觉缺陷	Caliper 尺寸	Tear 撕碎	Porosity 透气度	Other 其他	Total 总计
A	270	94	-	162	430	364	1320
B	120	33	-	612	58	137	960
C	95	78	380	31	74	62	720
D	82	103	-	90	297	108	680
E	54	108	-	246	-	62	470
F	51	49	39	16	33	142	330
总计:							4480

依据以上数据，可以按产品画帕累托图：



也可以按产品和缺陷类画帕累托图：



帕累托图除了能帮我们分析数据外，也能用于与高层沟通，让他们一目了然更

好理解问题的主要源头，希望得到高层的支持。

开发项目工作量（成本）分布

参考 Capers Jones 先生 2012 的例子，汇总成以下比例：

	30%	25%	20%	15%	10%
66%	测试与评审 100 (26%)	编码 83 (22%)	文档 66 (17%)	开会沟通 50 (13%)	项目管理 33 (8%)
13%	发布后修改缺陷 20 (5%)				
21%	维护 32 (8%)				

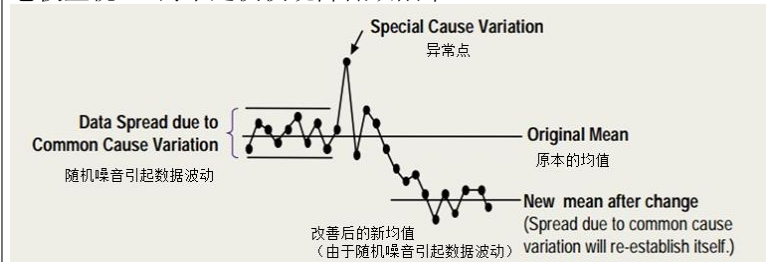
测试与评审一般占开发工作量的 30%，测试与评审一般占总质量成本（包括发布后维护与改缺陷工作）的 66%，把所有工作量都加起来，测试与评审还是占最大 (26%)，编码第二 (22%)。

注意：测试与评审包括所有与缺陷相关的成本，包括单元测试、静态扫描、评审与相关的缺陷修正，而编码只包括设计与编写代码部分。例如有些人会觉得比例应该是开发 30%，测试和 Bug 修改 25%，需求和设计 20%，项目管理和沟通 20%，文档 5%。但如果按上面的定义，开发部分很可能已包括单元测试、静态扫描与改正缺陷工作，如把这些都归到测试评审里会变回类似上图的比例。

公司高层不一定关注质量

问：在传统 IT 的公司，无论是缺陷 (Bug) 率还是其他质量指标对业务的影响的相关性都不容易度量，只有重大事件才会让公司在客户面前失去信任，所以高层不会真正的重视质量。

答：理解，但软件 BUG 的暴露越往后，返工的工作量就越高，而且是几何级数的增加（例如在单元测试或评审发现都可以 1 人时内解决，在系统测试时发现通常要花 20 人时，在客户使用后才发现便更高）。但在很多 IT 公司，大部分缺陷都是在系统测试、甚至到验收测试才发现。如果能把软件缺陷的发现前移，把通过系统测试、验收测试发现的缺陷减半，便可以大量降低质量成本，提升开发生产率。所以我建议利用缺陷数估算返工的工作量来引起老板重视。（而不是仅仅说降低缺陷率）



Chapter 11

团队协作分析根因

公司主要为全国不同的医院开发医疗行业的软件产品。六年前，公司规模还不到五百人；四年前，公司办公地点扩大了一倍，占地 3000 平方米，总人数也增加到接近 900 人。尽管公司骨干比较稳定，但新人的流动性很大。公司制定了很多量化指标，如需求一次性通过率、开发速度、测试缺陷密度等，并不断监控，但并未取得明显改善。

技术总监问：“请教一个一直困扰我的问题——如何激励团队，让他们更有积极性？”

“现在你们员工的收入底薪和奖金的比例大概是多少？”

总监：“大概七成底薪，其他的就按每人每年的表现分发。”

“主要依赖主管的打分吗？我完全可以理解为什么是这个结果了。因为在这种安排下，员工的收入既不会太差，但也不会特别高，所以他们自然缺乏动力去改进自己的工作表现。”

总监解释：“也不完全依赖主管的主观判断。我们制定了很多与绩效相关的度量指标，以根据这些指标客观地计算员工的奖金多少。”

“理解了，你觉得这样便能客观吗？一旦把度量与绩效/奖金挂钩，就会导致收集不到真实的数据。因为只要数据与收入有关联，高层需要什么数据，下面都会提供，这完全失去了度量的本意。收集度量数据本来希望像镜子一样，让团队可以看到现在的真正水平。团队成员相互合作非常重要，所以日本公司通常不会发奖金给某个人，而是发给团队。团队的奖金应该与团队为公司提供的价值（换算成金额）挂钩，这样才客观透明。大家可以预估到如果团队有了提升，能为公司赚多少钱，自己收入大概会增加多少。除了金钱上面的奖励，也不要忽略非金钱的赞赏，例如，定期让项目团队分享改进成功的故事，也可以帮助团队持续改善。你们每个开发团队里面都有明确的分工：需求、开发、测试等都有相关的度量指标。但岗位之间缺乏交流，导致团队的整体质量和效率没有显著提升，例如团队的缺陷密度和返工率一直没有明显下降。”

接下来，我讲述了 60 年代美国费城的 Weisbord 先生针对印刷公司订单管理部使用 XY 理论改革团队的故事：如何从每一步骤都要按小主管 (supervisor) 安排的传统管理模式 (X 型管理)，变为由 4-5 人自我管理的独立团队 (Y 型管理)，最终把每天处理订单量提升 50% 的成功改革故事。（详见第 2 章故事）

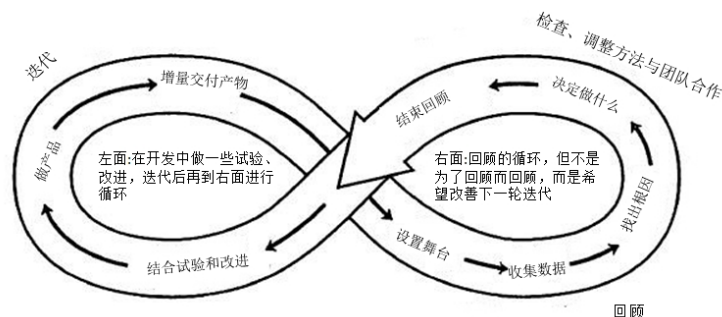
总监说：“听完这个故事，确实感觉我们团队的管理模式类似于 X 型管理。虽然我们团队每两周进行一次迭代，但没有真正做到团队合作。测试人员只负责测试，开发人员只写程序，然后交给测试人员去测试。因此，从这个角度看，或许每个过程的指标，如测试和开发都很好，但团队的整体能力没有任何提升。”“从这次观察来看，每个迭代中，系统测试的缺陷数保持在 40-60 个左右，六个月前是这个水平，现在依然如此，现在还是这个水平，没有任何进步。为什么？因为大家缺乏团队协作和互补的团队精神，导致没有改进的动力。各岗位只顾管好自己，确保指标达标，不被扣绩效就行。”因为我也对团队缺乏具体改进感到失望，所以不管总监是否高兴，我直言不讳地指出问题。当敏捷开发团队建立起协作互补的基础后，就可以开始使用根因分析。

回顾流程

回顾流程冲刺可以按以下五个步骤进行，确保每位成员都全心投入参与，并通过分析形成行动，达到改进效果：

设置舞台——“热身”，获取对游戏规则共识

1. 收集数据（除了收集“硬”数据，如缺陷、工作量、进度偏差等，也要收集“软”数据，如团员感受）
2. 分析，找出根因
3. 决定做什么（必须明确下个迭代的具体改进行动到人、任务、时间）
4. 结束



冲刺回顾本应是团队分析根因在下一轮冲刺改进的最佳时机，但很多团队不了解改正 (Correction) 和纠正措施 (Corrective Action) 的区别（详见之前第 9 章），团队在回顾时只讨论改正，导致以前发生过的问题还是会再发生。

太空穿梭机（航天飞机）灾难（Space Shuttle disasters）1986: 挑战者号（Challenger）航天飞机升空后，于发射后的第 73 秒爆炸，机上 7 名宇航员全部罹难（相关视频可以在网上搜到）。直接原因是用来密封火箭液态氢气燃料缸的 O 型橡胶环在摄氏零度低温不能及时膨胀。但这事故并非偶然。升空前，供应商已经警告过低温会影响橡胶的性能；以前的实验也发生过橡胶环有些部分半径只能达到预期的 2/3。但管理层为了按计划预期升空都漠视这些安全隐患预警。

2003: 哥伦比亚号（Columbia）航天飞机在返回地球时，因为左翼的隔热保护胶在 16 天前升空时被固体火箭助推器外层脱落的乳胶击破损坏，导致航天飞机返航时进入大气层的第 1000 秒钟，在 35,000 米高空因过热解体，7 名宇航员全部罹难。

Ride 在 2003 哥伦比亚号灾难事故调查回顾说：我很诧异这两次事故的原因非常相似（她也是挑战者号事故调查组员），86 年引起挑战者灾难的陋习又再出现：为了赶上计划升空日期，而忽略了之前性能报告的发现，没有注意前线技术工程人员提出的安全隐患，也缺乏安全保障措施。其实从哥伦比亚号在 1981 年首次升空开始，每次都有出现升空时燃料缸外的保护乳胶掉落的故事，但管理层一直都没有关注，工程师因为赶进度，每次升空前，都没有时间预先处理好。

虽然调查报告有找出事故的根因，但过了几年后，管理层只关注项目的成本与进度，忽略了质量与安全，没有再注意调研报告指出隐患，事故还是重复发生。

因为发生了这两次致命事故，航天飞机计划最终被叫停。

听完这故事，你可能会反驳说：“NASA 资源丰富，内部不缺经验丰富的工程师，也未能做好根因分析，避免同类问题再发生，敏捷团队未能在回顾时未能做好根因分析，不就是理所当然了吗？”

“要做好 PDCA 背后就要分析根本原因，讨论纠正措施。其实不需要高深的统计分析，只须使用二八原则识别改进方向，然后大家头脑风暴，比如使用鱼骨图记录主要根因，讨论纠正措施，制定计划，最后判断是否得到显著提升就可以了。你觉得开发团队在能否回顾时用上？”

“听你这样说应该不难。”

“其实甚至一家日本俱乐部里由 7 位年轻女服务员组成的 QC 圈在 84 年已经能做到。如果不信，可以发你相关分享文章。”

根因分析 (Causal Analysis & Resolution CAR) 的主要步骤：

1. 选择要分析的结果, 利用二八原则识别引起大部分问题的最主要因素
2. 分析背后的主要根因
3. 对应改进措施 (改进建议)
4. 评价改进效果
5. 总结成根因分析报告 (A3 报告)

可参考附件 A:“绅士俱乐部过程改进” 了解团队如何按以上步骤做过程改进。

但不要误以为只要使用各种根因分析方法，如帕累托图、鱼骨图等，就能做好根因分析。以上的根因分析都是以定性讨论为止，没有利用相关数据做分析。没

有数据，就无法谈改进，要做好根本原因分析也必须利用数据。如收集了缺陷与返工工作量数据，并利用二八原则便能找出应该针对哪方面入手，而不是指靠个人的主观判断和经验，直接制定改进措施。

应收集哪些数据？数据不是越多越好，因为收集数据花精力，所以是反过来越少越好。应针对收集哪些数据呢？就应考虑改进目标，例如按上章想降低返工，便应问以下问题：

业务目标	信息需要（问题）	度量目标	度量类型	基本度量项实例	衍生度量项实例
不影响成本的前提下交付给客户的产品的缺陷降低10%	在交付之前缺陷在哪里注入以及哪里被发现？	在整个产品生命周期中，评价缺陷检测的有效性	质量	按生命周期阶段注入及发现的缺陷个数 产品规模	生命周期阶段的缺陷检出比率 缺陷密度
	返工成本有多少？	确定修复缺陷的成本	成本	按生命周期阶段注入及发现的缺陷个数 修复缺陷花费的工时 劳动力成本单价	返工成本

除了每个过程的返工工作量，也要有缺陷排除率。

缺陷排除率

从需求到设计、编码、单元测试、系统测试、验收，整个开发过程大家都很熟悉，需求、设计、编码后可以通过评审来排除缺陷，但仅仅做评审不一定能确保质量。因为最终验收缺陷数取决于每个步骤能否有效排除当前的缺陷。下面我们介绍一下怎么从引进量化管理来提高开发质量：

设定量化目标。例如希望最终遗漏到验收发现的缺陷数降多少？

- 1. 并设定中间阶段目标缺陷数，从而预测能否达到最终目标，不要等到最后才知道不满足。

可用缺陷排除率 (Defect Removal Efficiency DRE) 来判断每一个过程的质量：

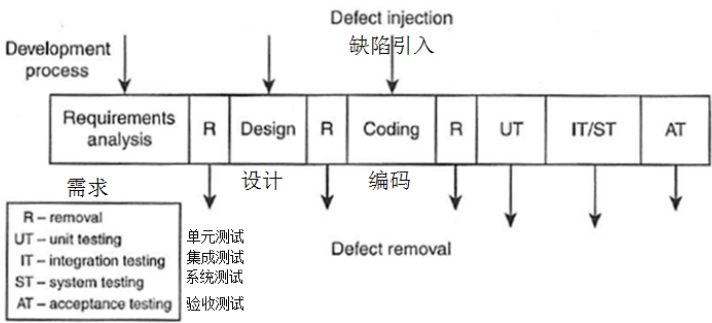


Figure 7.1 Defect injection and removal

$$DRE = \frac{\text{Defects found by the QC activity 缺陷排除数}}{\text{Total defects in the product before QC 缺陷总数}}$$

如果要最终质量好，缺陷排除率就要高。但计算缺陷排除率，必须要等到整个开发完成才可以计算。如果团队收集并分析本迭代的缺陷，如下：

	Req需求	Design设计	Coding编码
RR	5		
DR	5	5	
CR	8	5	10
UT		6	26
IT+ST		5	17
AT	2	1	5
总数:	20	22	58

然后使用 DRE 方程式计算各缺陷排除率:

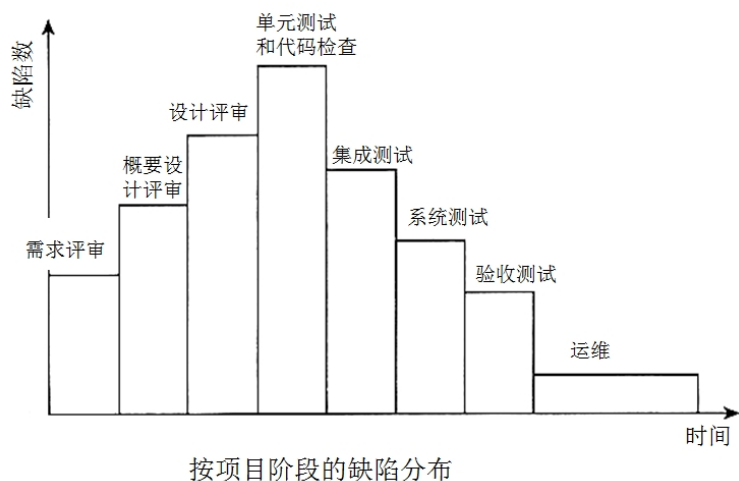
例如: 需求评审发现 5 个缺陷, 但当时共有 20 个从需求引发的缺陷, 所以 DRE 是 25%。

设计评审共发现 10 个缺陷, 但当时共有 20 个需求引发, 加上 22 个设计引发的缺陷, 减除 5 个已经在需求评审处理的缺陷, 所以 DRE 是 27%。

需求评审	RR DRE =	5/20						DRE	
								25.00%	
设计评审	DR DRE =	(5+5) / (20+22-5)						27.03%	
代码评审	CR DRE =	(8+5+10) / (100-5-5-5-5)						27.06%	
单元测试	UT DRE =	(6+26) / (100-5-5-5-8-5-10)						51.61%	
系统测试	ST DRE =	(5+17) / (100-5-5-5-8-5-10-6-26)						73.33%	
验收测试	AT DRE =	(2+1+5) / (100-5-5-5-8-5-10-6-26-5-17)						100.00%	

如果我们按上面各阶段缺陷利率百分比, 设计与需求排除缺陷 15%, 代码评审与单元测试 55%, 集成/系统测试 22%, 验收测试 8%, 下面按缺陷总数为 100, 得出算出设计与需求排除共 15 缺陷, DR+RR=15, CR+UT=55, IT+ST=22, AT=8, 求出各阶段的缺陷输入与缺陷排除率, 把参数输入预测模型 (XLS 模板)。

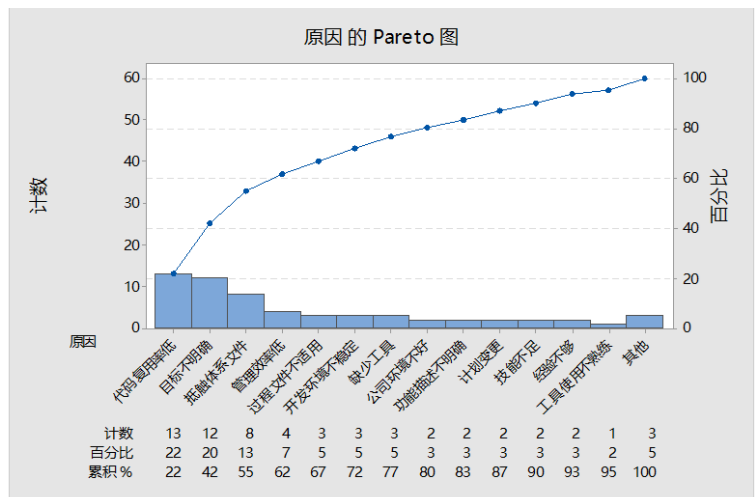
得出下面的表, 缺陷分布是中间最高头尾低, 右面与左面不同, 有条长尾巴, 类似估算软件开发项目工作量的 Rayleigh 曲线。



由于不同项目甚至不同迭代冲刺的缺陷数和缺陷率各异，难以进行比较，但如果方法和团队人员相似，缺陷排除率应该是可比的。通过分析以往的缺陷数据，针对最薄弱的环节讨论根本原因，并在下一个迭代中进行改进，提升缺陷排除率，并可以预估各个阶段（如评审、测试等）的缺陷发现数。例如，如果需求评审的缺陷数远低于预期，就应立即进行纠正，而不是像以前那样仅仅提醒大家要重视评审工作。

根因分析的误解

请千万不要误以为只要使用了根因分析的方法（例如：二八原则）就能做好，很多团队在开始时对其理解可能只是一知半解，需要不断纠正和改进。某公司过程改进组（共 4 人）对过去一年的项目进行了根因分析，旨在改进交付质量，减少交付验收时的缺陷数。



我：请问这

帕累托图的依据？

组长：针对客户缺陷密度较高的现状，我们做了敏感度分析。发现需求引入的缺陷数跟它相关性最高，接下来我们使用鱼骨图分析，请 4 个人一起头脑风暴，共识别出十几项主要原因种类。然后按每人 3 票的原则，各自单独投票得出排序。最后根据二八原则，我们识别出前 3 项主要原因，接着，我们根据这 3 项原因又进一步发现，这些原因是导致需求缺陷的原因，包括流程图画不好等。

香港启德国际机场 1996 年航班延误统计：

Cause 主因	Freq次数
Late baggage to aircraft. 运输行李到飞机有延误	340
Bad weather. 恶劣的天气	240
Too few gate agents. 登机口工作人员太少	66
Late cabin cleaners. 清洁工打扫机舱晚到	21
Poor announcement of departures. 出发通知混乱	12
Gate agents not motivated. 登机口工作人员没有动力, 影响工作	103
Late aircraft arrival to gate. 飞机晚到	68
Late or unavailable cabin crews. 空乘(飞机服务)人员迟到或因故不能上岗	21
Passengers bypass check-in counter. 旅客绕过出登机牌柜台	62
Late passenger cutoff too close to departure time. 最晚乘客登机时间太接近起飞	44
Late food services. 飞机餐饮服务晚到	37
Gate agents undertrained. 登机口地勤人员训练不足	116
Desire to "protect" late passengers 希望“保护”迟到的乘客	57
Late issuance of boarding passes. 打印/发放登机牌发生延误	33
Late weight and balance sheet. 飞机重量平衡表(为了确保飞机重量平衡, 保证飞机安全) 迟交	40
Late or unavailable cockpit crews 驾驶舱人员迟到或因故不能到岗	19
Confused seat selection. 座位安排混乱	66
Late fuel. 燃料延误	35
Gate agents arrive late at gate 登机口工作人员晚到登机口	19
Late pushback tug. 拖车延误	75
Checking oversized luggage. 检查超大行李	20
Desire to help company's income. 希望帮助航空公司增加收入	37
Air traffic control delays. 航空交通管制导致延误	166
Poor gate location. 登机口的位置不好	130
Mechanical failures. 机械故障	26
Delayed check-in procedure. 登机牌柜台手续 / 过程发生延误	31
Gate occupied by a late departing plane. 停机位被前面晚点起飞的飞机占据了	22

然后我解释:

我们不应直接把几十条原因按他们对问题的影响度从最高排到最低做帕累托图, 而应先对原因进行分类。以机场延误为例, 我们需要区分哪些是不可抗力的天然原因(如天气原因), 哪些是与闸口管理相关的问题, 哪些是与机场打印登机牌相关的问题。例如, 如果识别出与闸口管理相关的问题最多, 后续就可以针对这些问题进一步分析根因。分析才能有针对性 and 意义。

你们的分类和帕累托图依据的“数据”都不过是用数字来表达你们头脑风暴的主观判断, 所以这种帕累托图其实与直接用头脑风暴做出的结论没什么差异。你们只是用投票数据使结果看起来“量化”了, 但其实并没有客观数据支撑。你们的分类存在问题。例如只有“功能描述不明确”, 难道“非功能需求描述不明确”(如性能、安全性等)就不需要考虑吗?

你们现在的分类有不少是重复的, 同样一个问题有可能归属于 2、3 个分类。你们的度量操作定义也存在问题。例如, 如何根据历史项目的数据判断哪些原因归属于“功能描述不明确”?

建议你们应该首先识别出与想要解决的问题相关的度量项, 针对这些度量项收集数据, 然后再做分析。收集客观数据才是根因分析的第一步。

+++++

我: 你们现在的找出的其实并非根因, 只是浮在水面上的现象, 你们有听过 5W 吗?

组长: 有, What When ..

我: 不, 你说的是 5W+1H, 5W 是五个为什么 (Why), 你们应该不断追问“为什么”, 才能更好地识别出背后的主因, 并针对主因采取纠正措施, 避免同类问

题重复发生。
接着，我向他们讲述了丰田汽车大野耐一先生的 5W 故事（详见第一章附件“5Why 例子”）。
除了通过问“为什么”之外，还可以通过画整个流程，识别有哪些失效点。
但 5W 只算是定性分析，简单介绍几种依据数据的量化根因分析方法：

- 分析全过程 (Process) 例如，用 FMEA 分析搭不上飞机的根因为例子（详见附件），先画出整个过程，如果是软件开发，可以从需求开始到最终交付的全流程，然后按历史数据，分析哪个环节的影响交付的质量，并估计影响度、发生概率等参数，针对高风险的“失效”分析根因
- 也可以从时间点的角度，使用控制图，从时间顺序查看有没有异常点。后面 24 章会详细介绍控制图

软件开发绝大部分靠人来完成，也可以从“人”的维度分析，例如：

- 下面是 22 位来福枪组装工人从 2 月到 7 月的缺陷数统计，他们每人负责完整安装一支枪：

（缺陷：试验后无法打开，取出空弹壳。导致必须从组件重新安装。）

组装工人	3月	4月	5月	6月	7月	8月	总计
1	4	1	0	0	0	0	5
2	1	2	0	5	1	0	9
3	3	1	0	3	0	3	10
4	1	1	0	2	2	4	10
5	0	1	0	10	2	1	14
6	2	1	0	2	2	15	22
.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.
17	18	8	3	37	9	23	98
18	16	17	0	22	36	11	102
19	27	13	4	62	4	14	124
20	6	5	2	61	22	29	125
21	39	10	2	45	20	14	130
22	26	17	4	75	31	35	188
总计	234	146	34	496	239	241	1390
%缺陷百分比	10.6	6.6	1.8	22.6	10.9	11	10.5
最好的5位	9	6	0	20	5	8	48
最差的5位	114	62	12	265	113	103	669
比率	13	10	∞	13	23	13	14

发现每月的缺陷率差异很大（从 1.8 到 22.6）；但发现 6 个月里最差 5 位与最佳 5 位缺陷数的比例差不多。所以不是个别人员能力的差异，而应是有原因。

观察最佳和最差工人的工作方法发现质量好的工人都会用金属打磨器对负责的逐渐打磨。最差的工人没有这样做。

如想多了解应如何分析与对应各类与人员相关的缺陷，请看除了要附件里“如何分析与对应各类由于人员引起的缺陷”。

当敏捷开发团队除了要用各种方法分析根因时，还应考虑让各团队成员全心投入参与。只有这样，过程改进才能持久。下章会举例说明。

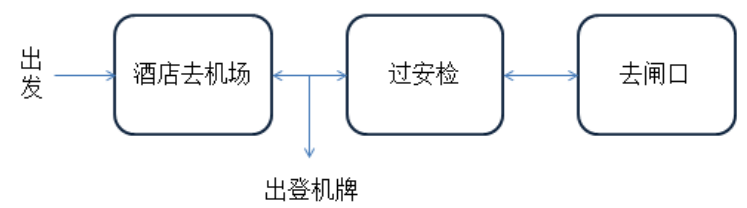
附件

FMEA 实例

(FMEA: Failure Mode Effects Analysis)

大家可能都经历过因为没有管理好时间而导致迟到的情况。以坐飞机为例，从离开酒店到登上飞机的过程中，很可能因为一些风险导致最后没搭上飞机。当你出发时，可能选择不同的交通方式，如出租车或机场大巴。如果你不能在起飞前 45 分钟到达机场办理登机牌，你就无法登机。但即使你拿到了登机牌，也有可能最后没能搭上飞机，因为机组严格执行起飞前 15 分钟关舱门的规定，所以你还需要在起飞前 15 分钟到达登机口。

图 1 登机过程



要赶上飞机其实是个过程，中间有很多环节会导致最后失败，所以我们可以通 FMEA 过程分析来看如何减少失败的概率。

图 2 FMEA 例子

Process / Product Failure Mode and Effect Analysis															
Process or Product Name		1		Process Responsibility		1		FMEA Number		Page 1 of 1		Prepared By:		FMEA Date (Orig.) (Rev.)	
Model Year(s)/Vehicle(s):		Key Date:													
Core Team:															
Process Step	Potential Failure Mode	Potential Effect(s) of Failure	Severity	Potential Cause(s)	Occurrence	Current Process Controls	Detection	RPN	Recommended Action(s)	Responsibility & Target Completion Date	Action Results				
											Actions Taken	Score	Occurrence	DRPN	
2	4	5	8	6	9	7	10	11	12	13	14	15			
酒店去机场	赶不上45分钟前柜台取登机牌	搭不上飞机	10	堵车	6	出发前问酒店前台预计时间	6	360	出发前（网上地图）查路况 计时间（因前台可能随意说	1/ 统计时间 2/ 领早上网查					
过安检	赶不上15分钟前到达闸口	搭不上飞机	10	寄存行李要开包检查 安检太久	2 8	出登机牌后，等3分钟 自己看手表控制时间	8 4	160 320	必须起飞前35分过安检 个人控制制放在一固定地方（某一口袋）	1统计时间 2/ 换数字手表					
				登机牌错误，票从打	2	自己替好	8	160							
去闸口登机				安检后去闸口很远	6	如熟悉机场，熟悉路线	2	120	预先问机场地面人员，估计所需时间						

图 3 打分参考

		Severity	Occurance	Detection
高 10 ↑ 低 1	危险没有警告	非常高的	几乎不可避免的	不能检测到或非常低的检测概率
	丧失主要功能	很高，反复失败		远程或低检测概率
	丧失次要功能	中等的失败		低检测概率
	小缺陷	偶尔的失败		中等检测概率
	没影响	极低		必然的概率

以第一个失效为例：如果发生就会坐不上飞机，所以严重性最高，评分为 10。发生的概率比较高，评分为 6。是否容易预防、预警？由于不熟悉当地情况，加上主要依靠酒店前台提供信息，而有时他们也不清楚，所以评分为 6。RPN = 10×6×6=360。

从以上登机的例子可以看出，FMEA 是以整个过程进行风险管理的。比如在出发前，就要查询一下各个交通工具要花费的时间，例如，在出发前，需要查询各个交通工具所需的时间。在南京，从酒店坐地铁需要转车，要花费一个多小时以上，如果时间紧张就会来不及，那么宁可多花钱打车，以控制风险。

即使拿到了登机牌，还需要经过安检并到达登机口。有时机场很大，到达登机口也要花很长时间，因此需要提前问好路径并计算好时间，避免误点。

如果从安检到登机口需要超过 10 分钟，那么就需要在起飞前 35 分钟通过安检，才能来得及。这些都可以通过 FMEA 的形式把整个坐飞机的过程识别出来，找出各个阶段可能出现的问题，从而知道如何控制。

以这个坐飞机的风险为例，我自己就多次没赶上飞机。原因很多，但总结一下，都是因为习惯没有改过来。我应依据以往差点赶不上的经验，回顾一下，确保每一个过程都控制住，就不会后面再次出问题，这个和企业做风险管理概念一样。有度量才有管理。人和公司一样，很多事情看似是自己主动去想的，其实很多都是潜意识的习惯。如果没有设定一些量化的控制手段，就不会提高这方面的风险意识，还是会错过飞机。因此，我通过回顾以往几次搭飞机的情况，把每次到达机场的时间和到达登机口的时间记录下来（详见下面图 4）。

图 4 上面是机场柜台关闭（45 分钟）前到达时间，下面是关闸口前到达时间（分钟）（X 代表赶不上飞机）

航班	NJ-SJZ	HKG-CD	CD-SJZ	SJZ-CD	CD-BJ
到机场	x	12	10	6	10
到闸口		1	2	1	x

最近一次赶不上飞机是因为未能在关闸门前到达，而之前也有两次类似情况——我刚到闸口，就到时间马上关闸了。

如果我把经验教训记下来，下次做好时间管理，就会避免后面的延误：有了数据，我们便可以更有效在回顾时做好根因分析 (CAR)。

以这登记延误风险为例，可以使用 FMEA 分析每个失效点，例如过安检（因没有预留足够时间）和从安检到闸口所需时间太长等的发生概率都很高（前者为 8，后者为 6）。

为了避免问题再发生，就要定一些具体的计划，最终希望把误点减到零。

在登机这个环节，可以利用以下有效方法/工具来帮助改善：

- * 每次出发去机场前都查询各种交通方式的时间与风险（概率），预留足够时间，降低失效发生的概率
 - * 拿到登机牌后，计划好在起飞前 35 分钟通过安检
- 在多次错过飞机后，我发现传统手表没有正负 5 分钟的准确性，而使用电子手表可以精确到正负 1 分钟，这有助于更好地把控时间。

图 5 平常用于培训/评估计时的电子钟



经过这次误机，我就买了个电子手表，取代传统针式手表，希望对日后不迟到有帮助。

效果：这件事发生在 2019 年 9 月，之后我按这些计划行动，再没有出现赶不上飞机的情况。我每次都提醒自己必须在起飞前 60 ~ 90 分钟到机场，30 ~ 45 分钟前到闸口。分析的目的是提高个人风险意识，避免问题发生。

如何分析与对应各类由于人员引起的缺陷

- 可大致分为 4 类跟人相关的缺陷和对应措施

类	分析后发现缺陷模式	可能的根因	解决办法
1	某些缺陷，发生模式是随机的，不容易看出哪类缺陷经常发生。	错误是由于疏忽造成的。	完善过程，防止人为错误。
2	某些缺陷，有些人员经常犯错，但另一些人员总是“很好”。	错误是由于缺乏技巧造成（能力、知识、技能等）。例如某些窍门只有部分人知道。	挖掘和推广窍门，避免出现小众才知道的秘密。
3	有些人员总是在各种各样的缺陷上容易犯错。	潜在的原因：故意不遵守标准。天生不能完成这类任务。缺乏培训。	激励措施，换人，提供培训。
4	某些缺陷，所有工人都容易犯错。	缺陷是应该由管理层控制。	语言标准化：提供翻译，词汇表。达到自我控制标准。

- 本文里的来福枪故事是上面第 3 类的标准例子
- 例如，下图统计了 6 位技术人员，按 29 种缺陷统计：

文件:Juran t5.8.jpg

分析：

- * 虽然第 3 类缺陷出现共 19 次，最多，但大部分是源自员工乙，但他除了犯第三类缺陷外，他其他方面的缺陷数很少。很可能是他误解了里面某些关键过程（上表的第二类）
- 第 5 类缺陷属于上表的第 4 类，所有员工都有犯错
- 员工戊的缺陷数最多，占了缺陷总数接近一半，是上表 2 类的例子

Chapter 12

软硬兼施

某次量化根因分析培训里，我跟各现场同学说：

“前面我们说可以用鱼骨图分析根本原因，但我不建议在迭代回顾时用它找根因。知道是什么原因吗？”

请问你是读那所高校？当时是否自己选的？”

“XX 大学。当时是我自己挑选的。”

“现在回想，是否觉得 XX 大学挺不错？”

“是的。”

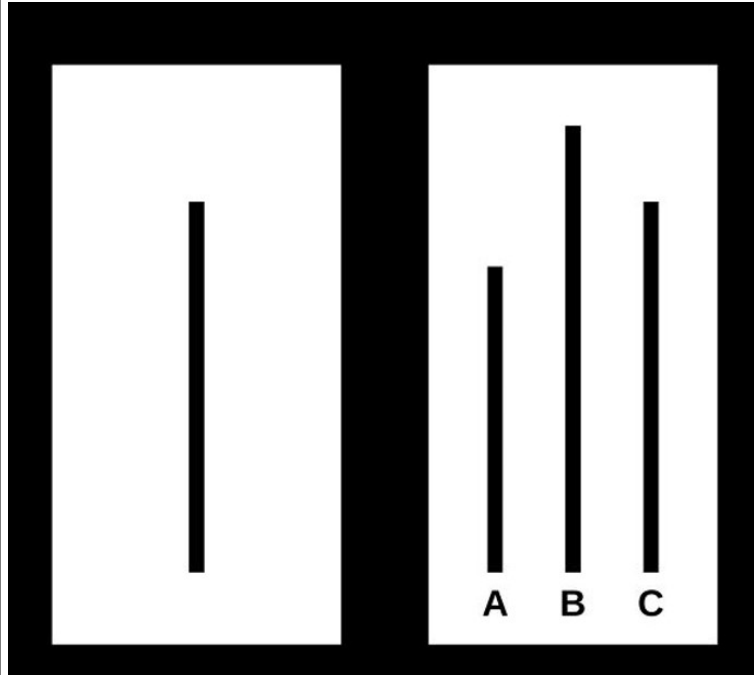
“如果是自己选择的，都会觉得比其他好。1956 年，Brehm 决策影响实验（详见附件）验证了这心理。所以如果用鱼骨图分析根因，大家都会觉得这只是组长的分析，并非我自己的想法，后面便没有动力执行。”

让团队自己寻找改进方案

前面第二章里的粮食分配实验也验证了团队只有兴趣执行自己集体讨论的分析结果，而非听从“专家”意见。所以若要团队做好迭代回顾，取得效果，要注意必须让大家参与，一起分析、讨论，才有动力采取行动。建议使用 KJ 分析法（详见附件）：每人都有一支水笔 + 便利贴，一起找根因，让每人都可以写上自己的意见。如果能让每位团队成员都有充分机会表达意见，不但能集思广益，做到更好的对应方案，也能提升团队后面执行改进方案的积极性。

KJ 分析法，让每位参与者写便利贴还能控制另一个心理因素。

想象你在大学读书时被邀请参加某教授的实验。你进入教室，里面已经坐了 4 位年龄较大的同学，都不认识。
教授投出下图：



然后问：“右面 ABC 三条线，那条的长度与左面的一条最近？”

第一位用了 10 秒钟仔细考虑后说：“我选 B。”

第二位考虑后也说：“我选 B。”

第三、四位也选择 B。

现在轮到你（第五位），你会选什么？”

“C。”

“你选 C 的时候心里怎么想？”

“有些疑虑。虽然很清楚正确答案应该是 C，但为什么其他 4 位都选 B？会否有些他们懂我不懂的秘密。”

“这经典群众压力心理学实验从 50 年代开始已经重复做了几百次（详见附件），如果在真实实验环境下，如果前面有超过 3 位都选了错误答案，参加实验者会有 30% 错误率，证明大部分人都会受从众影响。”

做鱼骨图分析时都是某人（通常是组长）拿着水笔自己说自己在墙上的大白板上画，大部分的原因其实都是组长自己说自己写，其他团队成员都不会主动说自己发现的原因。

但如果给每位参与者便利贴和水笔，大家都愿意把想到的写出来。



所以在回顾时，也要想办法避免受到群众压力的影响，导致不能真正挖掘问题的原因。所以在回顾时需要大家放开，以下方法可以减少团队压力，让大家更投入，能畅所欲言：

- 回顾开始时（设置舞台，“破冰”），尤其是团队首次在回顾里做集体根因分析，建议用互动游戏，让团队放开顾虑，全心投入讨论（可参考附件里“游戏：应与否”）。

让每位团队成员都有充分机会表达意见，集思广益，得出更好的纠正措施。

整个团队（所有相关干系人）参与

整个团队分析全局问题。例如如果只是从测试人员的角度，他可能以为缺陷都是来自开发；需求分析人员（或产品经理）会觉得自己做得很好，因需求评审里没有什么问题被发现。但如果团队所有成员聚在一起（包括项目经理、需求、设计、编码、测试），把所有的缺陷列出来，让团队所有岗位一起分析，才可以全局看问题，才可能发现不少问题是因为需求没有明确，导致后面做出来的功能并不是客户要的。后面可扩大参与回顾的干系人，例如客户代表，可以更全面分析。

看全局，寻找大家的共同点

避免局部最优 (Sub-optimization)，避免盲人摸象，最终希望可以全面从每个角色的视角全面分析。（这点可看成是上点整个团队参与的延续）

某公司管理层一直非常关注度量，对各过程，包括需求、开发、测试等，每个过程都订了七八个 KPI 指标，并每月监控。但各个过程按指标做好不一定代表总体效果会好。

每个过程都会影响软件开发最后遗漏到客户的缺陷数，迭代回顾的时候，团队可以用缺陷排除的预测模型让团队“看见”如果代码评审排除率提升如何影响每个过程的缺陷数，不仅仅是定性的讨论分析。

有数据才有管理，但软件开发与工厂不同，很多数据必须依靠员工提供，例如

缺陷返工工时。因为一个迭代通常 2-4 周，如果每位成员都能每天自己记录自己的工作量分布（像前面第 5 章，个人记录每天工时），回顾会时，大家便可统计本迭代每个过程的返工工作量，有效做到：

回顾让数据提供者能即时收到反馈

- 软件开发与工业生产不一样，大部分的数据必须开发人员自己收集，不能单靠机器/系统（例如：返工工作量）。
- 如果回顾时分析根因是依据迭代数据，除了能帮助根因分析更具体外，也让团队成员有动力收集数据（因他们知道会一起分析数据）。

因为软件开发是知识性工作，例如，某活动所花的工时，必须靠个人记录（具体步骤可参考第 5 章“个人量化管理”：个人记录每天工时）。

- 但记录数据要花精力，如果不了解收集的数据，后面有什么用途，便难以维持不断收集数据的习惯。
- 当大家都清楚到迭代回顾时会一起分析团队数据，并制订改进措施，就有动力继续迭代里统计数据。

如果使用传统方式，依赖公司级数据分析专员收集并分析团队数据，因通常几个月后才收到数据分析的反馈，公司永远都收不到真实的数值。

团队要做好回顾，除了要注意以上 4 点外，也要注意如何利用实际数据分析根因。

团队迭代回顾根因分析常见问题

1. 没有利用缺陷数据做帕累托图分析，不清楚应针对哪一类，不清楚哪一类问题最多，导致根因分析讨论没有针对性。有些缺陷分类没有定义好、不明确或同一缺陷归到几个类别，导致分析结果没有参考作用。
2. 不清楚什么是根因，只找到表面看到的现象，例如人经验不足，对技术不熟悉，对业务不熟悉等。没有抓到系统的根本问题。
3. 没有量化目标，只有定性的根因分析，下轮迭代难以判断效果能否达到。
4. 回顾时间不够（足够的时间，最少 3 小时）。

以上前 3 点可以利用培训加强上章讲过的根因分析方法。

最后一点需要领导理解如果想要有效，必须收集数据，并让团员投入讨论。

研发总监：已经做了很多年过程量化考核，起初能给团队带来活力，因为至少比以前好，有路径可以量化他们，员工觉得做有所值，公司也看得见。前几年执行得很好，产出和上线频率比过去好很多。但是现在感觉逐渐拉不出差距来，因为大家都非常熟悉这套游戏规则。

我：理解，比如现在你们团队开始做迭代回顾，团队一起分析根因，持续改善。您觉得效果如何？

研发总监：挺好，但是还是感觉他们开回顾会，耗时太长了，每次都要 2 小时以上。

我：因需要团队收集数据、分析、讨论下迭代措施，所以通常要 3 小时，熟练后会快一点，底线是改进的节省应大于回顾的投入。

高层的支持是任何改进的重要成功要素。所以首先要让管理者赞同迭代回顾能为公司带来回报（省钱），可以节省的成本（工作量）大于回顾的人力投入；也要让他理解任何改变都有个混沌的过渡期，要经过几轮回顾后，团队才能自己持续改善。

如果回顾只能 1 个或 1 个半小时的话，就不够时间让团队针对缺陷分组分类、画帕累托图做分析，没有根据实际数据分析根因，也影响到团队没有动力在后面迭代花精力去收集数据。例如缺陷返工数据一直都非常难获取到的，也需要在迭代根因分析时给时间团队讨论统计，所以通常完整的迭代根因分析回顾不会小于 3 小时。要做好这块如果有内部教练可以更好控制整个回顾的节奏。不会在某一个环节耽误太多时间，也不会太长，也确保团队人员都全程投入讨论。

所以要团队做好迭代回顾，内部教练不仅仅教大家各种“硬”技巧，也要考虑影响大家的心理，必须让每位成员都积极参与，才有动力执行和以后继续积极参与。如果只是组长一言堂，反而会妨碍团队成员的动力。但是只依赖大家的经验和感觉，识别根本原因也不科学，所以也必须有数据支撑（硬的内容）。

当然也需要高层支撑——提供资源和时间，如高层觉得整个团队花半天来做回顾太浪费资源，也无法推行。

下一章我们会分享一些团队对回顾的误解和常见问题。

附件

Asch 1951 群众压力实验

实验设计：随机邀请几位志愿者（包括你）参加，实验之前与（除了你以外）所有人预先说好，都选 **A**，并且要假装成经过深思熟虑。实验时，你是最后一位作答。

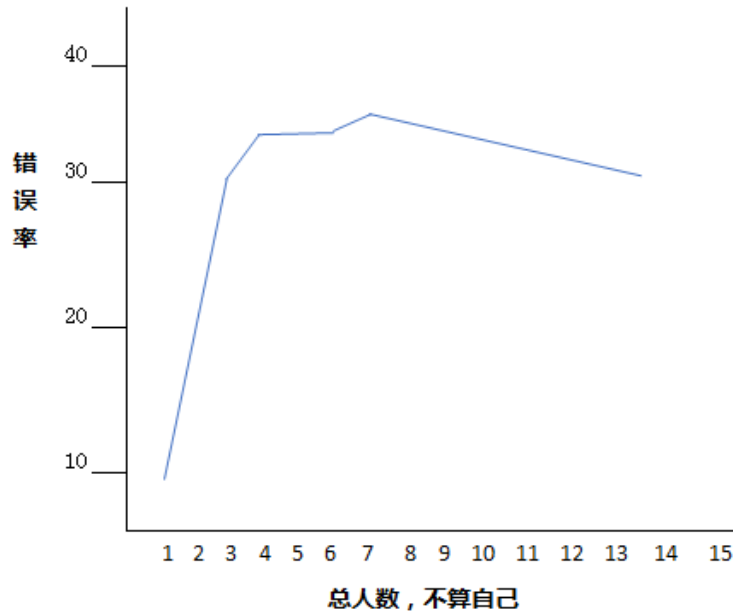
结果：

- 有群众压力时，大概 **37%** 会从众选择错误答案（虽然不同人有差异，但总体只有 **25%** 能坚持自己的正确答案）。

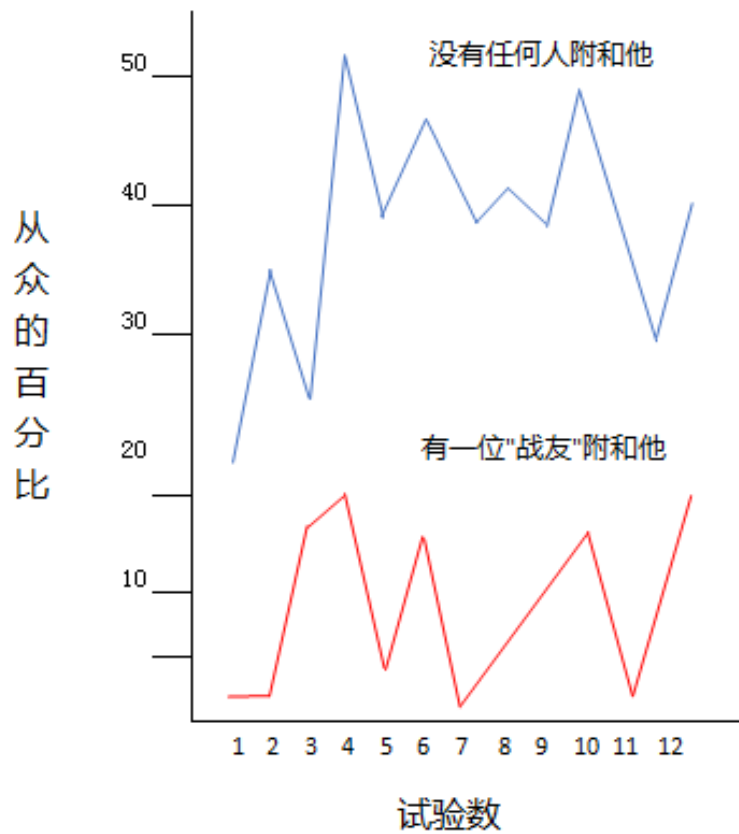
- 个人自己作答，接近 100% 选对，错误率少于 1%。

影响因素:

- 前面选择错误答案的人数越多，错误率越高:
 1. 一位：接近 0% 答错
 2. 两位：大概 14%
 3. 三位：32%（详见下图）



- 如果前面有一位“战友”选了正确答案，你的错误率就显著降低（大约原本水平的 1/4，看下图黄线），并让你感到与他更亲切。



- 尴尬场景 (例如迟到), 你的错误率会更高。
 - 实际你没有迟到, 只是让你觉得确实迟到了, 觉得尴尬。
- 如果不是要你说出答案, 而是让你把答案写在纸上, 错误率会降低 1/3。

Brehm 1956 决策影响实验

实验设计:

1. 提供 10 多种不同礼物, 包括台灯、多士炉、挂钟、收音机等。
2. 请你按自己喜好对礼物排序。例如最喜欢的选一, 如果同样喜欢两件礼物, 可以不分高低, 写同一个数字。
3. 但当你马上就要离开时, 让你从两件同样喜好的礼物挑选其中一件, 让你拿走。
4. 在你最终离开之前, 请你再对所有礼物按喜好排序。

结果:

- 绝大部分人都会抬高自己选择拿走的那一件礼物排序, 调低非选择的那件礼物排序。

- 但如果不是自己选择，而是由老师选好给你，你后面的礼物排序就没有变化。

结论：

- 人都会觉得自己的选择比较好。
- 例如选大学、选汽车、选伴侣。如果是由你自己决定选择（非被安排），你会不会都会觉得自己的选择较好。

KJ 分析方法步骤

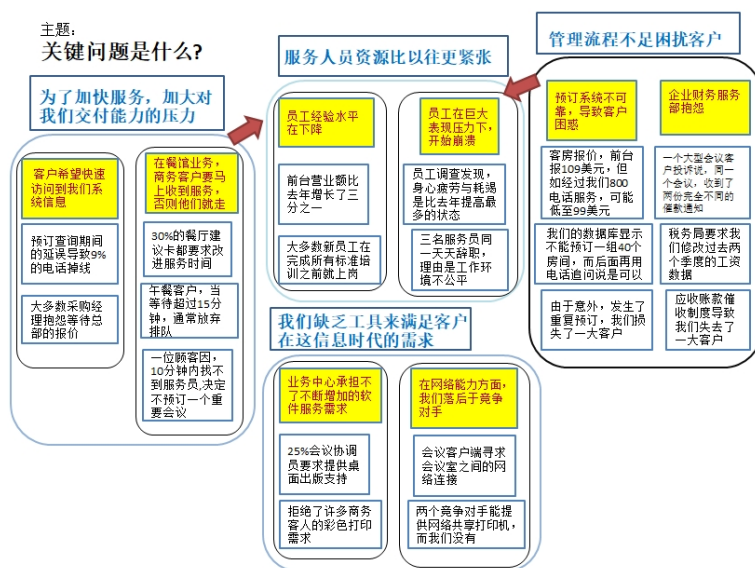
让团队了解大家的事实，一起分析根因，一起做分析报告

- 大家都对着墙上的大白纸
- 每人依据主题写“事实”并写在便利贴上，贴到大白纸上
- 轮流移动上面的便利贴，经过1~2轮后，逐渐看到归类
- 大家一起挖掘隐藏在低层次事实中的更通用的重点



一种头脑风暴方法，每人把自己想法写在便利贴上，一起轮流把所有便利贴排放分组

1. 把主题以问题形式写在大白纸的头顶
2. 把相关的事实写在便利贴上面，用黑色。
3. 搜集相关的事实，如果是一组人的话，分散到不同人手上。
4. (团队) 查看描述是否不清楚，是否要整理？
5. 每人轮流把相关的便利贴组合在一起。
6. 当每人都已经调整过便利贴组合后，一起把每一组便利贴加上一个题目，用红色标识。
7. 红色的标题下包含相关事实的内容。
8. 把相关的红色题目（最好不超过 3 个）组合在一起。
9. 给这大组一个题目——蓝色。
10. 在题目下放上相关的红色小组。
11. 把蓝色的题目和剩下红色的小组或单独没有分类的便利贴都在墙上放好，也可以用一些箭头描述它们的因果关系。



利用便利贴做 KJ 分析让大家都充分参与不仅仅适用于迭代回顾。我每次培训，如果有时间，尽量最后腾出一个小时，让大家归纳刚刚学过的东西，按自己日常工作不足可以改善之处，分析根因，并制定改进计划。本来上课时无精打采的学生都会立马精神过来，积极参与讨论。后面有些同学在填培训反馈表时，还建议应再多分配时间给学生互动讨论，减少讲课和个人做练习的时间。

游戏：应与否

在迭代回顾中使用。

目的

帮助建立有效沟通的思维模式。帮助参与者抛开指责和判断，以及对指责和判断的恐惧。

所需的时间

8 ~ 12 分钟，取决于小组的大小。

描述

在描述这些模式之后，参与者讨论这些沟通模式的意义。

步骤

1. 将注意力集中在“应与否”(见下图)，并简要阅读。
2. 组成小组，每组不超过 4 人。要求每组用一对词来定义和描述，有 4 对以上的词组。如果有多个组有相同的词组也可以。

3. 让每个小组讨论他们的 2 个单词的意思和它们所代表的行为。让他们描述各自对团队和回顾的影响。
4. 每个小组向整个小组汇报他们的讨论情况。
5. 询问大家是否愿意使用左面的方式。



用角色扮演为做好回顾打基础

首次现场培训时，我都会要求角色扮演：学员用各种角色扮演什么是吵架、什么是交流，让观众和表演者更能感受到量化回顾先要有对事不对人的心态。

除了能提高学员的参与度，也能让大家能更好理解左面与右面的区分。

Chapter 13

从团队实验到持续改善

“发现我讲过的方法总是需要重复解释和提醒，是不是我教的方法有问题？”与某老师交流时，对方提出。

“这很正常，绝不要怀疑方法本身，我也经常遇到类似的情况。实际上，这是一切学习的必经过程：起初只是根据学到的技巧初步应用到项目中，而要实现持久应用，不仅需要团队持续实验才能真正理解，还离不开‘奴、徒、工、匠、师、家、圣’这个不断提升的过程。但更关键的是高层的认同与支持，不然很容易会出现你说的情况。”

项目团队分析根因

某家专门做企业管理产品的软件开发的 Y 公司：

总经理听说如果能做好迭代回顾和根因分析，不仅可以大大减少后期返工的工作量、提升产品质量，还能够提高整体开发效率，因此表现出了很大的兴趣。

8 个月后再见面时，总经理说：“虽然我们现在已经要求团队多做回顾和根因分析，但每次迭代的系统测试缺陷数量还是降不下来。比如某产品线，每次迭代的缺陷数量大约在 50 个左右，也曾降到过 30 个左右，但不久之后又回到了之前的水平。能否跟我们团队谈谈，了解一下情况？”

从他们提供的缺陷数据、缺陷分析和系统测试报告等材料来看，其实他们并没有找到根本原因，只是对缺陷进行了分类，并针对缺陷的现象提出了修正措施。

问：第二条根因是什么？

答：编码人员把某个参数搞混了。

改进措施：提醒开发人员要注意，避免同类问题再发生

第一条：测试人员不清楚需求，没有设计出充分的测试用例，导致未能在测试阶段预先暴露问题

改进措施：加强人员的培训，更了解业务需求

问：测试本身是不会产出缺陷的，为什么你们识别测试是根本原因的源头呢？

以上是团队自认为在复盘时已经做了根因分析的例子，但不要以为经过根因分析培训后就能一帆风顺了。

某家专门做医疗软件系统产品的公司，虽然团队已经接受了根因分析的培训，并且在每次迭代回顾中都在使用，但依然遇到挑战。以下是项目团队分享会里的交流：

“第一条根本原因写的是‘开发人员不理解需求，没有按照规格要求，导致后续测试不通过。’请举例说明。”

“我们这个项目是系统升级，针对医生在手术前的很多准备工作，不仅包括麻醉师，还包括护士、设备等都需要准备就绪。而且大小医院要求不同，大医院分工很细，小医院可能由一个岗位负责多个任务。所以，要避免此类缺陷，首先要画出所有可能的处理流程图，明确有哪些不同的配置项和参数，并与熟悉流程的产品经理（甚至医生）确认。”

“这些内容是否都应该归入根因分析的范围呢？还记得我们之前讲过的 KJ 分析方法吗？所有团队成员需要用便利贴写下所有相关的因素和问题，并在大白纸上画出它们之间的关系，这才算是完整的根因分析。你现在写的“根本原因”只是浮在水面上现象。

再看对应的纠正措施：记得我们一直强调，改进计划也要像项目管理一样，用 WBS 将任务分解成不超过五天的可执行部分，并明确任务负责人。每个任务都必须有具体的产出物。如果只简单写成“一个月后由开发组长负责完成”，但由于缺乏实际行动，最终一个月后很难取得理想的效果。”

	根本原因	纠正措施	行动计划
1)	开发人员不理解需求，没有按照规格要求，导致后面测试不通过	开发与需求改善开发前的沟通，加强相关业务培训	7.17 (一个月后)，张XX (开发组长)
2)	接口测试出错，没有使用正确接口，不清楚接口参数 (2349, 2321)	代码评审时检查接口相关检查	7.17 (一个月后)，张XX (开发组长)
3)	开发没有充分自己测试好，很多非一般，但可能出现的情况，报错或显示效果不好，开发要加强自测 (2419, 2421, 2412, 2433)	对开发人员加强相关测试培训	7.17 (一个月后)，李XX (项目经理) + 王X (系统测试)

“我们分析根本原因都应有要数据支撑，不应单依赖个人主观判断。这里第三项根因只是写了四个缺陷号，是什么意思？”

“因为我们没有时间把几十条缺陷都列出来分析，我们只能抽几条把缺陷号记下来。”

“你们总共有多少时间做迭代回顾？”

“一小时左右。”

“明白了。确实时间不够，所以管理层必须给你们充分的时间，不然的话，无法做好根本原因分析。”

“第二条写‘没有使用正确接口’是什么问题？”

“这系统有很多外部接口与医院现有系统对接，有些缺陷是因为开发一测试用的接口版本不同，导致测试时不通过。”

“记得我们一直强调，开发、测试与客户的生产环境必须一致，因此这是一个关于环境的问题，接口只是环境的一部分。那么，为什么会出现版本错误？是否不应仅靠人工记录最新版本的对应关系？你们现在代码开发版本在 Git 管理；但

相关文档，包括配置，在 SVN 管理。依赖人工记录这些之间的相互对应，很容易出错，这才是根因。”

分析全过程

为北京某企业进行差距分析时，发现大量缺陷在系统测试阶段才被发现，因此建议加强迭代回顾中的根因分析。

质量经理：你说我们上次做了回顾还没有找到真正根因，但不知道如何改善。

我：数据是根因分析的基础，若要做好分析，首先要有充分和足够数据。

例如，团队只依据系统测试缺陷数据进行分析，由于没有缺陷对应的返工工作量的数据，只能从缺陷类型的数量来分析，这使得难以识别出影响最大的缺陷类别。

没有考虑从需求设计开始的评审、编码人员自测，也没有考虑各阶段的缺陷排除率，大家便只能从系统测试缺陷的症状分析，结论是编码人员疏忽、能力不足，不会考虑这些缺陷是否可以预先避免和预防；

此外，没有考虑从需求设计开始进行评审、编码人员的自测，也没有关注各阶段的缺陷排除率，结果大家只能从系统测试缺陷的症状进行分析，结论是编码人员疏忽或能力不足，而不会考虑这些缺陷是否可以在早期预防和避免。

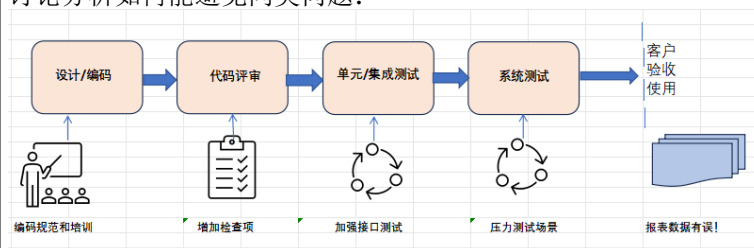
因此，在每次迭代回顾时，我们必须给团队足够的时间来收集数据，绘制全流程图以识别失效点，这对分析也有帮助。

流程图不仅能帮助分析过程，还能帮助找出根因。绘制流程图后，就可以利用 FMEA 方法针对每个失效点找出根因和相应的纠正措施。

某公司专门为电信供应商做客服管理软件系统，因客户对质量有要求，每次发布后都分析有哪些发版后的问题。有些软件开发相关问题不容易排查根因，例如某次发布后发现报表的数字对不上，但这些错误不是立马使用时就出现，开始的时候没问题但使用时间长了，就会产生错误。

团队花了很长时间分析整个过程，最终发现错误是由于有些内部接口，没有考虑到所有传输数据是否正常，有些有错但没拒收，导致开始时没问题，但累积多了就会出现最后的报表错误。最后也针对这问题改正了。

讨论分析如何能避免同类问题：



按系统产品集成的流程，识别有那些点可避免问题发生（FMEA 思路）。可以在集成测试时和系统测试时，增加类似的场景，加大类似的压力测试，应可预先暴露问题；集成测试时，如果都测试所有接口传输的数据，也可以避免问题；如果我们前面做好设计和代码的评审（根源与设计有关），也应该可以避免。例如如果针对这种问题，完善评审检查单的检查项，以后应可避免同类问题再发生。

从这个实例可以看出，首先绘制总体流程或路线图，并识别每个环节的潜在风险，有助于更好地挖掘问题的根本原因。

除了 FMEA 方法外，缺陷排除预测模型也可以帮助团队分析从需求到交付的全过程。如果团队对此不熟悉，可以先安排一天的培训，以帮助他们了解根因分析的重点。

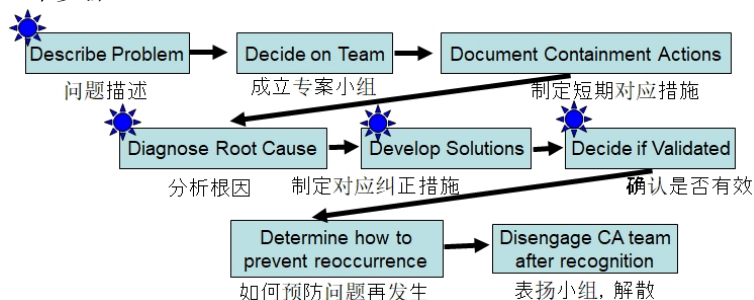
如何提升软件开发质量TrainingPlan	
9:30	用案例介绍根因分析的五个步骤 与技巧:帕累托图、鱼骨图等练习
10:00	根因分析小组互动
11:00	休息
11:15	迭代回顾场景(由公司内部教练做角色扮演)
11:45	总结,并介绍缺陷排除率模型
12:00	午休
13:00	介绍缺陷排除率模型
13:15	量化根因分析模拟练习,包括: 分析模拟数据,估计缺陷排除率与返工工作量 针对主问题,团队所有成员做KJ根本原因分析 从根因、讨论对应改进措施
15:15	休息
15:30	设定量化质量计划模拟练习 利用MonteCarla工具 对原本迭代数据做3点估算 再依据前面制定的方案 利用工具寻找最佳方案
16:15	总结,确认团队后面的具体工作

以上一天培训让团队了解以下重点:

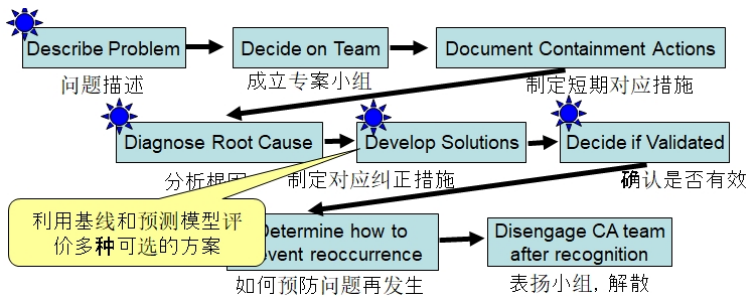
改进目标:降低返工工作量(同时也会降低缺陷后期才被发现的数量)

分析根因:不仅要分析后期发现缺陷的分类与数量,还需对从需求到最终客户验收的全生命周期进行缺陷排除率分析,以找出最薄弱的环节并深入挖掘根因。利用数据进行互动练习是最有效的学习方式,因此建议从以往的缺陷数据中抽取 70-150 条进行小组分析。

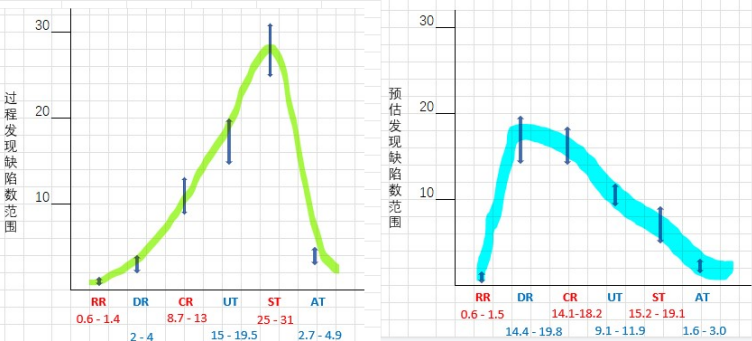
预测效果:以福特 8D 根因分析框架为例,缺陷排除与返工预测模型可以帮助团队预估新方法的效果(例如,通过代码静态扫描加强代码评审)。福特 8D 的 8 个步骤:



缺陷预测模型可以在分析根因和预估效果步骤里贡献:



例如，若大部分缺陷集中在系统测试阶段被发现，模型可以预估下一轮迭代中的缺陷分布。如果代码评审中发现的缺陷数量未达到预期，团队应进行内部分析，避免等到系统测试阶段才找出缺陷。（想了解如何利用缺陷排除预测模型预估改进后的效果，详见附件里的“缺陷排除预测模型实例”）



”团队自己收集数据”：此外，团队成员应开始每天或每周自行统计返工的工作量（包括评审缺陷，所有数据均记录在缺陷管理系统中）。

BUG #	系统测试BUG描述	开始	结束	总修复人时	备注
5384	当客户选护照（非大陆身份证），大陆地址留空，系统报错，无法继续	7:30 - 1:30	14:30 - 16:30	10	没有穷举各种场景，分析需求有误
5198	输入错误结束日期，但系统没有报错（估计程序没有校验输入是否正确）	17:00 - 18:00	20:00 - 21:00	2.5	
5365	输入-9.99 查询，结果是阳性，不对	19:00 - 21:00	9:30 - 11:00	5	内部接口，过系统没有处理
			18:30 - 20:00		

内部教练如何做好现场辅导

培训后，过了 6 周，质量经理电话联系我。
质量经理：我们计划下周三与项目组进行迭代回顾。该团队有 7 到 8 人，两周前曾与另一团队使用便利贴进行了根因分析，确实听到了更多不同的声音。应该如何准备，才能让团队放开并有效地做好根因分析？
我：

如果只有缺陷数量和工作量等“硬”数据，但缺乏如团队成员的感受、积极性、主动性等与人性相关的“软”数据，也难以真正分析出能够避免同类问题再次发

生的根本原因。例如，在冲刺回顾时可以用时间表（Timeline）让大家回忆过去四周的经历，并用颜色点（Color Code Dots）让每位成员标记每个事件的感受，这是一种收集软数据的方法，详见附件：敏捷回顾互动练习

要做好回顾，准备工作至关重要。例如需要准备大白纸，并确保有足够的空间。如果团队成员围坐在一个大桌子旁讨论，效果往往不佳。相反，我们在墙上贴上大白纸，并提供便利贴和大号、小号水笔等，当每个人都有自己的一支笔时，团队成员会更加投入。他们会走动、讨论、写字、画图，并不断交换位置、互相帮助，这样能更好地体现团队精神。

因此，辅导的重点不是在各种技巧上，不是教他们每一步怎么做，只需提醒时间（时间的管理很重要。每次进行小组练习时，我都会用电脑投屏显示剩余时间（分钟），让团队即时掌握剩余的时间）。

先定好目标，让团队自主发挥与讨论，效果会更好。只要他们清楚需要做什么，大部分团队都能自行分工，比如有些人负责画帕累托图，有些人做总体根因分析。

质量经理：你的意思是减少干预，尽量让团队自主思考。但因未做过，可否再举些例子？

我：正确。不如我反过来，建议你应该避免说什么：

1. 其实你们团队和我见到的其他团队相比好或者差（因这都是个人观点、感觉，非事实）。
2. 本来只需要你们画帕累托图和鱼骨图，为什么你们还画了个脑图？
3. 你们都已经讨论了很多细节，要立马抓紧时间做最后总结。
4. 你们讨论好像都只是针对机场闸口的问题，是否应该也考虑其它？

为什么要鼓励团队表达不同意见？

团队要改进，首先要确保成员充分理解彼此的差异。经过充分讨论后，团队成员找到共通点，才有机会一起改进。

如果大家在回顾时能够开放讨论，充分表达各自具体的看法，就能让大家看到全貌和问题的真正情况。

因此，内部教练应鼓励多样化的声音。心理学家 Asch 在 1951 年的从众实验中证明，只要有一个人率先提出异议，其他人也会陆续表达自己的意见。因此，内部顾问应确保每个人都有发声的机会。意见越多样化，大家越能了解真正的问题，找到大家认可的改进方案（详见附件：功能性分组帮助团队成长实例）。

例如，如果发现团队在大型会议上因人数众多而不愿发言，可以考虑分成小组讨论，这通常会更有效。

对于一些“慢热”的团队，需要给予他们更多时间，不要急于打破沉默。等一等，一旦有人开始发言，接下来的讨论就顺利了。

从迭代复盘到持续改进

当某些团队已经能做好回顾和根因分析，也能利用数据改善下一轮的开发质量时，如何将这种做法转变为团队或公司的持续改进呢？

深圳有一家公司专门从事内部 IT 服务，因为对质量要求严格，新版本在正式投产（发版）前必须通过测试验收。公司每两周进行一次发版，并对发版成功率进行统计。因为都做了很久，依据以往水平，要求发版成功率不会低于 93（如想更了解这案例的数据变化，详见第 24 章里‘控制图的应用’）

虽然个别项目组的迭代回顾能够有效改善质量，但这些成果仅仅是高层关注下的“种子”。尽管通过工作量和缺陷的改善能够显著减少持续浪费，但如果高层不关注并投入资源，那么这些“改善”可能只是短暂的闪光，很快大家就会回到以前的工作状态。

与 Y 公司总经理汇报

我与本章第一个案例做企业管理产品的 Y 公司里的团队交流，了解问题原因后与总经理汇报说：

“您之前问我为什么我们的缺陷数量总是降不下来。通过与您的团队沟通，我发现他们在根因分析上存在很多误解，虽然也做了复盘，但是过程和做法跟之前没有任何变化。”

“有什么好建议？”

“任何有效的质量改进必须从高层的关注和支持开始。如果管理层认同改进的重要性，并认为它对企业有价值，就必须将其当作项目来实施。这个项目需要高层领导和监督，成立改进小组，指定小组成员并明确具体的任务，制定项目管理计划，确保高层每月监督进展。”

“我们高层是非常支持改进的，但员工手头的任务已经很多了，很难再要求他们腾出时间来做计划与监控。既然他们已经有复盘的习惯，是否可以先做些根因分析的培训？”

“没有问题，我当然可以按你们安排做培训。开始时某些团队可能会做出些较好的分析，并有效果，但过了几轮，因缺乏公司持续质量改进的支持，不久又会返回原本，因为要改变人的习惯是非常难。”

培训不能改变团队的思路和做事的习惯，虽然培训很重要，但培训本身无法给企业带来可持续质量改善的公司文化。

改进效果

香港 A 公司成立了过程改进小组，并由高层领导担任组长，负责监督每月的小组会议，代替原来的传统会议。尽管每次会议都有纪要和待办事项，但三个月过去了，进展并不明显。

于是，我建议他们回顾之前培训过的根因分析方法，并采用 KJ 法，请大家用便利贴进行讨论：

在新的会议形式中，每个人手持便利贴和小号笔，一起围绕着墙上的大白纸讨论分析，并制定改进行动和目标。开始时有些反对意见：

“我们之前开会讨论大家都有提出各种意见，不一定要用这方法。”我解释对此，我解释道：“当大家坐下开会时，一旦有人提出意见，其他人若有不同想法，他要先想好如何提问，并如何应对对方的反对等，所以选择不提意见。但如果将意见写在纸上，就能避免这种心理顾虑。”

起初，部门经理们持观望态度，但当他们看到高层对过程改进的决心而非一时的口号后，也就开始关注改进小组的工作了。

由于项目经理也是过程改进小组的成员，项目团队在迭代回顾中也采用了便利贴和大白纸的讨论方式。

四个月后，改进小组的组长（即高层代表）回顾道：“起初缺乏经验，但后来改用了便利贴的互动讨论方式，看到大家开始愿意积极参与。从每个月完成的改进项数量和客户缺陷密度数据来看，确实有了按月的改进。”

对比世界级企业如丰田汽车，自二战后至今，他们一直在持续改善，超越了原本的美国汽车巨头。

Dr. Juran 引用了丰田的例子说明什么是持续改善的企业文化：“在 90 年代，丰田汽车拥有 8 万名员工，每年提交约 400 万条过程改进建议，平均每名员工每年提出 50 条。”

也正是这种改善得动力让丰田超越竞争对手，成为全球最大的汽车公司。

公司持续改善必须有高层的关注并提供资源，把过程改进当成项目来管理。然而，万事开头难。在获得管理层支持后，初期需要通过软硬兼施的方法“解冰”，即让团队成员动起来。如果在开始的几个月里不能改变他们的习惯，后续的改善将更加困难。反之，如果能让小组尽快动起来，就能逐渐培育出像丰田那样的持续改善的企业文化。

附件

功能性分组帮助团队成长实例

系统，包括团队、公司的成长都依靠不断集合内部的差异，如果完全不能接受不同意见，就无法进步。基于心理学家 Yvonne Agazarian 系统中心理论 (Systems Centered Theory)，团队必须先从传统（按岗位、功能）分组 (Stereotype Subgroup) 变为功能性分组 (Functional Subgroup)，才能成长。要让大家觉得有共同点，基于这基础才可以听从不同意见。

场景实例：

团队讨论如何完善社区里面的教育，大家讨论个人的希望。

A：好像我们大家的想法都很类似。

内部教练：可否举个例子？

B：我们都想为自己和我们后代有个终身不断学习的机会。

C：好像我们都没有人提过大学，不知道为什么。

D：但很多人负担不起大学学费。

E：我觉得每个人只要想受到教育，都能做到，我就是靠自身努力。最后完成大学教育。

F：（开始跟 D 形成小组）是的，但教育确实越来越昂贵，我估计我们当中不少人负担不起那些美国东岸名校的昂贵学费。

G：（有成见，觉得那些不能完成大学教育的都是没有动力，没有上进心）我还是觉得任何人，只要有动力都能做到。

如果没有人附和 E 的观点，就可能变成他的个人独角戏。后面有两个可能的发展场景：

场景一 C 附和了 E 的观点。

C：我的经历确实和 E 说的类似，辛辛苦苦最终完成了大学教育，但确实很艰苦。如果我当时没有舅舅的支持，肯定完成不了。

场景二没有人附和 E 的说法。

内部教练：有没有其他人也有经过自己努力完成大学教育的经历？

（注意：内部教练千万不能说：“有没有人赞同，人只要有动力都能进大学？”因为这只是个人观点，并非事实。）

A：我确实也完成了大学教育了，但也经历了很多困难。

G：我读了一年，因为再得不到奖学金，没有办法，只能退学。

到了这个点，整个分组就比较复杂，虽然生成了新小组，E 得到一些回应，但也有些人提出不同的经验，需要有说法把新小组归纳。

接下来，

C：我看来这个很简单，有些人能完成大学教育。有些人虽然很希望完成，最终还是完成不了，但不是所有人都想或者必须完成大学教育。所以我们的改进任务应该是帮助所有人学到他希望学习的东西。

有了以上 C 的说法，整个团队就可以进一步成长。

从以上实例看到，首先要让大家愿意接受不同意见，不然的话就会变成争吵，不会有效果。有人提出不同意见后，必须有人附和，成为他的战友，才能形成新的小组。有了新小组以后，团队需要把不同小组的立场合并起来，变成一个新方向、新思路，整个团队就有成长、进步。

敏捷回顾互动练习 1: 时间表 (Timeline)

在较长的迭代、发布或项目回顾中使用此方法收集数据。

目的

促进大家记忆，从多个角度回看之前的工作。大家一起回看谁在什么时候做了什么工作，与当初的假设对比。看是否有什么规律？或者生产水平有没有变化？

用来表达事实和每人的感受。

所需的时间

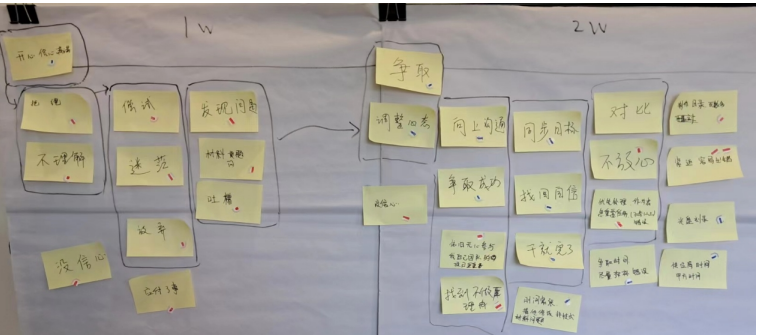
30 到 90 分钟，这取决于团队的规模和迭代周期长短。

描述

小组成员写卡片来标示那些很有印象、特别有意义或其他重要事件，然后按时间顺序贴在时间表 (Timeline) 上。教练鼓励团队讨论事件，以了解已发生的事实和感受。

变化：功能性泳道 Functional Swim Lanes

沿时间轴的背景线纵向绘制行 (假设你不打算直接在墙上张贴卡片，然后使用丝带或胶带划分行)。为每个部门或功能做一行。每组只会把他们的卡片放在自己组的泳道里。



敏捷回顾互动练习 2: 颜色点 (Color Code Dots)

与时间表 (Timeline) 练习结合使用，在更长的迭代回顾中收集团员的感受。

目的

展示团队成员对经历事件的感受。

所需的时间

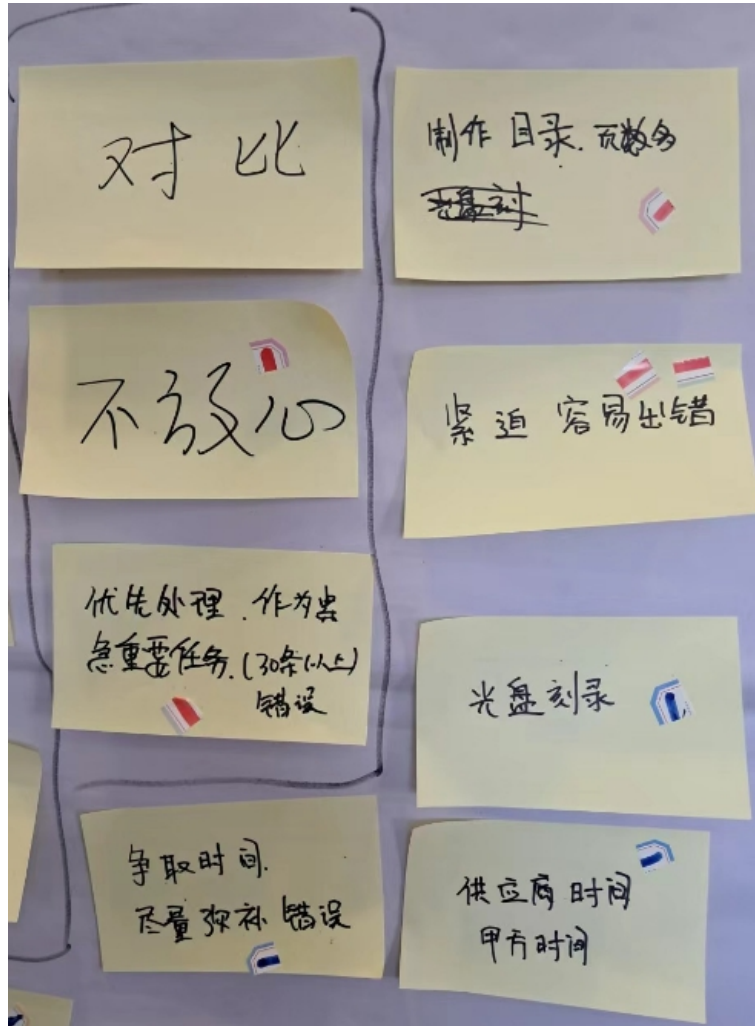
5 至 20 分钟。

描述

团队成员使用颜色点来表示在时间轴上事件的心情高低。当所有的事件都已经展示在时间轴上，团队也已经回顾后，每人使用颜色点来显示他们的情绪是高还是低 (见图)。

1. “我们已经一起看了事实，让我们也看看大家工作时的心情，状态。” 作为开始之前说，让大家理解游戏目的。

2. 给每个人提供两种颜色的点。每人从 7 到 10 个点开始，如后面不够用，可提供更多。解释哪种颜色代表情绪高，哪种颜色代表情绪低。
3. 让每个人都用点来表示哪里情绪高，哪里停滞、衰弱或低潮。



缺陷排除预测模型实例

- 团队分析迭代缺陷数据，缺陷是源自需求/设计/编码，得出以下分布表：

		Req	Design	Coding	
RR		1			
DR		1	2		
CR		2	3	6	
UT		2	5	10	
ST		3	9	16	
AT		1	1	2	
		10	20	34	

- 按缺陷排除率 (DRE) 公式，计算出各个过程的缺陷排除率，并把参数输入水晶球模型。例如第一行 Quality 那行：10 (共有 10 个缺陷是源自需求)；第二行：10% (需求评审缺陷排除率是 10%)。(本来迭代缺陷参数都标 Infosys 识别。)

RR	$1/10$	10%
DR	$1+2/(10+20-1)$	10.3%
CR	$2+3+6/(64-4)$	18.3%
UT	$2+5+10/(64-15)$	34.7%
ST	$3+9+16/(64-32)$	87.5%
AT	$1+1+2/(64-60)$	100%

[illegible]



	① 修复缺陷 总工时	② 缺陷数	① / ②
RR+DR	16	4	4
CR+UT	80	28	2.9
ST	160	28	5.8
AT	80	4	20

- 从团队工时表估算各过程的缺陷返工工作量。例如最下面一行 Quality 那行输入: \$20,000 (因为验收测试缺陷平均修复工时是 20, 假定每工时成本为 \$1,000); 系统测试那行: \$5,800(因为系统测试缺陷平均修复工时是 5.8)。

Zone 4 AE2:AL60		Costs	Unit Costs	Min	Most Likely	Max
Requirements Development	Effort	\$71,888	\$966	\$700	\$800	\$1,000
	Quality					
		Costs	Unit Costs	Min	Most Likely	Max
Reqs Review	Effort	\$8,416	\$828	\$700	\$800	\$1,000
	Quality	\$16,768	\$887	\$3,400	\$4,000	\$4,800
		Costs	Unit Costs	Min	Most Likely	Max
Design	Effort	\$67,688	\$798	\$700	\$800	\$1,000
	Quality					
		Costs	Unit Costs	Min	Most Likely	Max
Design Review	Effort	\$6,308	\$764	\$700	\$800	\$1,000
	Quality	\$27,002	\$1,418	\$3,400	\$4,000	\$4,800
		Costs	Unit Costs	Min	Most Likely	Max
Code	Effort	\$88,301	\$844	\$700	\$800	\$1,000
	Quality					
		Costs	Unit Costs	Min	Most Likely	Max
Code Review	Effort	\$11,682	\$885	\$700	\$800	\$1,000
	Quality	\$71,888	\$1,877	\$2,486	\$2,800	\$3,336
		Costs	Unit Costs	Min	Most Likely	Max
Unit Test	Effort	\$144,168	\$976	\$700	\$800	\$1,000
	Quality	\$83,814	\$7,643	\$2,486	\$2,800	\$3,336
		Costs	Unit Costs	Min	Most Likely	Max
Integration Test	Effort	\$4	\$826	\$700	\$800	\$1,000
	Quality	\$3	\$3,831	\$3,000	\$3,600	\$4,600
		Costs	Unit Costs	Min	Most Likely	Max
System Test	Effort	\$79,004	\$801	\$700	\$800	\$1,000
	Quality	\$124,782	\$7,303	\$4,800	\$5,800	\$8,670
		Costs	Unit Costs	Min	Most Likely	Max
Acceptance Test	Effort	\$62,786	\$834	\$700	\$800	\$1,000
	Quality	\$131,887	\$17,801	\$17,000	\$20,000	\$23,000

- 估算质量总成本：水晶球会依据质每过程的缺陷分布和单位成本分布，预估质量成本的分布。按本来迭代需求引入缺陷数 = 10 (3)，需求评审缺陷排除率等于 10% (4)，设计引入缺陷数 = 20 (2)，设计评审排除率 = 10% (4)，代码引入缺陷书 = 34 (4)，代码评审排除率 = 18% (1)，单元测试缺陷排除率 = 35% (2)，系统测试缺陷排除率 = 88% (1)，验收测试缺陷排除率 = 99% (3) 来估算质量成本分布。
- 如果我们把优化后的缺陷分布与本来迭代的缺陷分布比较（本章正文里比较前后两迭代的缺陷分布左右两图比较），能看出改进后，本来在系统测试才发现的缺陷大多数可以预先在评审暴露，降低返工工作量，也可以预估评审缺陷范围，如果实际评审缺陷数未能达到范围，便可预警，尽快纠正。

如果没有预测模型，我们只能主观估计会降低，有了预测模型，便可以估计缺陷分布的变化，也能帮我们挑选最佳搭配，并预估下一轮的缺陷分布范围。

解读预测模型最佳搭配选择：

模型以总成本最低可以防止局部最优。例如需求评审没有选用排除率高的新方

法，因为新方法会使用较大评审工作量。如果用质量最优（质量成本最低）便必然会读选用缺陷排除率最高的方法，但用总成本来比较便可以综合考虑工作量 (Effort) 和质量 (Quality)，避免局部最优。

参考 Reference

1. Juran, Joseph. "Juran's Quality Handbook "(1999) 5/e

Part VI

代码质量改进的基础

这部分参照 Jeffery 先生总结极限编程 (XP) 的三层图 (在第 1 章里), 利用实例, 先从内层的测试驱动开发 (TDD)、重构 (Refactoring) 与结对编程 (Pair Programming) 等核心最佳实践开始, 然后到中层的持续集成 (Continuous Integration), 最后是外层的团队各角色协作/互补 (Whole Team) 和如何做好需求分析等。

Chapter 14

测试驱动开发

开发人员问：为什么要这么做？写完程序后，然后再写测试用例，不是更正常吗？

为什么我们要测试驱动开发

精益是敏捷中很核心的概念。最近发生的一件事帮我回答了以上问题，也让我体会到 TDD 与精益的概念是如何协同的。

我有一位拥有三十多年电机工程经验的老同学李先生，他在对 Linux 的研究方面一直都很有兴趣，这几年又开始玩树莓派，目前他对此非常精通。

最近我找到他，希望在树莓派上安装 Ipython, Jupyter Notebook 等开源应用软件。一直以来，李先生的理念就是尽量用最简单、最轻易的方法去解决客户的问题，而不是浪费资源。

例如 95 年开始尝试建立 WIKI 服务器给客户使用的时候，他就建议我用树莓派来建。

他说：“树莓派不仅省电而且快，也不知道你这个概念是否持久，我不会浪费时间——在大型服务器上安装 Linux + Mediawiki，如果你觉得这（树莓派）可以，便继续用。如果觉得它性能不够了，下一步我帮你换更大型的服务器。”



一直到现在，已经 7 年了，我们一直还是继续用树莓派来协助评估（树莓派从原来的 Pi2(1G 内存)，现在已经是 P4(4G 内存)，已经服务超过百家客户了，这证明他的思路是对的。

回到我刚才说要安装 Jupyter Notebook，他安装好那个程序包后，一直没动。我问他什么原因？他说：“很简单，我不熟悉软件工程，也不懂如何测试。肯定要找你来测试一下，我才知道有没有做错。这样可以避免无谓的返工。”

所以我们就约好周日我回到办公室跟他远程测试。

李先生打开树莓派后，我用 VNC 客户端进入他的树莓派，打一些最基本的 Python 指令，验证没问题。

下一步我要跑一些比较复杂的、要用到 Pandas 包的 Python 程序，却发现还没有安装 Pandas 包。他马上到网上去找安装介质以及树莓派如何安装 PANDAS 的方法。处理好后，我再跑对应的程序，都跑通了，包括之前跑过的指令。前后 2 小时，按本来计划，测试成功。

我回想一下，其实他的思路就是精益。用有限资源、以最低投入，达到客户要求，不多做。每当李先生做完一步，如果没有通过测试，就不会浪费时间走下一步——这也是精益和测试驱动开发的原理。

每小步验证

工程类的本科生都要在最后一年做毕业项目。当时是 1983 年，导师建议我们尝试参考一些常用的语言发声算法，在专门做信号处理的芯片上写程序，读出一些英文字或句子（与现在不同，当时这项技术还没有成熟，很多大学还在研究），虽然我预计有不小的技术难度，但觉得这很先进，很感兴趣，便与另一位同学合作，开始制订项目计划。

我们很努力，全情投入，一面要研究语言发声的算法，一面还要并行设计电子线路和软件等。我们用了 2 ~ 3 个月的时间做了整个电子线路板的硬件，同时还设计了整个软件架构，并使用计算机模拟，验证每个字实现算法后的发声效果，因为最后一年课程很多，时间过得很快，从 9 月开始准备一直到次年 3 月，软硬件的设计与开发终于都完成了，但是不知道什么原因就是没有声音，更不要说能读出一些字和句子了，最后项目以失败告终。

在这之前的一年，我有幸在大学的第三年进入大东电报局 (Cable & Wireless) 实习，实习的部门 (负责电报业务运维) 正如火如荼开发一套新的电脑系统以取代本来基于 UNIVAC 的电报系统，总工程师让我用半年时间，在一个微机上编程，做一个系统，以从那些电脑上收集重要信息，如果发现异常就警报。头 3 个月都是花精力做整个系统设计，也买了一些展示电子版，准备用来展示，但由于经验不足，半年后最终什么都展示不出来。

到了 2000 年，在我兼读软件工程硕士课程时，开始接触到敏捷开发，才了解到两次项目失败的主因：不应该花大量时间去做前期设计——希望有一个完美的设计，而是应该一步一步迭代，先做一些最基本的简单功能，逐步优化。例如，在我的毕业项目中，应该先做出最基本的硬件、软件，起码能够发出声音，因为没有前人做过，整个项目是从未做过的实验。

敏捷大师 Dave Thomas 先生在 2015 年演讲里提出敏捷软件开发的核心是：

1. 向你的目标迈出一小步。
2. 从反馈调整你的理解。
3. 重复。
 - 当两种或以上选择的价值大致相同时，选一条让未来更容易修改（软件）的路径。

这些经历让我体会到为什么我们在写程序时，必须先想一下该怎么测试。

用 TDD 开发程序

程序员问：为什么我们要针对这么小的部分都做单元测试？

答：

1. 小的单元测试容易写，比较简单，不会花费很多时间。
2. 因为任何程序都会按时间变化，越来越复杂。有了单元测试，后面的测试就有依据，知道改动是否出错。没有单元测试的话，就没有信心去改程序，不知道是否有问题。

(这和我们工程的概念一样，把工程细分，每个子系统都要确保质量达标，尽量找出缺陷，才进行下一步)

如不相信请看附件“从 MIT OCW 学写程序”，我从头学编程的实例。如果老师没有提供单元测试和相关数据，学生写完代码后是无法验证代码是否通过，是编程的基本要求。为什么专业程序员的程序反而可以不需要先通过单元测试，写完直接便交给测试人员做系统测试？

网上有关于是否要 TDD 的争论:

- 一派是坚持要先写测试用例，再写编码。
- 另一派觉得 TDD 太多余，如果功能测试做得全，也不一定要有单元测试。

这个争论让我想起以前教 ACP 敏捷管理的一个原则，叫“守破离”。

守破离

英文翻译成“Shu -Ha -Ri”，意思是学习合气道，要先从基本功出发（守），按规则一步一步做。当你理解以后，就能融会贯通（破），融合管理到一定境界就是大师级了（离）。

像宫本武藏（江户时代著名剑术家）一样，一生赢了 60 多场决斗，然而到了晚年，他在《五轮书》中总结：不要局限于某种刀法，最重要是抓住原理。

敏捷大师 Mr Cockburn 有一次到企业做 Class-responsibility-collaboration (CRC) card 咨询培训，因为他很熟悉 CRC 方法，他觉得只要有助于面向对象设计，不一定要按原本的方式，可以简化。但学生都不熟悉，一直在问“请你告诉我们从头到尾，每一步如何做，我们照着做”。大师没办法，最终按最原始的方法，一步步来教如何做，学生才能懂，这是守破离的“守”。TDD 测试驱动开发也一样，把 TDD 看成“守”，当没有概念的时候，还是要从最基本的概念入手，我们每个程序都应该有测试，所以自然的顺序是先写测试用例，再写代码，这也能更好地帮助你理解程序的功能。

所以 TDD 就是做好程序基本功的基础，当你已经掌握这些基础了，就可以灵活处理，可能不一定要每次先写测试用例，但底线是每个代码都要有单元测试。

但是有些人觉得，只要有功能测试，单元测试可以不做。

功能测试是黑盒测试，从客户的角度测试总体功能，无法真正判断代码是否正确。反过来，单元测试是白盒测试，从代码的功能角度，验证代码是否正确，所以功能测试是不能替代单元测试的。尤其是当代码有功能上的变化时，就更需要有单元测试来验证这些改动是否有误，所以单元测试的复用率非常高。

大家正越来越多地使用自动集成工具，如果有单元测试，那么每天的自动集成就不仅仅是看编译是否通过，还需要通过单元测试，才能更好地保证代码质量，这就是很有效率的回归测试。（回归测试：为避免因代码修改，导致原先通过的测试不通过，所以之前通过的测试用例也要重复再测。如果手工测试，为了节省测试时间，只重跑部分重要测试，但如果自动化，就可以轻松地重跑所有测试用例）

单元测试与 TDD 的好处

上周在杭州跟一位资深的开发主管对话。他回国前在日本工作了接近 10 年。他说很佩服日本注重开发质量，例如很注重单元测试，所以回国后在团队严格要求要有单元测试。

我问：现在有很多静态代码检测，为何还要单元测试？

他答：目的不同，静态代码检测只是用过去的历史经验去检查常见的语法、命名问题，但如果是设计本身的逻辑问题就无法查出。以他多年的经验，必须通过单元测试才有信心确认这个程序是否正常，单靠集成测试、功能测试是无法确保程序的每一部分都是正常的。所以他严格要求团队，代码经过检验后（自动静态检验或代码走查），还必须做单元测试才能进入下一步的集成、系统测试。

单是靠集成测试、功能测试是无法确保程序每一部分都正常。

他还说 TDD 有 3 点好处：

1. 让开发人员在编码前去理解设计和需求。
2. 让开发人员知道代码可验证性的重要。
3. 强迫开发人员主动与设计和需求人员沟通，否则无法设计出单元测试用例，但 TDD 对团队成员素质要求较高。

结束语

开发人员用 TDD 编程，体现每走一步前必须验证通过，减少返工，精益思路，是中初级程序员健康成长之路，养成好习惯。

但若你是九段高手，因已经过了以上基本训练阶段，可能便不再需要，好比合气道“守破离”成长路径。

有些敏捷大师，已经累积有 20 年以上编程经验，例如上面提到的 Thomas 先生，他已经很少写单元测试，因为他觉得已经不需要依赖单元测试来验证代码是否正确。

附件

从 MIT OCW 学写程序

第四天早上 10:00 我终于把所有功能写完并通过自动单元测试！

(趁十一长假做编码练习题。上次做题已经是春节后，在集中隔离期间做过两题。这次的练习题只有十个功能，本来预计可两天完成。)

练习题 (Problem Set)——分析美国过去温度变化，提供了美国超过 20 个城市的每天平均温度。(从 1961 年到 2015 年)

- 先写基本功能：
 - 画散点图，回归分析，计算标准差与决定系数 (R^2 Coef. of determination)
- 分析过去的五六十年数据，来判断美国或全球是否在暖化

给学生的文件包：

1. ProblemSet5.pdf：分成 A、B、C、D、E 部分，详细说明每部分要开发的功能
2. PS5.py 程序模板，学员填入代码
3. PS5_test.py 自动单元测试模块，自动测写好的 PS5.py

回顾一下，像我这种 Python 新手，因为不熟悉 Python 语言的属性与特性，必须按部就班，一步一步来写，欲速则不达。

更重要是体会到个人如何记录数据并分析：

记录时间

十多年前我在美国考 CMMI 培训师，被观察时，观察老师会在培训后告诉我，每一个模块用了多少时间，与计划对比。从此以后每次培训我都会习惯记录每个模块所花的时间。

怎样做

培训时，我都会在桌上放一数字电子钟，记录每一个模块的开始时间与结束时间。上完一天课，晚上在电子表单计划时间旁边写上每个模块的实际时间。这三天半，我也是用这种方式记录每个模块的时间。

记录缺陷

因为每个功能都很小，不到 20 行，所以没有正式把缺陷与返工工作量记下来。尤其是一些小的语法错误问题立马就改正了。但影响较大的重大缺陷，我会记在模块旁边，算是这个模块花的时间。

个人记录实际时间没有想象中这么困难，只要每天晚上简单按当天本子的数据更新电子表单，避免遗忘。

	开发功能	开始	结束	人时	缺陷	LOC	修改 LOC	备注
1)	generate_models	10:30	12:00	3		7		
		13:00	14:30					
2)	r_squared	14:35	16:41	2	数据类型	13		
3)	evaluate_models_on_training	19:15	21:20	3	前后顺序	13		
		9:45	11:00		不熟画图规则			
4)	Problem 4.I			1		12		
5)	Problem 4.II			1		10		
6)	gen_cities_avg	11:00	12:30	4	数据类型	15		
		16:38	17:54					
		19:36	20:34					
7)	moving_average	17:00	18:00	2	误解语言	14		
		20:10	21:10		边界值			
8)	rmse			1	复用，有些变量没有改好	7	2	复用 2)
9)	gen_std_devs	9:45	13:30	10	错误理解需求	15		
		14:30	17:00					
		8:25	10:00					
10)	evaluate_models_on_testing			1		13	1	复用 3)
				28				

回顾分析

整个编程结束后，写上每一个模块的实际代码行数。注意：如果模块是复用其他模块代码，便需要注明改动的代码行数。例如：evaluate_models_on_testing13 行只修改了 1 行。

分析：除了标准差模块（`gen_std_devs`）以外，其他功能都能在 1-3 个小时之内完成。

计算标准差的功能花了 10 小时，问题出在我编码时没有详细看需求。开始以为是求每个城市的温度标准差。然后再求标准差的平均数。

测试不通过后，也没有再看需求便假定求当年几个城市所有当年每天温度的标准差，还是不对。到了第四天早上再看看 `ProblemSet5.pdf` 需求描述才知道我一直误解了这功能需求。

`gen_std_devs` 对应每一输入年份，返回一个数值，依据以下计算：

- 1) 按输入的所有城市，计算当年每一天的平均温度（所有城市的平均）
- 2) 计算当年每一天平均温度的标准差

所以如按我本来的算法，算一个城市是正确，但两个或者多个城市的时候，答案就比正确答案大。

针对这个计算标准差问题，一直以为是语法问题公式计算错误问题。但我用一些简单的数据检验去，未能发现任何不正常。来来去去都没有实际的进展，差点想放弃。

经验教训

- 最佳实践：尽量用最小的行数完成功能
 - 像 Python 这类成熟的语言，绝大部分的功能都已经具备
 - 尽量避免基础方法编写一个已有的功能 (reinvent the wheel)，除了耗费时间外，还会写多错多
- 写任何程序前必须先有自动化测试用例
 - 虽然自己可以先想一些数据自己简单测一下
 - 但替代不了另外一个人写的测试用例。跑通才有信心程序正确（所以自动单元测试是使用最多的程序）
- 架构与设计很重要
 - 校方除了提供测试脚本外，还提供了写程序的模板 (`PS5.py`)
 - 每个功能都明确，输入是什么，什么数据类型，输出是什么
 - 学员自己只需要填写实现的代码部分
- 先把握好新功能的用法与结果
 - 前面发现不少缺陷，是因为不清楚某功能的用法
 - 在写代码之前，自己应开个沙盘，用小程序先试验一下这计算功能有什么参数，会出什么结果
- 先评审后测试
 - 在跑程序之前，屏幕展示或打印出刚写好的代码，用铅笔在白纸上按程序的逻辑，从头思考一下每个数据的种类和中间的变化
 - 盲目地去跑程序或测试，最终只得出测试失败，但还不清楚错在哪里，如何修改。所以不如在跑程序或测试前，自己先动脑筋模拟一次，仔细想通每个数据的变化

Chapter 15

代码重构

重构代码的主要目的是使代码更容易维护，其他人更容易读懂。SOLID 原则是做好面向对象设计的基石，例如随便搜索下面 SOLID 表格的关键字，可以找到很多参考资料。(SOLID 是取下面五原则的首英文字母)

SRP 单一职责原则	Single Responsibility Principle
OCP 开放封闭原则	Open Closed Principle
LSP 里氏替换原则	Liskov Substitution Principle
ISP 接口分离原则	Interface Segregation Principle
DIP 依赖倒置原则	Dependency Inversion Principle

也有很多参考书，如 Gamma 和 Fowler 的经典书，例如”Design Patterns: Elements of reusable Object-Oriented Software” 里面有对每一种设计模式的介绍与详细例子。

如何学好 **SOLID**，打好基础做重构

培训经验：以前我们传统教学方式做重构设计模式培训，分别解释、举例说明二十几个设计模式，学生听完后，很快就忘记了，没有任何效果。

要有效学习，程序员必须积极动脑动手，不能单靠理论课。从温伯格先生教程序员的例子（详见附件）看到，如果先在讲课时强调某些重点，然后配上编程练习，学员才能真正吸收，后面能在项目中用上。

如何提升学员的兴趣与动力

我们后面改用工作坊的培训方式，减少理论讲解，要求学生在电脑写小程序，效果比本来仅仅是讲理论好。有些领悟能力比较强的学生，都可以很好完成一系列的练习题，但还是有 40%-50% 的人跟不上。我们在想问题在哪里？如何可以改善？

跟得上的学生本身领会力就比较强，软件开发的基础也较好，所以可以很容易跟得上我们设置的编程练习，在课堂上就可以按我们的设计完成大部分的编程

练习。问题出在那些比较弱的学员，本身对编程和面向对象一知半解，开始时候觉得写小程序跟不上那些其他高水平的同学，后面就放弃了。

培训新入职员工也面临类似问题，而且如果未能在头几个月把重点学好，养成习惯，后面便难以改正。

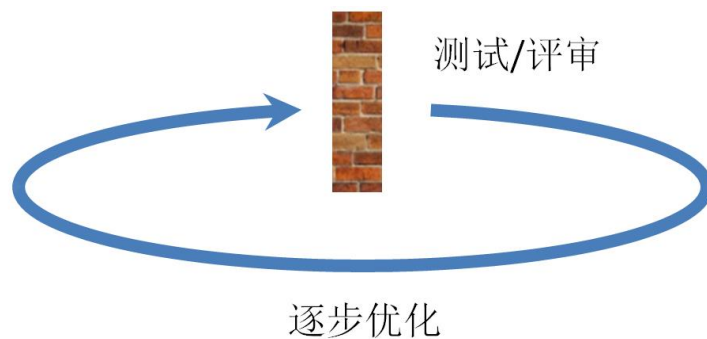
七八十年代，麻省理工科学家 Parpert 先生，他本来一直研究人工智能，他发现很多美国小学生都跟不上学校的代数与几何课，根因是老师用传统教学方法，学生没有兴趣学，他后面就利用乌龟几何 (Mindstorm) 做实验，让小学生可以轻轻松松用电脑编程、玩游戏学习几何（详见附件）。

所以后面我们就利用乌龟学数学的思路，设计了一系列的基于设计模式的练习题。设计模式最大的好处就是方便后面需求的更新。所以我们把那些常用的设计模式写成模块，就像乐高积木一样，让学员可以调用，然后学生就可以像乐高积木，尽量利用那些设计模式的组件，针对课题目设计程序。

做完了第一段后，就有些改变：新增或者减少需求，教他们怎么可以再利用设计模式更新程序，达到题目的要求。我发现利用这种方式，学生后面在实际工作中就更能用上设计模式，因为与之前单纯对应题目编程不同，学员有更大空间设计总体的架构与功能，与前面提的美国小学生一样，因有兴趣，便能很快学到面向对象的编码原理。

什么时候做重构

写程序与写文章类似，都是一个逐步优化的过程，初稿可能先写一些重点，里面的细节需要不断地评审、优化，就更不用说语法、错别字了。



虽然现在我写程序的时间不多，但还是依据同样思路。先把程序写出来，基于测试驱动开发的原则，通过单元测试，但通常第一版程序有很多不足，比如不能达到 SOLID 等面向对象要求，所以会依据设计模式原则优化代码，并利用自动化单元测试确保每次改动后程序可以跑通，所以写代码也是逐步优化的过程，只是代码错了一点就会跑不通，文章因为容错性强，错个别字不影响读者读懂。专业软件工程师不会等到最后关头才去重构，而是持续的小重构，一直考虑软件的维护性、扩展性，如果达不到质量水平，宁可不发布，好比具备专业手艺的陶瓷工匠会砸碎不达标的陶器，或作曲家宁愿烧毁自己不满意的作品而不发布一样。

代码规范

制定编码标准是其中一条 XP 实践，因为若要写好程序、减少错误，必须预先把常见问题罗列出来，提醒大家。

客户：我们一直没有，可否直接参考其他公司的标准？

我：某公司想快速通过认证，但公司一直都没有自己的标准过程，顾问说：“不担心，我们可以提供其他已通过认证公司的过程。”你觉得公司最终会有质量改善吗？

客户：应该没有，因过程不是从本身实战经验出来，对团队提升质量没有用。

我：非常赞同，所以编码标准也必须归纳自己动手的实战经验。像刚才乌龟几何一样，让学生自己通过解决问题并获得及时反馈，才能真正学会。编码规范很重要，但规范不应从人家那里复制过来，而是从自己常见的问题总结出来，对团队才有意义。

持续集成

文章、书本等文字性的不像代码、软件，错一个字都会导致跑不通的问题。所以几十年前人们无法想象现在可以做到这么大型的软件系统开发都起码要几个月，系统规模也不会太大。现代软件开发有很多辅助工具，可以一边写一边检测，也有各个层次的测试来确保软件没问题，并能自动化，让团队可用每天发布，也能支持大型分布式系统。下一章节，我们看看如何利用自动化工具帮我们做到持续集成、持续发布。

附件

小孩利用乌龟玩几何游戏 (Turtle Geometry) 的例子

美国传统怎么教小学生学代数、几何。传统是用纸和笔的方法，小学生就没有兴趣学，导致跟不上。原因很简单，我们以代数为例，现在很多小学生学数学，还是我们几十年前那种，背九宫表算乘除。以前我们的年代没有计算机，所以逼着人要背那些表，就跟古代学习用算盘计算一样。但现在计算机已经可以随时买到，且很便宜，手机也有计算功能，所以他们就认为这种传统的学习方式无用。几何也一样，如果学生比较擅长数学还好，但假如有跟不上的，就会觉得自己不是学数学的人才，后面就很可能放弃。

于是 Parpert 先生教小孩利用乌龟玩几何游戏 (Turtle Geometry)：



学生可以简单利用往前往右往左的命令，画出正方形、三角形：

```
TO SQUARE  
FORWARD 100  
RIGHT 90  
FORWARD 100  
RIGHT 90  
FORWARD 100  
RIGHT 90  
FORWARD 100  
END
```

```
TO SQUARE  
REPEAT 4  
  FORWARD 100  
  RIGHT 90  
END
```

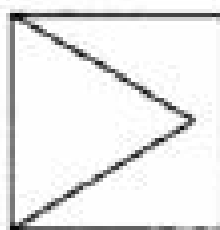
```
TO SQUARE :SIZE  
REPEAT 4  
  FORWARD :SIZE  
  RIGHT 90  
END
```

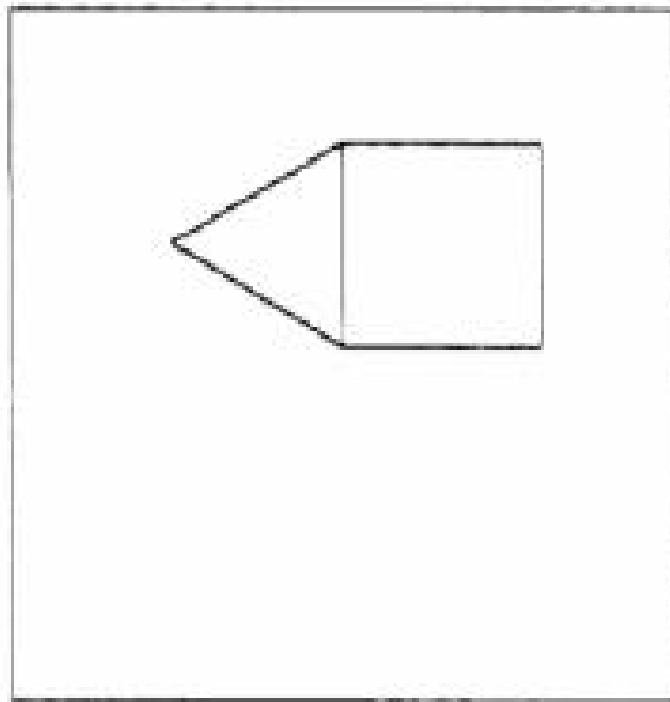
```
TO TRIANGLE  
FORWARD 100  
RIGHT 120  
FORWARD 100  
RIGHT 120  
FORWARD 100  
END
```

```
TO TRIANGLE :SIDE  
REPEAT 3  
  FORWARD :SIDE  
  RIGHT 120  
END
```

编码有误，学生自己调代码，把三角形的方向改正，才能画成一间屋：

TO HOUSE
SQUARE
TRIANGLE
END

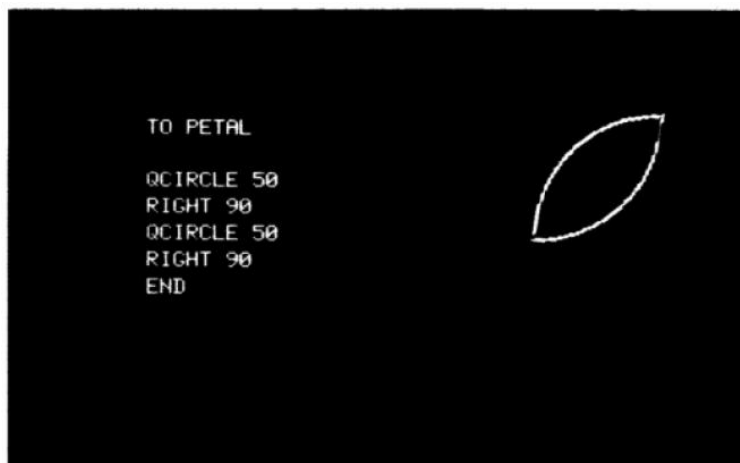




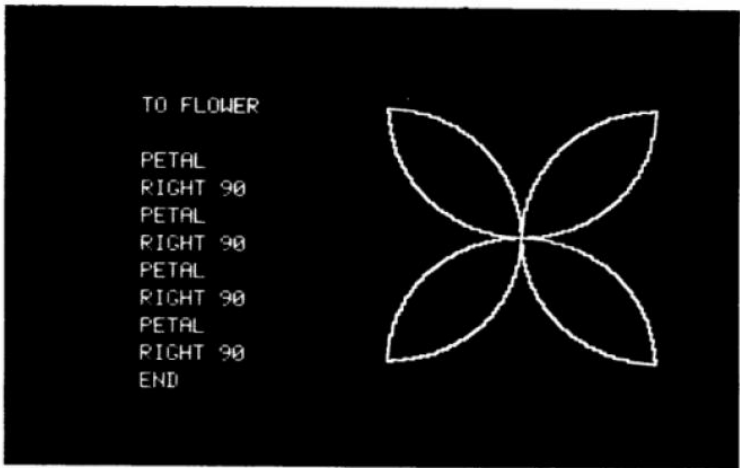
画一朵花的实例

可以用一段圆弧作为花瓣的一边，将它复制后旋转 180° ，这 2 个圆弧便组合成一个完整的花瓣：

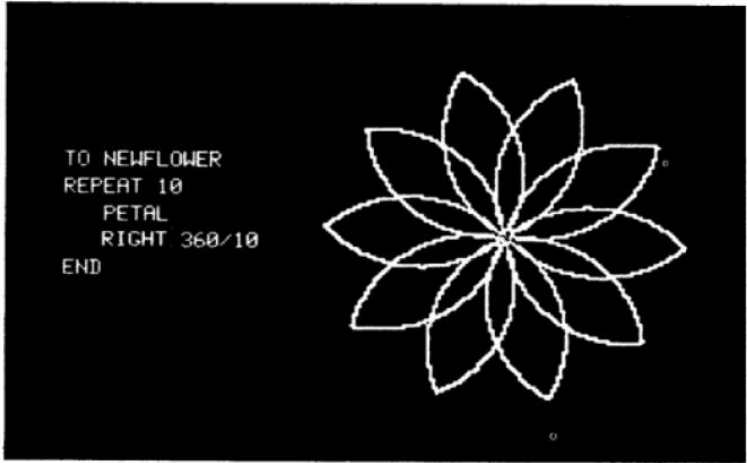
甚至圆形，在这个基础上，他们可以继续画一朵花，通过这种方式，学生就觉得自己可以掌握。



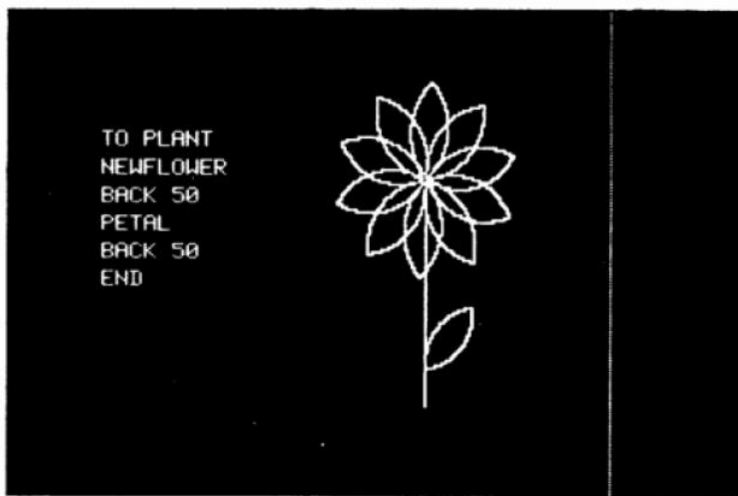
基于花瓣小程序，顺时针旋转 90° ，重复 3 次，便可以画出含 4 个花瓣的花朵：



同样思路，基于花瓣小程序，花顺时针转 36°，重复 9 次，便可以画出含 10 个花瓣的花朵：



基于 10 个花瓣花朵的程序，在其中心点往下走 50 单位，画出一条直线作为花枝，在花枝旁加个花瓣作为叶片，便能画出下图：



乌龟几何游戏不仅让学生发挥无限创意，觉得自己可以掌握写程序画画，同时也学到软件功能复用的原理。

温伯格的教学实验

温伯格先生在 60 年代教程序员时，就发现很多人没有动手，导致上课学过的东西无法用在工作上。为了解决这个问题，他在教 IBMOS360 时，把本来的理论课改成一些工作坊教学形式。他还做了一个小实验，在一班讲课的时候强调要注意一些 OS 语言的常见问题，例如如何避免错误的空格导致编程失败，他发现平均下来还是有 83% 的错误。而另外一个课堂上没有强调这些细节的班级的错误是 87%。他还做了另外一个实验，对第二个班学生增加越来越难的练习题。到了第 10 题，10 个练习，17 个学生中有 13 个基本上就掌握了这技巧，没有再犯同样错误。从这两个实验可以看到，要有效学习，程序员必须积极动脑动手，不能单靠理论课。

参考 References

1. Gamma, Erich et al. "Design Patterns: Elements of reusable Object-Oriented Software." (1994) Addison-Wesley .
2. Papert, Seymour. "Mindstorms: children, computers, and powerful ideas." (1980) Basic Books
3. Weinberg, Gerald. "The Psychology of Computer Programming: Silver anniversary edition." (2021 / 1971)

Chapter 16

持续集成

如果想避免发生第 13 章中哪类客户使用后才暴露的棘手缺陷（可能要花很长时间才能发现并根治的问题），可以依据 Thomas 先生的“每小步验证”原则，开发大型复杂系统，都应先拆分子系统/模块，先开发并测试子系统/模块、集成、再测试，按部就班地完成整个软件开发。

验收测试

验收要满足需求，需求要可验证，所以通过相应需求的测试成为验收通过的必要条件。需求确定后，就可以规范验收通过的准则，开始准备验收测试了。

很多 QA 人员以为验收测试只需要做黑盒测试：

- 把开发出来的系统看成一个黑盒，开发人员已经做好本身的所有自测（包括单元测试集成测试等），然后系统要通过我们的系统测试用例，包括功能与非功能。

常见问题

当开发人员完成了所有模块的开发，开始进行系统测试的时候，往往已经到了项目（或迭代）交付预期的后期，这时如果发现大量缺陷，开发人员没有时间做好修复，即便修复后，也没有充足的时间做回归测试（重新跑本来通过的测试用例，确保不会因为开发人员的修改，而引起新的错误）。

改善方法

要避免这种恶性循环，减少发布延期的概率，QA/测试人员就要关注前面评审和测试（如单元/集成）缺陷排除率，以降低质量成本的概念，尽早在项目的前期暴露缺陷并及时解决，不要等到系统测试时才处理。所以不能单靠开发人员自觉做单元测试/集成测试。

从开发人员的视角，他不会想主动用测试找出问题。
最佳场景：开发完成，交给系统测试。没有发现缺陷，通过并结束，再开发新的模块。
“精明”的编码人员思路：“如果对测试/缺陷没有要求，多严峻的开发进度目标，我都能轻易达成。”

所以要做好验收测试，QA/测试人员不仅仅是做最后的黑盒系统测试，而是要从单元测试开始，设定好测试通过准则 (Definition of Done DoD)，然后交给开发人员执行，要看到所有测试，从单元测试开始，都完全通过，才算通过验收，以避免上面的问题。如果 QA/测试人员的技术能力有限，起码要明确所有测试的准则（测试用例）。反之，QA/测试人员仅做最后黑盒测试，不能按时交付/交付质量问题，始终难以解决。

测试策略小建议：

1. 在项目一开始便要开始写测试用例（在设计、编码之前）。
2. 代码每次有任何变动时，利用持续集成系统自动跑测试。
3. 有了自动化就可以轻松实现回归测试。
4. 以上不仅仅是覆盖功能测试，也包括非功能，例如安全性、性能负载量等。

目的：尽早让开发人员获得反馈，以此驱动他们能随时达到一个可发布的状态。自动化让测试人员可以集中精力针对一些特有的测试，减少重复性测试的工作量。

测试自动化

与设计开发人员探讨他们的产品集成过程：
问：你们有没有用一些自动集成的工具，例如 Jenkins 集成工作自动化，原因是集成很花时间，现在也越来越流行持续集成、每天集成，所以如果靠手工集成、打包是很花工作量的。
答：没有，我们还是手工。
问：你们在手工集成之前怎么确保这些模块已经可以？
答：我们有测试。
问：做什么测试，可否举些例子？
答：我们开发完以后，会输入一些查询，然后看是否能正常使用。
问：我不是指整个系统功能的测试。例如，要通过你们 Java 开发，有自动化单元测试，可以写脚本，如果你单元测试通过，就变绿，不然的话就红，你们有做吗？
答：没有，我们都是手工测试。其实我们现在到了产品升级、增加功能的维护期，记得当初首次开发的时候，我们开发人员是有用 Jenkins 自动化集成，但现在维护期我们就没有。

从以上对话可以了解到，很多开发维护团队基本就没有再做一步一步集成的过程，只是针对修改的功能，开发完就测试一下，然后便交到系统测试。

如果只是依靠对整个系统的集成测试：

- 不一定能找到所有问题。当我们改动了一些代码，却没有通过模块的单元

测试，就很难确保模块的每一次改动都是正确的。所以，这将导致如果后续确有问题的话，就很难定位或者难以修复。

- 如果没有单元测试的话，你其实是不敢改动代码的，不知道改完以后对不对。

所以单元测试很重要，也因为单元测试高复用度，所以需要自动化才能节省大量手工测试工作量。

集成的概念，就是先写自动化单元测试用例，然后才写代码。通过单元测试才可以进入下一步集成测试，两个模块之间是否通。到最后进行总的系统测试，一步一步去做。如果团队像刚才那种情况，绕过了那些步骤，直接做总系统或集成测试。就会有很多隐患，到最终的验收测试才被发现，付出大量的缺陷修复代价。产品集成很重要、很花时间，所以绝大部分的团队如果注重软件质量的话，都会把产品集成自动化。改了代码之后，每天靠脚本来通过单元测试，比如晚上自动跑产品集成发现的问题，第二天暴露给开发人员修改，修改后再集成看看有没有问题出现。有些公司除了要求单元测试通过以外，也会要求通过代码的静态扫描，道理都一样。只有用这种一层一层进行产品集成的方式，才可以有效确保软件的质量。正如 Cunningham、Fowler 等敏捷大师所说，没有做好集成过程里的测试会产生很多债务，产品交付给客户以后，因为质量问题导致还债的代价就更高（详见后面“技术债务例子”）。

持续集成

持续集成的道理很简单，每次提交代码做的修改，系统都可以自动构建整个系统，并跑通相关的自动化测试，如果发现自动构建出错必须立马停下来修改缺陷。（详见附件“持续集成步骤”）团队要开始持续集成，起码要具备以下 3 个条件：

1. 版本管理：所有的代码测试脚本、数据库脚本/构建脚本等都放在版本管理系统管理，不仅仅是代码。
2. 利用工具实现自动构建（例如 Jenkins），虽然很多 IDE 集成开发环境都有这方面的功能，我们建议还是要用命令式（Command）去跑构建脚本（像跑程序一样），每一次构建都能有详细记录（也随时可以重跑）。
3. 持续集成不仅仅依赖工具自动化，还需要整个团队高度付出、参与和自律，每个人频繁以每个小步交付他的开发部分。James Shore 先生在“Continuous integration on a dollar a day”文章里提出不一定依赖自动集成工具（例如 CruiseControl 是另一种类似 Jenkins 的集成工具）只要每个团队成员都做好自动化单元测试，每次提交代码都使用版本管理系统合并，也能做到持续集成。



不要以为每个开发人员每天提交（Commit）代码到版本管理系统就算做到持续

集成了，还需要：

- 每次提交代码要确保已经测试过，没有问题。
- 所有部署发布后的问题都能在 10 分钟之内解决。

不然就不算达到持续集成的基本条件。所以每天提交只是持续集成的第一步。

所以要做到持续集成得符合以下 3 个条件：

1. 是否频繁把变更更新到主干去。
2. 有没有对应的自动化测试，确保集成后的代码是能通过所有测验。
3. 如果构建失败不能通过测试，必须立马停下修正代码，直到测试通过。

有人会质疑为什么要这么频繁去更新主干的代码，自己分析测试好不也一样？很多持续集成的团队只做到第 1 点，部分能做到第 2 点，但能把 3 点都满足的很少。

如果只是自己测试，不能尽早暴露团队成员之间的代码冲突；如果可以利用自动化测试，频繁进行自动构建的话，就能及时暴露这些冲突。而且，因为每次变更的范围比较小，所以比较容易修复，反之，如果等写了几千几万行代码后再测试，就可能积累了很多的集成问题，难以有效修正。自动构建有版本管理系统支持，我们可以知道每一次变更的内容，也可以回滚到之前某稳定版本。

团队协作

当多位开发人员协作编码时，首先必须让他们做到“同步”，不然无法做到持续集成，先分享以下我的培训经历。

早上讲完结对编程的基本知识，下午让学员按结对编程做编码练习：

互动练习分 4 个阶段，每阶段都有明确的产物，计划总共花一个小时。（为了强调结对非固定，要更换，所以规定每个结对只有 15 分钟，然后换人继续 15 分钟编码。）

因为刚好有一半学员是一直做开发，另一半是做测试（公司已经推动自动化测试好几年了，所以测试人员也懂开发）。我们把开发归 A 组，测试归 B 组。

过了 20 分钟 A 组完成任务，B 组还没有完成。

为了两边同步，我们让 A 组等等，但过了 10 分钟 B 组还未完成，没有办法，我们让 A 组继续做下一个阶段，最终 A 组用了 1 小时 20 分钟完成所有阶段的编码，但 B 组到最后连第一个阶段都还未完成。（我们其实一直 B 组，最终到了 1 小时 45 分钟才终止。）

我们在旁观察，主要原因是 B 组有两位成员能力很差。

经验教训

分组原则：不仅仅看团队成员间能否顺利沟通，也需要他们的技术能力接近。

信息辐射器

当团队的能力相当、节奏可以同步后，便可以让团队自己设定奖励/惩罚机制，然后在大家都能看到的墙上，使用日历监控每天的情况。

敏捷大师 Martin 先生的建议：如果哪位成员提交的代码破坏了团队构建，他便要穿上一件脏且臭的 T 恤，上面写着“我打破了构建”(I Break the Build)，并且要在墙上日历当天贴上红点，如果当天全部构件都成功，贴上绿点。团队首个月还是红点居多，过了两个月后便逐渐反过来，变成绿点较多了。

持续交付

早在 90 年代，XP（极限编程）的创始人 Kent Beck 先生带领瑞士某保险公司的团队做软件开发，他们当时已经可以做到每晚发布了。越来越多的软件团队（尤其是互联网产品类公司）已经按照“尽量缩短交付时间 (Cycle time)、尽快得到反馈”这样的思路在做了。

因为软件从完成编码到可以在客户投产环境成功部署，中间很多地方可能出问题，编码后必须经过构建 (Build)、单元测试 (Unit test)、集成/系统测试，最终在接近现场客户环境完成验收测试，才有信心能成功部署。

把软件系统分成多个子系统/模块，分到几位开发人员并行开发，各模块必须整合，并通过集成/系统测试，所以持续集成是持续交付的基础。

要做到持续交付，除了整个部署流程要自动化外，版本 / 配置管理也非常重要，不仅包括代码，也包括例如数据库 (DB schema)，脚本 (Script)，环境配置 (Configuration) 等，因为如果这些变动发生任何问题都可能会影响交付。

你可能会觉得持续交付太理想，难以达到。实际上有些面对全球客户的互联网公司 (例如 Amazon) 已经做到每天部署，但不要误会持续交付很容易做到，这些成功案例都已经经过多年的不断过程改进，并配合自动化工具，才能做到。

附件

持续集成

有很多开源的软件可以下载来用，选好你要用的持续集成 (CI) 软件后，你就开始要安装使用，希望利用工具来实现自动构建，跟安装其他软件一样，开始的时候会遇到困难，你可以在你项目的 WIKI 记录下来，让其他人知道，避免以后重复错误。接下来，大家可以开始用服务器持续集成：

1. 当你准备好提交 (Check in) 你的最新变更时，要确保它是否能正常运行。
2. 它可以正常运行并通过测试，你应该把你的代码从你的开发环境提交到你的版本管理 (Version Control Repository) 系统去，也看有没有更新。
3. 跑构建脚本和测试，确保所有过程都可以在你的电脑正常操作。
4. 如果你在本地构建成功通过，就可以把你的代码提交到版本管理系统。
5. 让持续集成工具自动构建你提交的更新。
6. 如果不通过，你要尽快在自己的机器上修改问题，返回第三步。
7. 如果构建成功，你就可以进入下一步：完成。

如果团队成员都按照以上的简单步骤，团队便有信心使 (开发的) 软件在任何电脑上，以同样的配置都能成功运行，一些持续集成的注意点：

1. 定时不断提交 (Check in Regularly)，可以想象你写了很多代码才发现问题，后面你会花更大精力来找出问题
2. 创建全自动化测试套件 (Create an Automated Test Suite)。以单元测试为例，很多编码人员觉得写脚本式自动化单元测试浪费时间，宁愿手工单元测试，因他们以为只要在写完代码后跑单元测试，证明代码没有问题。当后面集成/系统测试发现缺陷，只要修改代码的错误部分，并能通过集成/系统测试便可。但修改后的代码还能通过本来的单元测试吗？不一定。所以任何代码修改后，必须再通过整个部署流程 (Deployment Pipeline)，里面包括单元测试。如果利用流程自动化实现持续集成（包括单元测试），便可节省用于手工测试的工作量。如果编码人员了解这个道理，便不会再说自动化测试浪费时间了。（注）
3. 保持较短的构建和测试过程，如果可以把测试和构建过程缩短的话，就不会等到一大堆问题才要解决，就像我们把复杂的系统分成几个子系统，模块逐个开发的道理一样。
4. 管好自己的开发工作区，不要只做好代码的版本管理，配置管理应包括测试数据、数据库脚本、构建脚本、安装脚本等，因为每一块都会导致你的软件运作不成功。

（注：为什么单元测试很重要已在《TDD 测试驱动开发》里说明）

技术债务例子

员工：总监，我们过程改进是以高管的关注点出发的，请问你有什么需求？
总监：我们的产品刚发布后，客户做了软件安全性检测，发现有漏洞，这种情况就表示软件质量有问题，客户感受也很不好。我们后面也很难去修正，不知如何入手，你有什么好建议？
我：你们对产品有没有安全性的需求规范？在开始设计或者开发时，有什么对应措施？我们都很清楚，软件产品到了交付阶段，已经把各模块集成在一起了，如果这时候发现缺陷，解决起来是很困难又费时的。你现在只知道有安全漏洞，但不知道漏洞源自哪些模块，就很难知道怎么去改，改任何代码也可能会影响其他的系统功能。所以在交付后，才发现缺陷的返工工作量是很高的，可能要花好几天时间才可以正式改正。但如果你们在开发之初，就设计好对应安全需求的单元测试、集成测试、系统测试等的自动测试用例，每次变更代码都在持续集成、自动构建里通过测试，便可以避免后面这种验收才出现问题。

有些编码人员争辩说：“写自动化测试工作量很大，我们也不是开发产品，都是定制化开发。”但当他们理解到这些测试不是到最后跑一次，他们才会理解不能靠手工测试，必须自动化。

只有依赖自动构建、自动集成，程序员才敢改动本来的系统的代码。如果没有单元测试、集成测试把关，只靠最后对系统做功能上的测试，无法知道中间的改动会不会导致一些本来良好的功能整出问题，最后便导致客户投诉有安全问题。当系统已经是维护阶段，因为大部分代码都不是维护工程师写，如没有这种一步一步的自动测试来保证，程序员根本就不敢改动任何一行代码，软件开发行业称为“死”软件。

参考 References

1. Humble, Jez. "Continuous Delivery." (2010)
2. Shore, James. "Continuous integration on a dollar a day" (www.jamesshore.com)
3. Martin, Robert C. "Clean Agile - Back to Basics." (2019)

Chapter 17

结对编程与同行评审

客户：我们的客户以银行为主，他们很注重质量，所以一直很注重评审。他们对需求评审、代码走查等也很赞同，也能找到缺陷，对提升质量有作用。但他们最困惑的是通过设计评审很难发现缺陷。

我：你听说过敏捷的结对编程吗？

客户：听过，也给客户做过培训，但是一般管理层都接受不了用两个人干一个人的活，所以几乎都没有团队在实际工作中使用。

我：当写完了几千行代码后，就很难在评审时要求开发改动。程序员会觉得又不是不对，测试也跑得通，为什么要改。所以代码的质量问题应在写的时候就避免，这是结对编程的原理。你可以把结对编程看成是一个提高团队之间互相交流的效率的工具。因为结对编程是在编码时针对具体问题集思广益进行解决，很容易落地，效果会比培训或评审好。

比如我跟另一位老师看一些团队的代码，他们都写了起码几千行。我们的结论就是：虽然代码的设计很有问题，但已经到了这个地步，除非重写，否则单靠修正部分代码并没有帮助。这也是设计评审常常难以做好的主要原因。而且大家都要面子，如果在评审里，直接说他设计有问题，也可能会影响到大家的关系，所以在设计评审时难以找出缺陷，这很正常。

客户：你说的很有道理，但为什么这 20 年没有流行？

我：有很多人不了解怎样做好结对编程。例如：

- 以为结对编程必须要找一位合适的搭档，人与人之间是否合得来最重要。
- 以为结对时只是在旁边看。
- 以为仅仅是培训，教同伴应怎么做。
- 以为只是看，作为人手编译器，只看代码是否写对，能否通过编译。

以上都并非结对编程的目的，也正因为这么多误解，才导致结对编程没有效果，后面就放弃了。代码规范很重要，你同意吗？

客户：同意。

我：代码规范本应是依据自己面对过的问题，形成清单，提醒自己要注意哪些问题——从实战中累积出来的检查清单，才有实用性。但很多团队的没有自己的代码规范，或只使用其他公司的。但如果团队结对编程，便能相互碰撞交流，

可以更好地识别出规范应包括的检查项。

客户：我也很相信结对编程有用，但有什么好的方法去说服老板，得到他的支持。

我：你觉得结对编程后主要的好处是什么？

客户：代码的质量应该有所完善。

我：同意，但能否具体一点呢？例如之前有什么质量的问题影响项目客户。

客户：比如最近我们有个项目，测试都通过了，但是代码难以理解，读不懂。

我：同意，怎么解决这个问题？

客户：最终我们花费了大量时间对代码添加注释，还写了很多文档来解释。

我：后面加注释，添加支持类文档已经是亡羊补牢了，本该在写代码时就注意可读性。例如你觉得下面附件 A 与附件 B 代码，哪种较好？

客户：附件 A

我：其实附件 A 和附件 B 都没错，但附件 A 比附件 B 容易理解。所以如果团队用结对编程，就可以解决写代码时的质量问题，确保可读性。减少后面再后补文档或者再重构的时间。如果你觉得代码可读性和后面难以维护是问题，让我们探索一下具体是什么问题，怎样解决。通常你们通过评审能发现多少问题 (Bug) 并在评审后修正、改善？

客户：几乎没有从评审找出可读性问题，我也带过一些初级的程序员，确实出现了不少这类问题，但项目时间很紧，我只能暴露那些最主要的大问题，这是可读性问题，确实很多时候就没有时间去处理了。

我：你可以估计一下，如果用了结对编程，要达到质量要求，后面可以减少返工。

客户：懂你的意思了，应该像前面 API 里那样，估算结对编程能为公司节省多少返工成本，并估计能带来多少回报。

我：是的，但估算节省时请注意不要误以为结对编程只能改善可读性，它只是其中一类。

客户：如老板也赞同，应该怎么开始？

我：因大家都从未试过，可以先做一天培训，先感受一下应如何结对编程。因为没有培训的话，很多原理大家还是无法弄懂。

培训

结对编程目标

- 尽早识别问题，减少后面的返工。
- 促进知识分享和团队合作文化。
- 让两人可以相互协助。
- 同伴可放心评判对方的弱项。

先用代码实例让学员了解当前编码的不足，提出以下改进目标

- 促进代码规范。
- 提高编码可读性。
- 方便后期复用。
- 有单元测试 (测试驱动开发)。
- 更好了解需求。

用小组互动练习，让学员体会以下原则

1. 一边编码一边说话。
2. 不断地同步问问题，比如这个方法是否应放在这个类上面。
3. 每个人都要带小本子和笔。
4. 最后应该不断地重用、复用。

工具

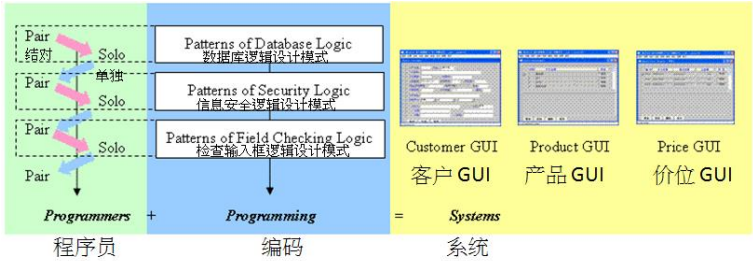
工作环境很重要，例如可以用一个大屏幕让两个人一起写和看代码：



或者用 Teamviewer 共享工具，让两位程序员各自用自己一台笔记本电脑：

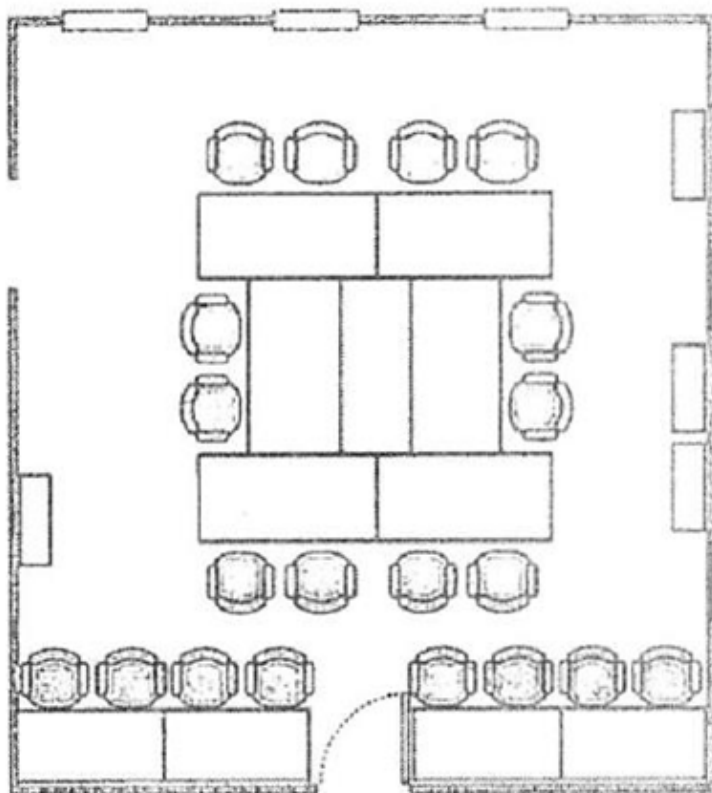


不一定全程都是结对，可以与单独交替



不断替换

如果结对编程总是由某一位编码，这容易开始，但是很难持续，这种互相分享知识的空间就越来越低。通常我们有好几种方式去对工作进行分工来做结对编程。例如最简单的是结对跟单独的交替或者共用跟私有结对等，也要考虑工位的安排。例如下图，有些座位可以让他们做到一起结对。另外有些是让他们私人做编程，减少干扰。



培训后的经验分享

培训的目的是让团队开始了解结对编程的精神：

1. 我们先定一些小目标，例如提高代码的可读性。
2. 代码能更好应对需求变化，尽量复用，但不是拷贝粘贴，尽量使用设计模式，而不是自己弄一个新的设计方法。

所以我们首先要有一个基本的代码规范，而且要求都写清楚。大家都了解以上原则后，可以让团队开始实战，通过练习感受结对编程的过程。很多时候学员都不习惯一边写代码，一边讲或者沟通，所以必须利用互动练习帮助学生提升这方面的能力。

结束语

经过合适培训后，结对编程比评审更能有效地帮助团队写好代码，提高软件质量，并减少返工成本。

如老板赞同很多软件工程师经验少、没注意代码质量，导致后面大量返工，软件产品无法维护，我们除了可以以节省成本为由外，还可以用以下理由说服他：

- 不是所有代码都结对编程，而是针对重要代码结对编程。例如项目初期搭

建架构和写偏底层一些的代码，以及同类功能用于当样板的代码。

- 结对编程, 因两人一起解决实际难题, 很适合用于以老带新的工作方式, 效果比讲解/培训好。

附件

代码可读性例子

A

```
tmpDate="2013-02-01"
for (k in 1: n)
{
    tmpDate=tmpDate+1
    .....
}
```

B

```
for (k in 1: n)
{
    tmpDate="2013-02-01"+k
    .....
}
```

A

```
bonus      = if (0==overwork) 3 else 6
monthPaid = basic + bonus
```

B

```
monthPaid = basic +
            (if (0==colConDex) 3 else 6)
```

A

```
time>=(as.Date(today)-30) && time<=today
```

B

```
(as.Date(today)-30) <= time && time<=today
```


Chapter 18

软件需求

客户：公司准备推行软件需求与研发分离。

我：为什么要分开？

客户：公司希望可以提高需求的独立性，也希望需求可以与研发相互制衡。以前需求是研发团队的一部分，很多时候就会倾向于从研发团队的角度来看需求。分开后可以更加独立，更多地从客户的视角看需求。

我们严格要求做需求评审，可否用评审结果来评判需求的质量？

我：我常常问团队，需求评审找出的缺陷是越多越好，还是越少越好？如果有人说是越少越好，我就会继续问是否 0 缺陷就是质量最好，所以我们难以单纯地用评审发现的缺陷数去衡量需求的质量。

让我先分享一下是如何来衡量过程质量。例如如何衡量系统测试的质量。看产品交付后有多少缺陷遗漏。如果没有任何遗漏的缺陷，那就表示系统测试做得很到位。如果往前倒退一步，可以从系统测试或验收测试发现的缺陷数来评判需求、设计或者编码的质量，如测试出的缺陷多就表示质量不好。但测试工程师无法判断那些系统测试的缺陷是因为代码还是需求引起的，所以如果要判断需求的质量，就必须在团队迭代回顾时，让所有成员（包括需求、设计、编码、项目经理等）一起分析缺陷源自哪里。所以过程的质量必须要从它的下游结果来判断。

我：你们把需求与开发分家以后有没有遇到什么问题？

客户：遇到问题可不少，例如研发团队会说，你需求写得太简单了，没写清楚，无法开发。而需求人员却说，你要写到多细才可以开发，你可以给我一个标准吗？甚至有些较极端的研发人员，明知需求有问题，但是还是按需求的描述做开发。

我：你记得我们解读软件工程时，说需求与开发是不断优化、完善的循环过程。需求不可能一次性写好，而是先依据客户提出的一些初步的概念，内部与开发讨论，然后再与客户交流，慢慢细化的。但如果把需求与开发切开，就切断了这个交流的过程。

软件开发一直困惑要怎么写好需求，但一直都没有找到能写出完美的需求文档、开发出完美的软件产品的秘方。有些团队为了写好需求文档，花了很多精力，但后面开发出来的产品还是问题多多，甚至失败。所以在 2001 年时，17 位敏捷

大师针对当时瀑布式开发侧重文档的弊端，提倡敏捷开发，希望更有效地开发出客户满意的软件产品，而不是把时间浪费在前期文档上。敏捷开发模式也能更好地应对当时需求经常变化，导致后面项目延误和质量不好的问题。所以如果硬要将需求、研发分离，就是回到之前那种瀑布式开发的思路：花大量的精力希望写一个完美的需求文档。（敏捷开发的思路：软件开发的蓝图不是设计文档，也不是需求文档，而是代码本身。只有一个跑得动的代码，才是一个可交付、持久的东西。）

50 年代针对英国煤矿生产问题的研究（详见附件）证明，要提升生产率不能单靠科学化的过程管理来实现，还必须有团队内部的相互合作。采煤这类传统低科技行业尚且需要团队合作，软件开发就更依赖于知识工作者的团队合作了。

客户：我们公司领导希望利用标准化的流程跟一些相关的度量指标去控制整个过程。我就是负责制定这些指标的。但有个问题，比如我们觉得研发做出来的产品应该得到客户的认同、让客户满意。所以客户满意度应该是个很重要的指标项。但研发组长就拒绝了 this 指标，他说现在需求不是我们负责，我们只是做研发，为什么最终那个产品的满意度要由我们研发负责，因为我们控制不了需求的质量。如果我们要需求人员或者产品经理对最终的客户满意度负责，体现在 KPI 指标，他们也会说，研发不是我们管，我们只做需求，开发做出来的产品客户不满意，为什么让我们买单？其实两边都有道理。如果我们希望只通过一些指标，比如针对研发的指标，反而会对总体的效果有负面影响，比如客户满意度。但是大家也知道，如果客户对一个产品不满意，那开发肯定有问题。所以只度量某些过程没有意义，就好比如果评审或者测试的缺陷数越高，奖励越多，聪明的团队可以很轻松地变成百万富翁。人很聪明，针对指标，他们都会有方式达到要求。

客户：你说得很对，我们那些团队就很懂怎么朝那个指标作假。

我：所以要有合理的度量，尤其是和奖励挂钩的度量就必须是客观、项目间可比和全面。例如最终盈利多少比较客观，或客户满意度指标也可以很好地激励团队之间相互合作的积极性，对公司、对团队都有好处。但是如果只是用个别过程的指标去衡量，反而可能影响团队行为。

客户：公司觉得今年改革之前做新产品开发项目的立项太宽松，缺乏有效的管理和监督，这导致很多新的研发项目都是没有回报或者失败，浪费了很多资源。所以现在用新方式，所有立项的研发项目，都应该由中央委员会审批才能立项和开始做，我觉得这个倒是挺好的。

我：你觉得好吗？

你听过苹果公司的故事吗？苹果公司本身就是由两人创办：一位是我们都认识的乔布斯、另外一位是沃兹 (Steve Wozniak)，他本来是惠普的工程师，在 75 76 年间研发出了苹果电脑 (Apple 1)，本来希望获得惠普高层的支持，就申请内部立项，但多次都被拒绝。乔布斯看到这个产品后，觉得很有市场潜力，就跟沃兹跑市场，最终获得 Byte Shop 老板 Paul Terrell 的首张订单，购买了 200 套苹果电脑 (Apple 1)。当时 Apple 1 都由沃兹手工焊接而成。创新产品都必须具备以下重要元素：

1. 了解市场需求的人员，乔布斯就是这个角色。
2. 研发工程师沃兹。

两者相互合作才把苹果产品做成。如果把需求跟研发切开，估计苹果的传奇故事永远都不会诞生。正是两人的团队合作，一起去满足个人电脑需求的市场才

会成功。从这个例子可以知道，要创新就不能用传统的工厂化流程，靠一些指标去管理。所以若单依赖一些质量指标去管理需求和研发过程，是不利于团队与公司的健康发展。

惠普公司很优秀，内部肯定不缺资深专家，评判的指标也应该很完善，为什么沃兹的内部申请立项，多次都被拒绝？惠普一直都是专注于做度量和测量仪器，但个人电脑是全新的概念，所以单靠现有专家的指标是无法鼓励项目创新。发明全新的东西必须是需求配合技术开发工程师，再有客户的接触跟确认，才可以成功实现。

客户：有道理，我们现在在新方式下几乎都没有什么创新了，都是按以前的方式做。

我：但这正是软件开发公司最致命的问题，没有创新，后面就会被竞争对手超越。

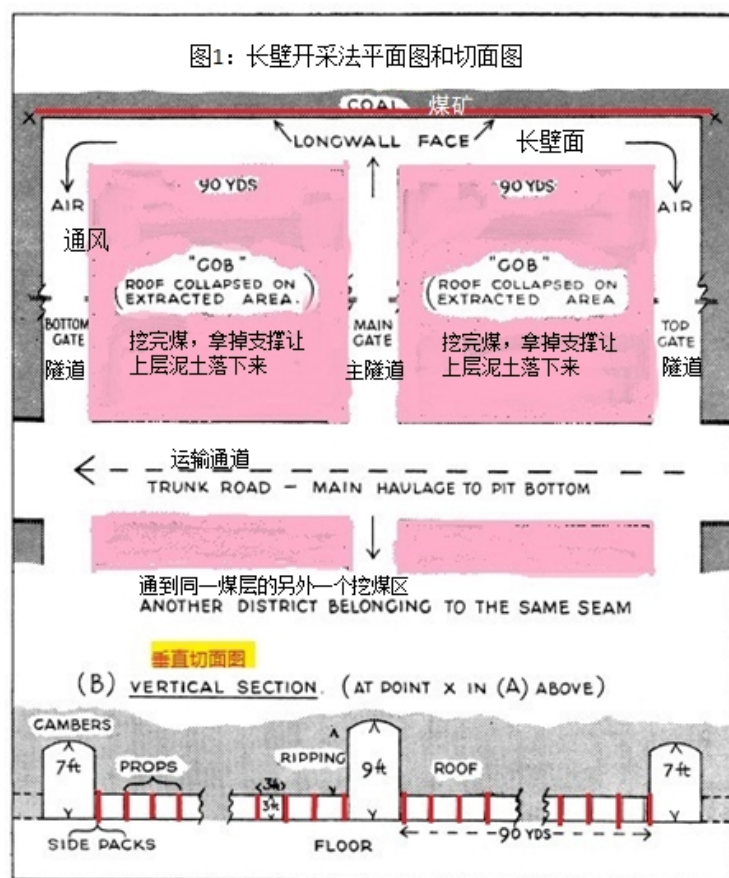
附件

英国煤矿生产问题的研究

在机械化之前，英国采煤是以小组作坊式工作，通常是两人一组，再加上一位做清理的助手。小组会直接跟矿场合作，专门负责某一块煤矿，小组以互补的形式做各种工作，自己管自己，独立性很高。因为采矿工作非常危险，所以选择伙伴很重要，都希望有稳定的伙伴关系，不会轻易换人。当某工人受伤或者去世，他的伙伴都会尽力帮助他的家人。这种小组作坊式工作方式后来被机械化生产线模式取代，但机械化生产却引起很多新问题。

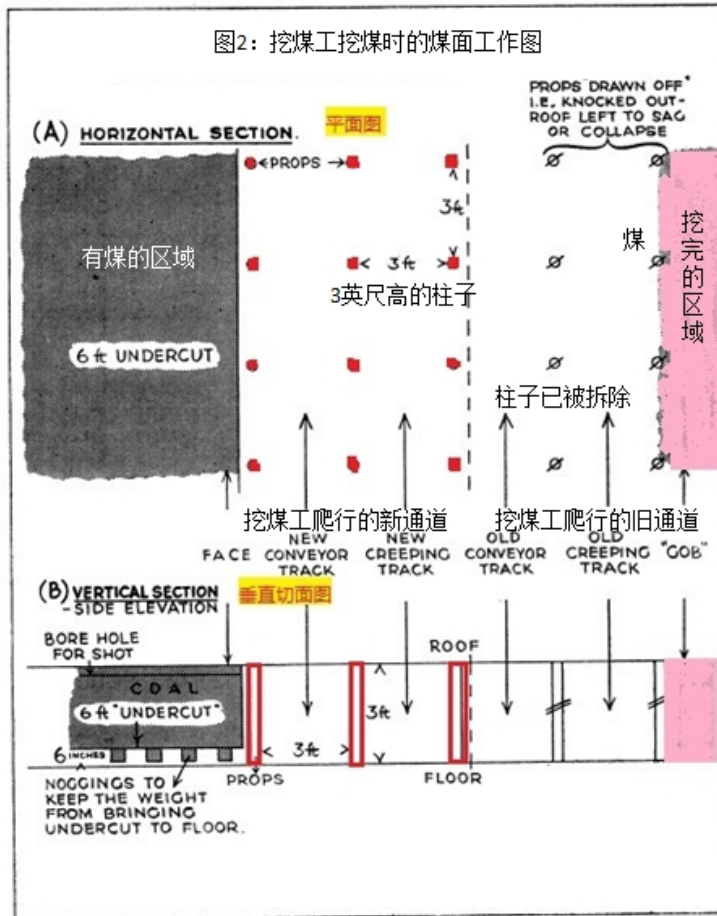
先了解一下长壁（Longwall）开采法是怎么利用机械化大规模开采煤矿。

煤是古代的植物经过很多年的碳化累计下来的产物，英国的煤层一般不深，最多可能一米左右，上面覆盖着泥土。用长壁机械化采煤方式就可以大面积采煤。我们看看它是怎么操作的。



(上图 A 部分是煤矿俯视图, B 部分是俯视图中右到左横虚线的切面图, 都能看到是左、中、右 3 条隧道)

共 180 米宽的长壁是为了大面积采煤, (看上图) 为了通风和运输, 会有 3 条隧道, 之间距离 90 米, 中间隧道是比较高, 大概有 3 米 (9 英尺), 左右隧道比较矮, 大概 2 米 (6 英尺) 高。这些隧道都是固定, 让人或者那些机械也可以在中间运输。看上面平面图, 粉红色是已经采完煤的泥土。从下面的切面线可以看到中间的和两边两个隧道。每个采煤的煤层, 只有 1 米高。也可以看见红色的柱子, 柱子是用来顶住上面的泥土, 防止泥土在煤挖空以后塌下来。



(图 2 说明：与图 1 类似，A 部分是俯视图 (平面图)，B 部分是横的切面图)

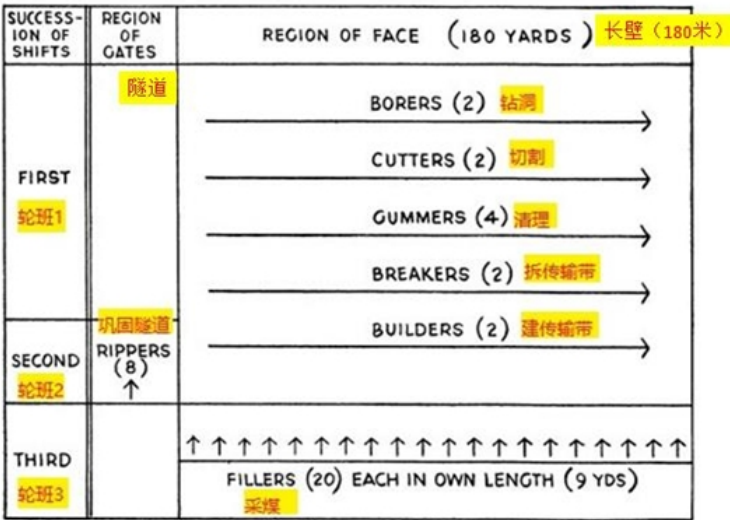
可以从上面那平面图理解整个长壁 (Longwall) 如何操作。右面就是已经采完煤的那些泥土，我们用粉红色标注，叫 GOB。右面有两条垂直的过道，都是 1 米宽。前面两米宽的那条过道，是之前已开采完的，它左面会有一条传输带运作的通道，右面是一条只有 1 米高的通道，以便采煤工人爬过去工作。你可以想象环境多么恶劣。左面那个通道就是一条新建的通道，同样它左面是传送带，右面是矿工爬行通道。每一次采完右面的煤以后，有工人会把那些柱子拆掉，放到新的通道里面，然后再依据这条通道挖左面的煤，一直往左边伸。你可以想象在第一个图的左、右面 90 米宽的长壁，就是一个连续往上升的一条线，矿工按分配的工种和轮班去挖煤。为了配合这种机械化挖煤方式，工人就不能再用以前的作坊式小组工作。工业工程师设计了 7 个工种，请参照下面的轮班表。每循环会包括 3 个轮班：

- 第 1 个轮班主要是做转动和切割和清理，比如让那些切割的煤有空位落下来，让后面那些采煤工人容易可以采到煤。通常第一个轮班都是下午班或者晚班。

- 第 2 个轮班就是巩固隧道和安装传输带，通常是下午班或者晚班，这 2 个班总共就会大概有 20 个工人。细分到不同的工种，比如有 2 位只钻洞、2 位只切割、8 位专门管理通风隧道的巩固工作，分工很细。
- 第 3 个轮班主要有 20 位采煤工人操作。第 3 个轮班只有早上班和下午班，所以他们需要依赖前期技术人员做好准备工作，然后自己按大概 9 米的范围去采煤。收入是按采煤量计算，比如 20 个采煤人每人负责 9 米，加起来刚好等于整个长壁的 180 米。

分工设计很科学，如果都按正常操作，每次循环可以采到 200 吨煤。

图3：长壁工人的岗位和在煤矿的工作



但是你估计，这种机械化的大规模连续采煤操作会有什么问题？生产力效果怎么样？

以上的工作关系很密切，例如：

- 如果洞钻得不够深，采煤工便采不到煤。
- 如果切割没做好会导致采煤工可采空间不足，影响产量。
- 如传输带没有安装好，不成直线就导致后面的传输停顿，采煤工无法操作。

各种状况都可能发生，加上煤矿是天然的，本身就有很多变数：例如地层有断层，或者地下天然气等。各种人为因素或天然因素都影响采煤工能否正常采煤。加上采煤的工作环境很恶劣，导致旷工问题很严重。例如某采煤工因前面 2 个轮班工作没做好，导致煤太硬，采不了，就需要用机器、工具去采，很辛苦，效果也不好，导致他工作 2 天就放一天假，然后也不补班。有些轮班因为人手不够，剩下的采矿工人就需要在地下多做两三个小时，才可以把整个长壁做完。最后当有些轮班时，采矿人数确实太少了，其他人也撂担子，导致无法正常开工。因为采用机械化大规模采煤的各种问题，有些矿场开始引入自主团队组织架构取代原本的轮班分工（下面称为传统分工）

表 1 是两个不同的组织架构，技术都一样，也是在同一个环境。左面用原本长壁的方式设计工作分工，右面的加上自主团队负责制分工。左面矿工只需要做一项简单工作，比如采煤，与其他工人没有任何关系（在前面已详细举例说明）。右面就需要组员合作，协调应对采矿的各种特殊情况。从这个例子看到同样一个采煤技术，可以有不同的组织架构支撑，效果完全不一样。所以团队组织架构必须与技术部分相配合，不能单独设计工人工作。

	Conventional 传统分工	Composite 自主团队
Number of men 人数	41	41
Number of segregated tasks 任务数	14	1
Mean job variation for members 每成员的平均任务/变化数:		
Tasks work with 要处理的任务:	1.0	5.5
Main tasks worked 主要任务:	1.0	3.6
Diff. shifts worked 不同的轮班:	2.0	2.9

从下面表 2 可以看到右面的组织架构更灵活，生产率比传统的单一工作设计高。右面的自主团队架构更能适应变化万千的采矿、采煤环境。采煤的步骤安排，也必须按照环境的不同来应对。但如果是左面的硬性单一分工安排，缺乏这种灵活性（除非是专业性强的技术工种，必须把工作细分，每个人做某一件事）。但在采煤机械化、批量生产这种技术没有这个限制，所以采煤工人经过一些学习可以兼任其他工作。

	Conventional 传统分工	Composite 自主团队
Productivity(%) 生产率	78	95
Ancillary work at face(hrs per man-shift) 辅助工作 (小时/每轮班)	1.32	0.03
平均后备人力/总人力 (%)	6	no
Shifts with cycle lag 轮班延迟 (%)	69	5
最长连续轮班数 (没有轮班有问题导致取消)	12	65
average per cent of coal won each day 平均每天获得的煤炭%		

团队组织架构除了让工人更好做好采矿工作以外，也可以更好满足工人的个人心理需要，包括互相支持，有团队合作的概念。但是如果按左面传统的工业化分工，就缺乏合作。导致很多采煤工只能单打独斗，孤立无援，导致心理压力很大。这个也可以从表 3 的缺勤率看得出来，会更容易无缘无故请假等，背后主因是分工设计导致工人心理压力太大，承受不了。团队互相协助的环境可以减轻这方面的压力。

	Conventional 传统分工	Composite 自主团队
Absenteeism (% of possible shifts) 旷工率 (可能轮班数之百分比)		
Without reason 没有理由	4.3	0.4
Sickness or other 病或其他	8.9	4.6
Accidents 意外	6.8	3.2
Total 总数	20.0	8.2

Chapter 19

客户参与

常见问题

某软件外包开发中心专注承接某政府医疗机构的 IT 系统维护工作，因为工作量变化很大，而且需要快速反应，一直都是使用敏捷 (Scrum) 开发方式，每两周一个迭代。但因为甲方提需求的代表都是医护专业，缺乏软件开发和业务分析经验，很多时候，每轮迭代提出的需求都很粗。例如护士办理病人住院：只发出一张表，列举了各种人群患者的入院要求、步骤与条件。

乙方本来只使用敏捷开发的用户故事 (User Story) 方式，“护士办理病人住院”只算一个用户故事，难以判断：

- 需求是否完整。
- 与其他功能的依赖关系。（例如如可能有高度传染病，必须先做某些化验，如果阳性便必须去隔离病房）
- 各种异常情景应怎样处理。

为了避免开发出来的功能不能满足需求，必须靠乙方开发团队先做好详细分析，但他们反而不太懂业务，导致只能从技术角度分析，未能全面挖掘业务特性。

改进方案

- 使用功能点识别实体与行为
 - 便能容易识别出需求中，哪些地方甲方没有考虑，例如，除了新增，是否也需要有查询和删除功能。
- 再利用用例 (Use cases) 与各种场景 (Scenarios) 和对应系统页面原型图，与甲方代表讨论各种场景，系统应如何处理

改进效果

- 挑了两个医疗相关的团队做试点，在开发前预先用了这些方法与甲方充分讨论，3 个月 6 轮迭代后，UAT 里的需求相关缺陷数平均降低 43%。

从以上案例看到，客户代表除了要参与，还要有具体措施做好需求，例如可利用功能点方法，用例与场景来挖掘分析需求。

识别利益相关者

和杭州客户领导吃晚饭，领导就跟同桌的项目经理开玩笑说：“你们刚刚完成内部项目管理自动化工具，做调研的时候好像没有找过我？其实我是其中一位经常要使用这系统的管理者，下面项目的监控、申请都是经过我，但我发现这个系统很不合用，对收集我需要的项目信息没有作用。比如没办法处理一些批准信息。”

从上面对话，可以了解如果没有全面识别项目的利益相关者，可能会影响到项目的成败。例如我接触一些有些具备开发经验的需求人员，但他们通常只注意功能需求的技术细节。

问他们：“哪些是你的项目干系人？”

很多人都会说：“甲方有协调员，要访谈哪些人都是由甲方协调员安排，我们听他安排。”所以为了避免未能识别所有干系人的风险，就要主动跟甲方一起策划。我们说沟通计划必须是甲乙双方一起合作做出来，而且会牵涉甲乙双方各个层次的人。

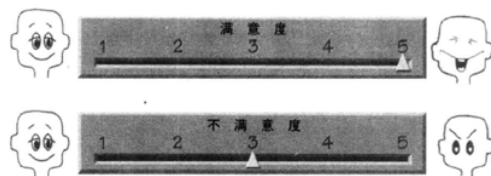
例如要听甲方出资人的需求，可能就要乙方的总经理出马，乙方的需求人员顶多可以跟甲方对口的项目组人员沟通。（如何可以做好识别干系人，并制订沟通计划，可详见附件。）

用户故事卡

用户故事卡目的是让用户（业务）与开发沟通交流。

虽然 Kent Beck 先生强调用户故事卡片上的东西并非交流的全部，也不应该包括太多细节。但 XP/Scrum 的用户故事卡片未能全面包括需求的各元素，例如缺乏以下内容：

- 来源：每项需求都应可追溯到源头。
- 冲突：确保需求之间的一致性。
- 顾客满意度 / 顾客不满意度：用两个数比单纯用优先级能更全面反应客户声音。



满意度和不满意度评分反映了客户对最终产品是否实现某项需求的看法和考虑。满意度高分表示如果能成功交付这项需求，客户会非常高兴；不满意度高分表示或者如果产品不能实现这项需求，客户会很不高兴

（如想多了解为什么要这样分，请看附件里的“客户声音: Kano Diagram”）

卡片例子可参考 ROBERTSON 夫妇的需求卡片模板：

需求编号: 75	需求类型: 9	事件/BUC/PUC编号: 6
描述: 如果一个气象站传送读数失败, 产品将发出警告		
理由: 传送读数失败可能表明该气象站失效并需要维护, 并且用于预测结冰的数据可能不完整		
来源: V Appia, 道路工程师		
验收标准: 对于每个气象站, 当每小时记录下来的各类读数个数不在制造商规定的范围之内时, 产品将通知用户		
顾客满意度: 3	顾客不满意度: 5	
依赖关系: 无	冲突: 无	
支持材料: Rosa 气象站规格说明		
历史: VA 在 2013 年 7 月 28 日提出		

白雪卡上的一项完整的功能需求

也可以用于非功能需求：

需求编号: 110	需求类型: 11	事件/BUC/PUC编号: 6.10.13
描述: 产品应该对道路工程师易于使用		
理由: 工程师不必为了使用该产品而参加培训课程		
来源: Sonia Henning, 道路工程管理者		
验收标准: 一个道路工程师将在首次接触该产品的一小时内, 能够成功地执行指定的用例		
顾客满意度: 3	顾客不满意度: 5	
依赖关系: 无	冲突: 无	
支持材料:		
历史: A.G. 在 2013 年 8 月 25 日提出		

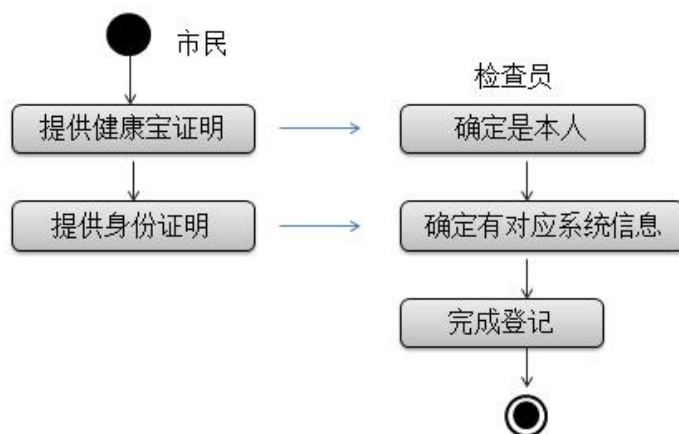
一项非功能需求，这是一项易用性需求

- 千万不要误以为卡上的所有信息都需要一次性与用户获取。
- 挖掘需求永远是持续，并不断细化。
- 卡片上增加了这些部分，可提醒我们不要忽略这些元素。

用例与场景

用户故事针对“做什么”与“为什么”（“What”，“Why”）；用例与各种场景针对“如何做”（“How”），所以它们之间是互补，没有冲突。

下面用大家都熟悉的简单系统“在核酸检查点做检查并提交结果”来举例说明，如果考虑不周全，也会导致需求相关问题。



正常情况：使用身份证时，用读卡器获取实名身份信息，与系统记录对应，记录信息也齐全，包括手机号码。

其他可选情况：没有身份证，可用其他证件，例如港澳通行证、国外护照。

异常情况：市民身份信息不能用读卡器自动获取，需要手工输入，但检查员输入信息错误。例如护照号码错误，手机号码为空，被删掉或者没输入。

如果问需求分析师、产品经理有什么异常场景，一般都会回应“没有”；部分回应一些最基本的异常，例如“只有输入信息错误、报错，要求重新输入”，我便会以我的经验举例，说明必须要识别各种异常场景，才能算全面挖掘需求；识别正常场景是基本工，能识别各种异常场景才算做好。

我每次在北京做免费核酸检测，因为非身份证，检测员都要用手机手工输入我的信息：姓名和港澳通行证号（因为系统已经有我的手机号，所以通常不需要再输入手机号），只要我确认以上信息都输入正确，通常都没有问题。后面不知道什么原因？不再用手机，改用笔记本电脑配电子读卡器阅读电子身份证和电子扫描器读二维码。

检测员不再要手工输入我的信息，靠我预先进系统填好所有信息，生成二维码，只要检测员验证了我的二维码信息与系统信息对应，便可完成。

一般正常操作是没有问题的。

但我最近在某采集点检查完，之后第二天未能在健康宝正确展示出核酸结果。（开头以为出现了“混管阳性”，但之后几次核酸结果又回复正常。）

我遇到的问题可能是因为那个采集点使用了笔记本电脑，信息先存在电脑，最后才批量上传到服务器。但因为非身份证的情况，没有检查清楚信息是否都完备，例如，可能某些信息被检查员错误删掉了，但电脑没有报异常，最终批量上传时，服务器便无法识别我的检查记录。以上只是我的猜测，估计永远都不会知道真正答案，但无论如何，如果当初系统分析时能全面识别所有场景，应可避免这类问题。

如果使用银行交易系统概念，应可避免这类错误。比如我从自己账号转钱到另一个人的账号，中间会经过多家银行，我提交后，系统都会确保中间每一轮交易都已成功完成，并确认对方银行最终收到款，才算完成这个交易（通常前后经过几分钟）。中间任何环节出问题，都会撤回整个交易，能避免出现我那种不了了之的情况了。

如果信息有误或者不成功，导致最终无法提交上线，怎么处理？因总服务器本身或网络出问题无法连上，怎么处理？

如果要充分考虑各种异常情况，应仔细询问以下一系列问题：

通过检查正常情况的每一步并询问以下问题，可以发现异常情况。

- 如果这一步不能完成，或没有完成，或得到了错误，不可接受的结果，会发生什么？
- 在这一步会发生什么错误？
- 会发生什么事情，阻止工作到达这一步？
- 是否有一些外部实体可以打断或阻止这一步，甚至这个业务用例？
- 实现这一步的技术是否会失败或不可使用？
- 最终用户是否会不理解对他们的要求，或者错误地理解产品提供的信息？
- 最终用户是否会采取错误的行动（有意或无意），或者没有做出响应？

另外一类情况，有几十人排队等待做核酸，原因是服务器出了问题，停机了，排队人非常不满。工作人员也很尴尬说已经想尽办法用手机、电脑，但是都没办法。如果提前想好这种场景，就可以在本地把相关信息记录下来，等服务器恢复后一次性上传，就可以避免刚才的情况。

除了正常，也可以考虑一些假设的场景，让干系人、客户可以更开放地考虑各种不同的情况，更创新。例如是否可以考虑像超市付款一样，自主处理？当很多人排队的时候，就不会因为检察员一个人忙不过来，导致排长队。

例如取登机牌也可以用机器，让乘客自主一组办理。其实做核酸也可以这样处理。

所以从以上案例看到用例 + 场景不同于用户故事，可以弥补后者。Kent Beck 先生特别提倡用户故事，因为当时很多客户都觉得用例太细、太偏技术，不愿意写，甚至也不愿意读；但客户大多都愿意用自然语言写用户故事，可以很容易触发客户与开发交流沟通。

但如果需求分析师只考虑现在业务流程的各种场景，开发人员开发系统，把所有的场景都覆盖好，是否便能成功？不一定，请看以下例子：

举例

你们都有申请过护照吗？比如我们护照更新，以前是要到柜台手工办理，如果我们把那个过程数字化，应该怎么做呢？

某国家的做法是本来的手工填写模板直接变成系统页面（每个输入与手工表格一一对应），申请人在系统里按本来模板填写并提交，上传个人新照片，也是经过系统，我有一次尝试用系统线上填电子表单申请，但因表单很繁琐，有很多护照原有信息都需要重新填写，光是填那表就花了我接近一个小时，最大问题还不是在我花时间填手工表，而是最后要上传照片，因为照片像素高的话就很大，需要很好的网络才可以传得上，如果照片像素小，便导致模糊不清，不能通过。最后，一个半小时后，我用尽所有方式都还是无法传上照片，我最终放弃了在线上提交申请，直接预约去在柜台做！

客户：有好方法解决这个问题吗？

我：有，另外某国家的做法就很简单多了：

之前描述的整个过程只是把原有的手工步骤信息化，和原本的申请手续一样。但线上办理和在柜台现场办理不同，在现场你可以要求对方直接把照片给你，也可以要求对方提供老护照，但在线上办理，应很容易从系统里找到个人护照信息，所以很多本来在柜台要手工填写的老护照信息就不需要再填了。

客户：怎么可以简化整个过程？最困难应是照片的更新？

我：有些国家是这样做法，比如你申请续证，只需要填上老证件的基本信息，系统就立马能识别出本来的证件你确实有那个“旧”证后，便可立马提交申请。跟在网上购物一样，你确认过内容没问题，就在网上付费，然后打印申请表并在表上亲手签字，然后附上几张符合规格的照片，邮寄到政府机关。他做好新证件以后再邮寄回你。或者你自己到他规定的地点领取也可以。这样就能很简单地利用“低”科技解决方案，解决了刚才上传照片的困难。

从上面例子看到，我们应不仅是把那些本来手工的流程自动化，应该全局看要解决的问题本身，哪些过程自动化，哪些不应该自动化（比如传照片）。

甲方对怎么可以在线上做这个过程也没有概念，他只是知道，本来手工需要填表。乙方也不知道，他只是做开发，也不知道有什么方面可以不用 IT 方式，而用其他方式更合理。必须要一起探讨才可以有最好的解决方法。

所以作为业务分析师，你很可能要改变用户思考问题的方式，例如利用业务过程模型，配合场景与页面原型，与利益相关者一起探索问题的本质。软件系统必须为拥有它的人提供最理想的价值，构建软件系统本身（例如只是把现有流程自动化）不一定能解决客户业务问题。

结束语

除了有客户参与团队，快速反馈，也要：

- 全面识别所有利益相关者。
- 利用功能点分析，了解范围，确保需求完整。
- 利用用例与各类场景，全面了解各业务事件都可度量、可测试。
- 利用需求卡片记录用户故事（因与干系人沟通，获取需求是持续逐步细化过程）。

附件

利益相关者

利益相关者计划检查单 (**Stakeholders Plan Checklist**):

1) 列出所有潜在的利益相关者 (List all potential stakeholders)

1a. 类型 / 分组 (What are the base Segments)

1b. 可否再细分 (Any sub-segments)

2) 把他们分为 F (友好), I (忽略), U (不友好)

Assign F (Friendly), I (Ignore), U (Unfriendly) to them

3) 他们最关注什么?

What is important to them?

4) 学习目标 (Learning objectives):

针对某利益相关者, 我们需要了解什么

For each stakeholder, what will we need to learn?

5) 怎样沟通 (How)

6) 什么时间 (When)

7) 抽样计划 (Sampling plan)

如何招募 (How to recruit)

如何能获得承诺 (How to get commitment)

[针对以上第一至第三项, 参阅以下实例/解读]

实例/解读 (**Examples / explanation**)

某公司专门设计、开发新一代手提电脑产品。

1/ 全面考虑各类利益相关者 Stakeholders

出资人 (**Sponsor**): 出资人为产品的开发付钱

顾客 (**Customer**): 顾客购买产品。必须对他们有足够的了解, 理解他们认为什么有价值, 所以会购买什么产品。

用户 (**User**): 确定用户的目的是为了理解他们所做的工作, 以及他们认为哪些改进有价值。

在开发消费产品、大市场软件时, 应该考虑用一个“假想用户”。假想用户是一个虚拟用户, 他是大多数用户的原型。

类型/分组的例子:

未来笔记本电脑的潜在用户:

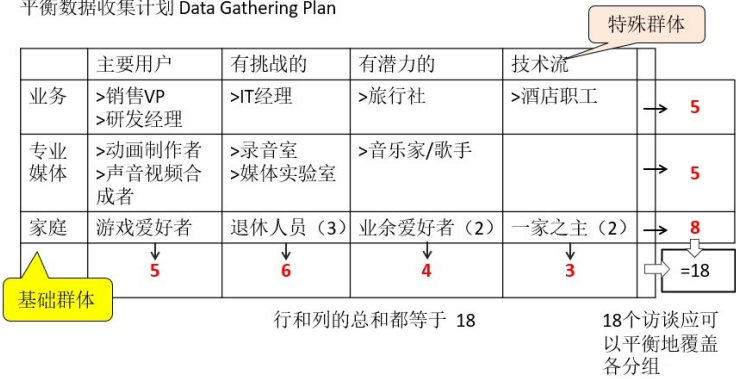
大类:

1. 商业人士 (Business)
2. 媒体专业人士 (Media Pro)
3. 家庭用户 (Home)

细分：

- 1. 主要用户 (Lead User)
- 2. 有极高要求者 (有挑战的 Demanding)
- 3. 潜在用户 (有潜力的 Potential)
- 4. 追求技术完美者 (技术流 Tech-Phobic)

客户矩阵 Customer Matrix
平衡数据收集计划 Data Gathering Plan



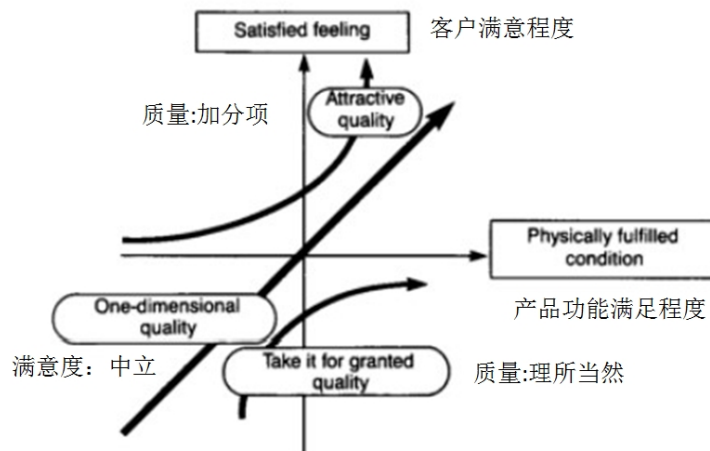
- 2) 策划包括哪些用户 (User inclusion strategy)
- 依据设计人员会怎么对应，把不同识别出来的利益相关者分成 **F**、**I**、**U**
- 例：以针对笔记本电脑创新产品
- F(Friendly)** 友好，比如家庭用户、商业用户、媒体专员
- I(Ignore)** 忽略，比如忽略残疾人士
- U(Unfriendly)** 不友好，比如黑客、小孩（禁止他下载游戏、玩游戏）
- 3) 他们注重什么（主要关注点）

干系人	角色/概况	质量关注重点
商业用户	长期出差者 -坐长途飞机 -做演示 -保护某些秘密文档	-待机时长 -屏幕清晰度 -安全性
专业媒体	创造性工作；并要协同。录音和录视频	数字带宽（声音和视频）电脑速度和内存
家庭用户		

- 以上有那些在调研之前已知，其他有那些需要挖掘。

客户声音：Kano Diagram

可以用下图分析用户对需求优先级：



解读上下两个箭头应怎样看：

* 下面的箭头代表理所当然 (Take it for granted)，如果缺乏，客户会很不满意，包括觉得是理所当然 (例如满意度：中立 1，非常不满 5)。

- 上面的箭头代表是加分项 (Attractive)，如果包括会非常满意，但如果缺乏不会觉得不满意。(例如满意度：非常满意 5，不满意度：中立 1)。

所以“需求卡片”用两个系数：顾客满意度 + 顾客不满意度，能更好判断某功能属于哪类功能需求。

参考 References

1. Beck, Kent , with D. West. "User Stories in Agile Software Development", Ch.13 of "Scenarios, Stories, Use Cases: Through the Systems Development Life-Cycle" edited by F. Alexander(2004)
2. Cohn, Mike . "User Stories Applied. "(2004)
3. Martin, Robert C. "Clean Agile - Back to Basics."
4. Robertson, S. "Mastering the requirements process. " (2006) 2/e

Chapter 20

团队协作

客户：极限编程 (eXtreme Programming) 强调团队要共同拥有代码，这条看起来很容易理解，但又很难理解。

我们一起写程序当然是一起负责，道理很简单，但为什么会有这一条原则？背后有什么原因？然后这章节会探索作为程序员和管理员，怎么才算做好这一点？

我：讲需求与开发已强调团队合作很重要，但没有探索针对软件开发团队心理问题，之前的英国煤矿“长壁”机械采煤案例证明不能单靠正式的组织架构和科学管理方式设计工作流程，来管理团队。因为实际的情况变化万千，必须依赖团队成员之间相互合作，才可以有效提高生产力和矿工的心态健康。

我：现在公司新人也很多，怎么识别谁是开发？

客户：很简单，不喜欢说话，不跟人家交流，每天沉迷在电脑前面那些人，肯定是开发人员。

我：一般人有这种概念，就是开发都是单独和富于创作性的，他们喜欢自己创作，不希望被打扰。员工专注做好工作，原本是好事，但如果做得过的时候，反而会产生负面影响。

客户：为什么？

我：因程序员越来越觉得程序是自己的作品，好像自己写了本书，在封面有自己的大名一样，但是软件和艺术作品不一样，如果有反对声音，觉得他作品不好，艺术家会认为是你不懂。但是软件程序不一样，它最后是在电脑运行，运行有错误就是有错误，所以导致程序员都会想尽办法证明自己的程序没错，问题不是出自自己。

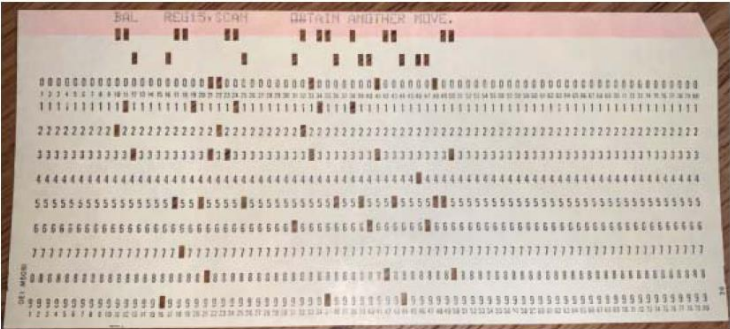
一般人都会觉得自己比他人出众，如果做得好，觉得是自己棒；如果做错了，就是题目太难了，不是自己的问题。

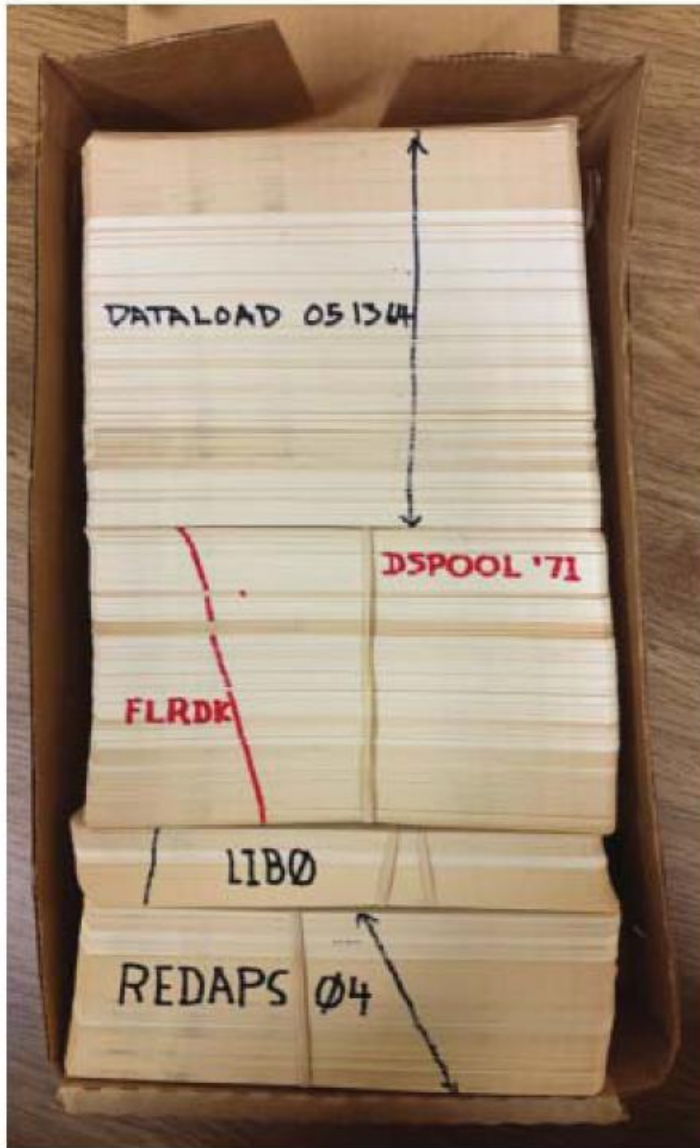
为了平衡这个心理，一般人会想尽办法找理由说明问题不在自己。

程序员的心理（比矿工）更加高深莫测，这问题从 60 年代软件开发的萌芽期已经存在，不信可先看看以下故事：

各自负责自己开发的模块

60 年代，当程序还需要操作人员用机器把程序打在一堆卡片（一行一张卡片），然后利用读卡器把程序输入电脑的时代，如果程序有误，编程员会说是输入员出错，或者说是卡片的顺序错了。





你可以想象，如果程序员有这种心态，对整个团队的质量和交付都不好。所以我们每个人要抛弃拥有自己程序的心态，不要认为这是个人的作品，而是属于大家的，他会更能接受错误，对软件的质量有好处。

在实际工作环境中，大部分的软件都是有一组人合作写的，很少能靠单一个人完成。

团队合作

当项目需要多人合作，各有所长和所短，也出于更好满足需求，自然会形成一个团队合作模式，各人互补。当这个团队形成以后，团队成员会互相依赖，不轻易换人。例如某个团队预计公司会有计划解散这个团队，队员就开始寻找新的工作机会。最后这个团队选择一起离开，入职另外一家公司。新公司的经理觉得过来这些人都挺靠谱的，软件交付质量不错，当他有一些比较重要不能延误的开发工作，都会交给团队的其中一位。但他有所不知，虽然他是交给其中一位小李，但实际上工作是由团队内部相互合作完成的，不是一人的工作。

从这个例子看到，一旦团队形成相互合作模式，它的作用要大于每个人相加，也正因为如此，也正因为这个原因，团队会很慎重处理新成员的加入。假如有新人进来的时候会先告知新人，必须遵守团队的交付规则。例如后面加入了一位新人，他有 3 年开发经验的“老”程序员，觉得自己有经验，不需要遵循这一套规则，还是用自己的方式去开发，但越来越发现他的交付质量跟不上团队。最后有一次他交付了自认为是“良好”没有缺陷的程序给团队。为了避免再出事，他的程序被交给一位与他同时加入团队的实习生审查（这实习生加入后一直按团队规则交付，所以进步很快），他觉得很没有面子，最后他无法接受就辞职了。这种情况不仅在软件开发团队存在，在合唱团或弦乐四重奏团也一样，非常依赖互相合作，所以在选择新成员方面非常谨慎，不要求个人英雄，更重要是要能跟其他人合作。

从以上例子看到，成功的团队是软件开发公司的宝贵资产，但管理者应该怎么去管理呢？要注意管理程序员这些知识工作者，不要用管理工人那种心态。比如下面这两个例子。

按时上下班

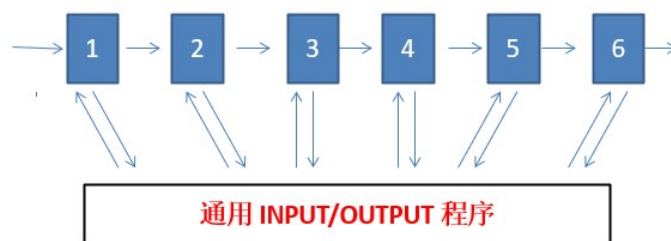
某大型项目中途换了部门经理，新加入的经理发现经常有程序员早上 10 点半才上班，他没有去问原因（其实有些程序员前一天晚上工作到凌晨两点钟），觉得这些情况直接影响团队管理，要立马改正，避免大家漠视公司规则偷懒，立马发通知要求大家要按公司规定时间上下班。

程序员立即集体反应：

完全按照经理的要求准时上班，也准时 5 点 15 分清理好桌面下班，并一起排队去打卡。但因为本来这些团队都默认按轮班制，有些早到、有些晚去对应电脑时间的限度（60 年代机器没现在发达，写好程序打完一堆程序卡片后，都是要排队让电脑按序运行）。经理发了这个通知后，生产率立马降低一半，很多程序员白天没事干，等机器有空才可以工作，然后电脑也按公司的时间运行，到下班时间就立刻停机。

60 年后的现代，也会看到同类故事在软件开发公司发生。

什么导致系统崩溃



有一个大型项目，希望把一个以前在老版本主机运行的应用软件系统更新到新硬件平台，总费时超过两年，团队平均人数在 10 位左右。这系统本来已经运作很成熟，所以整个开发最关键的部分就是更新硬件的接口部分，其他模块的开发相对简单。项目比较大，必须要分成 6 个阶段，其中有人负责这个重要的接口部分，其他人就负责原有模块的软件升级。过了一年半，一切的测试进展都很顺利，到了第 5 个阶段的验收测试就发现出问题了，整个机器就停下来了。距离交付只有不到 2 个月的时间，大家就赶紧去找问题，仔细查看所有第 5 阶段的相关代码和接口部分的代码，测试之后和原有的比较没发现任何问题。

没办法，就在加人手研究之前的第 4 部分，还是没有任何进展，时间也越来越紧了，大家很着急，又加了一些新的人来加入研究，因为开发时间比较长，有些之前阶段的人员都已经离开了。有一次，小张（他刚进入团队）与一位原本团队成员喝咖啡聊天，听到那位“老”员工回忆刚开始分工时，有一位特别厉害的程序员唐工，他非常不满意分工安排，他认为自己是团队里面最有经验最有能力的，他不满意接口部分的编程交给了李工负责。唐工一直闷闷不乐，一周之后才平静下来。

小李听完以后立马有了灵感，就问唐工当时负责哪个阶段？哪个模块？那个老员工说：“他负责第二个阶段的那些模块。”这个小李回去立马查看第 2 个阶段模块的代码，很快便有发现，程序一开始就在内存里面读一个变量，然后把整个程序从那个接口调到唐工写的模块运行。原来唐工因为不满安排，写他负责部分的程序时，绕过李工负责的接口模块，让自己的第二模块直接对接，不用本来的接口。因他编程很厉害，做完这个动作后还让原本的变量原封不动，大家很难看出来。但因为他改变内存里那些记录，外围那些机器，如磁带机，因内存记录错误，就被搞乱了，所以一直到第 4 阶段都没有异常，但到第五阶段的时候就引起整个系统崩溃。

从上面的例子看到，软件协作并不是一件简单的事，外人很难看出其中做了什么手脚。如果我们分工时，没有真正让每个成员满意，也没有注意某些成员的心理不平衡，可能会导致灾难性后果。

技术团队的持续性

保持团队持续性也很重要，因为成员会离开，当处理不当的时候，将会引起灾难。

例如公司小李很有魄力和经验，下面有两位实习生帮助他。但是为了这个程序，他长期离家到偏远的现场工作，想尽快回家，于是向经理申请：“咱公司小张在这块也有经验，如果可以让他过来学习这个怎么做，我就可以派去做其他项目。”

但碍于小张正在做另外一个项目，经理不想调走小张，因为又要找到另外一个人代替。所以为了满足小李的要求，经理给他多增配一位实习生，但这对小李一点帮助没有，增加实习生只是增加他的负担。所以小李就再去对经理申请说：“不行了，你还是快点找小张过来帮我吧。”

经理最后的解决办法是给小李增加了 25

经理最终没有其他选择，只能派小张接管这个项目。但因为他没有受过任何相关的培训，还是一团糟。过了几个月，这个项目出不了预期的效果，最后被取消了，对公司造成很大的损失。

从刚才的故事可以看到，一般经理以为薪水可以代替一切，其实不一样。对小李来说，加薪水就是希望他留下来，但是他就是不想留下来，所以这个加薪水反而变成他辞职的催化剂。从这个案例有两个经验教训：

- 每个项目每个岗位都应该有备份，如果有任何一个岗位不能有人代替的话，后面的风险极大，必须立马处理。
- 我们对程序员不能用一般管工人的心态，不是用钱就可以解决一切问题。

从以上的例子看到管理开发团队要注重成员的心理状态。这个道理其实对任何团队协作的工作都适用，但对于软件开发这种偏知识性行业则特别重要。因为每个开发人员都会觉得自己是高手，如果这块心理得不到平衡，他会用各种方式去平衡，甚至会对整个团队的表现产生负面影响。

客户：听完你这些开发团队的小故事，我作为管理者是否应该放手尽量少干预，让团队自主管理，类似庄子那种无为而治道家思想就好了吗？

我：不一定，如果你只是放手团队自我管理，没有任何要求，团队很容易就会没有动力，后果也很严重，可以看看以下实例。

某回国技术总监的经历

刚回国不久，还帮日本公司带领小团队做开发，这时有个朋友找我，说他的团队经常被客户投诉，请我帮忙看看。我就到这家公司，因为大老板以前是学校的老师，没有投入足够精力管理公司。本来这公司的团队管理模式很自由，每个人和团队都非常独立，大家互相不干扰。团队一直帮客户做开发与维护，客户觉得这个合作习惯了，也不想转变或换人，导致开发人员的效率和质量都不高。因为管理层也比较放松，开发人员也没什么压力，只要服务到客户不投诉就行。没有任何规范和指标。我观察了一段时间，帮他们做一些辅导咨询，后面老板比较信任我的能力，于是邀请我来管理团队，我答应了。我发现加入以后，管理他们更难了。比如我希望他们按我的要求有计划地加强管理，他们并不听。有时我觉得有些员工能力不够，需要调走，但他和客户关系好，客户立刻和我说，不能动这个人，否则我就不再继续签合同。这些开发人员已经是“老油条”，知道用什么手段保护自己，尽量不发生任何改变。我没有办法影响他们的话，团队就无法进步，于是建议大老板说，我们废除以前根据主管打分决定员工薪水的方式，而是做成“奖金池”，用客观的方式来判断员工的贡献，而不是靠主管的主观判断。奖金是按开发出来的功能点数量，开发越多奖金越高，功能点规模是由独立的 QA 依据开发的软件客观算出。开始时，效果挺好的，因为有客观数据的衡量。但是几年后，他们熟悉了之后，就开始玩花招。听你说团队持续改进的方式，我觉得可以尝试，让团队有下一波良性发展，提高他们的开发和创新的能力。

作为管理者不是插手、替代团队做他们的工作，例如编程、计划等，但需要设定高的提升目标，帮助他们自己提升，才能持续保持公司和团队的竞争力，不然这很容易被自然淘汰。在这个过程中，也要注意开发人员的心态。激励不能单靠奖金，尽量要让他们形成团队，相互合作，对公司、个人都有好处。当团队成员能相互合作后，便可以帮助公司做好项目，满足客户需求。我看到很多团队成员在项目中间会被调走，这是常常影响到团队质量（与生产率）的主因。

客户：没办法，客户是上帝。比如他们有紧急维修要做，而只有一些有经验的开发能解决，就必须占用他们时间，否则客户不满意。

我：但这种临时抽调的做法，会影响到他们正在做那些项目，也会引起客户不满。

客户：你有什么好建议？

我：我认为可以考虑下面敏捷大师 Robert Martin 的建议，他鼓励尽力保持团队不变，发挥团队相互合作关系的作用。

如何保持团队稳定

因为要让团队成员相互合作，变成有效的互补团队不容易，团队是公司的宝贵资产。所以当项目结束后，不应该解散团队，而是要分派新的项目，让团队可以继续合作。但有时候，项目的规模大小不一定能与团队匹配，可以考虑以下建议：

假如某团队的生产力是 50 个单位，就可以安排以下 3 个项目来让这团队负责，比如 A 项目需要 15 单位，B 项目需要 15 单位，C 项目需要 20 单位。如果某些项目后面突然间需要紧急加快速度，也可以很容易策划、改变分配的比例，这样会比平常增减人员简单很多。因为软件开发特别需要团队之间沟通，如果某项目需要加快速度，通常增加人手是没用的，原因是团队成员会花更多精力让新加入的成员融入。本来团队要教他们，在这种后面增加人员的情况，收益还不如在过渡培训的投入。有时候项目延误不是团队能力的问题，而是因为有紧急需要导致某些开发人员被调走，留下来的开发人员要兼顾老项目其他项目，同时要服务两三位项目经理，导致无法专注做好编码工作。所以太多骚扰会影响总体效率、生产率和质量。

应怎么安排呢？

要避免这种临时的抽调。但确实有些客户的紧急问题，只有某些开发人员熟悉，其他人不懂。

项目不应该有任何事情只有单一人懂，缺乏备份，要避免临时抽调，就必须策划好开发过程，团队不仅要做好产品开发跟交付，也需要把知识传递给软件维护团队，到后面出现事故时，就可以让维护团队处理，不打扰本来的开发团队，开发专注开发，维护专注维护。就可以避免这类对公司不利的抽调发生。

附件

平衡心态 (Cognitive Dissonance)

- 探索当行为和心理有冲突时，人们如何对应。
- 被安排做一些很沉闷的任务 (例如扭螺丝钉)。
- 试验者会收到 1 美元或 20 美元，要他告诉下一个人这个任务很有趣，很有意义。
- 试验完成后，再问实验者对任务真正的感觉。

1957 年实验，要求有些学生 (被实验者) 做一些很无聊的工作，比如反复钻螺丝、包装，后面给他 1 美元或者 20 美元。要他告诉下一位，刚才那个工作很有趣、很值得。实验后再问他们这个任务是否有趣？发现给 1 美元的人，大部分会觉得工作有趣。而给 20 美元的反而不同意觉得工作无聊。

原因：当你给他 1 美元的话，金额很小，他心里觉得不是被贿赂，所以会相信他自己说过的话。而当给了 20 美元，他觉得自己是收了那个钱才这样回答，否则肯定不会这么回答，所以心理上就会抗拒，不同意这个工作有趣。

分析实验结果：人会尽量说服自己“我做的是对的”，来取得心理平衡，所以即

使他确实做的事很无聊，因为他跟人家说了这个事很有意义，他会尽量先说服自己，觉得这个是有意义，使自己心理平衡（收了 1 美元场景）。但反过来，当因为这件事收到 20 美金，他觉得这个说法不是自愿的，只是因为收了钱。反而就不需要动力去平衡自己的心态。

参考 References

1. Weinberg, Gerald. "The Psychology of Computer Programming."
2. Martin, Robert. "The Clean Coder."(2011)

Part VII

度量与分析

当团队开始养成每天收集数据的习惯，在迭代回顾时分析数据，公司便可以开始准备度量与分析。（注）

要做好度量与分析，跟过程改进一样，要针对目标，制订度量计划。

本部分开头会以问卷调查为例，介绍度量计划的主要元素，然后会介绍假设检验、控制图等数据分析的基本功。

请不要误以为度量与分析必须由管理层驱动。若要团队能从定性升到定量管理并能持续，而且有效果，必须从团队开始，而不是靠管理者总体策划。

Chapter 21

做问卷调查

要做好数据分析，便要制订计划——收集哪些数据、如何收集、如何存储数据，准备后面如何分析数据。

思路类似如何设计问卷调查，先看以下案例。

案例：为某全国快餐连锁做问卷调查

美国一家连锁快餐店，全国有 3 万员工。但员工流失率严重（接近百分百）。人事部估计聘用、培训、建档案等，每个新员工要花费 300 美元成本。粗略估计每年由于员工流失就产生 1000 万美元的成本，差不多是整个集团的盈利。虽然高层一直知道，但觉得大量员工流失率是快餐业的通病，所以没有注意。咨询顾问针对这问题，详细查看流失率的分布，发现不同店铺的员工流失率差异很大，从最低 6% 到最高 800%。

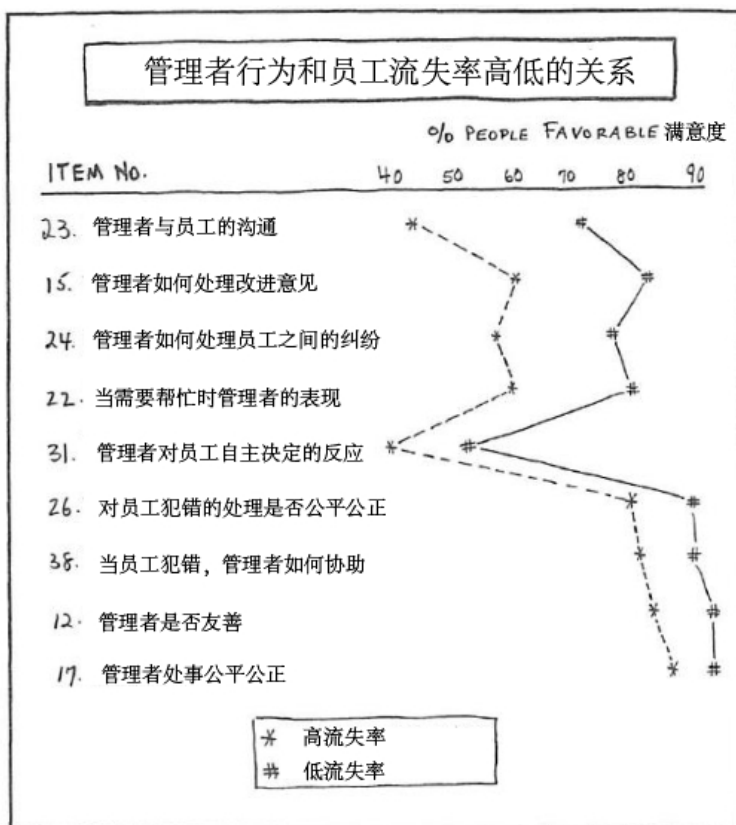
详细分析

抽看其中 100 家餐厅，把每年流失率在 120% 以上的当成高，60% 以下的当成低，分析数据发现在各种职位：前厅、后厨、维修工人等。发现员工流失率与工种无关，主要是跟餐厅有关。

研究高流失率的原因

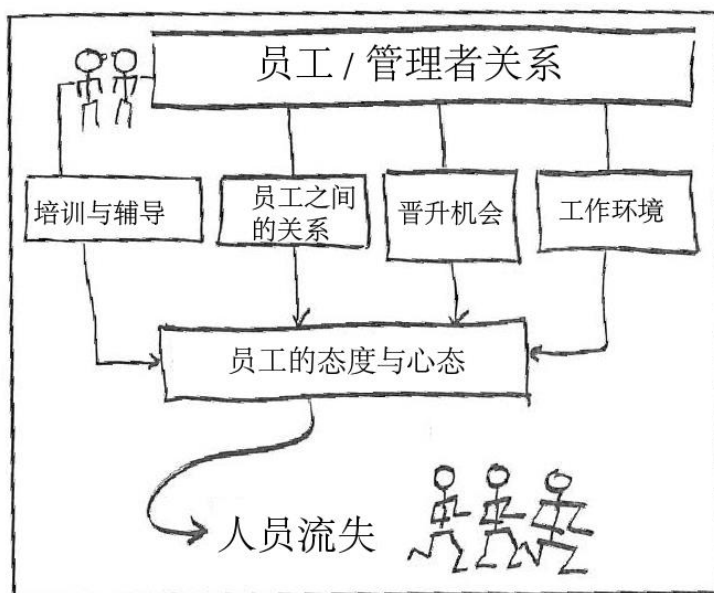
- 随机抽样 10 家餐厅，4 家属于低等（每年流失率 40%~60%），2 家属于中等水平（60%~120%），4 家属于高等（每年超过 120%）。
- 做问卷调查，有 39 条问题。
- 初步分析发现高与低流失率被抽样单位，发现它们的员工的年龄、被餐厅聘用了多少年、工种、是否轮班等分布都很类似，所以这些都不是原因。
- 主要原因是在管理者的态度。比如低流失率餐厅的员工，在 39 条问题里，关于管理层的 31 条都比高流失率餐厅员工较高分。

- 低流失率餐厅的员工觉得他们得到较好的辅导，经理比较乐意帮助他们，甚至更有机会升职。
- 低流失率餐厅的员工更满意工作和薪酬（其实无论高与低流失率餐厅的工资水平大致一样）。
- 下图是 9 条最有代表性的问卷结果：



从分析到行动

把那些关于管理者的态度如何影响餐厅员工流失率总结成下图，让管理层更了解主因：



因为经理与员工的关系影响到培训、员工的成长机会、晋升的机会、员工之间的关系等，直接影响到员工流失。

针对发现开始做实验

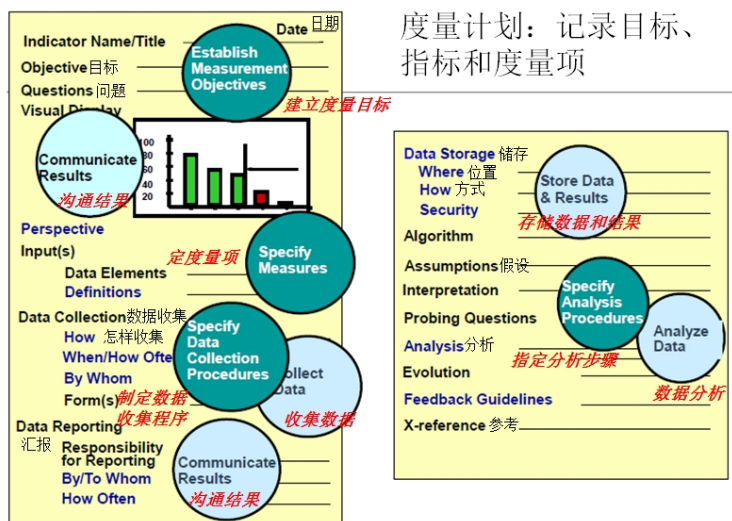
为某区域的餐厅经理做针对性培训，也让他们看到从问卷调查，他们店的成绩与全国其他餐厅的比较。在 4 天培训里面，我们针对问题解决、领导能力等做针对性培训。这些培训不仅仅是老师讲课那种，大部分是利用角色扮演，首先列出来他们希望新员工要学到哪些点，然后用角色扮演，在那个新员工的入职培训里面如何执行，自己评判效果或者表现如何。他们也跟高层经理交流，可以反映一些他们在自己餐厅单独解决不了的问题。

比较培训后效果

- 发现在 27 家餐厅中，参加过培训的 24 家后期员工流失率显著降低。例如其中一家有 29 位员工，以前是一年走了 37 位，但做了培训后，没有流失。整个做实验的区平均下来，流失率减少了 50%。

从以上案例可以看到依据目标制订一些度量项——就是问哪些问题。之前也有一些基本的分析，比如以什么维度去分析，比如前面发现跟工种无关，主要是针对不同餐厅，问卷调查也验证了本来那个猜测，这是经理跟员工之间的关系影响到流失。问卷调查证明了这个关系后，就有对应的改进措施。措施实施后，效果是否有显著的完善。

从以上案例看到如何针对改进的目标设计问卷、收集数据，利用数据分析帮助找出根本原因，制定针对根因的改进措施，然后评判改进效果。



看上图，度量计划应包括：

度量目标为何要度量，要解答什么问题。

度量项收集什么度量（对什么对象，问什么问题）。

收集数据，分析数据用什么形式来度量，如何分析，这个是在度量前要想好，有些很重要的因素忘记问，后面无法补。所以我们度量与分析也是这个思路。

结束语

度量分析的计划应该包含下面要素：

1) 明确度量目标。希望改进什么，然后依据目标或者预估影响结果的可能因素。例如，是否跟管理者的领导和沟通能力相关。然后依据这思路，制订问卷内容。

2) 度量项的操作定义。如果定义不清楚，可能不同团队因为理解不一样，提供的数据就不同了。

- 如何确保数据的质量。因为如果数据本身不对，后面什么分析都没用。如果有系统记录，可以利用系统验证数据是否正确。但反过来，如果数据都是凭经理在项目结束最后填一个表给你的话，就会怀疑这些数字有多少的可信度。

3) 数据怎么获取。尤其是在软件开发中。如果他们没记录缺陷或者工时，很多数据就无法在回顾时获得，没有数据就没法做后面的分析。然后在做计划时，要想假如那些数据收集到之后要怎么分析？如果问卷没有收集一些餐厅的基本数据，比如区域、员工数量、入职时长等基本数据，后面就无法用这些维度去分析。

4) 数据存储。为了方便后面的统计分析，应该形成一个数据表的格式，比如一个项目一行，每一列表示不同的变量，方便后面累加数据，可以不用修改就直接分析了。

度量与分析常见问题：收集度量要花精力，如果有一堆历史数据，但都不知道哪些度量与分析的目标关联，就浪费了。所以度量计划首先要明确度量分析的目标再定需要收集哪些数据。

Chapter 22

教小孩统计分析

如果你觉得统计学很高深，可以看看这个如何教小孩统计分析的故事。

从考试成绩到贸易逆差

孩子：爸爸，你是做什么工作的？我明天要在班上介绍自己父母的工作。

我：帮客户做统计分析。

孩子：统计分析？

我心里想你这初中孩子，要如何给你解释统计分析呢？

我尝试用一个他可以想象的场景来介绍——假如下面是你班里同学的数学成绩，你会用什么方式把这些成绩表达出来？

孩子有点迷茫。

20 名学生在数学考试中的分数 (满分为 **100** 分，按大小排序)

30, 35, 37, 40, 40, 49, 51, 54, 54, 55

57, 58, 60, 60, 62, 62, 65, 67, 74, 89

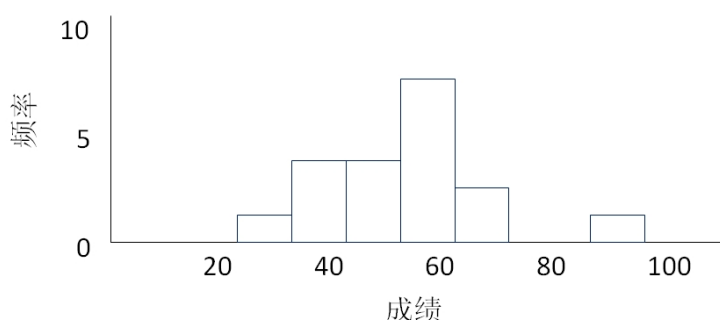
我：任何样本数据，都可以用图加平均值、范围等来表达。

举例：我们可以画茎叶图 (Stem and Leaf Plot)(详见附件) 表达这些学生成绩：

3	0			
3	5	7		
4	0	0		
4	9			
5	1	4	4	
5	5	7	8	
6	0	0	2	2
6	5	7		
7	4			
7				
8				
8	9			

把 20 组数从最低排到最高。排在第十的学生和第十一的学生的平均数 56，叫

中位数 (Median)。排第五和第六成绩的平均值 44.5，就是 $Q1$ ($1/4$)。排第十五名和第十六名的平均值 62，就是 $Q3$ ($3/4$)。这个例子中茎叶图再加上 $Q1$ 、 $Q2$ (中位数)、 $Q3$ ，[44.5、56、62] 便可以总括表达这二十个学生的成绩分布。也可以用下面的柱状图 / 直方图表达：



也可以利用 $Q1$ 、 $Q2$ 、 $Q3$ (统称四分位数) 画箱线图来表达 (详见附件)。除了中位数，也常用平均值 (Mean) 来表示一堆数据的中间趋势，平均值就是把所有数据加起来，然后把总数除以数量，比如上面 20 名学生数学成绩的平均值是.....

孩子充满自信，打断我说：这个平均值简单，我懂。

我说：好啊，你这么懂平均值，我问你一个小问题？

问题：5、10、350、355 的平均数是多少？

孩子拿了计算器算出：180。

我：你知道那些数是代表什么吗？

孩子：不知道。

如果这些数是代表角的度数，它的平均值应该是零，而不是 180。这是统计分析很重要的概念——你必须知道那些数的背景，否则你的分析没意义。如果你把那 4 个数字当成是距离来算均值是 180，与 4 个角度数求均值的结果就完全不一样了。

从以上例子，你就知道了解问题背景 (Context) 很重要。

孩子：这些统计分析有什么用？

我：例如有了这成绩的分析，你就可以知道自己的成绩在整个班是处于什么位置？也可以帮你比较不同科目的成绩。

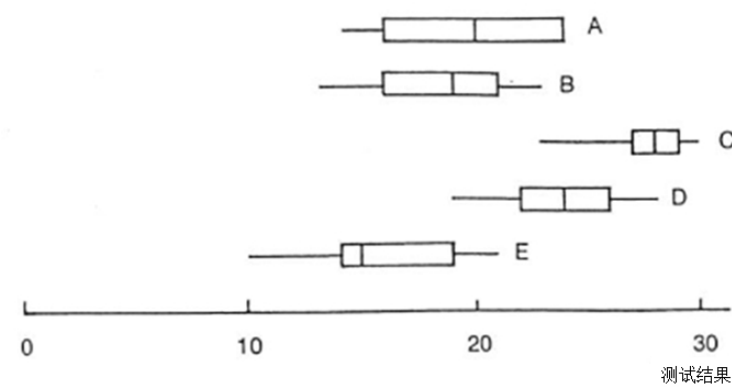
比较 3 种教学方法

45 名学生被随机分成 5 个大小相同的组。两组 (A、B) 采用目前使用的方法 (控制组)，另外 3 组 (C、D、E) 采用 3 种新方法。实验结束时，所有学生参加了一次标准测试，结果 (满分 30 分) 见表 1。关于教学方法的差异，可以得出什么结论？

表 1

45 名学生的测试结果									
A 组 (控制组)	17	14	24	20	24	23	16	15	24
B 组 (控制组)	21	23	13	19	13	19	20	21	16
C 组 (赞赏组)	28	30	29	24	27	30	28	28	23
D 组 (责骂惩罚组)	19	28	26	26	19	24	24	23	22
E 组 (不理睬组)	21	14	13	19	15	15	10	18	20

我：你可以用刚学过的四分位数 (Q1、Q2、Q3) 配合箱线图表示每一组数据分布，然后看看有没有差异。这样你便可以比较 5 个班的成绩。
他过了 15 分钟就画了 5 个组的箱线图：



我：能看到显著区分吗？
孩子：看得出是 C 组最好，D 组也比较好，E 组就比较差。
我：是的，但如果我们只是比较 5 个组成绩的平均值，没有考虑每一班成绩范围，便不能全面比较。例如：上面数学考试成绩的 Q1、Q2、Q3 是 (44.5、56、62)，如果 C 组的数学成绩是 (32、57.5、63)，你会认为 C 组比你们好吗？虽然中位数比你们班高，但 Q1 比你们低很多，所以不能单看中位数（或平均值）比较。

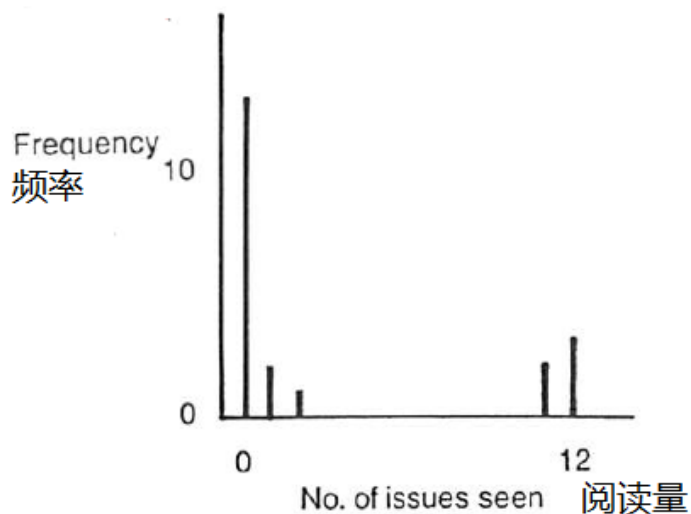
看到他对这些挺有兴趣，我便问他如何表达以下数据。

20 人一年内阅读月刊的数量

0, 1, 11, 0, 0, 0, 2, 12, 0, 0
12, 1, 0, 0, 0, 0, 12, 0, 11, 0

我看他开始使用同样方法，我便说这些统计数据跟前面学生成绩的分布不一样，它一头一尾最高，这表示大部分人要么就是不看杂志，要么就是每个月都看，

这种分布就不合适用刚才那些方法去表达，只能说它有两个高峰，或者叫众数 (Mode)，再配上一个柱状图。如果我们用中位数或者这些数的平均值，或三分位数来表达的话，反而是误导读者，不能正确的表达那些数据的分布。



当总体 (Population) 包含非常多数据时，就只能抽样，用抽样来估计 Population 分布。但是要注意，如果样本不是随机抽样 (Random Sample)，可能会导致出来的参数偏离、产生误差。

我看孩子一头雾水，但我记得他很喜欢研究二战的战斗机，我就用下面这个例子说明，这样抽样出来的结论是没有意义。

取样偏差 (Survivor's Bias)

错误的抽样会导致错误结论。

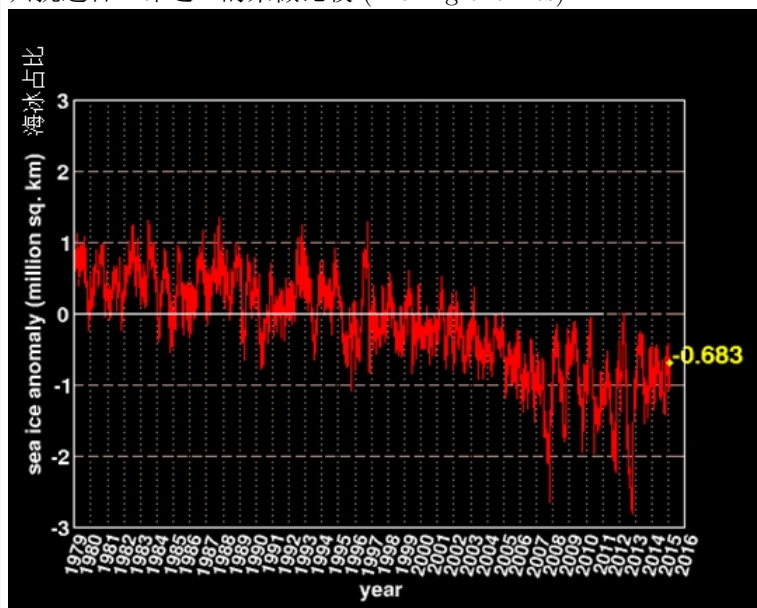
第二次世界大战时会对那些没有被击落的战斗机，研究哪个部分被德国军队的攻击最多，针对这些部位去加强防护，希望增高飞机的存活率。你同意吗？我们只是抽样了未被击落的飞机，被击落的都未被抽样。一个比较正确的抽样是所有两类都抽样才比较合理。其实被击落的飞机哪个部分被击中更重要，但是我们无法得到那些抽样，只能从没有被击落的飞机去看，这是抽样的错误。所以分析得出的结论没有意义。

二战时期战斗机



我：你看看下面例子，便会更了解随机抽样的重要性：

只挑选合“味道”的来做比较 (Picking cherries)



从上面 80 年代到现在全球海洋冰量的统计很明显看到地球在不断地暖软化。但你还可以抽两个时间，例如：2012 年 4 月份的冰量比 1988 年 6 月份多，来说明“全球变冷”！

孩子对统计分析好像越来越感兴趣。

我：现在你开始知道什么叫统计了吗？明天可以跟老师讲故事了吗？

孩子追着问：可不可以再给我一个练习题？我觉得这统计学也没什么困难，我还可以把你给我的那些题目拿给我同桌小李试试，看他懂不懂（男孩总是要当英雄，出风头）。

我：好啊。下面的身高统计数据，请你用刚才的方式来汇总一下，看看你学会没有？

20 名妇女参加某种疾病研究，她们的身高（以米计）分别为：

1.52, 1.60, 1.57, 1.52, 1.60, 1.75, 1.73, 1.63, 1.55, 1.63

1.65, 1.55, 1.65, 1.60, 1.68, 2.50, 1.52, 1.65, 1.60, 1.65

他便拿着纸，用刚才学过的方式计算中位数、三分系数等，蛮有自信地画图、交卷。

我：你认识 2.5 米高的女性吗？

小孩：真的好像没有。

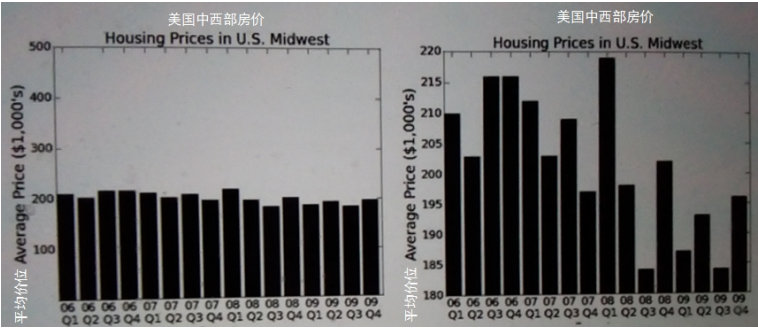
我：所以里面的 2.50 肯定是不对。你为什么没有疑问，直接去做？统计分析必须要判断数据是否正确、是否合理。

如果数据本身不可靠，那么怎么分析都没用，我们称之为“垃圾进，垃圾出 (Garbage in, Garbage out)”。

我：也有很多人利用图表误导读者，所以我们看一些统计专家的统计数据分

时也要小心。

美国中西部房价趋势图



上图是 1906 年到 1909 年每个季度，美国中西部的房价。如果我们看右面那个柱状图，你会认为房价波动很大。但是如果同样这组数据看左面柱状图的话，你就会觉得没有太多变化。所以我们要注意看左面坐标有没有标明数字。不应该仅仅看图。

虽然数据都一样，但如图形不一样，传达的信息便完全不同。

孩子：你说的我都懂，很简单。

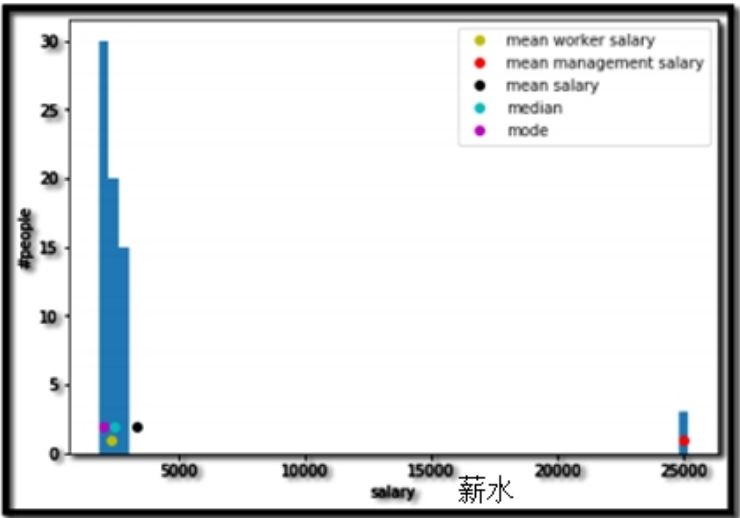
我：用统计分析骗人的故事可多。你不是说要今年暑假找暑期工赚钱，下面是一个去工厂找工作的故事：

{| class="wikitable" |-| 小李刚中学毕业，希望去工厂找工作。

问：工厂员工薪水是多少？

人事部经理面带笑容地跟他说：我们工厂员工平均月收入大概 3300。

小李进了工厂后，才发现绝大部分的工人月薪是 2000。有多年经验的工人才可以拿到 2400。但他们有少数的管理层，他们的平均月薪是 25,000。



所以人事部经理没有“骗”小李，工厂员工平均收入是每月 3309。

}}

分析例子：连续与分组数据

第二天，孩子乖乖地来找我，他说：爸爸，今天我跟老师说了你的工作是做统计分析，老师听到就发我一些高年级的生物实验数据，希望帮他做些分析，看两类实验有没有差异。你可以帮忙吗？

蚕豆植物 – 双样本 T 检验

以下数据比较 10 个剪枝和 10 个生根样本中某一化学物质的浓度：

剪枝:53, 58, 48, 18, 55, 42, 50, 47, 51, 45

生根:36, 33, 40, 43, 25, 38, 41, 46, 34, 29

我：你可以同样用昨天学过的方法（如柱状图）比较两组数。

孩子就按这个汇总了两个实验数据的分布。

浓度	
	15 - 19 20 - 24 25 - 29 30 - 34 35 - 39 40 - 44 45 - 49 50 - 54 55 - 59
剪枝	1 1 3 3 2
生根	2 2 2 3 1

我：你应该也看出：剪枝样本有一个很明显的异常点——18。你应该问老师，18 是否错误还是值得相信的数？如果 18 是不对，你就很明显看到上面剪枝的平均

浓度比下面生根高。如果不能“看”出来就可以用“双样本 T”检验比较两组数据是否有显著差异。但是如果有刚刚那种异常点，如直接用双样本 T 分析，因为第一组数据被那个 18 拉低了，得出结果是两组没有显著差异。所以还是直接看数据的分布更实在。

我：我们在统计学可以使用假设检验（注），其中包括双样本 T 检验比较, 方差分析 (ANOVA) 等，就是针对这类统计数据分析。但是在你这个实验数据，从画图一眼都能看出有显著区分，就不需要再采用假设检验，因采用那些方法不会得出额外的信息。
我接着跟他说：有些时候统计数据不是连续数据，像学生成绩、浓度、高度等，而是分组数据。例如下面这些国外人种眼睛颜色与头发颜色的数据，这种数据我们就需要用列联表 (Contingency Table) 来识别这两个变量之间有没有关系。

头发和眼睛颜色的不同组合
592 人的头发和眼睛颜色的不同组合：

表 1	头发颜色			
眼睛颜色	黑	深褐	红	金
棕	68	119	26	7
蓝	20	84	17	94
淡绿褐	15	54	14	10
绿	5	29	14	16

如果我们利用表 1 的数，把它那些每一行、每一列进行求和，我们可以看见每列或者每行的分析结果。例如棕色眼睛的占大多数，但金发例外，金发的大部分是蓝眼睛。其实也看到金头发的那列跟其他的一些分布很不一样。比如我们看每行的话，也可以看到棕色眼睛跟蓝眼睛的数量差不多。但在金头发和黑头发的列分布就不一样了。统计分析的卡方检验也可以帮我们分析分组数据间有没有相关。但刚才这类简单 4×4 数据表，计算出表 3 列联表便足够，卡方检验没有带来更多有用信息。
实际 (Obs.) 数量表:

表 2	头发颜色				
眼睛颜色	黑	深褐	红	金	总计
棕	68	119	26	7	220
蓝	20	84	17	94	215
淡绿褐	15	54	14	10	93
绿	5	29	14	16	64
总计	108	286	71	127	592

实际 (Obs.) vs 预计 (Exp.) 数量表 Contingency Table:

表 3	头发颜色								
	黑		深褐		红		金		
眼睛颜色	Obs.	Exp.	Obs.	Exp.	Obs.	Exp.	Obs.	Exp.	Total
棕	68	40	119	106	26	26	7	47	220
蓝	20	39	84	104	17	26	94	46	215
淡绿褐	15	17	54	45	14	11	10	20	93
绿	5	12	29	31	14	8	16	14	64
总计	108		286		71		127		592

注：双样本 T 检验、方差分析 (ANOVA)、卡方检验等都属于假设检验，下一章会详细介绍各种假设检验方法。

结束语

统计分析主要是希望利用数据分析、统计方式帮我们解决问题 (Problem Solving)，所以度量的重点是如何分析，帮助解决问题。

按目标策划收集哪些数据？如何收集？如何分析？

我们也应按以上思路，协助开发团队，更好地利用数据解决问题。

步骤	上面实例
	这些数字代表什么? 什么单位 (角度、还是距离)?
策划收集数据	怎样取样? 避免取样偏差
确保数据质量	2.5 米高女性? 异常数据?
初步数据分析 (描述性)	平均值、四分位数 (Q1、Q2、Q3)、柱状图、箱线图
数据分析	如果能简单看出来有显著差异, 便不需要用假设检验, 或其他统计分析方法
解读、沟通结果	当我们读统计分析报告要小心, 注意是否被骗

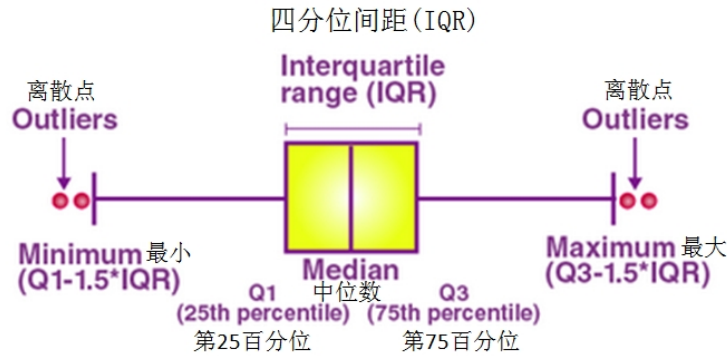
附件

如何画茎叶图 (Stem and Leaf Plot)

以文中 20 名学生在数学考试分数为例:

- 1. 把分数从小到大排序。
- 2. 从上而下画一直线。
- 3. 第一个数 30, 左边写 3, 右面写 0 (叶)。
- 4. 排第二是 35, 因 5 大于 0-4, 新一行, 左边写 3, 右面写 5 (叶)。
- 5. 37, 因 7 属于 5~9, 所以同一行, 在 5 右面写 7 (叶)。
- 6. 最终便可以简单利用数目字形成一个横放的直方图。

箱线图 (Box and Whisker Plot)



箱线图包括 5 个数:

- Minimum 最少 (Q0)
- 1st Quartile (Q1 , 25 percentile) 箱子的左面边线
- Medium 中位数 (Q2 , 50 percentile) 箱子中间的线
- 3rd Quartile (Q3 , 75 percentile) 箱子的右面边线
- Maximum 最大 (Q4 , 100 percentile)

有些箱线图直接把两头指到最大与最少, 不展示离散点 (Outliers)。

$$\text{Interquartile range (IQR)} = Q3 - Q1$$

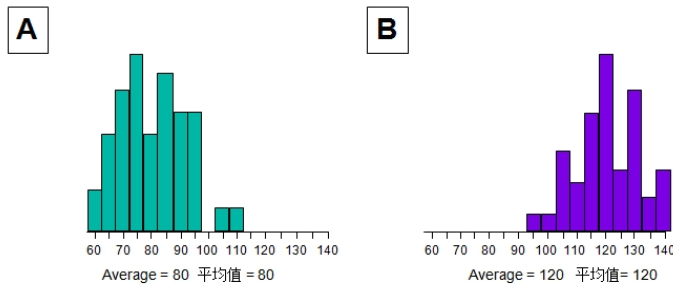
参考 References

1. Guttag, John V. "Introduction to computation and programming using Python." (2021) MIT Press
2. Chatfield, Chris. "Problem Solving: a statistician's guide."(1995) 2/e Chapman & Hall
3. Bluman, Allan. "Elementary Statistics." 10/e

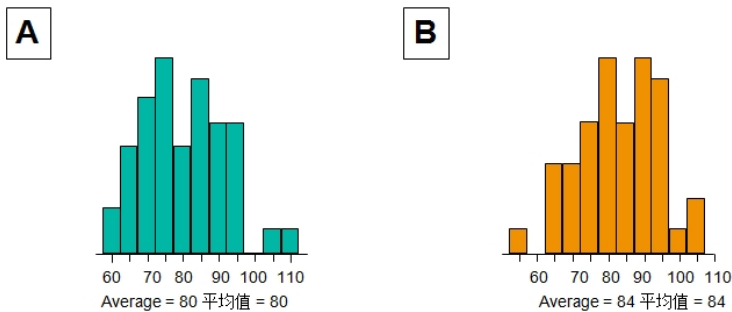
Chapter 23

假设检验

上章介绍了双样本 T 检验，是假设检验的一种。下面介绍假设检验，及它帮我们解决什么问题。



想比较 A、B 两组实验结果是否有显著差异。如果像上图，应该看到 B 明显比 A 高。
但如果像下图，就很难看得出来：



假设检验利用统计分析方法帮我们区分，两组数据是否有显著的差异呢？

假设检验的步骤

零假设: A 与 B 没有显著差异

备选假设: A 与 B 有显著差异

挑选对应的检验方法后, 可以利用工具计算出的 P 值来判断。

如果 P 值大于 0.05, 就不能拒绝零假设。

如果 P 值小于 0.05, 就可以拒绝。

因 A 与 B 都包含非常多样本数据, 我们只可以利用 A 的随机抽样, 和 B 的随机抽样数值, 来判断 A 与 B 是否有显著差异, 这便是双样本 T 检验。

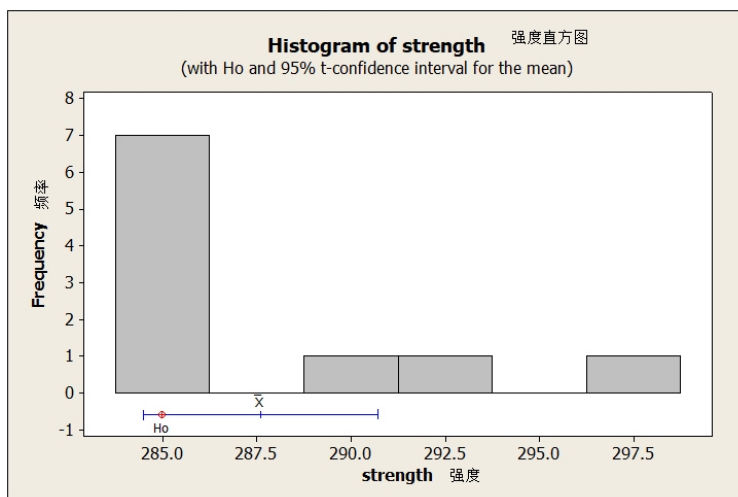
要深入了解便要用数据说话, 先举个简单的假设检验实例:

检验单个正态总体的均值

按历史数据, 某厂生产的钢丝折断力是正态分布 (平均值 = 285, 标准差 = 4), 更换了新的钢材供货商, 随机抽取以下 10 样本, 想判断更换供货商后折断力和原先是否有显著提升?

289, 286, 285, 284, 286, 285, 285, 286, 298, 292

从下面样本钢丝折断力柱状图看到 10 个样本中有 7 个接近 285、有 3 个是明显大于 285, 应如何判断是否有显著差异呢?



按假设检验步骤:

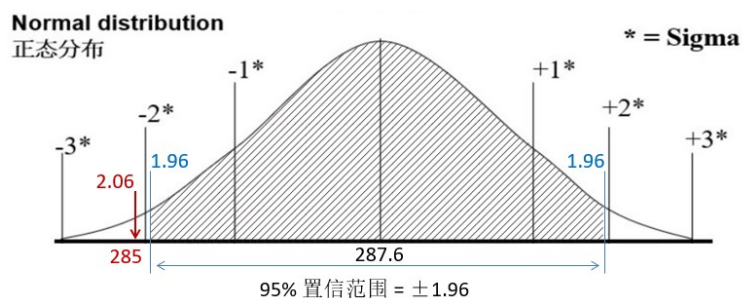
1. 零假设*: 新钢材的折断力小于或等于 285 ($H_0: \mu \leq 285$)。
2. 备选假设: 新钢材的折断力大于 285 ($H_1: \mu > 285$)。

(* 通常设零假设为原本情况, 没有变化, 备选假设为希望的情况, 有提升。)

这例子可以使用正态分布相关方程式，估计概率，如概率低于 5%，拒绝零假设。（折断力 > 285）

利用正态概率分布图，抽样的平均值是 287.6。

历史的平均值是 285，并考虑样本平均值、样本方差、样本数量得出检验统计量计算值 Z_n 为 2.06（计算方程式详见附件），超出了 95% 置信区间 ± 1.96 范围，所以我们拒绝零假设，样本与 285 有显著差异（提升）。



也可以利用工具，得出单样本 Z 检验 (one-sample Z test) 的 P 值，因 P 值 = 0.04, 少于 0.05，判断拒绝零假设，新供货商的钢丝折断力比原本 285 高。

要了解 P 值是什么？为什么低于 0.05 可以拒绝？首先要理解假设检验会发生两类错误。

检验可能犯错误，所谓犯错误就是检验的结论与实际情况不符，有两种情况：

1. 实际情况是 H_0 成立，而从检验结果选择了 H_a ，犯了第一类错误 (Type I error) 或“弃真”错误。
2. 实际情况是 H_0 不成立， H_1 成立，而从检验结果选择了 H_0 ，犯了第二类错误 (Type II error)，或称“取伪”错误。

估计是：

		Ho	Ha
实际是：	Ho	Right Decision 正确决策 	α Risk Type I Error α风险 第一类错误 
	Ha	β Risk Type II error β风险 第二类错误 	Right Decision 正确决策 

这两类错误 与 之间是有关系：

如果 风险（概率从本来 0.05 降低）下降，便会引起 风险提高，所以不能同时降低 与 。 不能无限降低，一般选 $\alpha = 0.05$ 。

		Ho	Ha
Ho	Ha		第一类错误α 
	Ha	第二类错误β 	

我们可以简单理解 P 值为发生第一类错误的概率，如果低于 5% 我们就有信心拒绝零假设。

使用假设检验比较两组数据有没有显著差异的例子。

比较两个正态总体的均值

一名研究人员假设大学为男生提供的运动项目的平均数量大于大学为女生提供的运动项目的平均数量。下表是对大学男女生提供的体育项目数量的随机抽样结果（各 50 位大学男女生数据分成两组，两组数据之间没有对应关系），设定临界概率为 5%（ $\alpha = 0.05$ ），是否有足够的证据支持这种说法？（假设方差都一样， $\sigma = 3.3$ ）。

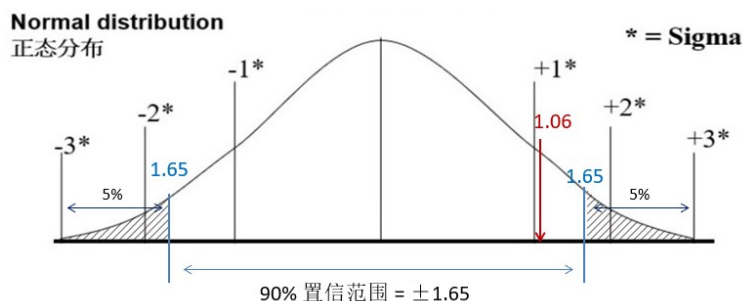
男生 Males					女生 Females				
6	11	11	8	15	6	8	11	13	8
6	14	8	12	18	7	5	13	14	6
6	9	5	6	9	6	5	5	7	6
6	9	18	7	6	16	10	7	8	5
15	6	11	5	5	16	10	7	8	5
9	9	5	5	8	7	5	5	6	5
9	5	11	5	8	7	8	5	7	6
7	7	5	10	7	11	4	6	8	7
10	7	10	8	11	14	12	5	8	5

1. 零假设：男女运动项目平均数量没有区别（ $H_0: \mu_1 \leq \mu_2$ ）。
2. 备选假设：男生运动项目平均数量比女生多（ $H_1: \mu_1 > \mu_2$ ）。

男生的均值是 8.6，女生的均值是 7.9，相差是 0.7。

假定使用 90% 置信区间：

正态分布 $\pm 1.65\sigma$ (标准差) 内的面积是总面积的 90%，计算出检验统计量计算值为 1.06，从下图看到 1.06 落在 90% 置信区间之内，所以不能拒绝零假设。



(有关计算 1.06 的方程式，详见附件。)

也可使用工具跑出来的 P 值是 0.146，大于 0.05，不能拒绝零假设。

以上两个例子都是假定：分布的方差已知。

但很多时候，我们不一定知道分布的方差或标准差是多少？我们便需要从样本数估计分布的方差（或标准差）。下面是一个结对 T 检验 (Paired T test) 例子：

检验单个正态总体的均值，方差未知

营养师正在观察如果每天饮食都添加某类矿物质，是否会改变人的胆固醇水平。对随机选取 6 位对象在实验之前进行了测试。然后对他们进行为期 6 周的测试，在他们的食物里添加矿物质补充，实验之后再测试，结果显示在下表 (血清总胆固醇水平以毫克每分升 (mg/dl) 为单位)。能否得出结论，在 $\alpha = 0.1$ 时，胆固醇水平是否发生了变化。

对象	1	2	3	4	5	6
前 (X1)	210	235	208	190	172	244
后 (X2)	190	170	210	188	173	228
(X1) - (X2)	20	65	-2	2	-1	16

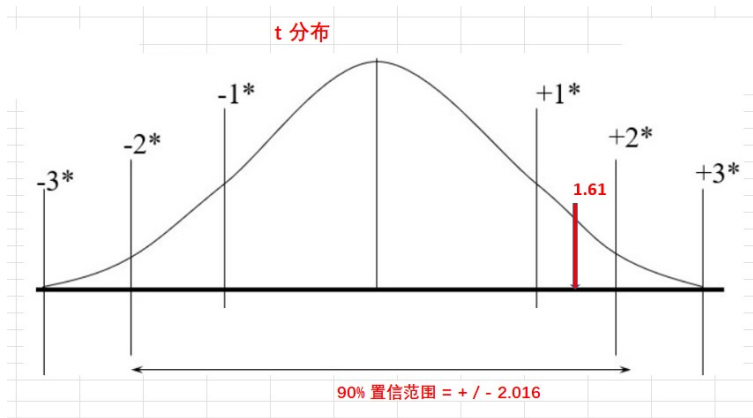
1. 零假设：没有变化 ($H_0: \mu_D = 0$)

2. 备选假设：有显著变化 ($H_1: \mu_D \neq 0$)

注意：

这例子与上面男女学生例子不一样，因前后是一一对应，我们需要用结对 T 检验 (Paired T test)。因标准差不是已知，是从样本估算，所以要用 T 检验，非 Z 检验。

可以使用相关方程式，计算出 t 值 = 1.61 (计算过程详见附件)。



从图看到 1.61 落在 90% 置信区间 (2.016) 之内，所以不能拒绝零假设。

也可以利用工具，计算双样本 T 检验 (Paired T test) 的 P 值，因 P 值 = 0.165，大于 0.05，判断不能拒绝零假设。

上面都是比较两组数据，如果想比较比较两组以上便要用方差分析 (ANOVA)。

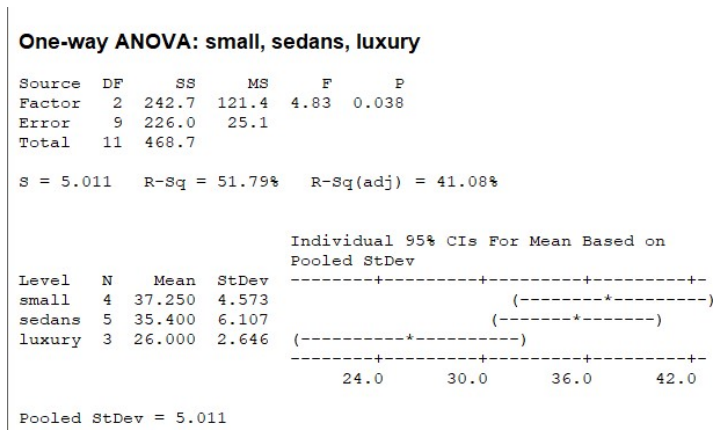
方差分析 (ANOVA test)

例子一

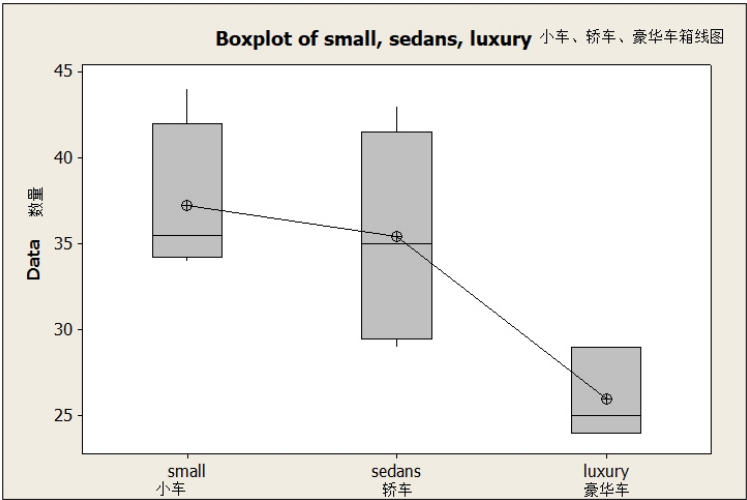
比较 3 种车型（小轿车，舒适型，豪华车）的省油系数有没有显著差异？

文件:5anovaDataScreenshot 2022-07-24 105955.jpg

用统计工具跑单因子方差分析，得出 P 值是 0.0385，低于 0.05，所以我们便可以拒绝零假设，3 车型有显著差异。



从下面工具的简单画图的也看到豪华型的系数范围比其他两种有显著差异：



方差分析只能识别有没有显著差异，但若想要明确差异在哪里，便要用 Tukey-Kramer 做多组之间比较。

例子二

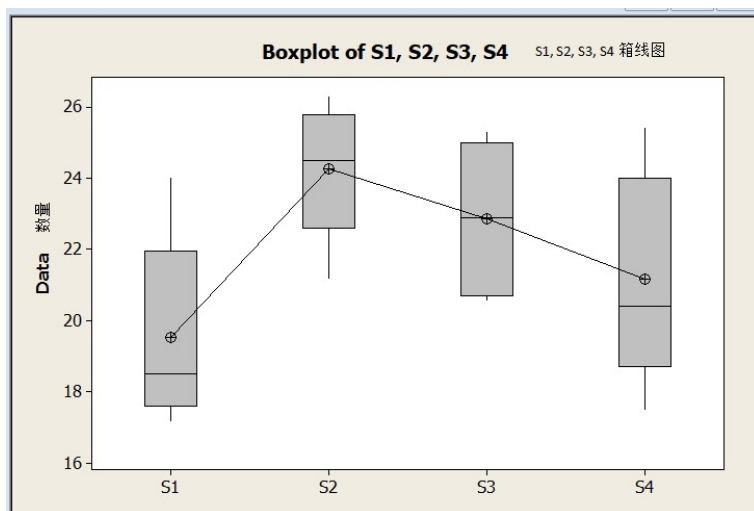
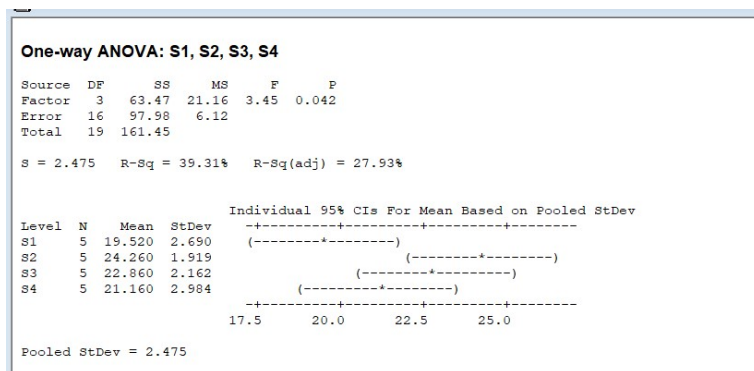
某降落伞工厂想比较 4 家布料供货商的布料拉力（因拉力不足会影响使用者的性命安全）。

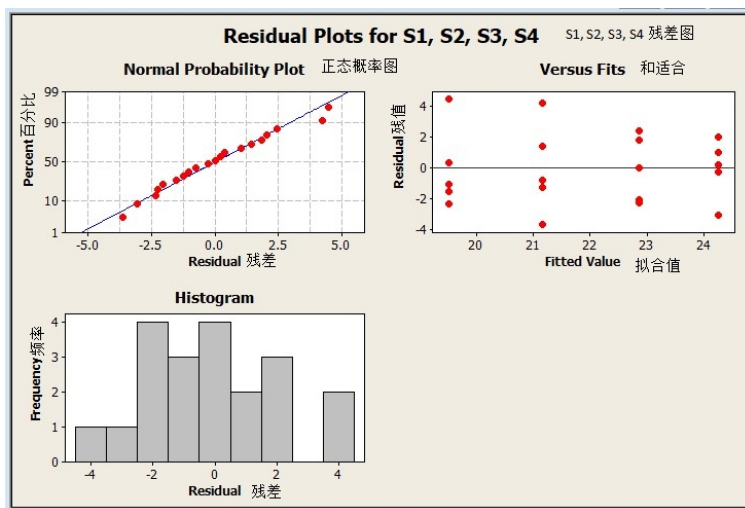
下面是它们随机抽样的结果：

布料供货商	1	2	3	4
随机样本拉力	18.5	26.3	20.6	25.4
	24.0	25.3	25.2	19.9
	17.2	24.0	20.8	22.6
	19.9	21.2	24.7	17.5
	18.0	24.5	22.9	20.4
样本均值	19.52	24.26	22.84	21.16
样本标准差	2.69	1.92	2.13	2.98

用工具跑方差分析 P 值是 0.042，少于 0.05，判断拒绝零假设。

- Minitab 出来方差分析结果表，与上面的总结表能一一对应。





- 从 4 组的分布，你可能看出第一组与第二组的差异很明显，其他差不多。如要明确哪里有差异，便需要用统计分析来判断。

Tukey-Kramer 多组比较

实例（比较 4 供货商布料的拉力）：

$$\#|\bar{X}_1 - \bar{X}_2| = |19.52 - 24.26| = 4.74$$

1. $|\bar{X}_1 - \bar{X}_3| = |19.52 - 22.84| = 3.32$
2. $|\bar{X}_1 - \bar{X}_4| = |19.52 - 21.16| = 1.64$
3. $|\bar{X}_2 - \bar{X}_3| = |24.26 - 22.84| = 1.42$
4. $|\bar{X}_2 - \bar{X}_4| = |24.26 - 21.16| = 3.10$
5. $|\bar{X}_3 - \bar{X}_4| = |22.84 - 21.16| = 1.68$

用 Tukey-Kramer 相关方程式（详见附件），得出临界范围为 4.47，所以判断只有供货商 1 与 2 之间的布料拉力是有显著差异。

分组（分类）数据

上面都是用于连续数据的例子，假设检验也可以用于分组（分类）数据。

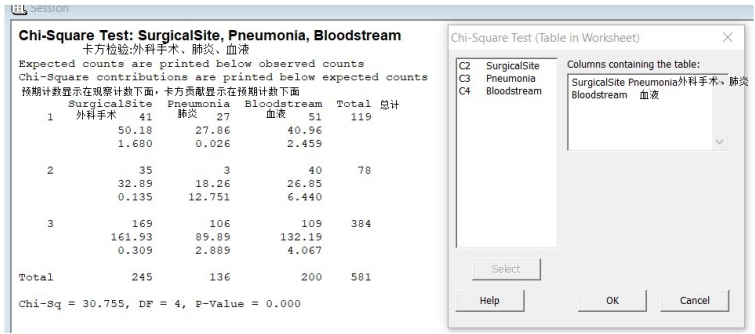
卡方检验 (Chi Square test)

研究三家医院 (A、B、C) 在 3 类最终导致患者死亡感染的主因（外科手术 Surgical Site、肺炎 Pneumonia、血液 Bloodstream）分布是否有显著差异？

↓	C1-T	C2	C3	C4
	hospital	SurgicalSite	Pneumonia	Bloodstream
1	A	41	27	51
2	B	35	3	40
3	C	169	106	109

1. 零假设：那家医院与感染主因没有关系。
2. 备选假设：两者有显著关系。

卡方检验得出 P 值等于零，所以可以拒绝零假设，医院与感染主因有关系。



• 为了计算卡方值 (Chi-square)，首先使用以下公式计算每个列联表的预计值 E(Expected Value)，

$$E = \frac{(\text{row sum}) * (\text{column sum})}{\text{grand total}}$$

再用下面公式从预计值与实际值 O(Observed value), 计算卡方值：

$$\chi^2 = \sum \frac{(O-E)^2}{E}$$

• 计算得出卡方值 = 30.696 （注），比关键值 9.5 高
(对应 DF=4 (= (3-1)*(3-1))， $\alpha = 0.05$ ，从 χ^2 统计表得出 9.5)，

所以拒绝零假设，医院 (Hospitals) 与感染种类 (Infections) 之间相关。
: 注：对应 Minitab 工具自动算出的 30.755。

附件

中心极限定理 (Central Limit Theorem CLT)

当总体 (Population) 均值为 μ ，标准差为 σ
随机样本均值

$\overline{X}_n = \frac{\sum_i X_i}{n}$ 的: (n 为每轮抽样样本数量)

均值 = μ

标准差 = $\frac{\sigma}{\sqrt{n}}$

- 样本均值的分布随样本数量增加越来越接近正态分布

检验统计量计算值 $Z_n = \frac{\sqrt{n}(\overline{X}_n - \mu)}{\sigma}$ 接近正态分布。

(正态分布图形, 详见第 25 章控制图里的噪音还是信号)

检验单个正态整体均值相关方程式

样本平均值分布的方差 = 样本方差 / n, 并可以用以下方程式估算 (如果样本数 n 越大, 样本均值的方差就越少):

$$Z_n = \frac{\sqrt{n}(\overline{X}_n - \mu)}{\sigma}$$

$$\overline{X} = 287.6$$

$$\frac{\sqrt{n}(\overline{X} - \mu_0)}{\sigma} = \frac{\sqrt{10}(287.6 - 285)}{4} = 2.06$$

比较两个正态总体的均值

男生:

$$\overline{X}_1 = 8.6; \overline{\sigma}_1 = 3.3$$

$$\text{女生: } \overline{X}_2 = 7.9; \overline{\sigma}_2 = 3.3$$

$$\frac{(\overline{X}_1 - \overline{X}_2) - (\mu_1 - \mu_2)}{\sqrt{\frac{\sigma_1^2}{n_1} + \frac{\sigma_2^2}{n_2}}} = \frac{(8.6 - 7.9) - 0}{\sqrt{\frac{(3.3)^2}{50} + \frac{(3.3)^2}{50}}} = 1.06$$

检验单个正态总体的均值, 方差未知

$$\overline{D} = \sum D/n = 100/6 = 16.7$$

$$(D = X_1 - X_2)$$

$\mathbf{H}_0$ 的拒绝域为:

$$\frac{\sqrt{n}(\overline{D} - \mu_0)}{S_D} > t_{\alpha}(n-1)$$

- 查 student-t 分布表, $t_{0.05}(5) = 2.016$

- 从样本估计方差

$$s_D^2 = \frac{\sum (D_i - \bar{D})^2}{(n-1)} = \frac{\sum D_i^2 - n \times \bar{D}^2}{(n-1)} = \frac{(20^2 + 65^2 + (-2)^2 + 2^2 + (-1)^2 + 16^2) - 6 \times (16.7)^2}{(6-1)}$$

$$= \frac{4890 - 6 \times 278.9}{5} = 645$$

$$s_D = 25.4$$

- 计算得：

$$t = \frac{\sqrt{n}(\bar{D} - \mu_0)}{S_D} = \frac{\sqrt{6}(16.7 - 0)}{25.4}$$

$$= 1.61$$

Tukey-Kramer 相关方程式

- 计算显著差异的临界值：

$$\text{Critical range} = Q_u \sqrt{\frac{MSW}{2} \left(\frac{1}{n_j} + \frac{1}{n_j} \right)}$$

对应布料拉力例子：

- 按 $\alpha = 0.05$, $c = 4$, $n - c = 20 - 4 = 16$

从统计表得出对应 $Q_u = 4.05$;

- $MSW = 6.1$ (如何计算 MSW, 详见下面”如何计算 MSW”)

代入上面方程式：

$$\text{Critical range} = 4.05 \sqrt{\left(\frac{6.1}{2} \right) \left(\frac{1}{5} + \frac{1}{5} \right)}$$

$$= 4.47$$

如何计算 **MSW**, 参考 **ANOVA** 方程式

- 总离差平方和 Total Variation = Sum of Squares for Total (SST)。
- SST 由组内离差平方和 (SSW), 和组间离差平方和 (SSA), 双加出来
- 如果组间离差远远大于组内离差, 表示组间差异大, 应拒绝零假设; 反之, 如差异不大, 便难以拒绝零假设, 没有显著差异。
- 单因素方差分析计算 $F = MSA / MSW$, 如果 F 值大于临界值, 拒绝零假设

统计工具, 如 Minitab, 能直接算出下面总结报告：

原因	自由度	离差平方和 (Sum of Squares)	(Mean Square)	F
组间	c - 1	SSA	MSA = SSA/(c-1)	MSA/MSW
组内	n - c	SSW	MSW = SSW /(n-c)	
总	n - 1	SST		

相关方程式

* 总离差平方和

$$SST = \sum_{j=1}^c \sum_{i=1}^{n_j} (X_{ij} - \bar{X})^2$$

注：总体平均数 $\bar{X} = \frac{\sum_{j=1}^c \sum_{i=1}^{n_j} X_{ij}}{n}$

- 组间离差平方和

$$SSA = \sum_{j=1}^c n_j (\bar{X}_j - \bar{X})^2$$

注：组 j 的平均值 $\bar{X}_j$

- 组内离差平方和

$$SSW = \sum_{j=1}^c \sum_{i=1}^{n_j} (X_{ij} - \bar{X}_j)^2$$

注：组 j 的平均值 $\bar{X}_j$

- 单因素方差分析均方和

$$MSA = \frac{SSA}{c-1}$$

$$MSW = \frac{SSW}{n-c}$$

$$MST = \frac{SST}{n-1}$$

实例（比较 4 供货商布料的拉力）：

- 组间自由度 = c - 1 = 4 - 1 = 3
- 组内自由度 = n - c = 20 - 4 = 16
- 总自由度 = n - 1 = 20 - 1 = 19

$$\text{总体平均数 } \bar{X} = \frac{\sum_{j=1}^c \sum_{i=1}^{n_j} X_{ij}}{n} = \frac{438.9}{20} = 21.945$$

$$SSA = \sum_{j=1}^c n_j (\bar{X}_j - \bar{X})^2$$

$$= (5)(19.52-21.945)^2 + (5)(24.26-21.945)^2 + (5)(22.84-21.945)^2 + (5)(21.16-21.945)^2$$

$$= 63.285$$

$$SSW = \sum_{j=1}^c \sum_{i=1}^{n_j} (X_{ij} - \bar{X}_j)^2$$

$$\begin{aligned}
&= (18.5 - 19.52)^2 + \dots + (18.0 - 19.52)^2 + (26.3 - 24.26)^2 + \dots + (24.5 - 24.26)^2 \\
&\quad + (25.4 - 21.16)^2 + \dots + (20.4 - 21.16)^2 \\
&= 97.5
\end{aligned}$$

$$\begin{aligned}
\text{SST} &= \sum_{j=1}^c \sum_{i=1}^{n_j} (X_{ij} - \bar{\bar{X}})^2 \\
&= (18.5 - 21.945)^2 + (24 - 21.945)^2 + \dots + (20.4 - 21.945)^2 \\
&= 160.7895
\end{aligned}$$

$$\text{MSA} = \frac{\text{SSA}}{c-1} = \frac{63.285}{4-1} = 21.1$$

$$\text{MSW} = \frac{\text{SSW}}{n-c} = \frac{97.5}{16} = 6.1$$

$$F = \text{MSA} / \text{MSW} = 21.1 / 6.1 = 3.46$$

能对应文中 Minitab 出来的结果：

$$\text{MSA} = 21.16, \text{MSW} = 6.12, F = 3.45$$

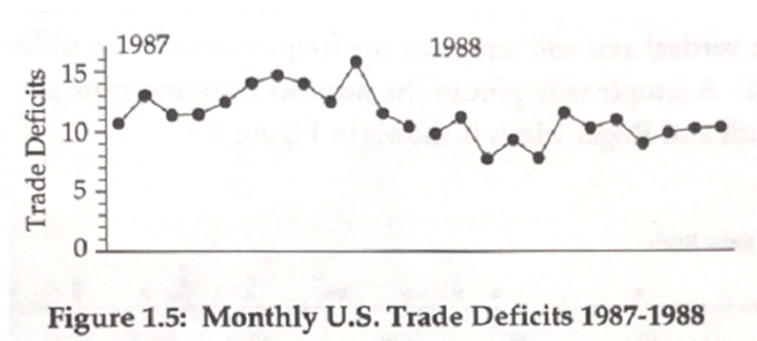
Chapter 24

控制图

我：请问你们公司来年的目标是什么？
客户：希望提升 5% 市场占有率，总体的毛利也提升 10%。
我：公司只是定个百分比是难以监控。原因是大家都不知道本来是多少，所以后面难以判断是否达到。有些公司不仅仅用百分比来制定总体目标，也用百分比监控生产，你可随便看某月报。
例如某 7 月份报表里包括当前 7 月的数字与上个月的比较；与去年 7 月份比较；比较累计值等，但变化百分比有正有负，你看完这种月报能了解生产的变化情况吗？是变好还是变差？
客户：很多时候看不出来。
我：所以我们有以下原则：——
''' 要了解确实的数据变化，应该有数字'''。

但是仅有数据也无法“看”到变化，所以也需要把那些数按顺序画趋势图；比如你看下面 1987 ~ 1988 美国国家总贸易逆差每月的数据趋势图，就更好了解美国贸易逆差这两年的变化。
美国按月贸易逆差，1987 ~ 1988 (\$, 亿美元)

	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
1987	10.7	13.0	11.4	11.5	12.5	14.1	14.8	14.1	12.6	16.0	11.7	10.6
1988	10.0	11.4	7.9	9.5	8.0	11.8	10.5	11.2	9.2	10.1	10.4	10.5



建立基线

有数据便可以建立公司基线（或标杆）作为以后参考。你看刚才 2 年的美国贸易逆差数据有变化吗？

客户 1：有变化，比如中间 1987 年 10 月就特别高，后面 1988 年的 5 月也特别高，应该是不正常。

客户 2：我看不是，好像是后面那些数比前面低了。

我：很好，但要判断分析数据有没有变化，很难单靠主观识别，需要有工具帮助。

噪音还是信号

我：假设一家生产矿泉水的企业（通常一瓶水 500 毫升），为了确保每次机器装的水不会太多，也不能太少，每一轮生产都会随机抽 5 个样本，然后记录抽样的均值和范围（最大和最小）。下图是过去 22 天均值的变化：

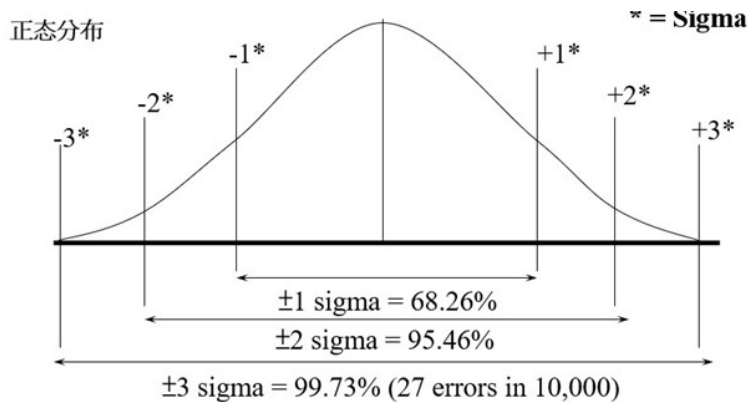


肯定是后面比之前的下跌了，但是如果等到最后 D 点才行动就太晚了，已经跌了太多，但是在那个点开始有变？

这也是 20 年代美国电话公司要解决的问题：很多电话线路与设备都埋在地下，调配维修工作很困难，如何能尽量减少无用的调配工作？公司也利用生产线大量生产电话设备，需要调试生产线的各种设备参数，但工程师发现难以把生产线调试到稳定状态，如果按系数低了调高，或者高了调低，反而会越调越乱。针对这难题，Dr Shewhart（当时在 Bell Labs 工作）发明了控制图，用来区分是

过程的噪音还是信号。你们高中的时候学统计学有听过正态分布吗？

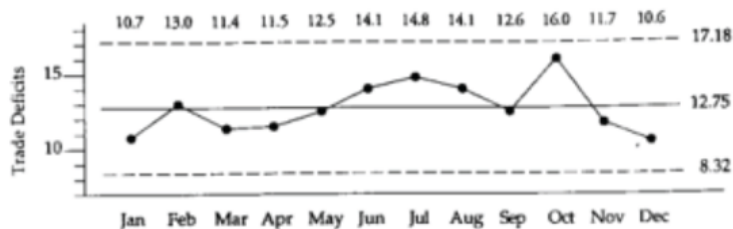
客户：有印象，但全都还给老师了。



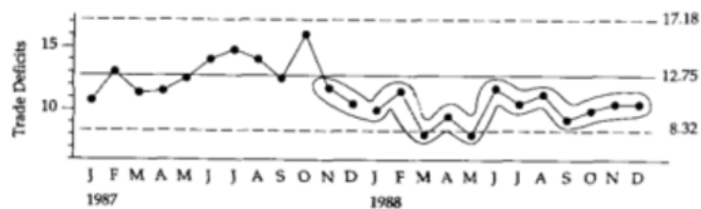
我：上图是正态分布，正负 3 个标准差包括了 99% 以上的面积。记得我们前面说过中心极限定理吗？任何分布如果随机抽样够多的话，它的均值就是接近正态分布。所以控制图按样本平均值的标准差，用方程式计算上限和下限，如果后面有些点超出了上下限范围（因上下限是按正负 3 个标准差出来，所以任何后面的点超出这范围，它发生的概率是少于 1%。）利用控制图帮我们区分那些波动是噪音，那些波动（异常点）是信号。（想了解各种常用于生产管理的控制图，参考附件。）

注意：虽然上面用正态分布来理解为什么选上下 3 个标准差为控制图上下限，但 Shewhart 先生的控制图方法没有规定或假定数据必须是正态分布。

回到两年美国贸易逆差数据，刚才不是说 10 月份超高吗，但是如果用控制图发现它还是在范围之内。如果我们多看两个月 -- 11 月和 12 月，发现贸易逆差又跌下来



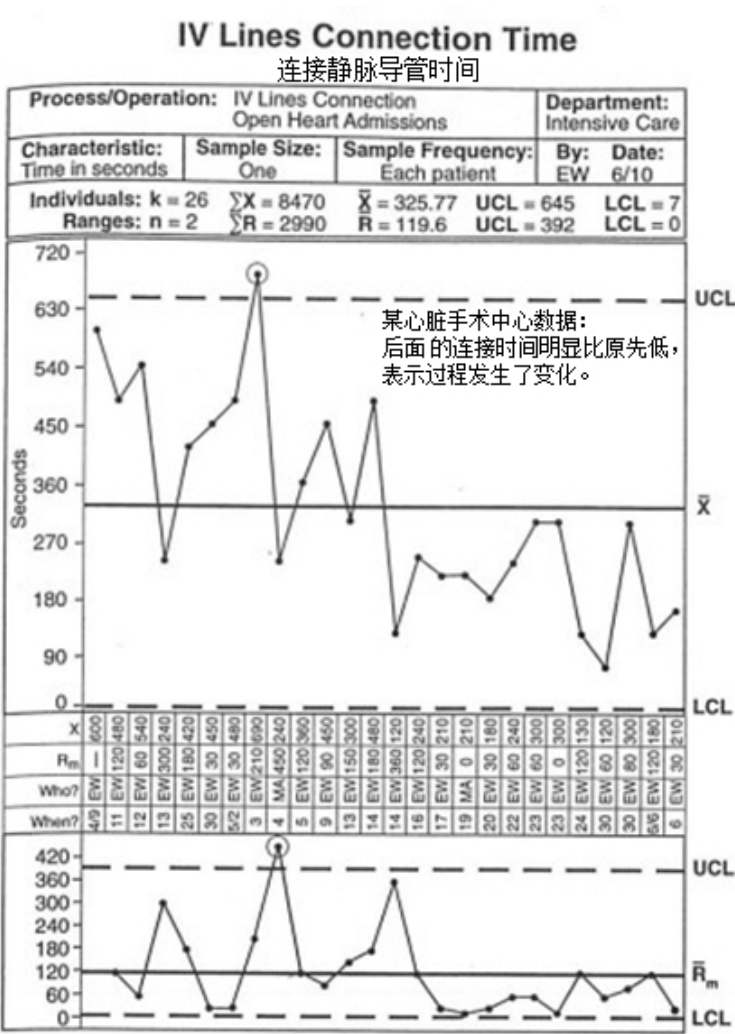
我们继续看，发现 3 月有异常点，后面在 5 月份也有异常点。如果我们再看后面月份的逆差数据，看见明显比前面降低。



客户：理解了。这个对于我们定基线有什么作用啊？

我：从刚才贸易逆差的数据，我们就不能把两年的数据的均值与范围（极差）作为基线，应该是以后面降低后的那些点，才能真正反应当前的水平和分布。

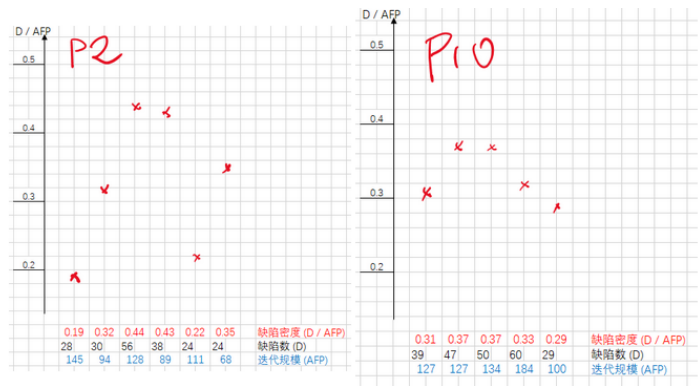
如果不是工业生产，不一定有这么多数据去随机抽样、求均值，怎么办呢？可以用 ImR 图 (或 XmR)，比如下图就是某个医院做外科心脏手术所花的时间的统计，想看看开心手术的时间是否稳定，因为外科手术特别危险，越长时间那个患者的存活机会就越低，也看到是有异常点。



(ImR 控制图相关公式，详见附件)

客户：可以用于我们软件开发吗？

我：是或者不是，比如一些已经进了维护期的产品都很稳定，你就可以用控制图来识别有没有变化。但是一些全新的过程可能变化很大，可能控制图就不一定适用。但如果是客服收多少投诉，每天都是很稳定、连续的，应该可以用。下面是两个项目的迭代缺陷密度趋势图，请问你觉得那个项目的数据可以用统计图？



客户：右面 P10 应该可以，左面 P2 的数据太散了。
我：是的，例如，你会发现 P2 的控制图上下限非常宽，必须首先要知道是什么原因导致，完善后稳定，才可以用控制图。

控制图的应用

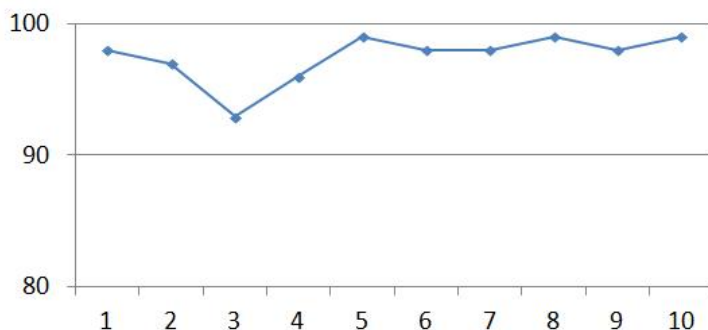
某企业非常关注交付有没有延误，每月都收集各个部门发生投产延误的次数，除以项目发布投产总数，统计并每月汇报高层。
从 2021 年初开始使用控制图，计算上限与下限 (0.98 , 0.93)，3 月份统计发现系数是 0.92 (0.92 是按时交付率，所以是越少越差)，超出了下线范围，改进小组启动根因分析。
每月的统计数是从 7 个模块（事业部）汇总来，改进小组利用 80/20 原则，识别出贡献最大的头两个模块已占总数之 73%，便专门调研这两个模块。使用问卷、面谈等，找出主因：

1. 新进项目经理较多，对里程碑管理、关闭条件等要求不清楚。
2. 项目有点难，和用户沟通效果不佳。

对应改进行动：

- 对项目经理的里程碑管理过程进行改进。包括：
 1. 培训（讲解项目管理工具的使用和里程碑管理要求）。
 2. 考核（明确如果出现里程碑红灯的考核标准）。
 3. 预防（建立专项沟通小组，对存在进度风险的项目，提醒项目经理尽快推进，项目经理和用户提前沟通，准备变更）。

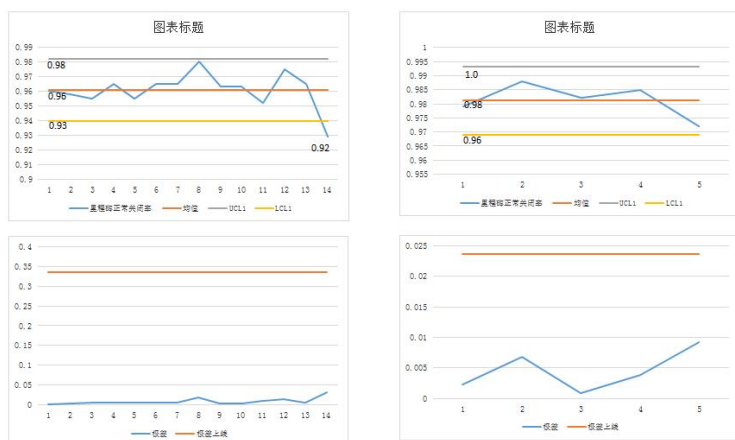
我问：效果如何？
过程改进组长：请看下图今年到 10 月份的趋势图，从 3 月份的低点后面就明显改善。



我：建议你用控制图方程式计算，从 5 月份到现在的统计图。

第二天，过程改进组长展示之前之后的两个控制图：

下面左图是 3 月份 0.92 与之前一年多的控制图，右图是改进后 5 月份开始的控制图。



我：从左右控制图看得出改进后的下限从本来的 0.93 上升到 0.96。除了画图，也可以计算过程能力指数 (Cpk) 反映过程能多满足客户要求 (Cpk 计算公式，详见附件)。

正与你说公司本来要求不能低于 0.93 (客户要求规格)，所以 3 月份之前的能力指数 $Cpk = 1.0$ 但现在过程收窄了， $Cpk = (1 - 0.93) / (1 - 0.96) = 1.75$ 上升了。

如果公司因看到提现，也是收窄的规格范围，要求把下限调高到 0.96，你们的能力指数 Cpk 又变回了 1.0。所以 Cpk 可用来代表过程满足客户要求的能力系数。

过程改进组长：从 5 月后数据更新的控制图一直都很稳定，没有异常点，是否表示我们就没有提升的空间，下限 0.96 已经是最佳状态？

我：不一定，你 3 月份分析时，不是说利用二八原则，识别出两个影响最大的板块吗？现在你的控制图是把所有板块的总数的控制图，建议你试试针对那两

个影响最大的板块，画他们的控制图看看。有机会能看到一些异常点。
 第二天，过程改进组长：确实看到有较大的波动，但还没有超出上下限，我们会继续观察。
 以上实例帮我们了解：

1. 基线

从历史数据得出基线范围能帮助我们区分后面哪些波动是自然噪音，哪些是信号。信号表示过程可能有基本变化，需要更新基线。控制图帮助我们做好这区分。

2. 过程能力 (Process Capability)

- 识别。
 1. 过程的声音 (过程的自然波动范围)。
 2. 客户的声音 (客户规格上限下限)。
- 计算过程能力指数 (Cpk) 反映过程能多满足客户要求。

3. 细分

- 不仅仅看总的综合控制图。
- 从总图细分，细分后的控制图可能发现异常点信号。(详见附件有一个细分例子)。

结束语

以上是如何利用统计图帮继续改地过程的典型案例，所以不要误以为控制图的目的只是为了控制不要发生异常点忽略了也要利用异常点看看过程有显著变化，是否启动根因分析，并采取纠正措施。

很多公司制定改进目标还是依赖主观判断，定一个“合理”提升百分比，使用统计图后便可以当前过程的基线范围作参考，制定有具体范围的提升目标。

附件

如何画 ImR 控制图

- 1/ 计算单值 (X) 平均数 (X-bar)
 - 2/ 计算移动极差 (mR moving Range) 均值
 - 3/ 利用以下方程式，计算 X 的上下限

$$UNPL = X\text{-bar} + (2.66 * mR \text{ 均值})$$

$$LNPL = X\text{-bar} - (2.66 * mR \text{ 均值})$$
 - 4/ 利用以下方程，计算移动极差的上限

$$URL = 3.27 * mR \text{ 均值}$$
 - 5/ 如果 X 或 mR 有超出范围，表示有过程变化的信号，过程不稳定
- Note:

UNPL=Upper Natural Process Limit

LNPL=Lower Natural Process Limit

URL=Upper Range Limit

例子:

系统测试缺陷密度	移动极差
0.14	0
0.14	0.01
0.13	0
0.13	0.01
0.14	0.01
0.15	0.01
0.14	0.01
0.13	0.03
0.16	0.04
0.12	0.01
0.11	0.01

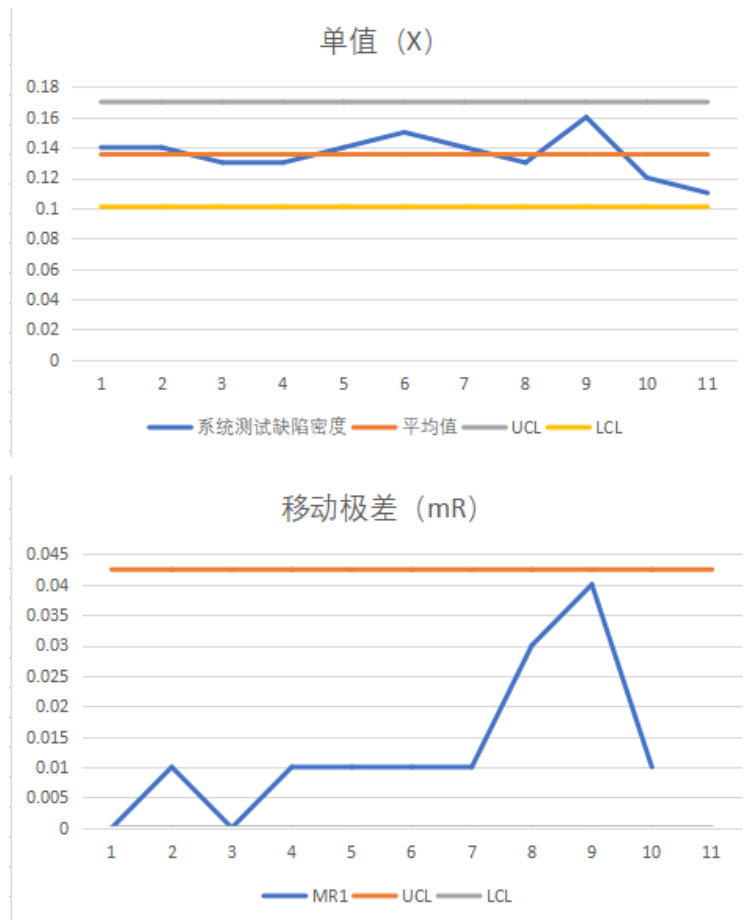
1/ 计算单值 (X) 平均数 $(\bar{X}) = 0.135 [(0.14+0.14+\dots+0.11)/11]$

2/ 计算移动极差 (mR moving Range) 均值如第一移动极差为 $0.14-0.14=0$; 第二移动极差为 $0.14-0.13=0.01$; 10 点的均值 $= 0.0128$

3/ 计算 X 的上下限: 用上面公式 $UCL=0.135+2.66*0.0128=0.17$, $LCL=0.135-2.66*0.0128=0.10$

4/ 计算移动极差的上限用上面公式 $UCL=3.27*0.0128=0.042$, $LCL=0$

5/ 下面 2 图已经加上上下限与平均线:



各种控制图

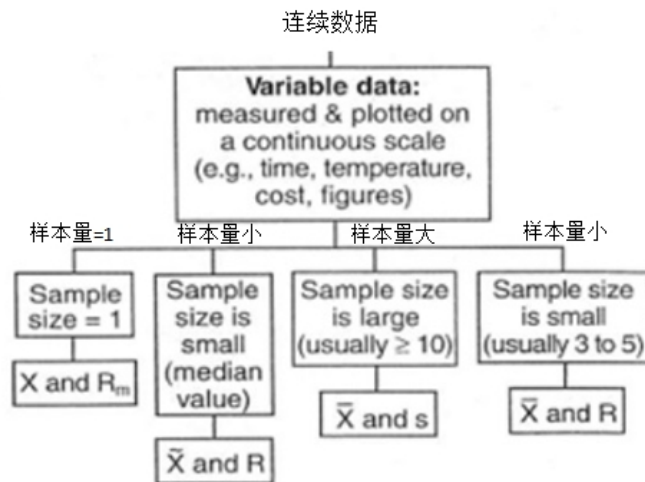
生产线用连续数据的控制图有两部分：

- 均值或中位数
- 标准差或者范围

两个图都要看。通常一些数据本身的价值有异常点的话，很可能它的标准差变化也会有异常点。

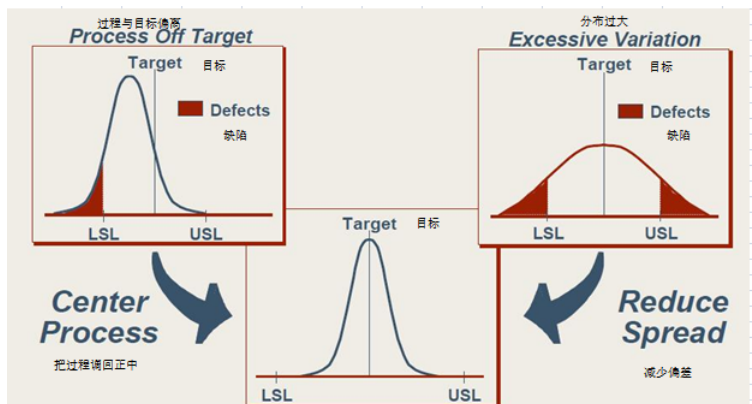
下面就是各种对连续数据的控制图方法，大部分都是按抽样样本的大小制定，比如抽样的样本数大于 10，我们就可以用平均值和标准差来做控制图。

基于样本多少选择恰当的控制图

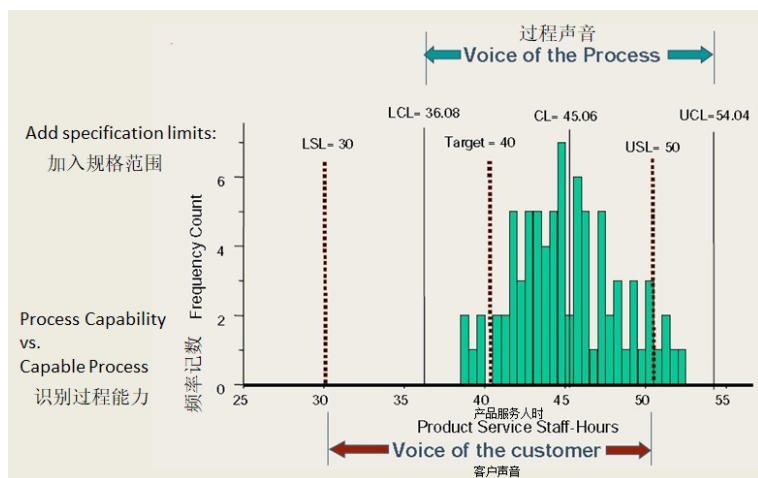


过程能力 (Process Capability)

过程能力反应能否满足客户要求:



中间的图就是客户的声音，USL 和 LSL 是客户要求的规格上下限，右上图就代表过程分布太宽，只有中间部分能满足客户规格要求，左图虽然过程变化范围满足，但偏离了，也不能完全满足客户要求。下图看到过程的范围是不能完全满足客户的规格范围要求:

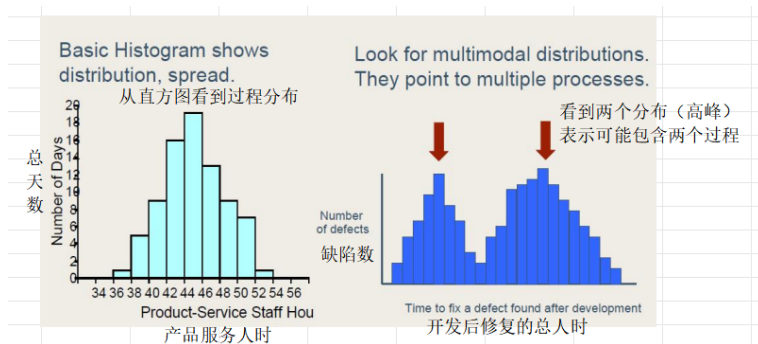


以下方程式可以计算过程能力指数。 $C_p = \frac{USL - LSL}{6\sigma}$ ，不考虑是否偏移，只考虑分布范围。

如果考虑偏移，计算 $C_{pk} = \min\left[\frac{USL - \mu}{3\sigma}, \frac{\mu - LSL}{3\sigma}\right]$ (选其中最少数)
指数越大表示过程越能满足客户规格（客户声音）。

细分例子

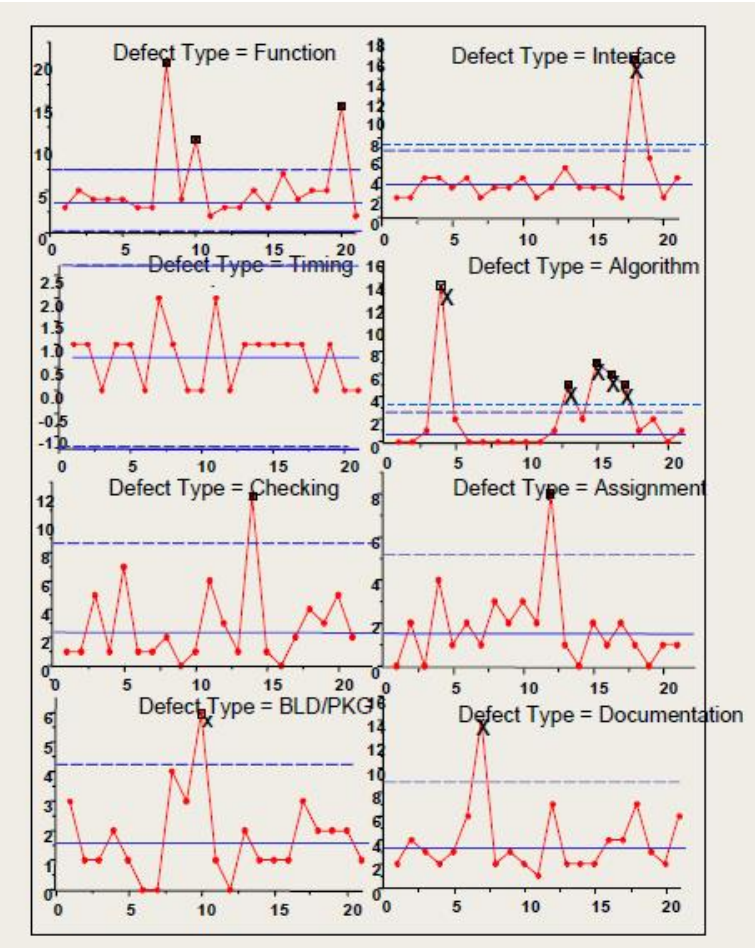
过程的分布柱状图也可以帮我们细分找出主要原因，比如下图右边这种柱状图表明这个分布可能有两个组成：



下表是关于客户的等待时间是多少天，可以看到 150 天以下和以上很可能是两个不同过程，所以我们分析的时候需要按这种细分，控制图也可以帮我们细分。

Component 组件	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	Totals 总数
Defects 缺陷数	12	16	18	32	22	16	23	35	15	27	16	25	20	26	20	23	23	36	22	27	17	271
Defect Type 类型	Number of Defects per Type per Component 缺陷数/类型 * 组件																					
Function 功能	3	5	4	4	4	3	3	20	4	11	2	3	3	5	3	7	4	5	5	15	2	115
Interface 接口	2	2	4	4	3	4	2	3	3	4	2	3	5	3	3	3	2	16	6	2	4	80
Timing 时序	1	1	0	1	1	0	2	1	0	0	2	0	1	1	1	1	1	0	1	0	0	15
Algorithm 算法	0	0	1	14	2	0	0	0	0	0	0	1	5	2	7	6	5	1	2	0	1	47
Checking 检验	1	1	5	1	7	1	1	2	0	1	6	3	1	12	1	0	2	4	3	5	2	59
Assignment 分派	0	2	0	4	1	2	1	3	2	3	2	8	1	0	2	1	2	1	0	1	1	37
Build/Pkg. 构建	3	1	1	2	1	0	0	4	3	6	1	0	2	1	1	1	3	2	2	2	1	37
Document 文档	2	4	3	2	3	6	14	2	3	2	1	7	2	2	2	4	4	7	3	2	6	81

比如上表，某产品有 21 个组件，统计了每个组件的缺陷数，如果只是看总缺陷数的分布，平均在 22.4，看不出有什么显著变化，没有异常点，但是如果按缺陷的 8 个缺陷种类，画成 8 个控制图（下图），就很明显看到有很多异常点：



参考 References

1. Wheeler, Donald. "Understanding Variation The Key to Managing Chaos." (2000)
2. Florac, William A. with A. Carleton. "Measuring the Software Process: Statistical Process Control for Software Process Improvement."(1999) SEI Series in Software Engineering

Part VIII

总结

前面的迭代根因分析只归属于问题解决，不算是创新，这部分会先回顾和分析个人和企业的创新案例，探索创新的要素。

个人的创新能力是基础，26 章北京手记让大家认识到为什么创新能力会因人而异。

最后 27 章会利用实例回顾人类从 20 世纪至今创新的根与翼，并探索软件开发团队能如何继承前人的根与翼持续改善。

Chapter 25

创新：从个人到公司

引言

创新，顾名思义，是指创造新的事物或提出与常规不同的见解。

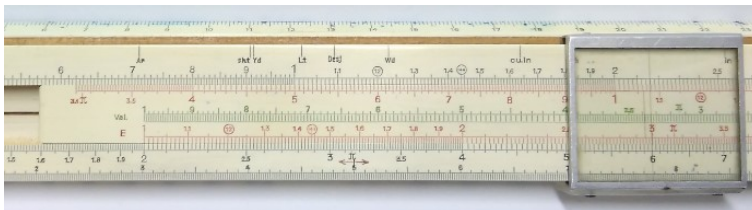
丰田汽车公司精益生产 (JIT) 的案例展示了丰田通过一线员工的智慧不断改善，最终实现了许多人最初认为不可思议的目标，如“零”缺陷和“零”库存。这使得丰田成为现代汽车生产的标杆。但这些改善都只属于解决问题 (Problem Solving)，而并非创新 (即使团队迭代回顾时针对缺陷排除率分析根因也不能算是创新)。

为什么有些公司和个人能够通过创新取得成功呢？接下来将首先回顾个人创新的要素，然后探索团队创新的方式和成功的关键要素。最后，从这些创新故事中探讨与敏捷开发概念的关系。

创新不仅需要独特的思维和洞察力，更重要的是能否有持久的毅力实现创新。在欧洲文艺复兴初期，一切都要从零开始，创新者必须克服重重困难才能成功。

400 年前的重大数学发明

如果你要求 345 乘 456 等于多少，你可以立即使用电子计算器甚至手机，一秒钟就可以得出准确答案。但在 70 年代之前（还没有电子计算器），是怎么计算呢？如果用手工计算，很费时间。那时候，许多读理科的学生都使用计算尺 (Slide Ruler)，通过移动计算尺上的滑动部分来求得答案。



对数是计算尺背后的原理。（如果不用计算尺，也可以查对数表得出答案。）

对数是 16 世纪纳皮尔先生 (John Napier 1550 – 1617) 用了超过 20 年时间“发明”的。

对数的简单例子：我们能在草稿纸上计算出“ $10,000 \times 100$ ”的结果，同时，我们也可以通过将“10,000”和“100”中的“0”的个数相加，得出答案为“1,000,000”。也就是说，把“10,000”看作是“10”的四次方，把“100”看作是“10”的平方，将四次方和平方的 4 加 2，即可得出答案。纳皮尔注意到了这一数字法则，总结出了对数的概念。

计算“ $10,000 \times 100$ ”的话，使用乘法会更快，但如果数字位数较大，需要手动计算时，使用加法运算会更加简单。纳皮尔先生按照这思路制作出对数表，简化了计算。

看到这里，也许有读者会想“这不就是指数运算的法则吗？”即是说，按照指数运算的法则“ $a^m \times a^n = a^{m+n}$ ”来思考的话，

$$10,000 \times 100 = 10^4 \times 10^2 = 10^{4+2} = 10^6 = 1,000,000$$

如此便可导出答案。然而，在纳皮尔时代并没有指数这种书写方式，指数的概念也不明确。纳皮尔的伟大之处也正在于此，他在没有指数这一概念的情况下发现了对数，并将其归纳为一个体系。

可以想象在 16 世纪的欧洲，数学研究还处于早期阶段，发现对数能够将原本复杂耗时的乘法或除法转化为加法或减法进行计算，这是一项非常不容易的成就。为了实现这个方法，他凭借个人努力，花了 20 年的时间手工计算出相关的对数表。现在我们还可以找到他当时算出的对数表，用现在的计算机验证，8 位数运算的头 7 位还是准确的。

纳皮尔先生的对数表：例如：第一行列出 $18^{\circ}30'$ 的正弦值为 0.3173047，对应数值为 0.111478920。

Gr. 18					
min	Sinus.	Logarithmi	Differentia	logarithmi	Sinus.
30	3173047	11478920	10948332	530594	9483237
31	3175805	11479237	10938660	531568	9482314
32	3178563	11479555	10929013	532543	9481390
33	3181321	11479873	10919364	533519	9480465
34	3184079	11480191	10909723	534496	9479539
35	3186837	11480509	10900090	535473	9478612
36	3189594	11480827	10890464	536451	9477685
37	3192351	11481145	10880845	537430	9476757
38	3195108	11481464	10871234	538410	9475828
39	3197864	11481782	10861630	539391	9474898
40	3200620	11482101	10852033	540373	9473967
41	3203375	11482420	10842444	541356	9473035
42	3206130	11482739	10832862	542340	9472103
43	3208885	11483058	10823287	543325	9471170
44	3211640	11483377	10813719	544311	9470236
45	3214395	11483696	10804158	545298	9469301
46	3217150	11484015	10794605	546286	9468366
47	3219904	11484334	10785059	547275	9467430
48	3222658	11484653	10775520	548265	9466493
49	3225412	11484972	10765988	549256	9465555
50	3228165	11485291	10756462	550249	9464616
51	3230918	11485610	10746944	551242	9463677
52	3233671	11485929	10737434	552236	9462737
53	3236423	11486248	10727931	553231	9461796
54	3239175	11486567	10718436	554226	9460854
55	3241927	11486886	10708948	555222	9459911
56	3244679	11487205	10699467	556219	9458968
57	3247430	11487524	10689993	557217	9458024
58	3250181	11487843	10680526	558216	9457079
59	3252932	11488162	10671056	559216	9456133
60	3255682	11488481	10661583	560217	9455186

当时为什么要计算这些复杂的正弦相乘呢？因为纳皮尔先生还研究球面三角学，可以利用他的纳皮尔公式探究角度与球面弧（距离）之间的关系。16 世纪的欧洲国家主要通过航海，发现新大陆进行发展。但海员在大洋中如何确定自己的位置呢？船员在大海中只能依赖天上的星星，利用六分仪估算出船只的经纬度。使用纳皮尔公式，他们就能够估算出地球球面点 A 和点 B 之间的弧长。

纳皮尔先生通过研究球面三角学，发现了角度与球面弧（距离）之间的关系公式，如下所示：

$$\cos \theta = \cos \varphi_A \cos \varphi_B \cos(\lambda_A - \lambda_B) + \sin \varphi_A \sin \varphi_B$$

海员只要知道两点（A 和 B）的经纬度（角度）便能使用以上纳皮尔公式，估算 A 和 B 点之间地球球面弧长。

例子-北京和巴黎距离：

A: 北京中心位于北纬 $39^{\circ}54'20''$ ，东经 $116^{\circ}25'29''$ 。转换为纬度 39.905556 ，经度 116.424722 。

B: 巴黎转换后纬度 48.8583 经度 2.29451 。

$$\begin{aligned}\cos \theta &= \cos \varphi_A \cos \varphi_B \cos(\lambda_A - \lambda_B) + \sin \varphi_A \sin \varphi_B \\ &= \cos 39.905556^{\circ} \times \cos 48.8583^{\circ} \times \cos(116.424722^{\circ} - 2.29451^{\circ}) \\ &\quad + \sin 39.905556^{\circ} \times \sin 48.8583^{\circ} \\ &= 0.767103 \times 0.657923 \times \\ &\quad (-0.40881) + 0.64152 \times 0.758084 \\ &= 0.28 \\ \theta &= 1.287 \\ (\text{rad}) \text{ AB 距离} &= \text{地球的半径 } 6,378 \text{ KM} \times 1.287 = 8,208 \text{ KM} \quad (\text{实际距离是 } 8,217 \text{ KM})\end{aligned}$$

当时没有电子计算机，他发明了对数来解决这类乘除难题（对数不仅仅用于航海，还有助于天文学的计算）。对数帮助我们解决复杂的计算问题长达 350 年，直到电子计算器出现才退出舞台。

创新 Creativity

并不是每个人都能像纳皮尔先生那样做出创新和发明，为人类做出巨大贡献。虽然他也是普通人，但又是什么驱动他花费 20 年的时间和精力来独创对数呢？

Robert Fritz 回忆：“60 年代，当我在 Boston Conservatory 学音乐作曲时，开始思考学的不应仅仅是对位、和音，更要了解音乐大师们的创新过程。”他毕业后继续做音乐创作，有一次参加创新专业人士聚会，这些专业人士中有作家、画家、音乐家和建筑师等，他发现这些人在本专业的创新能力都很强，但都没有想到用他们的创新能力去提升自己的生活。于是他举办培训课以帮助不同背景的人提升创新能力，让各行各业的人都可以学习什么是创作/创新。几年后，他把创作的重点写成了一本书（见 Reference 参考）。Robert Fritz 在他的书中，以贝多芬为例，说明创作并非 Problem Solving（解决问题），而是要有很高的目标/理想，然后不断地尝试比较，最终达到创作目的。

原创精神 Spirit of Creating

真想不通是什么原因驱动我能够在一个半小时内、在山西省图书馆一口气读完了《且听风吟》（日本著名小说家村上春树先生 1973 年的处女作品，共 144 页，共 40 章）。（快速读完的原因之一是：我没有图书卡，而图书馆会在 5 点钟准时关门。）

作者描述了自己亲身经历的场景——在酒吧与老朋友喝酒、听电台播放流行歌曲、购买唱片以及邀请女朋友吃牛排等情景。

虽然没有惊天动地的结局，也不是一气呵成的大作风格，相反，里面的场景很散乱。虽然出现了少女裸体、女朋友自杀和作家跳楼等情景，这些都不足以打动我。但总觉得这本书与众不同，很有特色，例如作者本身，以及书中的每一位角色都挺有性格，展现了独立的思维，都是活生生的“真”人。我不仅看到了他们的表面，也感受到了他们内心深处的世界。

创作并非被他人驱使，而是源自内心对创作的渴望，渴望创作出世界上最优秀的作品。只有认为自己的创作具有重要价值，才会尽全力努力创作。最终，希望自己的作品拥有独特的生命力，并受到他人的赞赏。

除了《且听风吟》，我还读过《挪威的森林》，也是采用第一人称叙述的手法，以作者渡边君的视角逐步展开各角色：例如他好朋友本月的女友直子，大学同学绿子，在疗养院照顾直子的玲子等。直子的优美、优雅、娴静，绿子的活泼烂漫，都是通过渡边的亲身经历逐步细化而来，就像使用显微镜观察一个细胞：起初由于焦距还没有对好，一片模糊，但随着焦距的调整，可以一步步看清细胞的每一个细节。每个故事的发展也时不时地出人意料，就像一部优秀的话剧，观众永远无法预料下一个桥段……这种情节的安排吸引着读者继续追读，让他们不愿停下来。（因为延续了《且听风吟》的写作风格，有些评论就认为《挪威的森林》运用了大量无意义的描写，这些描写空洞、无新意，仿佛只是为了拼凑字数与拖缓节奏而存在）

《海边的卡夫卡》于 2002 年出版。在这本书里，村上先生设立了卡夫卡和中田两条线，奇数章以卡夫卡的故事为主线，偶数章则以中田的故事为主线，两条主线齐头并进。起初两条线没有直接的交叉点，但后来它们产生了交集。像我这样的非文科读者可能会觉得两条独立的情节主线会很奇怪，但当它们开始产生交集时，便会感到耳目一新——从没有过类似的小说。

文艺创作，无论文学或音乐，不同于数学、科学，给作者很多原创的空间，贝多芬的创新推动了西方音乐从古典时期向浪漫时期的过渡。

贝多芬 (1770 -1827) 的音乐创新可以说是前无古人后无来者，深深影响着后面两个世纪的作曲家。

我们都熟悉他的《命运交响曲》，其实从 1795 年到 1827 年他去世前，他出版的每一首作品，不论是交响曲、室内乐还是歌剧，都堪称经典。他是完美主义者，例如《田园交响曲》的创作时间不少于 3 年，从他手稿得知，首乐章某主题他修改了不下 12 次，所以与巴赫、莫扎特、舒伯特比较，他的“生产率”不算高，平均每年出版 5 6 个作品。他出版《命运交响曲》、《田园交响曲》时 (1808)，已经出版音乐作品 13 年，包括 23 首钢琴奏鸣曲，9 首弦乐四重奏。贝多芬的早期作品风格与海顿和莫扎特还很相似，但到了中期，如《命运交响曲》和《田园交响曲》，就已经远远超越了他的前辈和同辈们，并启发音乐家们从古典主义逐渐发展出浪漫主义的新风格。

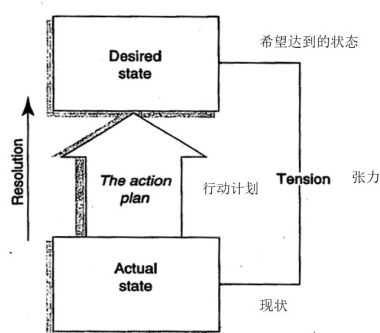
例如，在《第九交响曲》最后的第四乐章中引入了四声部合唱，这在传统的交响曲中也是前所未有的（传统交响曲通常只由乐队演奏）。

贝多芬晚期的作品，如钢琴曲和弦乐四重奏等，又展现出了与中前期不同的另一种风格。例如，他的作品 Op133 弦乐四重奏 *Große Fuge*，由于其思想深邃远超当时的水平，并未被当时的音乐家所接受，他们认为这部作品有问题，有的甚至批评他太老了，不仅聋了还疯了。但贝多芬仍然故我，表示：“这首曲子就是为未来而写的！”

很多人都听过贝多芬的钢琴奏鸣曲，例如《悲怆》、《月光》奏鸣曲等。我也觉得他的钢琴曲与莫扎特和他老师海顿不同，别具一格。

贝多芬对音乐创作的远景目标——希望创造前所未有的作品（他觉得当代的作品，甚至包括海顿和莫扎特的作品，离这远景还差很远）。由这崇高的目标驱动，加上他自身的音乐演奏和作曲能力，贝多芬不断突破，甚至连耳聋也阻挡不了他的创作力（他从 25 岁开始听力减退，到 45 岁已经完全失聪），将音乐创作推向了新的高度，对后来的作曲家产生了深远的影响。贝多芬追求完美，他有极大的魄力，通过持续不断地创新和完善，这才取得了超越常人的成就。

Fritz 发现大师们都拥有以下共同的特点：把握自己的命运，和主动追求个人目标。许多人很有才华，但为什么他们中的大多数却不能成为大师？他用下图解释。除了要对未来有远景目标外，了解现状同样重要。如果不觉得有差距（张力），就没有改进的驱动力。有些人有理想，但缺乏计划与行动，便只是梦想。



这个创新驱动模型不仅仅适用于音乐创作，也适用于数学。例如，纳皮尔先生为了解决正弦相乘的困难，发现可以转换成对数进行计算，把乘法简化为加法 (VISION)，为了实用，又花毕生精力计算出了对数表，也是从张力转化为行动 (ACTION) 并创新的例子。

从贝多芬和村上春树的故事中，我们了解到创新首先是由不满足于现状驱动、并由此长期不断完善的结果。如村上先生说：并非受人之托才写小说，而是因为“我想写小说”这种强烈愿望，深刻感受到这种内在动力，才不辞劳苦地努力写小说。

公司如何创新

您身边的人群中可能有超过 50% 的人都在使用这家公司的产品，涵盖各个年龄段。该公司的产品功能广泛，包括音乐播放、电话、电脑/平板、在线视频观看以及日常工作。您能猜到这家公司的名字吗？

苹果公司 (Apple)

我和全世界千千万万人一样每天都在使用苹果产品：iPhone、iPAD 平板电脑和 iTunes 网上搜索音乐歌曲，要了解苹果的成就，就不能不谈乔布斯 (Steve Jobs) 了。他的一生充满传奇，很多故事可通过阅读《Steve Jobs》一书进行了解。让我们先回顾苹果如何成为大众音乐市场巨人的故事。

==

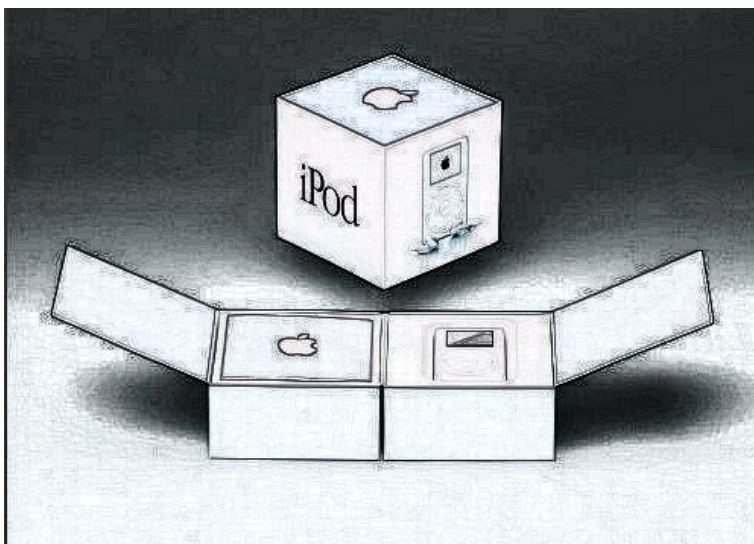
音乐播放器

自 80 年代末 MP3 格式问世以来，将歌曲数字化开始流行，市场上也出现了一些可以随身携带听音乐的小型设备。然而，由于技术限制，这些设备都存在不少问题。他们比较了当时市场上常见的播放器，发现或多或少存在各种问题，例如容量有限、难以按照个人喜好进行歌曲的排序等，如果想要从 CD 上下载歌曲到电脑，然后按照自己的喜好顺序在播放器上播放，过程相当复杂。乔布斯

本人很喜欢音乐，他看准了这个市场，并希望将苹果电脑定位为个人电子设备（包括音乐播放器）的综合中心（Hub）。首先要解决的是音乐播放器的便携性和存储容量。当时，小型显示器、锂电池和小型硬盘都尚未普及。几个月后，他们找到了合适的解决方案：东芝刚刚研发出一款直径只有一英寸的超小型 5G 硬盘，形状像个大银币。但如果将这些功能都集成到播放器中去实现，因为播放器的屏幕尺寸太小了，实现起来非常困难。他一直非常关注产品的易用性，当乔布斯看完内部软件研发工程师首次展示正在开发中的刻录软件后，他站到白板前画了一个框，说：“你们设计的软件就应该只需把歌曲拉到那个框里面，然后点击一个按钮就可以刻录。”当时软件工程师被吓呆了，觉得很难做到。这反映工程师往往不会从用户的角度来看问题，乔布斯后来要求播放器软件都必须给不懂技术的人使用，让她在 3 个步骤内，不需要查看说明书就能完成任务，这成为他的测试标准，如果做不到，他就不会接受这软件。播放器的设计也有创新，比如通过转动面板上的轮子来选择歌曲，用户只需转动轮子就能方便地跳到要听的歌曲，这样的设计极为便捷。iPod 的产品设计也体现了苹果公司一贯的设计理念——简洁。产品采用白色设计，包括耳机在内都是白色（当时，大部分耳机还是黑色的），甚至连充电器也是白色的。设计师 Jonny Ive 解释：“白色能够赋予产品高贵感，使其与其他廉价的播放器有明显区别，用户会觉得这样的设计有艺术感。”



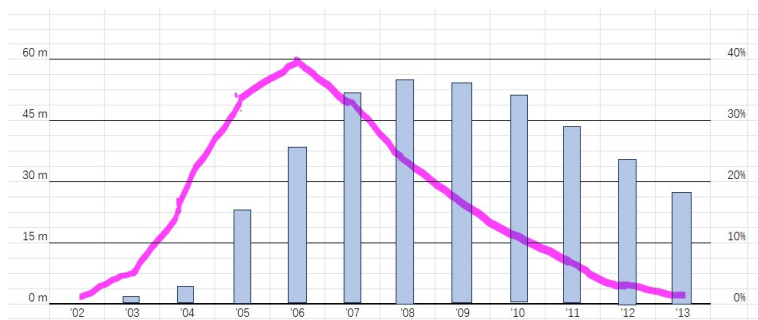
乔布斯也非常注重产品包装，iPod 的包装让你觉得：“在拆包装时，觉得有一件很贵重的东西在里面”。



虽然定价不低：399 美元，很多人觉得定价过高，但是 iPod 在 2002 年正式推出市场后一直热卖。

== == ==

从 2001 年到 2014 年苹果大概卖出了 4 亿件 iPod，2015 年后还继续卖出 50,000,000 件。iPod 播放器也成为了苹果的主要收入来源。2006 年，也就是苹果推出 iPhone 的前一年，iPod 的销售收入占了公司总收入的 40%。下图展示了从 2002 到 2013 年，iPod 的销售数量变化和收入占比变化：



iTunes 网上商店

苹果的 iPod 加上 Mac 电脑的 iTunes 应用软件很成功，但这只解决了从网上下载音乐并在便携播放器听歌的技术问题，还有更严重的问题需要解决——盗版，2000 年开始，越来越多网站提供免费音乐下载，这不单影响了唱片销售（2002 年销售额下降了 9

因此，他基于 iPod 和 iTunes 音乐播放能与电脑结合的特点，向各大唱片公司展示了在线平台如何运作、如何防止盗版。由于唱片公司缺乏相关的技术，很

快就有 4 家唱片公司表示有兴趣，愿意合作：利用 iPod、iTunes 等技术，筹建统一在线平台服务，使消费者可以在网上购买并下载音乐。

从 2002 年开始，乔布斯一直忙于周旋各类问题：除了技术方面格式问题要解决（例如各有自己的格式，需要统一），也有些唱片公司想保护自身利益（例如索尼，自身实力雄厚，有意建立自己的音乐平台）。

最终在 2003 年 4 月，iTunes store 正式推出市场，用户可以在网上用关键字搜索任何一家唱片公司的唱片或歌曲，也可以搜索相关的唱片。乔布斯的业务模式很简单，他从以前的经验知道首先要通过低价吸引大量用户使用，如果收费太高，只是小部分人使用，就流行不起来，也没有动力吸引新唱片公司加入，恶性循环。他的策略很成功。

2003 年苹果刚开始发布 iTunes 网上商店时有 200,000 歌曲可以下载购买。他本来预计可能需要 6 个月的时间才能售出 100 万首歌曲。但实际上推出 6 天就已经卖了 100 万，到 2003 年 12 月，已经累计卖出 2500 万首歌曲。

2004 年 7 月份共卖出一亿首歌曲（2003 年底的 4 倍）；2005 年 11 月（两年半后）苹果成为全美国十大音乐零售商之一，其他都是传统零售商，例如第一是 Walmart。

随着网上买歌商业模式越来越普遍，传统零售商越来越困难，到了 2010 年 2 月，苹果成为美国最大的音乐零售商。

因为网络技术发达，现在不再是当年的网上下载，但很多人还是喜欢只须要每个月交几十块钱就可以随时随地在网上搜索并听到自己心爱的歌曲，这商业模式也保护了艺术家、唱片公司的利益。

从以上乔布斯关于音乐的故事，我们可以看到与 400 年前相比，创新的速度发生了巨大的变化。在过去，像纳皮尔那样仅凭一个人独立完成创新可能需要 20 年的时间。到了 2000 年，一切都变得非常迅速，乔布斯的团队只用 3 个月就推出了 iPod 播放器，并在两年内创建了 iTunes 网上音乐平台，帮助苹果公司为个人客户提供数字中心（Digital Hub）平台服务。客户可以使用电脑选择喜爱的歌曲在 iPod 播放，也可以下载最新的歌曲，非常方便。这些就是我们 2000 年代看到的创新，这些创新不可能靠一个人独自完成，而是要团队合作。

公司创新与几百年前那些大师的创新最大的区别：需要整个公司所有人的协作，并确保他们的目标一致。这意味着整个组织的各个部门都必须协调合作，同时又要细分目标，以便不同的人负责执行不同的任务。

纳皮尔先生花了 20 年时间写了 3 本数学书，总页数不到 200 页（其中一本还是在他去世后出版的），贝多芬的产量也不算高，一年只出版几首作品。这些大师都是靠个人的努力完成整个创作过程。但是现代市场变化越来越快，要成功，公司必须快人一步，但因依赖团队合作（各有专长），也需要协调大家的工作，确保各个层次的目标与公司总目标一致。下面介绍一个简单的系统让企业管理整个公司的提升：

1. 识别公司的总目标。

例如在下一年，要一年内生产 7 个新的产品系统。

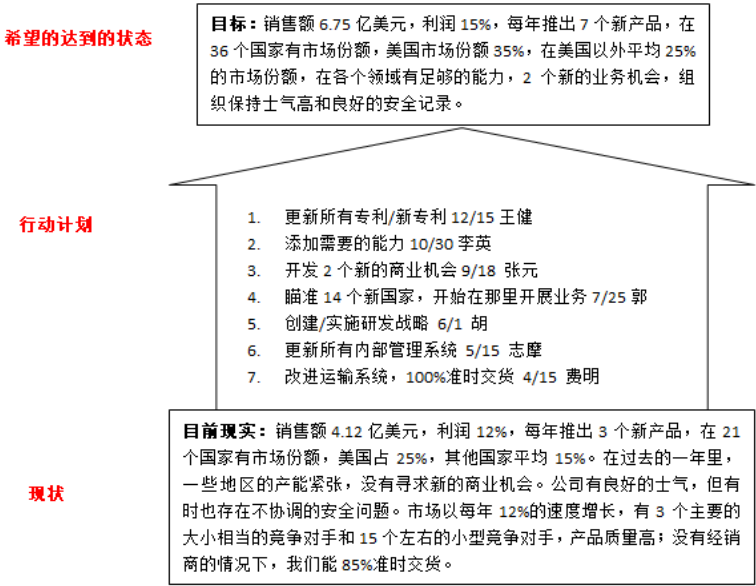
2. 描述公司的现状。

例如现状是：平均一年只能产出 3 个新产品，有 2 个开发团队。但开发人员的能力有限，质量也不好，导致用了不少精力，去维护已经完成的产品，没有时间来开发新的，招新人也不容易，比较难找得合适的，也没有时间培养新人，扩大工作。

3. 形成具体行动计划。

例如简洁、精简的开发流程，增加 1 个开发团队，加强代码评审和编码能力，来减少缺陷导致的返工，要跟市场部密切合作，确保新产品满足客户的要求，给客户带来价值。但真正公司的从上而下总计划会更具体。

4. 当我们定好明确的最终结果，也了解真正的现状，有一些行动计划，下一步是具体到每活动的完成日期和负责人。



同意可以用以上模式帮助公司创新吗？先回顾村上先生的 40 年创作经历：

村上春树先生《我的职业是小说家》

(2017 年出版, 他当时 68 岁, 从 73 年出版《且听风吟》开始, 他已经不停写书超过 40 年)

写长篇小说是我的“主业”, 我规定自己每天写 10 页稿纸, 每页 400 字, 形成规律。1987 年出版的《挪威的森林》, 全部的书稿是我在欧洲旅行期间写的, 因为不熟悉当地的环境, 每天都要费一番功夫才找到一个咖啡厅写稿, 但我依然保持每天 10 页的写作量。然后我进行了多轮修改。修改十分重要, 例如我在欧洲创作《舞! 舞! 舞!》(1988 年出版), 开始使用富士通便携式计算机写稿) 时, 临发稿时才突然发现有一篇我觉得写得挺好的章节找不到了, 只好按照原来的思路重写。书出版后, 又突然在电脑里发现了那篇章节, 原来是放在一个很少用的文件夹里。但我发现出版的版本远远比旧版好, 所以不断修改是非常必要的。写小说需要灵感, 因此必须阅读大量作品, 不仅仅只是名家的作品。我的小说主要描述各种人物, 所以我会细心观察他们的特征, 并将其展现在小说中。就像风景油画家必须仔细观察周围景色的颜色和光线一样。1979 年, 我的第一本小说《且听风吟》得了新人奖时, 我的高中老同学来到我开的小店说: “你那种玩儿都行的话, 我也能写出来。”他说完就走了。真奇怪, 虽然当时我很气愤, 但也自己想: “说不定真像那个家伙说的, 那种水平的玩意儿, 只怕谁都能写出来。”我仅仅是运用简单的文字, 把浮上脑际的东西记录下来。艰深的词语、考究的表达、流利的文体一样也没用到, 说来几乎无异于“空洞无物”。我从那“空洞无物”、简单的文体开始, 写出一部部作品, 注入血肉, 将规模更大更加复杂的故事塞进其中。从得新人奖的第一部小说起, 到现在已经过了 40 年, 我一直都在从事这项工作。到事后回顾才发现一直只是一步一步逐渐完善, 并不是从一开始就计划周全、按部就班完成的。

个人的创新已经不可以预先规划, 公司面对的外部与内部变数更多, 更无法预先策划。我们看看乔布斯另外两个公司的创新故事。

万事起头难

1985 年 9 月, 乔布斯被迫离开苹果公司后, 自己投资 700 万美元, 创立了一家专门针对高校工作站市场的新公司, 名为 NeXT。乔布斯开始时对公司的期望特别高, 当时他看到这市场主要被太阳 (SUN) 和 (DEC) 公司占有, 但这些工作站产品都是以输入命令为主, 与 Macintosh 用鼠标操作不同, 所以易用性不高, 所以他希望创造前所未有的新平台, 让那些高校的学者与研究人员可以简单地使用电脑模拟各种场景来做研究。

但是要达到这么高的理想, 要做的工作就非常多。比如第一次公司开创 90 天后就开始有一次度假村的头脑风暴会议, 希望可以在两年后推出产品, 以赶上学校放暑假购买电脑的时机。三个月后再次讨论时, 大家都意识到这个目标太高, 难以实现。但乔布斯没有放弃, 他一直用自己的魅力说我们可以按大家的专长达到这个目标。1986 年过去, 1987 年又过去了, 但新产品研发仍然没有显著进展。直到 1988 年, 公司才进行了第一次长达 3 个小时的产品演示, 但产品仍然处于初级阶段, 没有彩色显示, 并且速度较慢。尽管签下了曾为苹果推销 Mac 的全国电脑零售商, 但一年仅售出 360 台, 与预期相差甚远。(在 1984 年, 同

一家全国零售商却为苹果售出了 40 万台 Mac；而 NeXT 工厂的生产线设计标准是每月生产 1000 台产品。)

1990 年，到公司的第二次产品发布时，新产品开始用了彩色显示，速度也得到了提升。然而，尽管有所改进，该产品仍然被认为只是初级产品，尚未完全成熟。由于销售情况不理想，乔布斯再次投入了 500 万美元的资金，但很快就用光了。幸运的是，有一位外部投资者——德州企业家 Ross Perot 先生，他对乔布斯非常信任（他觉得以前没有来得及投资苹果，一直耿耿于怀），因此投资了 2000 万美元进入 NeXT。1993 年，公司开始裁员。600 人的公司，起初只裁减了几十人，随后又裁掉了 200 多人，接近公司总人数的一半，随后又有 200 多人被裁，最终公司将整个硬件部门出售给了日本佳能公司，只保留了操作系统部分 NextStep。因为苹果公司看中了该操作系统，NeXT 公司于 1996 年以 4300 万美元的价格被苹果收购，乔布斯也得以重返苹果公司（最初以顾问身份）。又经过了 15 年的努力，乔布斯带领苹果公司再度走向辉煌。

从濒临破产变为回报最好的投资

乔布斯在 1986 年用一千万美元投资 Pixar。当时，Pixar 只是 LucasFilm 旗下的一个由 30 多人组成的电脑部，由《星球大战》导演 George Lucas 创立，主要用于电影动漫效果的制作。乔布斯对美术领域一直很感兴趣，他通过 Alan Kay 的介绍去参观了 Pixar，被公司的产品深深吸引，认为其技术领先并远超同行。乔布斯最初只是一名投资者，并没有过多干预公司的软硬件产品开发，但乔布斯雄心勃勃，觉得公司开发的产品有特长，也希望能像苹果的 Mac 一样，大量供给动漫专业人员，进行 3D 设计。尽管产品的功能强大，但使用起来仍需要复杂的命令操作，不太适合一般的动漫从业者（Adobe 推出的软件，虽然功能不如 Pixar 强大，但更易于上手）。因此，乔布斯投入了更多研发资源，改善产品的易用性。为了推广 Pixar 产品，他开设了很多零售商店。虽然乔布斯对 Pixar 公司的投资不断增加，然而产品的销售情况一直未见起色，公司陷入了资金短缺的境地。到了 1988 年，他被迫采取了紧缩开支和裁员的措施。直到 1991 年，与迪士尼合作制作 Toy Story《玩具总动员》，Pixar 才得以起死回生。乔布斯在这之前已经投入了高达 5 千万美元的资金，相当于他从苹果公司股票出售所得收入的一半。因为与迪士尼的成功合作，Pixar 公司得以在 1995 年（与 Toy Story 首次公演同年）成功上市，获得了 15 亿美元投资。随后，Pixar 与迪士尼继续合作，推出了 5 部动漫电影，都非常卖座。2006 年，迪士尼为了防止 Pixar 与其他公司合作，以 74 亿美元的价格收购了 Pixar，也使乔布斯成为迪士尼的最大独立股东。（关于 Pixar Toy Story 制作故事，请看附件）

后来的事实证明，乔布斯认为一般动漫从业者会喜欢使用 Pixar 的 3D 建模软件是错误的，这只是他的一厢情愿。但他的另一个希望：如何结合电脑数字技术和艺术做创新，确保梦想成真，Pixar 与迪士尼合作制作新一代动漫片，从 1995 年的 Toy Story 开始，把动画片技术提升一个高度。

从以上两个案例可以看出，任何企业的创新往往会面临难以预料的挑战。因为市场的不确定性，超过 90

公司创新成功要素

回顾 iTunes 电子商店和之前 iPod 成功的故事，以及乔布斯与 NeXT 和 Pixar 的故事，可以总结出以下成功要素和注意事项：

灵活、敏捷

从达尔文物种进化历史可以看到物种可持续的原则：

- 最强壮/巨大的物种也会灭绝（如猛犸象）
- 只有最能快速适应环境变化的物种才能长期传承下去

从 60 年代开始，商业环境快速变化，也只有能快速适应环境的公司，不断求变，才能持久，成为百年老店。

从乔布斯的 NeXT 和 Pixar 故事可以看出，创建新产品或服务涉及诸多不确定因素，失败的可能性很大。因此必须随时调整策略以适应变化。如果发现最初的想法或计划没有产生预期的效果，就应立即做出改变。

这个原则适用于许多科技公司，比如谷歌，它也有很多失败的项目。谷歌的宗旨是“Fail Fast”，意思是如果一个项目在半年到九个月内未能达到一定的客户量，就会立即终止该项目。许多科技公司通常在年底才会全面评估某项业务是否可持续，但为时已晚。

这也是敏捷开发的原则之一：不将精力集中在长远的规划上。由于需求经常变化，敏捷开发将项目分解成小迭代，每个迭代都是一个冲刺，以有限的资源尽力做好每个迭代：这不仅提升了效率，而且产品质量也会更好。

专注 (Focus)

资源有限，公司应该专注于自身能够做好的事情，而将那些可以外包的任务尽量外包。举例来说，对于生产方面，苹果公司专注于发挥自身的专长——产品设计。（请不要误解苹果公司对生产的态度，它的产品生产都是按照 JIT 原则组织的，只是不直接雇佣生产工人。）

乔布斯回到苹果后，吸取了过去十年在 NeXT 和 Pixar 的经验教训。当时苹果公司内部存在着一百多个研发项目，乔布斯深知资源的有限性，因此开始对这些项目进行整合。他只保留了在市场上表现最优秀的项目，解散了那些不必要的研发团队，只保留与公司发展愿景相关的研发项目。

乔布斯 2010 年被访问时说：“我们选技术，只挑选有发展潜力的技术，如果什么都支持、都研究就会耗费宝贵资源。”

A 级团队成员

Pixar 公司虽然小，但每位都是行业精英。

我在美国的堂弟从小就对动画人物情有独钟，家中也都是星际旅行、星球大战等的模型。他来香港旅游时还特意光顾那些专门销售他喜欢的卡通人物模型的小店。他（读建筑专业）毕业后专攻动画片和主题乐园设计。他和其他 3 位设计师是公司外聘参加《超人总动员》制作设计的原始核心团队。其他 3 位都是行业中的有名高手，例如，Blackman 曾设计蝙蝠侠系列，Delgado 设计星际旅行 (Star Trek) 等。

他说“Pixar 公司在动画片行业非常出名。核心制作团队, 包括艺术总监和生产设计师等都是当时行业的顶尖高手，他们除了能力超强，对艺术创作的品味与兴趣跟我也很相似，能有这机会与他们共事合作真是非常幸运，像中了彩票一样。开始时，我们四位设计主要成员定期要跟导演 Brad Bird 开会，评审设计。”《超人总动员》非常卖座，美国本土票房收入已经超过 2 亿美元，也得了奥斯卡奖。

Pixar 公司这些动画片，能有这么好的票房，并得奖，不是没有原因的，因为公司有名气，可以吸引到当时最优秀的艺术家参加，有了这些人，我们才能看到像《超人总动员》这档次的高端制作。

因此，当 A 级成员遇上能力相当的团队伙伴时，能够激发更大的创造力，产生 $1+1>2$ 的效果。

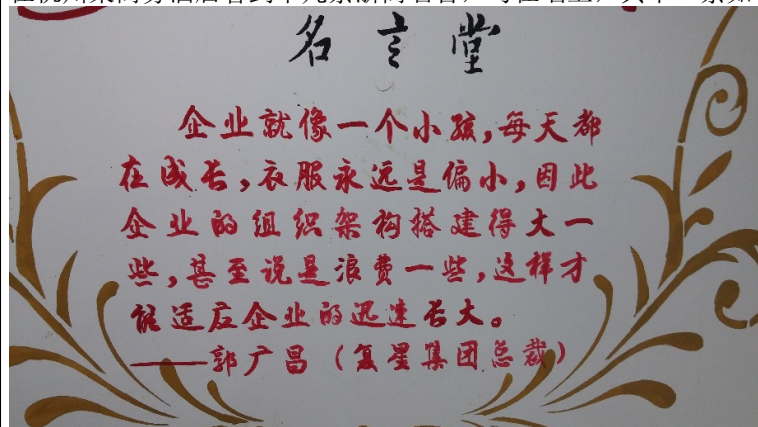
苹果公司于 1984 年推出的 Macintosh，产品在技术能力和设计方面（如视窗、鼠标等），远远领先于之前 IBM 推出的 IBM PC。该产品的开发团队成员也都能力超群：其中，Burrell Smith 负责硬件线路板设计，Andy Hertzfeld 等人负责 Mac GUI 操作系统的设计，Bill Atkinson 负责 Graphics 应用软件 QuickDraw 和 HyperCard 的设计，Joanna Hoffman 负责市场推广，Bruce Horn 负责 Resource Manager 应用软件的设计等。然而产品推出后，因销售情况并不理想，管理层也发生了变动，导致团队成员陆续离开。

乔布斯从 Pixar 的经验，深明高端球员 (A Players) 只愿意与高端球员搭档的道理。但为了吸引和留住这些精英，公司必须给有吸引力的报酬。

例如，乔回苹果不久就特别要求董事会修订员工股权安排（因当时苹果股价已经很低，之前的员工股权安排已经没有意义。）

极高目标，要做大做强

在杭州某商务酒店看到十几条浙商名言，写在墙上，其中一条如下：



乔布斯创立 NeXT 时是世界第一的工作站供应商，所以生产线是按每月能生产一千台设计！他重返苹果后，一直秉持着以创造世界第一的产品为目标，这在后来被证明是成功的，例如 iPod/iTunes(2001)、iPhone(2007)、iPad(2010) 等。这种思路使得苹果公司成为了美国市值最大的科技公司。

正如丰田的大野耐一所说：“容易达成的目标并不是好目标”，他给丰田员工设定的质量目标是零缺陷。

有非常高的要求，用心做

每位团队成员都应该追求卓越，不仅仅是完成任务，而是要创造出优秀的、世界一流的产品。只有这种抱负才能激发每个人付出最大的努力，把事情做好。乔布斯本人就是一个典型的例子，他判断有价值事就会全力以赴去做到最好。举例来说，在推出 iTunes 电子商店之前，他就不断地邀请各种音乐家、歌手和音乐制作人参与其中。他每次都会想尽办法去介绍他的好东西，有一次小号手 W. Marsalis 到家拜访，乔布斯就问他：“你喜欢谁的作品？”

Marsalis 说：“贝多芬”，他就一起展示如何在商店里面可以容易搜索到贝多芬的作品，Marsalis 后面回顾说：“其实我一直都对电脑不太感兴趣，但乔布斯他一直全心全意，全程用了两个小时很细心地给我介绍。过了一会，我的注意力不是放在他的产品或者电脑上，而是他本人，我深深地被他的激情和专注打动了。”

例如苹果的首席设计师 Jony Ive 用简洁设计思路，让消费者觉得苹果产品有艺术感，非一般电脑：



Ive 的父亲是位优秀工匠，从父亲那他学到要做一流工匠必须对质量有极高要求。

领导决策、快速行动

索尼一直以其在电子产品领域的创新而闻名，例如 Walkman，同时也拥有唱片公司；创建网上音乐平台的主要条件它都具备了，但为何索尼做不到，苹果却成功了呢？其中最大的原因在于索尼是一家庞大的企业，由各个事业部门组成，每个事业部门都要追求自身的盈利。正是这个原因，当需要各团队之间进行紧密协作时，许多精力就被耗费在部门之间的协调上，最终以失败告终。相反苹果公司不是这样的，它只关注整个公司的总体利益，而且乔布斯对产品有着非常高的要求。例如，在设计 iPod 的时候，工程部和软件开发部门完成了 30 个原型初稿后，乔布斯参与讨论后立即确定了设计方案，随后就立即着手实施了。（有些在其他大公司，比如飞利浦，工作过的员工会觉得很奇怪，因为像西门子、飞利浦那种传统大公司绝对不会在一个会议里做这么重大的设计决策。）

结束语

苹果的案例可以被视为将敏捷开发升级至企业级的最佳实践案例，从前面的故事中可以看出，整个公司都在按照精益的思路不断改善。

团队合作至关重要，而团队成员的能力要求也很高，这样才能在最短的时间内做到最好。强调岗位之间的相互合作，不是将工作分得很细，比如你负责测试，我负责开发，各自只需要做好自己的工作，而是要求团队成员相互协作，最终确保整个项目或产品的成功。不应计较是否在自己的工作范围之内，应该确保成员之间相互有效沟通，即使在需求或开发方面是最优秀的，但并不代表产品能做到最优。有些开发团队会抱怨说：“其实我们的开发工作做得挺不错的，但是由于需求混乱，我们无法做到最好。”

但是因为公司架构原因，部门之间各有自己的指标，导致各功能之间不能相互合作做好。

从乔布斯的故事中了解到，高层对产品质量有极高的要求这一点至关重要，敏

捷开发也同样。如果敏捷团队不注重产品质量，只关注按时交付，最后只能生产出一些不达标的次等产品。但反过来，当团队具备足够的能力，并且大家都有抱负、有追求，敏捷开发可以帮助团队快速响应不断变化的需求，就像苹果和乔布斯一样，成为世界第一。

1983—84 年，乔布斯（28 岁）在苹果公司带领 Macintosh 开发团队时，就不会向预计交付期限低头。如果他觉得产品未达到质量要求，他宁愿延后。他甚至鼓励团员要精益求精。（千万不要以为你在他管理的团队可以偷懒）

有人问我：“听说过 SAFe 这个框架吗？我们想用它在公司级别推行敏捷，你觉得怎么样？”

我：框架就只是框架，如果没有刚才提到的那些要素，任何框架都无济于事。因为框架只是把一些最佳实践的要素列出来，提供了一些过程和度量标准，但如果没有合适的人才和领导，没有苹果公司这样的企业文化，就永远无法打造出世界一流的产品。这个道理无论在国内还是国外都同样适用。

= = =

2005 年，乔布斯获颁史丹福大学荣誉学位，他在典礼上跟毕业生分享了三个故事：

故事一：

高中毕业后，我 17 岁进了学费昂贵的私立里德大学，但我只读了半年，一方面觉得学非所用，另一方面也不忍心花掉父母一辈子的积蓄，就选择了退学。但我并没有离开学校，而继续在学校里旁听感兴趣的课。我没有收入，就睡在同学宿舍地板上，同时靠捡玻璃瓶、可乐罐挣点小钱糊口。因平日吃不饱，每个星期天，我走路到距离七英里的一所印度寺庙去吃一顿施舍饭。

大学的美术字（Calligraphy）课程很有名，我去旁听后就立即迷上了。尽管当时我还不知道它将来会有什么用，但是在设计苹果的麦金托什（Macintosh）计算机时，我想起了当年在大学里旁听的美术字课程，并为 Macintosh 个人电脑设计了很漂亮的字体。

十年后回首，这些生命中的小点都串联在一起了，尽管在之前我无法预见到它们会串联起来，只是在事后回顾时才能看到。因此我们必须相信这些小点会在生命中的某个时刻连接起来，才有信心根据自己的信念勇往直前。

故事二：

在我 30 岁的时候被苹果公司赶了出来，当时感到非常失败，没有面子，也对不起其他的创业者。然而由于我对电脑和 IT 仍然充满兴趣，我决定自己创业，继续做产品研究，专注于在工作站方面开发世界一流的产品。回顾过去，如果我当时没有离开苹果，那么我可能就无法体验后来我做产品开发的许多概念。现在回想起来，我一点也不后悔。最重要的是，你必须对你所做的事情充满热情并爱上它，无论是工作还是寻找伴侣，这一点都是相同的。

“有些时候，生活会拿起一块砖头向你的脑袋上猛拍一下，不要因此失去信仰。我很清楚，支撑我一路走下去的，是那些我所爱的东西。你需要找到你的所爱，工作如此，爱人也是如此。你的工作将会占据生活中很大一部分。你要相信这份工作是伟大的，你必须先热爱它；你只有坚信自己所做的是份伟大的工作，才

能怡然自得。如果你现在还没有找到，那么继续寻找，不要停下。只要全心全意地去寻找，在你找到的时候，你的心会告诉你的。”

故事三：

我 17 岁时，听说应把每天当成你生命最后一天。所以我每天都会对着镜子问自己，回顾是否已经用好当天，做最重要的事情，如果连续几天都否定，就必须马上调整。现在回想起来，如果我能坚持这种想法，就可以不断地激励自己只做真正重要的事情。每隔几天，如果我发现时间被浪费在了不是最重要的事情上，就会立即进行调整。

“死亡是每个人必然的终点，没有人可以逃脱。死亡是生命中最好的发明，起新陈代谢作用。记住，每个人都会在不久后死去，这对我产生了重要的影响：当我做重要决策时，所有的顾虑，如其他人的期待，对失败的恐惧以及没有面子等，相对于死亡来说就都变得微不足道了。当你意识到生命有限时，就再也没有理由不去听从内心的指引，专注于做好最重要的事情，因为其他的事情都不再重要了。”

个人经验教训

乔布斯先生在大学毕业典礼上分享他的三个故事，回看我自己过去的人生旅程也有同类故事：

故事一：我预科考大学的成绩非常好，除了语文以外，物理、化学、数学都全 A，进香港大学选读什么系都没有问题。我父亲自己没有读过大学，基于传统观念极力劝我读医科（我祖父母的年代，香港大学可选择课目不多，学费也高，不是一般普通家庭可负担，所以父母都想子女读医，毕业后收入有保障，社会地位也高）。我自己一直对生物和化学的兴趣不大，反而对数学从小就很有兴趣，我本来一直想读麻省理工的计算机专业，但当时香港大学还没有计算机系，所以最终选择了电子电机工程。尽管父亲极力反对，甚至威胁说如果不选读医科就跳海，但我经过深思熟虑，最终还是坚持了自己的选择。因为香港工业的不发达，对科技也不重视，电子工程毕业后在香港难以找到好的发展机会。因此读书时和毕业后的十多年里，我会时常怀疑，如果当年听从父亲的建议选择医科的话，也许会有更好的发展。

现在回顾当年的选择仍然是非常明智的，如果选择了医科，我就没有机会在 5 年前重新学习线性代数、统计分析、大数据等我非常感兴趣的课程，也无法在过去 10 多年里在内地到处跑，长期出差。

1979 年，我在香港读预科时首次接触到《矩阵代数 (Matrix Algebra)》，当时觉得非常新奇，但并不知道它有什么应用。（但牛顿力学就实际多了，例如可以用来计算台球碰撞后的轨迹和速度也可以用来计算行星和月球的轨迹。）

1998 年在香港富士通工作时，开始接触到软件工程，对很多概念不太了解，但却非常感兴趣，于是兼读了软件工程硕士，开始学习敏捷开发、面向对象编程、软件的易用性、软件质量等。大学毕业后，在晚上兼读管理文凭时，我第一次接触到了 XY 理论，2010 年读了 MIT（麻省理工学院）教授

McGregor 的经典著作《The Human Side of Enterprise》，又开始接触和组织开发领域的各种实验和研究，还在看心理学的公开课视频。

2017 年开始接触大数据分析，发现要弄清楚大数据背后的原理，就必须了解线性代数（又称矩阵代数），于是开始在网上搜索相关视频。2018 年，在麻省理工学院公开课 (MIT OCW) 的视频中找到由数学教授 Prof. Strang 主讲的线性代数课程，共有 34 个讲座。我非常欣赏 Prof. Strang 的授课风格，他用粉笔在黑板上书写，结合讲解，所以我一有空就看视频。

我一直保存着读预科时的《矩阵代数》，作者也是 Prof. Strang。发现这本参考书依然非常实用，书中的内容完全不需要更新，因为数学与工程或技术不同，原理是不会改变的。除了再读《矩阵代数》这本书，我还购买了 Prof. Strang 为这门课程新出的书，经过几个月的学习，我看完了接近 30 个讲座。现在回想起来，以上每一点对我的咨询培训工作和写这本书都有很大的帮助。开始时只是因为感兴趣去研究，事后回顾时才能看到每一点的贡献和重要性。

==

故事二：我曾经在 40 岁那年被公司开除，当时担任富士通的业务经理。开始时很迷茫，觉得自己很沮丧——从刚毕业时的大学精英，一下变成没有工作的失业汉。现在回看，如果当时不是在 40 岁从零开始，估计会现在我会和其他同学一样，50 多岁就被迫退休，也不可能再找到工作。

==

故事三：大约 10 年前，我观看了话剧《最后十四堂星期二的课》，也阅读了原著《Tuesdays with MORRIE》。在与家人去爱尔兰度假时，开始思考生命的有限性，意识到人无法掌控命运的安排，随时都可能面临死亡，因此必须抓紧时间，尽量把想写的东西记下来。（Covey 先生经典作品《Seven Habits》中的习惯二 (Begin with the End in Mind, Principles of Personal Leadership) 场景：你参加自己的葬礼，看到你的家人在参加葬礼的亲友前读出自己的一生。）

我开始进行录音，写分享文章。起初，由于语文能力较差，写作速度慢且费力，只能尝试写一些简短的分享文章，随着阅读量与写作量的增加，我逐步完善，最终出版了自己的第一本书。（这个过程也像乔布斯的故事一样：只有事后才能知道最初的努力是否有用，而当时只能依靠直觉和兴趣来驱动。）

每人一生成就不是看是否名牌大学毕业，而是取决于能否找到最爱，并付出努力全身全力做到最好，并且定期回顾。因此我们应该将每一天都视作人生中的最后一天，只做最重要的事情。

“不积跬步，无以至千里；不积小流，无以成江海”——荀子《劝学》

附件

Pixar 制作第一部电脑动漫：玩具总动员 (Toy Story)

John Lasseter 出生于西岸 Hollywood，从小就非常喜欢卡通片，从加州美术学院（迪士尼出资创建）毕业后就进了迪士尼制作动画。和其他刚刚毕业的动画师一样，他渴望创造像当代《星球大战》那样的高质量作品，然而，他们的部门主管都非常保守，按照传统方式管理，经常发生冲突，最终被公司开除。

恰逢 Pixar 计划自制短片，以展示公司的软硬件技术实力，他就被招聘了进去（动画师都要经过多年的专业培训，收入不低，为了避免管理层的质疑，Lasseter 在入职时的职位是“界面设计工程师”）。

由于乔布斯对艺术情有独钟，而 Lasseter 又是公司里唯一懂艺术的人，因此自从公司被收购开始，两人的合作一直非常默契。

1986 年，公司决策要制作 2 分钟动画片展示公司的最新 3D 技术。

他就按自己所长，按他天天相对最熟识的桌灯制作动画短片。

同行看完短片初稿后反馈说，“虽然是 2 分钟短片也必须讲故事”

“短短 2 分钟，怎么可能讲故事？”Lasseter 想，但他还是按这建议，改成做成两把桌灯，一大一小，大的是父亲，小的是小孩。相互争夺一个气球，争来争去，最终气球被弄破了。（大家可能都看过，因为迪士尼 Pixar 制作的动画片通常都会在正片播放前放映这个短片。）

尽管在 NeXT 的工作繁忙，但乔布斯仍然抽出时间与 Lasseter 一起参加了 1986 年 8 月份的动画片大会 (SIGGRAPH)。结果这个短片大受欢迎，并获得了大会的最佳动画片奖。

乔布斯事后领悟到：Pixar 公司唯有这个短片才真正融合了艺术元素，而不仅仅是科技的展示。将艺术与技术相结合用于电影制作是公司的唯一出路，这与 10 年前他在苹果公司将技术和艺术结合创造出 Mackintosh 是一样的道理。

1988 年，尽管公司资金极度紧张，一直依靠乔布斯不断的注资，但他仍然为 Lasseter 的动画片部门拨款 30 万美元制作短片，但叮嘱必须做出优秀的作品。不负所望，《Tin Toy》得了 1988 年短片奥斯卡（学院奖），迪士尼高层想招聘 Lasseter 制作电影，但他不愿意离开 Pixar。迪斯尼与 Pixar 开始谈制作玩具总动员 (Toy Story) 动画片。尽管乔布斯是一位谈判高手，但由于 Pixar 是一家小公司，急需新项目来维持运营。经过几个月的商务谈判，双方签署了商务合同。由于迪斯尼拥有绝对的强势地位，Pixar 只是迪斯尼的合同工：迪斯尼拥有所有人物和电影片的版权，而 Pixar 则从票房销售中获得 12.5% 的份额。此外，迪斯尼有权继续以这个主题与 Pixar 合作制作两部电影，但迪斯尼拥有选择权，可以随时终止合作，而无需支付任何赔偿。

《玩具总动员》从 1991 年开始制作，但因为迪斯尼的负责人将里面的主角吴迪 (Woody) 变成一个有个性但缺乏人性的牛仔，导致团队辛辛苦苦做出来的初稿在 1993 年 11 月被迪斯尼高层叫停。然而乔布斯并没有解散团队，他认为仍然有希望继续合作，并鼓励团队按照 Pixar 原有的思路改善动画片。经过 3 个月的努力，他们再次向迪斯尼高层展示，重新获得迪斯尼的支持。1994 年 2 月，他们同意继续制作，但由于已经发生了大量的返工，最初的 1700 万美元预算无法完成了，乔布斯认为这些超支应由迪斯尼承担，因为这些超支是因为迪斯尼动画部门的管理失误引起的，但迪斯尼管理层不同意。最终经过多轮谈判与协商，调整了预算。

乔布斯本来没有太多参与玩具总动员 (Toy Story) 的制作，但后来他越来越投入，后期初稿出来后，他每次邀请亲友们到家里看更新后的动画片。有亲友回顾说：“每次去乔布斯家都要看他展示动画片的最新版本。虽然可能只是优化了不到 10%，越来越觉得有点烦。”

1995 年乔布斯被迪斯尼邀请一些大型的动画片宣传活动，他预计到可以依赖动画片制作让 Pixar 公司获得投资资金，所以同时筹备在 Toy Story 开片同年让 Pixar 公司上市（做 IPO）。然而许多投资公司对此表示怀疑，因为 Pixar 公司在过去的 5 年里一直处于亏损状态。乔布斯回顾道：“当时确实有些紧张。也有人劝我应该等到第二部电影上映后再进行 IPO。”但乔布斯解释道：“我们迫切需要这些资金，这样我们才能与迪斯尼就第二部电影进行谈判。如果我们没有资金，就无法在谈判中争取到更好的合同条款，我们将永远只是迪斯尼的合同工，没有发言权。”《玩具总动员》大获成功，首周票房达到 3000 万美元，已经超过了全部制作成本，随后一直保持着当年票房最高的排名，在美国本土的票房达到 1.92 亿美元，在国际市场达到了 3.62 亿美元的票房，同时影评也给予了极高的评价。

参考 References

1. Fritz, Robert. "The path of least resistance for Managers." (1999)
2. Isaacson, Walter. "Steve Jobs." (2011)

Chapter 26

北京手记

北京是全国文化之都，来到北京晚上有空便去欣赏各类表演节目。连续 3 晚：听了一场音乐会，看了两套话剧。

国家大剧院管弦乐团演奏马勒第三交响曲

到了最后第六乐章的最后 5 分钟，两位定音鼓手同步咚、咚、咚、咚，像为步兵打鼓（也像赛龙舟的鼓手），配合台上一百多位乐手，包括 6 位打击乐乐手、40 位铜管木管乐手，每位都专注着台前挥动着的指挥棒，大家没有一点放松，把全场听众带到最后高潮。

在大剧院演出音乐厅全场几百位听众已经经历了近 100 分钟的演奏，包括第 4 乐章的女中音演唱，和第 5 乐章的合唱团加女中音演唱。乐章之间都没有休息。回想上次类似的经历是在香港大学音乐厅，贝多芬第 29 钢琴奏鸣曲（“Hammerklavier”）的演出，全曲长达 45 分钟。当钢琴家演奏完，全场听众和演奏家都好像跑完马拉松一样，都想一起双互拥抱，庆祝可以一起完成这不凡之旅。无论家里有多贵多高级的 HiFi 音响设备和投影大屏，都无法取代这些现场音乐会的感受。

很赞同美国中提琴家 Kim Kashkashian 在 YouTube 被访问时说：“古典音乐的未来取决于有现场听众和演奏家一起的音乐会（而非单靠灌录唱片、视频、DVD）。”

因为与话剧一样，这种互动才让表演有生命力。

话剧：进入黑夜的漫长旅程 (Long Day's Journey into Night)

话剧表演结束，看看手表已经快 10:00，不知不觉连续没有休息整整看了差不多 150 分钟，现在才感觉到自己屁股有点疼，坐得太久了。

虽然只有一家人 4 位演员，父母亲 and 2 个儿子，但一场接一场的夫妻对话，父子对话，兄弟对话，让观众从他们早上的笑容背后，慢慢感受到每个人的痛苦、无奈、绝望。在父亲和小儿子 Edmund 晚上交心，父亲说出自己小时候一家人从爱尔兰移民到美国的穷困经历，后面他虽然赚到钱，但深知赚到钱不容易，却被家人认为是“守财奴”，如何不顾家等，我实在控制不住自己的眼泪，拿手帕

擦脸。

这部戏是美国的经典剧目，作者尤金 • 奥尼尔 (Eugene O'Neill) 先生生前 3 次获普利策奖，这部是他自认为好的作品，在他死后 1956 年出版，于 1957 年也获得普利策奖。

话剧：完美陌生人

虽然票价最贵 (¥300)，但是 3 晚最弱的演出。虽然 7 位演员都很专业，也非常努力，但未能感动我。剧本改编自一套 2016 年的意大利电影，讲述 7 位老朋友 (包括 3 对夫妻) 在其中一家人的家里晚上聚餐。聚会时玩了一个游戏，他们把手机都放在餐桌上，和在场所有人公开分享收到的信息和电话。结果像一个个炸弹不断爆炸：暴露出各种家庭纠纷，乱七八糟的男女关系等。开始时觉得很有意思，但越来越多就变麻木了，反而觉得这些故事是导演想象出来的，不真实。我也不喜欢演员都挂着麦克风 (microphone) 说话，好像晚上听收音机广播，没有与观众之间对话的感受。

创新来源于生活

唯一靠多听、多读、多看，不断扩大视野，才能提高个人创新（原创）能力。但千万不要局限在自己专业领域内的东西。

有人觉得香港大学的数学系归属于文学院，而非工程学院或理学院很奇怪。

虽然很多工程和科学发明是基于数学，但不能说数学与哲学、音乐等无关。著名数学家伊藤先生就说过：“其实是数学可以帮助我们发现大自然的壮丽和真理，比起很多哲学家的理论更落地。”

所以不应预设框框。

因为马勒的第一交响曲很成功，他就有了后面交响曲创作的蓝图：

- 超大型乐队
- 以歌曲作为主题
- 抛弃了传统德国交响曲的格式：例如奏鸣曲式 (sonata form)

他希望通过第三交响曲达到他的创作理想。他终于在奥地利萨尔茨堡 (Salzburg) 近郊阿特湖畔度假时，从周边的高山湖泊、鲜花巨木、万物生灵获得创作灵感。本来 6 个乐章都有标题：

- 牧神苏醒，夏季昂头阔步地迈进
- 草地上的花儿告诉我
- 森林里的动物告诉我
- 人类告诉我
- 天使告诉我
- 爱告诉我

但他在 1902 年首演前把所有标题都删掉：“希望听众直接从音乐去体验，不受标题影响。”

马勒的交响曲创作验证了创新章里提到弗里茨 (Fritz) 的创新理论：所有创作都是由很高但现在未达到的理想来驱动，觉得有差距（张力），从而产生计划与

行动。

如果只是借用人家的故事，内容很难有深度，必须基于个人的亲身经历。作者奥尼尔年轻时考进了著名的普林斯顿大学，但一年后就因为酗酒被学校劝退，所以他才可以这么深入地描述《进入黑夜》里长子 Jamie（一名酒鬼）的想法、心态和酗酒的诱因。其实他只是借用戏剧里的 4 位角色把自己的亲身经历反馈给观众而已。

田纳西·威廉斯（Tennessee Williams）的第一部成名话剧《玻璃动物园》（The Glass Menagerie）也是写自己的家：轻度残废的妹妹找不到工作，母亲想尽办法帮她找对象等。

要实践与坚持

一个月前开始启动香港某公司的过程改进，中间也遇到很多困难。比如培训已经过了一个月，跟顾问讨论，他还一直抱怨说推不动，客户里有些人就只想拿些模板，用最轻易的方式达到级别。我劝他我们只要尽力做好自己本分，因资源不在我们手上，如果客户自己不注重，不希望改好，我们做任何事也没用。我们只可以在主要关键阶段提醒，应该做什么，人家没去做，我们也没办法。这些真实的经历，如果没有及时记录下来，后面就很容易忘记了。假如我没有把刚刚连续看演出的感受记在纸上，过了一个月后肯定都变得模糊。所以我也尽力每天把过程记录下来。

所以除了多听多看外，还要多思考、多写，因为如果没落到纸上都只是心中的梦想。大家有每天写日记的习惯吗？

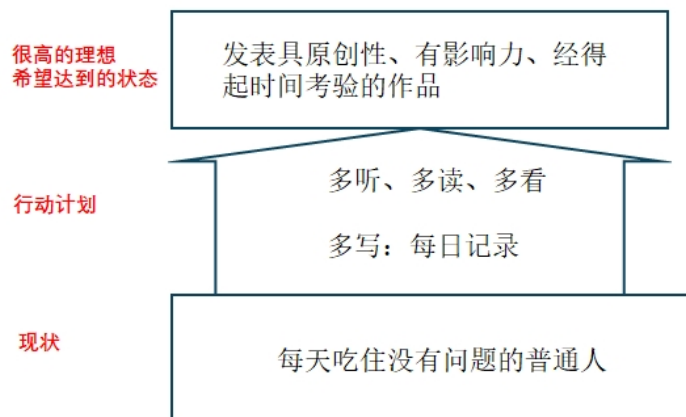
现在科技发达，比 120 年前民国前要用毛笔写字轻松多了，但当时很多伟人还是每天不停写日志。我们现在可以利用语音识别等，花 15 到 30 分钟把重点记下来。

我虽然每天记录、写草稿，估计最终能有十分之一留下来发布已经不错。

在美国，越来越多年轻女性参加以前只有男性的体育运动比赛，导致很多女性因缺乏准备、锻炼和热身，会腿部前十字韧带受损，做紧急手术。记者 Cynthia Gorney 就想按这个主题给纽约时报杂志写专题文章。整个过程像漏斗一样：先把各研究、访问得到的相关资料放进漏斗，到了交稿截点前开始整理资料，把零散的资料组织成有架构的文章，分章节、如何开头、如何结尾等。然后杂志编辑会按读者群体的关注点精简初稿，比如她初稿也包括研究过程中她觉得挺有兴趣的替换膝盖手术，她甚至特意飞到 Cincinnati 观察医生做手术，但编辑觉得这些跟主题无关，非读者关注，把这些部分都删除掉。经过多轮的过滤、修正，最后在杂志出版的文章 4000 字，共 7 页。

但是确实要靠这去芜存精的过程才可以保证发布文章的可读性和吸引力。

创新章里不是说过乔布斯在大学演讲的时候给学生提 3 个故事，第一个故事分享当初在大学学美术字（Calligraphy）时只因为很感兴趣，没有细想有什么用，到后来在苹果公司为 Macintosh 个人电脑设计字体时，才发现能用上。他用这故事说明很多时候过后回顾才知道以前那些点可以串联起来，对后面有作用。我也希望能在日后能用上之前自己的听、看、读、写。



和上面话剧、音乐会一样，必须与其他人交流，不断完善才能出好东西。总的来说，越接触各领域大师的创作，就越觉得自己渺小，但只要知道自己是今个月比上个月有进步，比半年前有进步就可以了。

不是每个人都可以成为乔布斯或马勒、尤金·奥尼尔，但都可以成为本职工作的专业知识工作者。

我喜欢看演出的另一个原因是想感受他们的专业性：

台上的音乐家、指挥、歌唱家、演员，和幕后的导演、戏剧作家、作曲家等。

专业和业余的区别：已经从多轮经验教训改善、精益求精，能让客户完全放心。

要专业，除了具备本职工作的知识技能，也需要有前瞻性。要不断改善，除了具备相关知识以外，更重要是态度：必须有自我不断完善的愿望 (self-motivation) 和个人规范 (discipline)，只有这样做才可以保持自己的专业地位。

在当今竞争激烈、需求不断变化的大环境下，企业每位员工都要准备好自己，做专业 $\times\times$ 师。员工要专业，也需要上面提到多听多读多看多写，打破现有的框框，全局了解客户的要求，动脑筋分析，不断改善，不然这些都只是梦想。

Chapter 27

根与翼

中国社会一直非常强调家庭价值观，希望实现家族的持续传承，家族有族谱，代代相传的关系对每个家庭成员的成长产生深远影响。我们每个人都只是人类进化过程中的短暂过渡。父母普遍希望把最好的东西传承给下一代。然而我们需要问自己，什么东西能够持久，仅仅是财富和金钱吗？

继承大量财富，但如果不懂得如何正确利用，可能反而会削弱个人的竞争力，适得其反，最终成为社会的蛀虫。

《七个习惯》的作者史蒂芬·柯维（Stephen Covey）在书的结尾（最后一章）中提到，我们可以传承给下一代并具有持久价值的，只有 2 样东西：根和翼。

“There are only two lasting bequests we can give our children - one is roots, the other wings.”

根

回看人类过去 130 年，除了打了 2 次世界大战外，也做了很多前所未有的创新：

- 电灯（煤气灯）
- 汽车（马车）
- 飞机
- 摩天大厦
- 半导体，集成电路
- 计算机
- 互联网
- 移动电话

回顾人类的创新历程，我们现在使用的许多发明，实际上都是“发明者”基于前人的经验不断演化而来的成果。我在爱丁堡大学的古乐器博物馆见到过各种奇形怪状的古乐器，它们都是现代乐队中铜管和木管乐器的前身，如双簧管、单簧管、巴松管、圆号和小号等。就像达尔文先生在《物种起源》中描述的物种进化一样，经过优胜劣汰，最终演化成现代的乐器。

例如我们熟悉的口琴和手风琴，其实是欧洲人在中国传统乐器笙的基础上改造而来的。

人类创新并不仅限于前述领域，这只是我们看到的凤毛麟角，在医疗、电影、音乐、数学等众多领域，同样展现出了非凡的创造力。这些都是祖先留给我们的宝贵财富。

举例来说，如果 19 世纪欧洲城市的居民能够坐上像 “Back to the Future” 那样的时间跑车，见到 200 年后现代人的生活，他们将感到难以置信。

正因为每一代人传承并发展了前人的成果，我们才能取得现在的成就。这，便是我们的 “根”。

翼

除了 “根” 的传承，祖先还留给了我们 “翅膀”，赋予我们自由思考、打破负面基因传承的能力，而不仅仅是简单地将基因传递给下一代。

以下面 2 故事介绍 2 种不同的策略。

飞行实验

自古以来，人类就渴望飞翔，东西方对此都曾有过诸多尝试，但大多以失败告终。传说明代火箭专家万户陶成道在凳子上绑了 47 根火箭，并绑上风筝进行飞行试验，最终却因爆炸丧命。15 世纪，意大利的达 • 芬奇也曾尝试模仿鸟类翅膀设计飞行器，但也仅仅是停留在图纸上，并未实现。

莱特兄弟（Wright brothers）在 1903 年成功进行了机动飞机的首次飞行。在 12 月 17 日的第四次飞行中，他们在空中飞行了 59 秒，航程达到了 259 米。



莱特兄弟1903年飞行成功

莱特兄弟并非工程学或数学方面的天才，但他们为了实现“飞翔”的梦想，不断从失败中吸取教训并持续改进。从 1899 年起，他们开始不断试验：

1900 年，他们首次试验滑翔机，但效果并不理想。

1901 年，他们将机翼面积从 15 平方米扩大到 26 平方米，并成功飞行了 120 米，但发现试验数据与计算数据不符。为了找出原因，他们制造了风洞来测量各种机翼设计的浮力，共试验了 200 种机翼设计。利用收集到的数据，他们开始制作第三架滑翔机。

到 1902 年 9 月，莱特兄弟已完成了 1000 次试飞，其中空中飞行时间最长达到了 26 秒。正因为有了这些经验，他们设计了飞机的舵，使飞行员能够更好地操控飞机，随后他们开始设计使用四缸发动机和螺旋桨的机动飞机。

1903 年成功飞行后，他们继续改善设计。

到 1905 年 10 月，飞机已经能够在空中飞行 39 分钟，并能在飞行员控制下完成转圈。

飞行就是在不稳定的大气条件下操控一台同样不稳定的机器，充满了各种变数，因此极其困难。然而，莱特兄弟通过反复试验，发明了许多关键部件，如机翼、舵、发动机和螺旋桨等，并研究了它们之间是如何相互影响的。

莱特兄弟的飞行故事告诉我们，他们的成功绝非偶然。除了强烈的意念和冲劲，更为重要的是他们从自己和前人的失败中汲取经验教训并不断试验，通过数据收集和持续改进，最终实现突破。

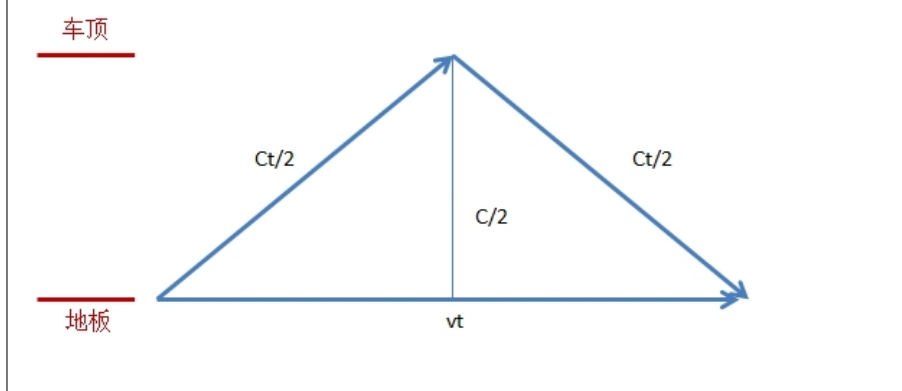
== ==

两年后的 1905 年，在大西洋对岸，瑞士伯尔尼（Bern）的一名 26 岁专利局员工发表了 5 篇科学论文，其中第 4 篇提出时间与空间并非固定，而是相对观察者的移动，这一观点打破了三百年以来对牛顿力学的传统认识。这篇后来被认为是物理界最具影响力的论文就是爱因斯坦的《狭义相对论》。

那么，这位年轻人是如何做到的呢？答案在于好奇心和年轻人无限的想象力。

他的好奇心源自之前物理学家在实验中发现光速是不变的，但这一发现与牛顿的传统力学相冲突。但他不是理学家，也没有实验室可供实验，只能凭借想象力，利用一系列假想的实验，创造了狭义相对论的公式。

他假设光速为 C ，高速火车从地板到车顶的高度为 $C/2$ ，光从火车的地板射到车顶后，再反射回地板，总共耗时 1 个时间单位。假设高速火车以高速 v 从左往右运行，一位静止的人从外面看这火车，也看到光从火车地板射到车顶，然后反射回到地板，因火车已经往右移动了 vt ，所以光线的总长度是 Ct ($= Ct/2 + Ct/2$)。如果光速 C 是永恒不变的，那么时间 t 必须大于 1 才对，并且随着 v 的增大， t 也越大。因此，在高速火车内部的时间比外部地面的时间要短。理解了因光速不变，时间相对必须变化的原理后，就能明白为什么会出现“太空飞行员乘坐高速火箭旅行 30 天后回到地球，发现地球上已经过去了几十年，亲人都已去世”的可能了。



(爱因斯坦在后来的采访中说：“我很骄傲的是，自己有大把的空闲时间。”因为在专利局工作，所以他有很多时间，可以尽情思考、想象。)

日本数学家樱井进先生从小就崇拜爱因斯坦，他总结道：

“爱因斯坦教会我珍视人类所拥有的‘想象力’。想象力比知识更重要，因为知识是有限的，而想象力则概括了世界上的一切。正因为有了想象力的翅膀，人类才能抵达无法到达的地方，看到不可见的事物。我们每个人都有这样的翅膀，可以展翅高飞。”爱因斯坦发表相对论时，大部分物理界学者认为这只是理论，无法验证，也缺乏实用价值。科学本身就是一种喜悦，是一场旅行，旅行者能看更多其他人看不见的风景。爱因斯坦教会我们享受数学与科学研究的乐趣，而将科学应用于实际则是另一回事。(随着科技的发展，相对论启发了后来的“宇宙大爆炸”和“黑洞”理论，并帮助我们能够精确地测量卫星的时间与定位。)

“我们都不知道。我们所了解的知识和小学生没什么两样。”爱因斯坦也曾说，“这个世界最难以理解的部分，就是去理解这个世界。”也正因为这年轻人敢面对这世界最难的挑战，使人类的物理认知上了一个台阶。他的策略是依赖年轻人无限的想象力，抛开前人留下的包袱，实现创新。

== ==

虽然我们都继承了祖先的根与翼，但并不是每个人都能为人类的持续发展作出贡献。要具备哪些重要元素才能做到呢？

To be a powerful transition person, one must first change his inner first.
 要成为有影响力的过渡者（a powerful transition person），无论采用哪类策略，我们必须首先进行由内而外的改变，只有这样的改变才能持久。
 （只有在内部发生转变，任何人才能在自己可控的范围内进行创新和持续改善。）
 以上源自 Covey 先生在《七个习惯》最后一章的总结。

“魔术师”乔布斯 1997 年重回苹果，13 年后再把苹果带上高峰

1985 年，乔布斯被迫离开苹果公司，独自创办 NEXT 公司，直到 1997 年 NEXT 被苹果收购，乔布斯重返苹果（当时苹果正处于破产边缘）。到 2010 年，他将苹果公司带回到美国市值最高的科技公司，这中间也经历了多次失败。

乔布斯在他的传记中说：“是什么力量推动了我？我觉得大多数创新者（Creative people）都希望感谢我们的前辈，感激他们对人类的贡献。

我并没有发明我用的语言或数学。我的食物基本都不是我自己做的，衣服更是一件都没做过。我所做的每一件事都有赖于前人的贡献。我们很多人都想回馈社会，在历史的长河中再添上一笔。虽然我们无法创作鲍勃·迪伦（Bob Dylan）的歌或汤姆·斯托帕德（Tom Stoppard）的戏剧，但我们可以用自己能掌握的方式去表达和体现。试图用自己的才能表达内心感受，表达对前人的感激并做出贡献，这就是推动我的力量。”

共通点

为什么莱特兄弟、爱因斯坦、乔布斯能在航天、物理、电子产品领域作出突破性重大贡献？他们的故事都有两个共通点：

- 专注 -- 例如：
 - 专心研究飞机
 - 解决物理难题，更好理解这个世界
 - 结合新技术研发针对个人市场的领先电子产品
- 长期锲而不舍 -- 从启蒙到成功都经过很多年不断努力，中间困难重重，必须内心坚定，也正是这心态让他们成为有影响力的过渡者，继续传承人类的根与翼。例如，乔布斯从被迫离开开始，前后经过 25 年，经过 iMac iPod iPhone iPad 等，最终才使苹果再创辉煌（因为这是公司级突破，不是个人，所需时间更长）。

“你的例子都是超级明星级大人物，根与翼概念不一定适用于我们这些普通人群。”

“为什么你认定自己不能成为有影响力的过渡者？”我问。

每次听到这类反驳，我都会接着讲我的故事。

因从小很喜欢看书，从小学到大学毕业，一切都很顺利，没有遇过什么挫折。例如，因 80 年代香港经济仍在高速发展，大学毕业后不难找到高薪岗位，每年还有两位数的调薪，所以对自己的能力信心爆棚，也看不到自己的问题和弱点。

但香港缺乏研发工作，1992 年我 31 岁，第一位小孩出生时，我的工程职业生涯已到了天花板。如果要继续提高收入，必须转到市场和销售岗。但转销售后，虽然能达标，但一直没有突出的表现，最终在 2000 年被公司辞退，沦为失业者。失业后我审视自己的问题和不足，发现自己一直以来保持着工程师思维，总觉得只要方案优良、产品有竞争力，客户就没有理由不买！但不知道产品只是商务购买中的一个考虑因素，市场如战场，有太多的因素会导致失败。最关键的是自己一直没有这种意识，也不觉得有问题。

现在静下心来想，自己一直都是先制定总体计划（最短也几个月），然后按计划执行的传统工作模式，这方式在 8、90 年代香港没有问题，但 2000 年后市场环境快速变化，思路必须转变过来，当时也刚读完软件工程，学过敏捷和精益等概念，自己的工作方式也转为每一小步试验，有效果才做下一小步的做法。例如写这本书里，每写完一个章节，必须找我的“写作伙伴”们看看，让他们给反馈，有些专门挑语法问题，有些能专门看出总体架构问题。经过他们的反馈，我再修正，然后才会在公共媒体上发布。

现在静下心来想，自己一直都是先制定总体计划（最短也几个月），然后按计划执行的传统工作模式，这方式在 8、90 年代香港没有问题，但 2000 年后市场环境快速变化，必须转变思路。当时我刚读完软件工程，学了敏捷和精益等概念，自己的工作方式也转为每一小步试验，有效果才做下一小步的做法。例如在写这本书时，每写完一个章节，必须找我的“写作伙伴”们看看，让他们给反馈，有些专门挑语法问题，有些能指出总体架构的问题。经过他们的反馈，我再修正，然后才会在公共媒体上发布。

想尝试创业，但创业很难，风险非常大。不能用传统方式，先制定大计划再执行，而是不断尝试各种能产生收入的机会，看效果。因此，开头几年我试过各种工作，比如给企业做项目管理培训，进高校当讲师，帮朋友公司做市场调研与分析，并写产品推广方案书等等，为自己带来一些收入。因因为刚读完软件工程，在富士通工作时，我也与一些 CMMI5 级软件开发公司合作过，就开始探索软件工程的质量改进并参与 CMMI 评估，慢慢学习（因毕业后一直没做过软件开发）。

从 2006 年开始，香港政府鼓励软件开发公司使用 CMMI 模型做评估与改进，并提供资助。我便开始专注于软件开发 CMMI 过程改进的培训与评估，并从长三角地区开始，与大陆公司合作提供 CMMI 服务（虽然我的成本较高，但 CMMI 源自美国，绝大部分材料都是英语，与大陆高校老师比较，我是直接读英文原文，比单看中文翻译能更好理解。）

十年后，2016 年左右，越来越多团队使用敏捷开发，我便开始深入比较各种敏捷开发方法。2018 年，正值美国 PMI 推出 ACP 个人证书，我开始与培训机构合作教授学生 ACP，发现如果团队能在每次迭代回顾中利用数据（缺陷和返工）分析根因，能够更有效地提升（背后其实是改善，CMMI，敏捷，精益等原则）。

随着接触的开发团队越来越多，我发现要有改进动力不仅仅是工程的问题，又开始探索人性相关问题（虽然以前读 MBA 时接触过，现在再细读 X-Y 理论、组织发展（Organization Development）的方法与理论），并在培训和现场辅导中不断实验有效的方法帮助团队提升。

学习敏捷大师和软件工程学院科学家与学者等前辈们的贡献，成为我的软件工程的根，没有他们，我不可能从零自己创造这么多东西出来。不固步自封，因市场环境变化，从传统模式改变为精益思路，是我的翼，这这些都是我自身根和翼的经历，让我成功走过 24 年的独自创业之路，也帮助我成功出版第一本书（也希望这本书能为软件工程做些微小的贡献）。根与翼如果能用得其所，确实能促进自己成为有影响力的过渡者。

所以任何人，只要把心态转过来，并能锲而不舍，都有机会。”我总结说。
“好吧。但你说这些跟敏捷开发、持续改善有什么关系？我没有期望能成为爱因斯坦！”

我认识的很多软件开发公司，虽然已经进行了多轮 CMMI 评估（每三年一轮），但一直没有任何实质的改善，是什么原因呢？

他们误以为 CMMI 是一套最佳过程，只要完全按照这个标准执行，开发质量便能得到保障。

美国软件工程协会在设计这个模型时，考虑到团队每个岗位都要有用，因此覆盖的范围很广，除了软件工程从需求到最终验证确认外，还包括项目管理、配置管理、PDCA 试点与推广、度量与根因分析等。虽然模型是面面俱到，但企业在使用这个模型进行 PDCA 过程改进时，每一轮试点应只针对最大的痛点进行改善，显著改进后再针对次要痛点进行改善，最终满足模型所有实践范围。因此，每当美国软件工程协会（CMMI 模型原创机构）听到某企业能在 12 个月内从零开始，迅速“高效”达到三级评估要求时，都会觉得这是不可思议的笑话。

与成为有效过渡者的要素一样，企业、团队、个人要有效改善，必须有针对性，大部分企业没有利用数据、找根因、做改进。所以开始时应只针对缺陷排除和返工工时，每次迭代回顾时分析根因。

“听你这样说，反观我们团队，我立马想到有很多地方需要改进，例如需求调研、开发流程、持续集成、测试自动化、配置管理等。为什么你提议只针对减少缺陷引起的返工做改进？”

要理解背后的原因，先听听质量大师 Dr Juran 回顾过去质量改善的成功要素：

- All Improvement takes place Project by Project 所有改进只能按项目逐个进行
- Backlog of Improvement projects is huge 潜在的改进项目非常多
- Quality Improvement does not come Free 质量改进并非免费
- Major gains come from the Vital few projects 主要效果（收益）来自少数重要项目
- Quality improvement extends to All parameters 质量改进可扩展到各类参数（如生产率、进度、安全性、环境等）

（以上源自 Juran 博士《Juran Quality Handbook》第五章质量改进。）

所以，如果从一开始同时对 4 到 5 个方向进行改进，反而会失去聚焦，难以尽早看到改进的效果。

但如果只针对降低缺陷引起的返工，目标明确，不仅让大家更容易接受，也更容易扩展到各个过程，例如需求、开发、产品集成，甚至配置管理。

因为能否引起高层对改进的兴趣非常重要，所以与企业领导首次见面时，我都会讲述团队应如何分析迭代缺陷数据，以此开始定量管理。

TED 演讲式总结

今年被质量经理邀请参加他们的质量沙龙，但只有 30 分钟，我就用了 TED 演讲思路，用了 17 分钟讲软件开发公司的常见质量问题和改进措施。

你觉得下面 4 种工作，哪一类工种占的工作量最多？

1. 编码与代码设计。
2. 交付后的所有工作，包括维护、更新与缺陷修正。
3. 交付前的评审、静态扫描、测试与缺陷修正。
4. 项目管理与监控。

从 2012 年 Capers Jones 对美国软件公司的统计中看到，编码只排第二（占开发工作量的 25%），而测试、评审和缺陷修改的总工作量则排第一（占 30%）。你选 1. 编码，其实大部分耗时的不是写代码，而是修正错误代码。软件开发有其特点，缺陷发现越晚，修复所需的工作量就越大。缺陷如果在早期被发现，可能只需要花费半小时或 1 小时进行修复。原因很简单，当已经集成为一个十几万行的系统，里面有很多组件。如果在完成集成后使用时才发现 BUG，首先很难找到出问题的代码（原因很简单，当已经集成为一个十几万行的系统，里面有很多组件）；然后，为了验证代码修正是否有效，必须先测试单元模块是否通过，再测试集成后是否正常，最后还要通过系统测试才能交付给客户。相对而言，若能在自测、集成测试和评审阶段时尽早发现并解决问题，将节省大量时间。

然而，大部分缺陷都是在系统测试或集成测试时暴露的，有些产品线的缺陷数甚至达到四位数。如果能够将这些缺陷减少一半，将为公司节省近一半的工作量，同时也能改善产品质量，对吗？

道理大家都知道，但为什么没有改善？怎样才能改变现状，提前发现缺陷？这种改进绝对不是通过下达一个命令或领导开会讲讲就能做到的。我先讲一个故事：在一家上万人的大型公司中，质量经理表示公司一直非常注重量化管理，为此制定了各种 KPI 考核团队，每月由主管分析数据。开始时确实有些改善，但过了一段时间后，效果就不明显了，甚至发现数据作假的情况。所以，我告诉质量经理，如果希望实现量化管理，就必须从团队提升开始，而不是仅靠总体数据分析和制定公司级改进策略。度量与分析最主要的作用在于帮助团队做好根因分析，制定纠正措施，并在下一轮迭代中进行提升。例如，你们现在按 2 到 4 周进行一次迭代发布，确实也有一些团队在复盘，但他们不理解纠正措施与改正的区别，通常只是针对个别缺陷进行修复，结果仍会因为未修复系统的基本问题而导致缺陷和事故重现。然而，如果整个团队在每轮迭代中分析数据、找出根因并制定纠正措施，并在下一个迭代中验证这些措施的有效性，就能养成良好的习惯。最重要的是，领导要给团队足够的时间进行复盘。要完成这种根因分析的复盘通常需要 3 个小时，因为要收集数据，并进行分析和讨论。

我们可以先从分析缺陷排除率入手，帮助团队养成复盘分析根因的习惯。缺陷有真实数据，让团队分析为什么需求引入的缺陷这么多？引导团队使用“五个为什么”方法找出根因。同时，团队成员需要了解当前的质量水平。通常，问需求人员自己的质量水平如何？他们会说需求评审中发现的 BUG 很少，就代表质量好，但这其实是一种误解。他们在评审过程中没有发现并不表示需求没有缺陷，等到客户发现时会更糟糕。

我们还可以利用一些预测模型和统计分析的方法。在迭代中，不仅要让团队制定纠正措施，还要设定定量化目标。例如需求评审应该发现多少缺陷？单元测试应该发现多少缺陷？如果未达到预期目标，说明可能还是会遗漏不少缺陷。通过设定这些目标，可以更好地推动团队进行前期的评审和单元测试。如果团队开始养成分析根因并制定纠正措施的习惯，而领导又关注团队的持续改进，时不时地询问“下次迭代是什么时候？改进了多少？”，就可以扩大根因分析的范围。除了分析缺陷外，还可以把根因分析扩大到其他改进方向。那么，如何开始呢？

只是把小孩扔进海里，他是学不会游泳的。因此，建议先进行培训，举办为期两天的工作坊，利用一些真实数据带领大家进行根因分析的互动练习，然后安排公司内部教练辅导团队，帮助他们在迭代中做好复盘。

如果有老师或教练教授各种分析技巧，并帮助大家客观地看到团队的水平，大多数团队会对降低客户缺陷率和返工的明确量化目标产生兴趣，愿意一起分析迭代缺陷和返工量的数据，制定下一轮的改进计划与措施（好比《汤姆历险记》里，Tom 认为给花园栅栏涂漆是件极富挑战，一般人难以做好的任务，他的朋友圈都会很想参加，甚至愿意付 Tom 礼物）。相反，如果目标仅仅是为了公司顺利通过每三年一次的评估，很多人可能会说后面的 3 到 6 个月很忙，没有时间。

但任何显著的提升同样需要有他们内心思维的转变。改变固有的思想和习惯并不容易。如果个人、团队或企业没有改变思维，满足于现状，觉得暂时不需要继续改善，敏捷大师提供的敏捷宣言、12 条原则和各种方法（如 SCRUM、XP），并进行相关的培训和技巧学习，最终都无法在敏捷开发中产生实际效果。

反之，如果个人和企业领导对当前的工作状态都不满意，从内心深处相信必须改善质量、有危机意识，认为这些是工作的重要组成部分，是唯一能真正完善的，那么在改善过程中，或许不必单纯依赖敏捷原则和方法，甚至可以创造出更好的方法与技巧。

团队和企业要健康成长依赖以下主要因素（与上一章创新的要素类似）：

* 高层要制订高的目标，也要提供资源。

- — 可参考开始过程改进之旅里面的获取高层支持，尽早估算改进能节省多少成本，要求立项，把过程改进作为项目来管理。
- 改进要有专注。
 - 例如针对如何减少缺陷返工，尽早在评审和前期测试发现缺陷，如何做好迭代回顾部分能帮助团队了解应如何开始。
- 团队成员的能力与动力。
 - 个人提升、自我管理、代码质量等部分能帮助提升。

有竞争才有进步，不知道自身的不足，缺乏危机感是改善的最大阻力。

东北虎作为濒危物种，目前在国内的数量估计不超过 30 只，因此国家动物保护组织采取了多项措施予以保护，其中包括人工饲养。专家曾试图将这些人工饲养的东北虎重新放归野外，但这一任务极为艰巨。它们是否能够在野外独立存活下来，仍然是一个未知数。有一部纪录片生动地讲述了动物保护组织在这一过程中所面临的各种挑战和困难。

要确保老虎在野外能独立生存，它们必须能够自行觅食。在自然环境中，它们主要的捕猎对象就是鹿，但鹿非常敏捷，时速可达 50 公里，而且具备很强的耐力。因此，专家需要评估老虎是否能够成功捕猎到鹿，否则就不能将它们放归到野外。在评估过程中，专家发现老虎的奔跑和狩猎能力明显不如野生老虎，因为缺少运动，它们缺乏狩猎的能力。因此，专家设计了多种锻炼方法，如使用车辆牵引轮胎，并插上鲜肉块来刺激它们奔跑。经过一系列的锻炼，老虎的体力得到了恢复，但能否成功捕猎到鹿仍然是未知数。接着试验是用汽车拖着鹿的尸体吸引老虎奔跑。然而，有些老虎因为在平时的饲养过程中并没有吃过鹿肉，所以对此并不感兴趣。最终只有两三只老虎追着汽车跑，试图抓到鹿。最后有 2 只年轻的老虎获胜了。但要将被杀的尸体作为奖励喂给获胜的老虎也并不容易，发现它们对着鹿的尸体不知道如何下口，因为以前都是由饲养员将肉切好后喂给它们吃。最终这 2 只老虎花了数小时才慢慢学会如何吃到鹿肉。

环境是影响团队能力与动力的重要因素。看完这部纪录片，我立刻想到了有些本来很有抱负的年轻程序员，也是同样受到环境的影响。公司大部分程序员都只是天天赶工，不注重质量，管理层也习惯了大量的缺陷等到后面测试阶段才暴露，大量返工这种生产方式，如何能希望这些年轻的程序员注重代码质量。跟保护区把饲养的东北虎放归野外一样，但这些新人几个月后也习惯了大家的工作模式，后面要改变习惯便非常难。需要经过艰苦的培训，才能提高大家的竞争力。关键是管理层必须认识到，如果自己也满足于当前的模式、质量水平，对产品质量和不断改进没有要求，那么公司的产品质量将永远无法提高，也难以与其他公司竞争。

针对软件开发，我们这一代继承了之前从 60 年代以来的各种硬件软件的创新，给予我们继续发展的“根”。

敏捷宣言指出了传统瀑布式开发和传统计划驱动开发的弊端，给我们“翼”，为有能力的团队提供了机会，可以更快速地开发出高质量的产品。

千万不要误以为这过程会一帆风顺，按计划执行就能达到目标。实际肯定会遇到种种无法预料的问题，但只要个人、团队、企业了解当前水平，分析根因，采取纠正措施、持续改善，像第一章丰田方式的习惯 4：“持续改善没有停止”——龟兔赛跑里的乌龟。

以上简单道理不仅适用于敏捷开发，但回顾周围，有多少人、团队能做到？

即使知道了最佳实践方法，如果没有动力（心态转变），最终只是梦想。

反观我的动力是什么？虽然软件工程发展历史不长（50 年代才出现第一台大型计算机），但发展速度惊人。环顾周围，几乎难以找到没有附带软件的产品。在此过程中，我们受益于软件工程的前辈，包括软件工程学院的 Humphrey 先生，

发表敏捷宣言的 17 位敏捷大师等，以及质量改善和组织发展的研究，如 Juran 博士、Lewin 教授、Weisbord 先生等。希望在我有限的能力和时间内，尽可能系统地总结这 20 多年与软件团队交流中的所见所闻和经验教训，解读敏捷开发最佳实践，供大家参考。

太太趁中秋长假邀请我跟她一起参加丽江香格里拉五天雨崩徒步旅行团。我们在丽江汇合后，晚上她就问我：“我的同事都说当你老公真辛苦，放假还要跟你徒步五天，没有休息。你朋友一般放假去哪里旅游？”

我没有回应，只是微笑，但心里想：“其实我认识的老朋友大部分都是选择去日本泡温泉、观光名胜、吃当地美食等，没听过参加这种徒步团。”因为海拔高超过三千米，而且要赶早班飞机，早上四点钟就要起床，全程五天我身体都不舒服。最终还是能按行程走完。旅程结束，她回香港继续上班，我也继续飞石家庄在大陆出差。我回想今次经历：虽然身体不适，但过程中深深感受到伴侣间的理解、关怀、支持，甜在心中。旅程过后，身体慢慢恢复后，我反思：

头疼、反胃、呕吐不代表心里觉得痛苦；

快慰、舒适不一定觉得快乐。

关键是如何选择面对，用心去克服困难，之后享受的喜悦更难能可贵，也帮助个人成长。

如果大家幸运，公司高层理解不断改善的重要性，给团队资源，提供培训，也希望你能和我一样能正面去面对过程中的遇到的各种困难，不仅是你的宝贵经历，也帮助你成长为 A 级队员。个人受益，公司也受益。

参考 References

1. Covey, Stephen R. *The Seven Habits of Highly Effective People*. (1990) Fireside Book
2. Isaacson, Walter. *Steve Jobs*. (2011)
3. Juran, Joseph M. "Juran's Quality Handbook" (5 ed., 1999) McGraw - Hill (2011)

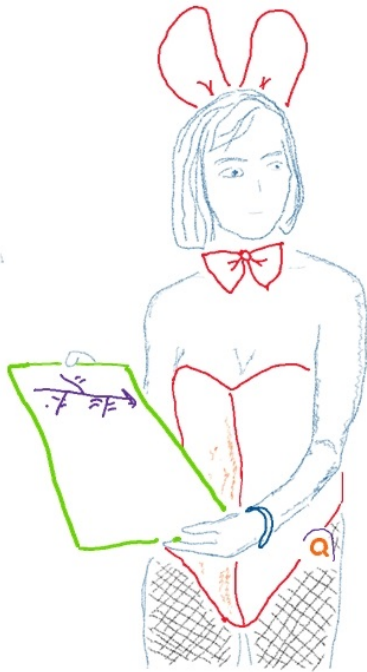
Part IX

附录

Chapter 28

A: 绅士俱乐部过程改进

美国学者 Dr Wheeler 到日本做培训，晚上好客的日本人请他去绅士俱乐部 (Esquire)。他发现年轻的女服务员腰上都有一个英文 Q 字牌子，便好奇地问原因。原来这家俱乐部刚完成几个月的过程改进，得到政府奖励金，所以都挂上这个 Q 牌 (Q 代表 Quality 质量)。



以下让我们看看这家俱乐部如何做过程改进。

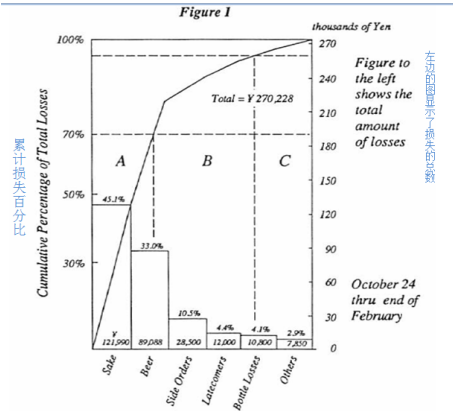
Esquire QC 圈

QC 圈成立于 1984 年 12 月，由 Sakae 领导，下面的女服务员。

当时，Sakae 43 岁，有七个月的经验。除了她，圈内 7 女服务员的年龄从 19 岁到 23 岁不等。

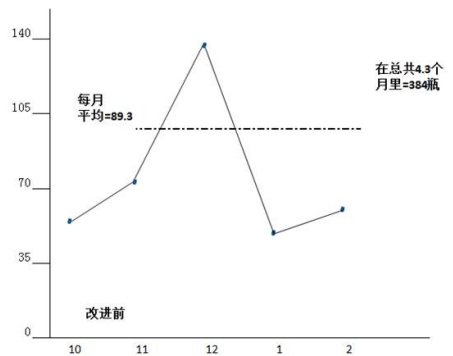
用二八原则（Pareto 图）识别主要源头

- 清酒 (Sake) 与啤酒 (Beer) 是最大的源头。

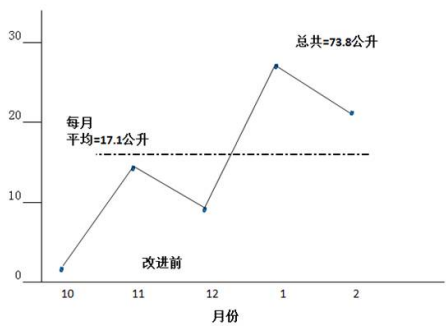


月份		从仓库领出的 瓶数	卖出的瓶数	损失 (瓶数)		从仓库领出的 瓶数	原瓶 (卖出的 瓶数)	差价	卖出的公升数	损失 (公升)
10	啤酒	409	350	59	啤酒	31.3	79	29.5	1.8	
11	啤酒	1047	972	75	啤酒	125.3	316	79	111.2	14.1
12	啤酒	1435	1297	138	啤酒	190.1	462	83	180.9	9.2
1	啤酒	883	835	48	啤酒	181.1	442	62	155.8	27.3
2	啤酒	844	780	64	啤酒	151	379	75	129.6	21.4

每月啤酒损失量



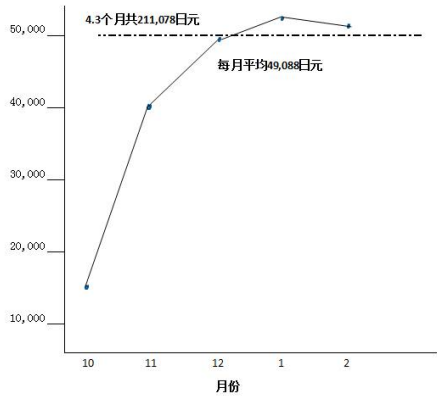
每月清酒损失量



结算出平均每瓶啤酒 232 日元，每公升清酒 1,653 日元

月份	损失 (日元)		每月损失 累加总数	支出 收入
	啤酒	清酒		
10	13,688	2,975	16,663	84.10%
11	17,400	23,307	40,707	85.40%
12	32,016	15,207	47,223	82.80%
1	11,136	45,127	56,263	77.60%
2	14,484	35,374	50,222	79.40%

啤酒和清酒每月总损失（日元）



从 10 月份到 2 月份，啤酒和清酒的平均每月损失约 49,000 日元。

建立目标

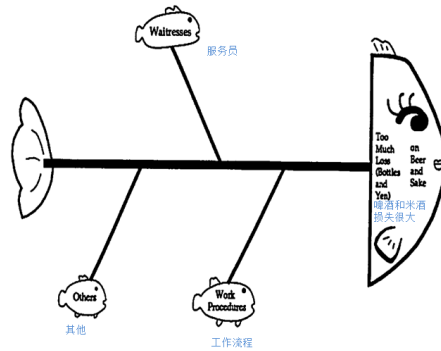
目标：把损失减半（少 50%）：从目前每月平均损失 89 瓶啤酒和 17 升清酒，降到（不超过）44 瓶啤酒和 8 升清酒。

根因分析 Root Cause analysis (鱼骨图分析)

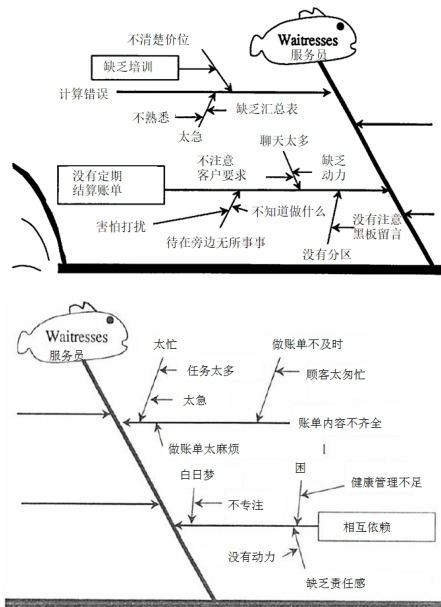
- 大家头脑风暴，讨论分析啤酒和清酒损失的主要原因。

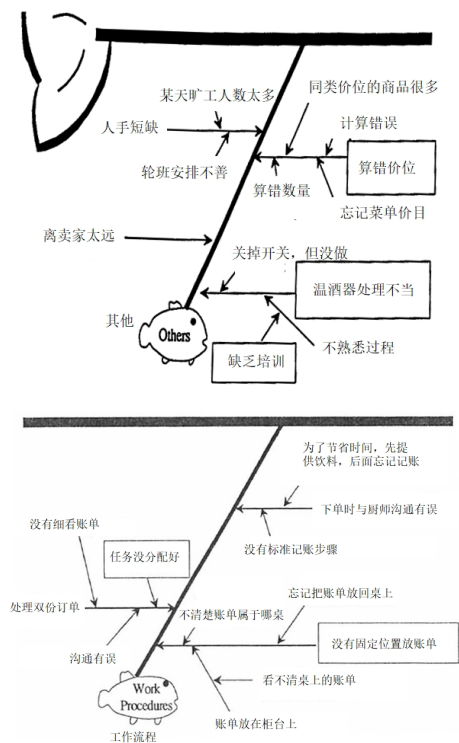
- 根据服务员、工作流程、其他等三个主线分析。

这三组构成因果图的三个主要分支。



鱼骨图每个分支的主要内容如下：





基于上面的鱼骨图分析，大家讨论并想到了以下主要理由/原因：

- 有具体的固定位置放置账单。
- 依赖别人。
- 处理热清酒器不当。
- 价格计算有误。
- 分工有误 / 没有分配好工作。
- 准备不足。
- 对客户桌结算（喝了多少并收拾空酒瓶）频率不足。

注意：以上每一项都是问题的根本原因，很多人误以为针对问题本身，找改正措施便算根因分析。“5 Why”方法可以帮助正确找到根因，详见附件“5 Why”例子。

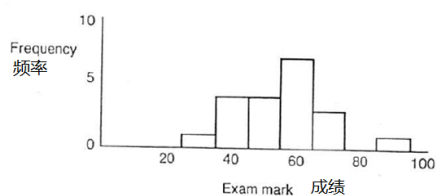
改进措施

- 要确保留票据在桌子上，不带走（在繁忙时段，在纸上标记消费了多少矿泉水、啤酒和日本清酒，贴在总单的后面，利于计算总账）。

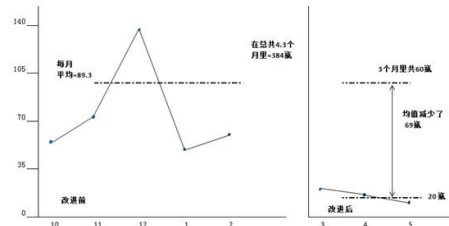
- 制定规程来分配各个区域的工作。每个小时每个区域负责人必须检查一次所有范围内的票据。
- 送热毛巾的那位服务员负责记录新加入的客人
- 谁接单谁负责记账，如果她要求其他人来协助记录，必须确保沟通好，检查是否已经做好记录。
- 管理者必须对新成员提供培训，如新员工小组学习。
- 以区域分桌，使大家清楚谁负责哪桌。

改进效果

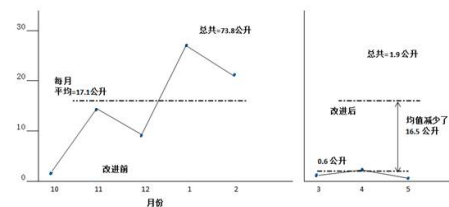
600px



改进后啤酒损失降低到平均每月 20 瓶

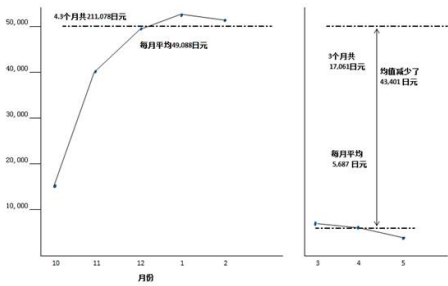


改进后清酒损失降低到平均每月 0.6 公升



结算出平均每瓶啤酒 232 日元，每公升清酒 1,653 日元

月份	损失（日元）		每月损失	累加总数	支出 收入
	啤酒	清酒			
10	13,688	2,975	16,663		84.1%
11	17,400	23,307	40,707	57,370	85.4%
12	32,016	15,207	47,223	104,593	82.8%
1	11,136	45,127	56,263	160,856	77.6%
2	14,484	35,374	50,222	211,078	79.4%
3	5,336	1,653		6,989	72.5%
4	8,472	1,157	6,029	13,018	74.1%
5	3,712	331	4,043	17,061	65.0%



以日元计算，啤酒和清酒的损失从每月平均 49,000 日元减少到 5687 日元，减少了 43,401 日元，即 88%。这一降幅远超过最初设定的 50% 的目标。

A3 报告

很多给管理层的报告都非常厚，信息太多导致管理者难以消化，所以丰田要求所有管理报告都必须精简到一张 A3 纸。照片中服务员手上的就是 A3 报告 --- 包含了根因分析的重点。例如你可以看见其中有鱼骨图。

参考 References

1. Wheeler, Donald J. : SPC at the Esquire Club (SPC Press 1992)

Chapter 29

B: 分析迭代客户满意度调查数据

案例背景

香港公司在广州的离岸软件开发中心，专门为香港的客户，如政府部门，做软件维护工作。从 2019 年，项目已经开始采用 SCRUM 敏捷开发方式，每两周一个冲刺，他们每次做完迭代后，客户都会填写满意度调查，然后分析数据。

迭代数据

每次迭代团队都会收集以下数据：

- 客户满意度调查（Cust Sat survey）
- 需求变更请求次数（Requirements and Change Requests）
- 系统测试缺陷数（Test defects）

数据分析

- Overall 满意度（整体指标）与 7~8 个因素相关，而与其中的 Deliverable Quality 相关系数达到 0.90，Deliverable Time 0.688 左右，其他的相关性都较低。

	P1A			
	Sprint49,50	Sprint51,52	Sprint53,54	Sprint55,56
Deliverable time交付时间	4	5	4	3.5
Deliverable quality交付质量	3.5	4.5	4.5	4
Overall satisfaction满意度	4	4.5	4.5	4

- 测试缺陷数和以下客户行为相关性如下：插入任务 0.51，需求模糊 0.66，初期没有需求问题 -0.45，需求不稳定引起返工 0.82。（详见下图）

(为什么初期需求模糊反而是-0.45：等需求明确，写成文档，导致后期才能给明确需求，反而比早期给个模糊需求好，因更可能引起返工。)

	需求澄清设计在UAT前或插入测试后澄清	需求澄清设计在UAT前或插入测试后澄清	需求澄清设计在UAT前或插入测试后澄清	需求澄清设计在UAT前或插入测试后澄清
qpr0010	1	1	1	1
qpr0011	2	0	1	0
qpr0012	4	1	1	3
qpr0013	2	2	0	3
qpr0014	4	1	1	1
需求澄清设计在UAT前或插入测试后澄清	0.0070	0.0070	0.4000	0.0033

若能写好需求，减少插入任务和变更，可以提升质量（降低测试缺陷数）。

改进措施

- 固化 Clarification 流程。当前是较为随机，由开发人员发起（统计数据中每个迭代 0~2 次），后面优化 Clarification 过程，要求每次迭代都必须做需求 Clarification，形式也更正式，明确甲乙双方哪些岗位、什么角色必须参与，在确认新过程有效后，就把过程固化下来。
- 把需求 Clarification 要具备的重点写成检查单，增加检查项，如：
 - 需求是否完整明确
 - 是否按简化功能点方法写清行为、实体
 - 场景是否明确
 - 能否有对应明确测试用例（是否可测试）
- 请甲方尽量减少插入任务和变更。

改进效果

除了客户满意度得到提升外，冲刺生产率也提升，例如：
生产率之前四分位数是 (1.07, 1.13, 1.16)，采取控制插入任务，减少变更，做好 Clarification 等措施后，有明显改善，变为 (1.04, 1.05, 1.08)。
(注：它们生产率 = 人天/功能点数，所以生产率系数是越低越好)